

МИЭМ

Департамент компьютерной инженерии

Москва
2026

Введение в ML 4: Глубокое обучение

В прошлых сериях:

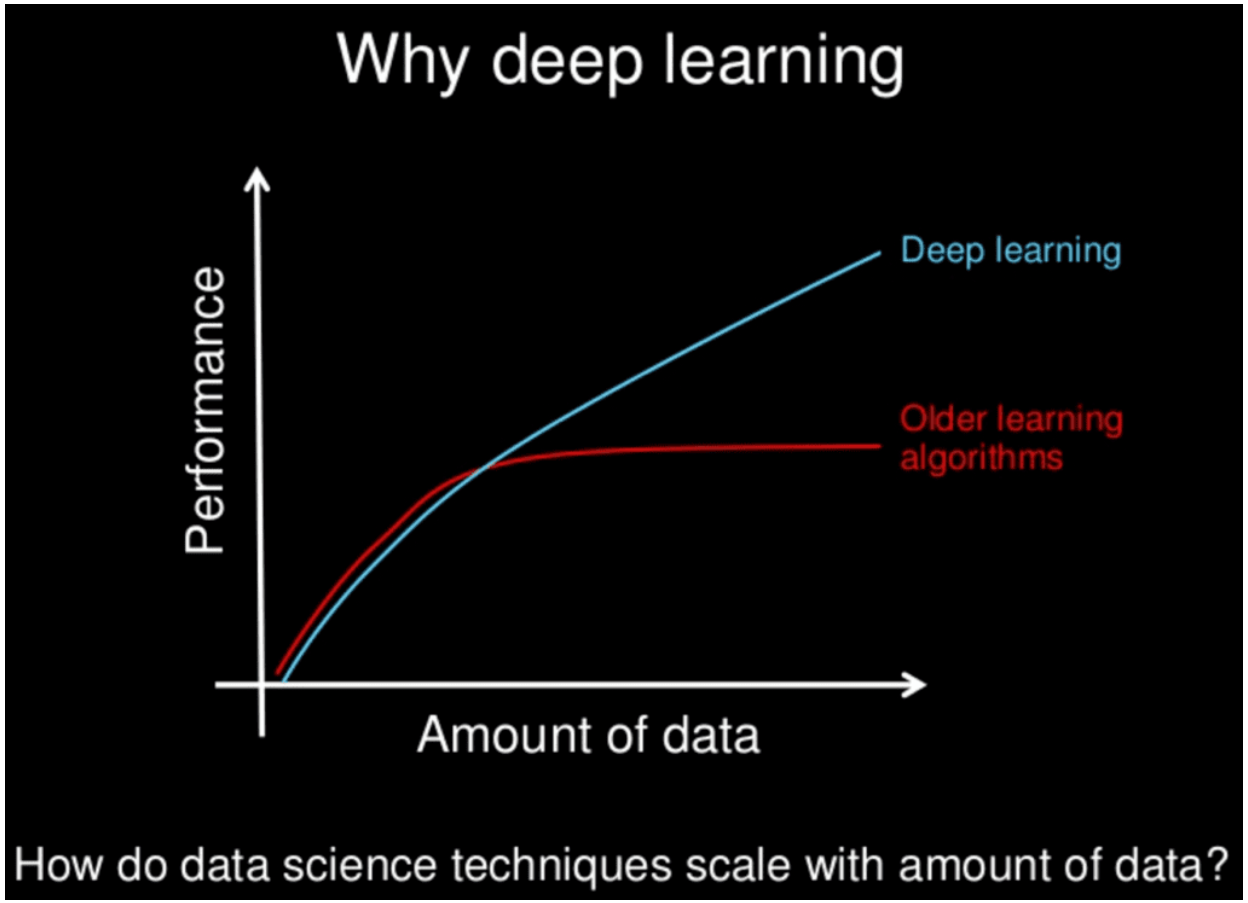
- Базовая задача ML – аппроксимация функций
- Обучение с учителем – восстанавливаем зависимость между признаками X и целевой переменной y
- Обучение без учителя – ищем в признаках X структуру

Опрос!

https://forms.gle/sBqoaJRNXN7428NJ8

Содержание

1. Обзор математики
2. Идея универсальных моделей
3. Нейронные сети
4. Градиентный спуск
5. Больше нейронных сетей
6. Глубокий двойной спуск



Математика

Скалярное произведение

- Скалярное произведение двух векторов представляет собой поэлементную сумму их поэлементного произведения.
- Мы также можем записать это как матричное произведение w_transpose и x.
  - Транспонирование вектора изменяет его из вектора-столбца в вектор-строку и наоборот.
- В нейронных сетях мы часто рассматриваем взвешенную сумму отдельных компонентов входного вектора, так что имейте это в виду.

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n = \sum_i w_ix_i = \begin{bmatrix} w_1 & w_2 & \dots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \overline{w}^\top \overline{x}$$

Матричное произведение

- Матричные векторные произведения можно рассматривать как повторяющиеся векторные скалярные произведения между строками матрицы W и вектором x.
- Это всего лишь расширение предыдущего слайда, за исключением того, что теперь мы накладываем скалярные произведения друг на друга, чтобы сформировать новый вектор.

$$W = \begin{bmatrix} — & \overline{w_1}^\top & — \\ — & \overline{w_2}^\top & — \\ & \vdots & \\ — & \overline{w_n}^\top & — \end{bmatrix}$$

$$W\overline{x} = \begin{bmatrix} \overline{w_1}^\top \\ \overline{w_2}^\top \\ \overline{w_3}^\top \\ \cdots \\ \overline{w_n}^\top \end{bmatrix} \overline{x} = \begin{bmatrix} \overline{w_1}^\top \overline{x} \\ \overline{w_2}^\top \overline{x} \\ \overline{w_3}^\top \overline{x} \\ \cdots \\ \overline{w_n}^\top \overline{x} \end{bmatrix}$$

Нотация векторных функций

- В нейронных сетях мы будем иметь дело с векторными/матричными функциями
- Обозначения, которые мы будем использовать на протяжении этой лекции:
  - x — Обычно это обозначает входной вектор
  - y — Обычно это обозначает метку, соответствующую x
  - W, b или θ (theta, тэта) — параметры, которые мы контролируем, обычно W для матрицы, b для вектора и θ для набора параметров в целом (также, когда нам лень и мы не хотим выписывать их все)
- В этом фиктивном примере ниже нижний индекс i в векторе обозначает i-е значение рассматриваемого вектора θ_i и θ_j — РАЗНЫЕ СКАЛЯРНЫЕ ПАРАМЕТРЫ, находящиеся внутри вектора параметров θ

$$f(\overline{x}, \overline{y}, W, \overline{b}, \overline{\theta}) = \sum_i [((W\overline{x})_i * \theta_i^2) - y_i + b_i]$$

Производные векторных функций

- Векторы состоят только из скаляров, поэтому нет причин, по которым мы не можем взять производные от выходных данных нашей функции (скаляра) по отношению к некоторому компоненту входного вектора (скаляра)
  - На практике в ML/DL обычно применяются библиотеки, где это уже реализовано
- В функции ниже вы можете взять производную скалярной функции по отношению к одному из элементов в векторе параметров или матрице, взятых в качестве входных данных

$$f(\overline{x}) = \bar{a}^{\top}\overline{x} = a_1x_1+a_2x_2+\ldots+a_nx_n$$

Скалярная частная производная векторной функции

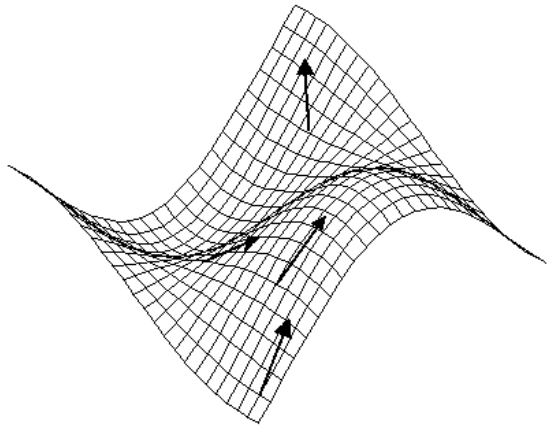
$$f(\overline{x}) = \bar{a}^{\top}\overline{x} = a_1x_1+a_2x_2+\ldots+a_nx_n$$

$$\frac{\partial f}{\partial x_i} = a_i$$

Градиент

- Это просто вектор частных производных по каждому входу функции
- Математический факт: в векторном исчислении направление самого крутого увеличения значения функции от точки — вдоль вектора градиента
- **Самый крутой спуск — вдоль отрицательного вектора градиента**
  - → Если мы хотим спуститься с холма, следуйте отрицательному градиенту, вычисленному в нашей текущей позиции

$$\nabla f(w_1, w_2, \ldots) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \end{bmatrix}$$

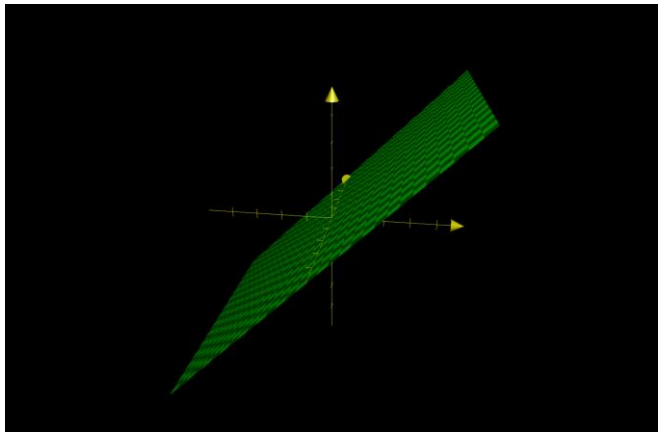
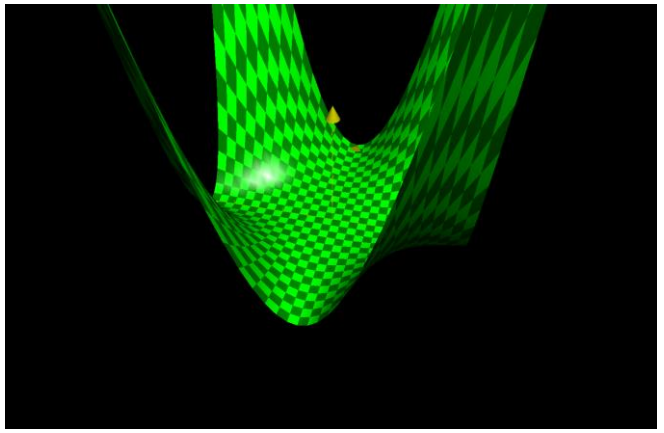
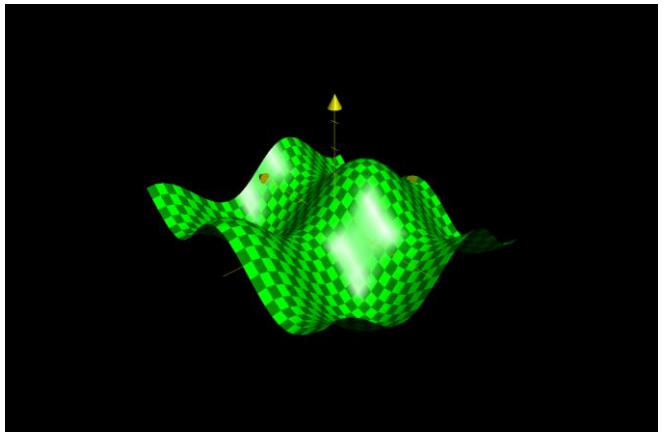


Gradient Vectors Shown at Several Points on the Surface of cos(x) sin(y)

Универсальный аппроксиматор функций

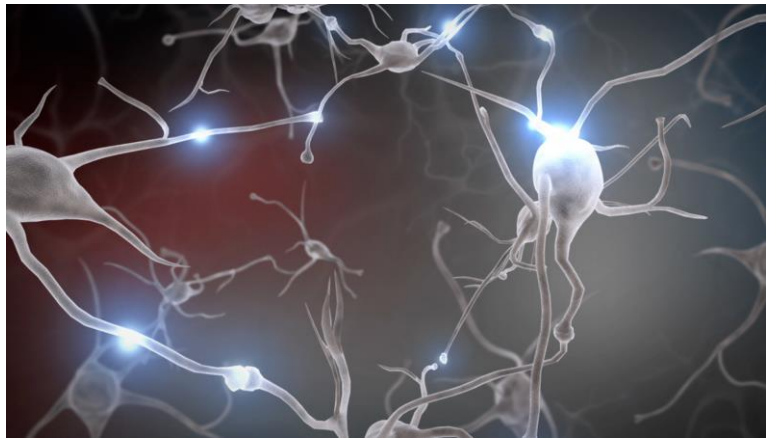
Мотивация к изобретению нейронных сетей

- Множество проблем, которые мы можем (или хотим) решить — регрессия, классификация и т. д. — часто вращаются вокруг создания функций, которые являются **очень** нелинейными
- Трудно понять, какой класс моделей лучше всего подходит для каждой задачи/набора данных
    - Существует ли какой-то класс модели, который может выполнять любую из этих задач практически сразу из коробки?

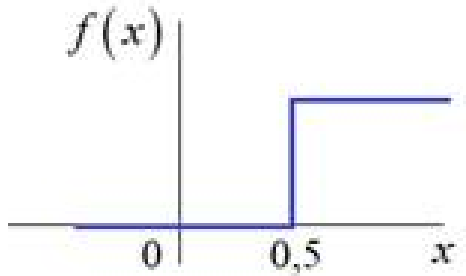
Мотивация к изобретению нейронных сетей

- Наш мозг способен выполнять множество задач с помощью одного и того же механизма
  - Что, если мы попытаемся создать модель мозга?
  - В нашем мозге есть нейроны
  - Нейроны принимают сигнал от окружающих нейронов
  - Нейроны выдают сигнал на основе количества принятого сигнала



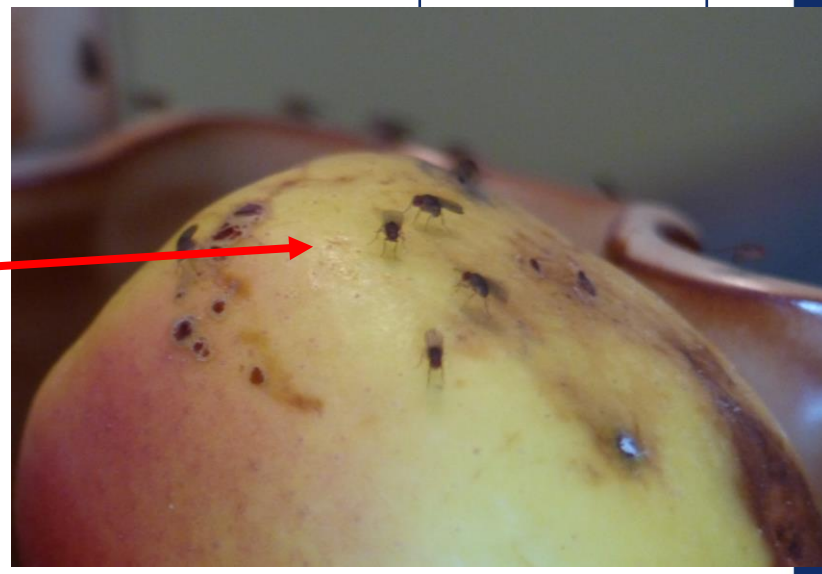
Пороговая функция

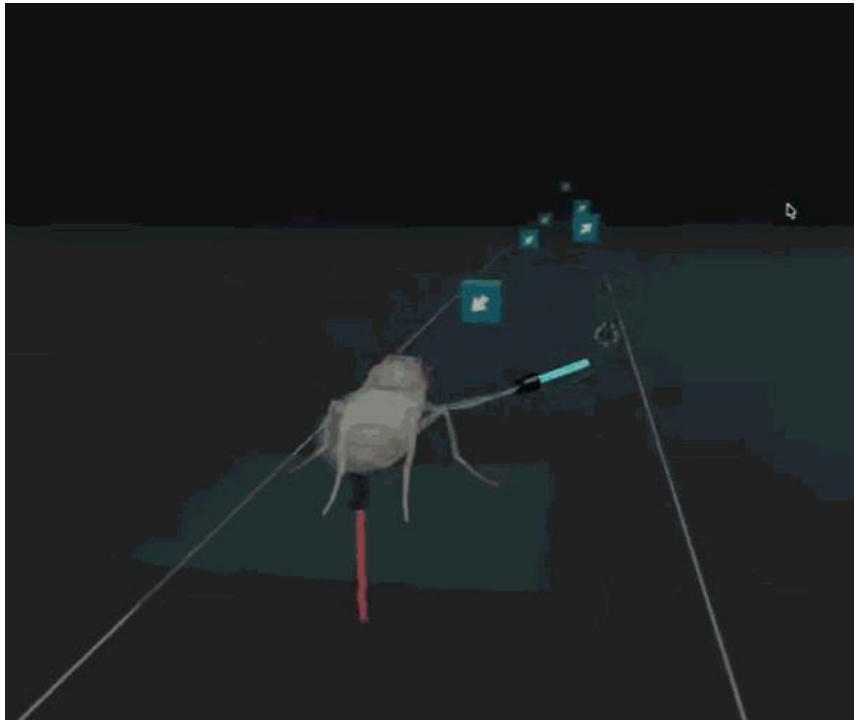
$$f(x) = \begin{cases} 1, & x \geq 0,5 \\ 0, & x < 0,5 \end{cases}$$

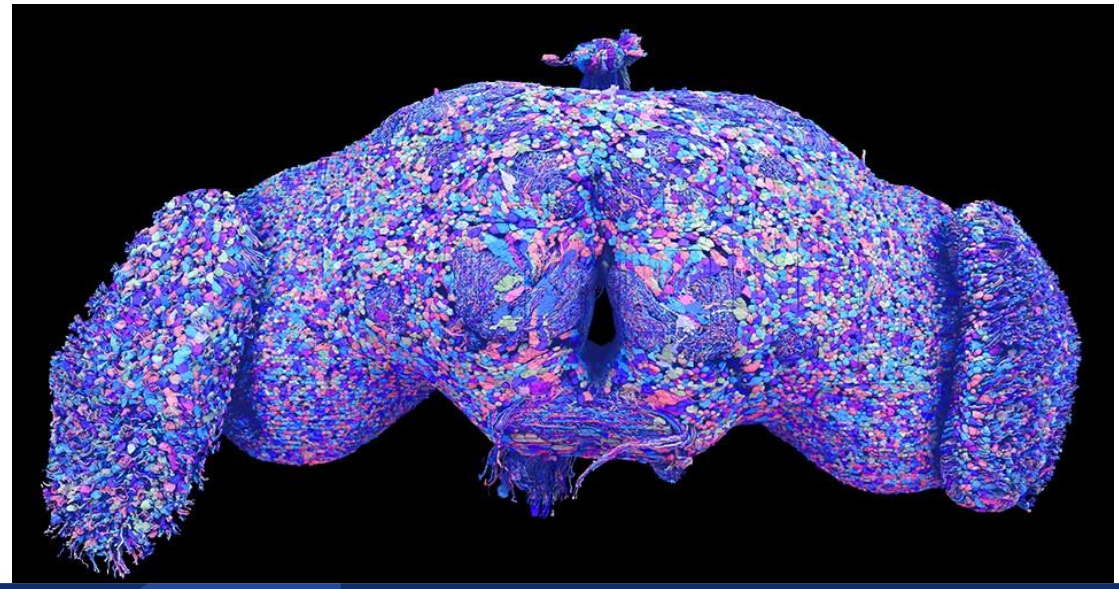


Отступление: FlyGM

- Оцифрованный мозг фруктовой мошки
- ~166 000 нейронов и более 124 миллионов связей





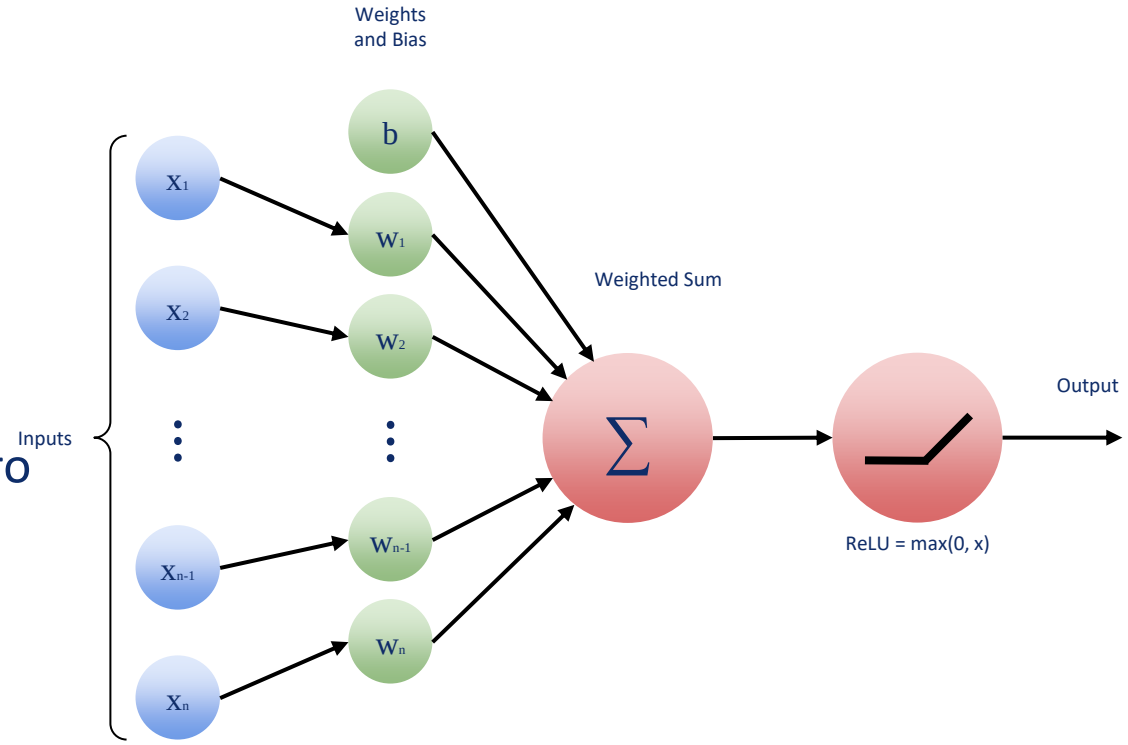


https://arxiv.org/html/2602.17997

Нейронные сети (графическое представление)

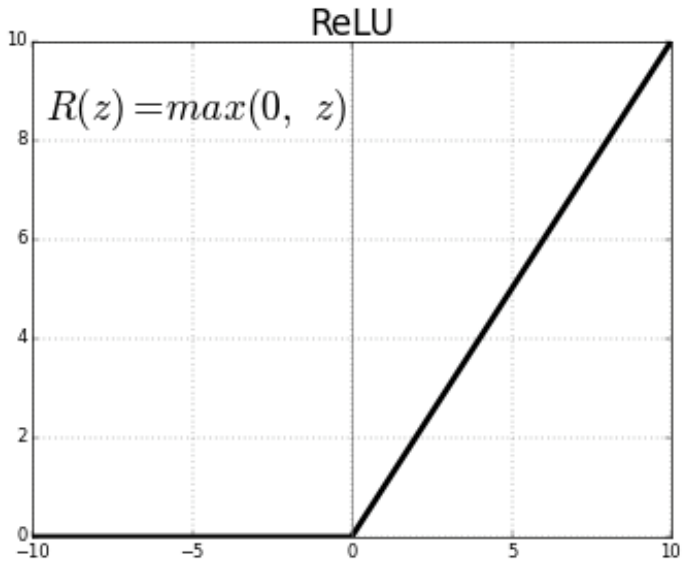
Перцептрон

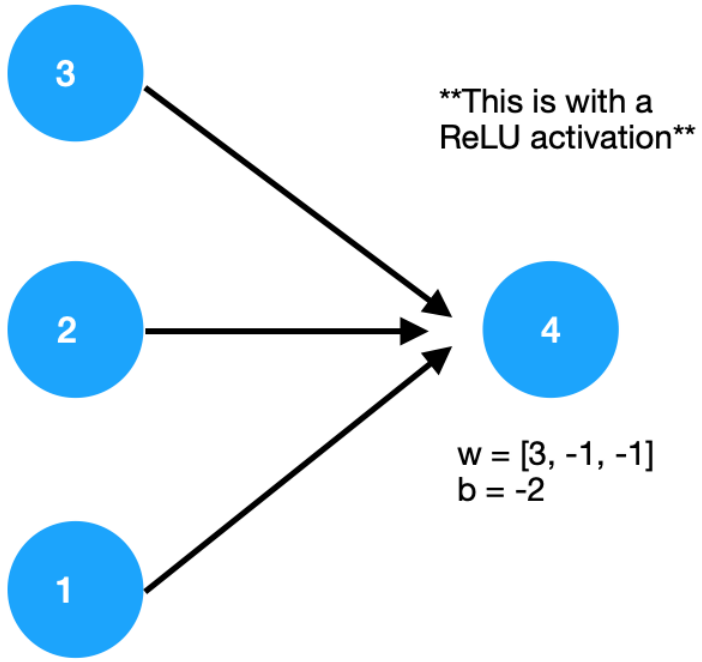
- Грубая модель нейрона
- Взвешиваем каждый компонент входа, затем «активируем» на основе суммы
  - Примечание: b — это просто скаляр, добавляемый к взвешенной сумме, и он не зависит от входных данных — мы называем его «смещение» (bias)
- Функция, которую мы используем для «активации» — ReLU
  - Мотивация ее выбора станет ясна позже

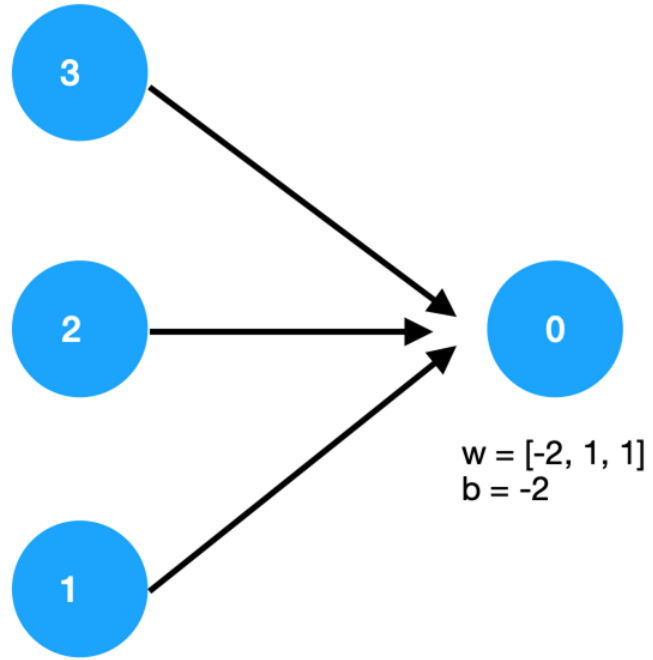


(Пока не беспокойтесь о том, где брать веса)

$$f(x_1, x_2, \ldots, x_n, w_1, w_2, \ldots, w_n, b) = f(\bar{x}, \bar{w}, b) = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b)$$







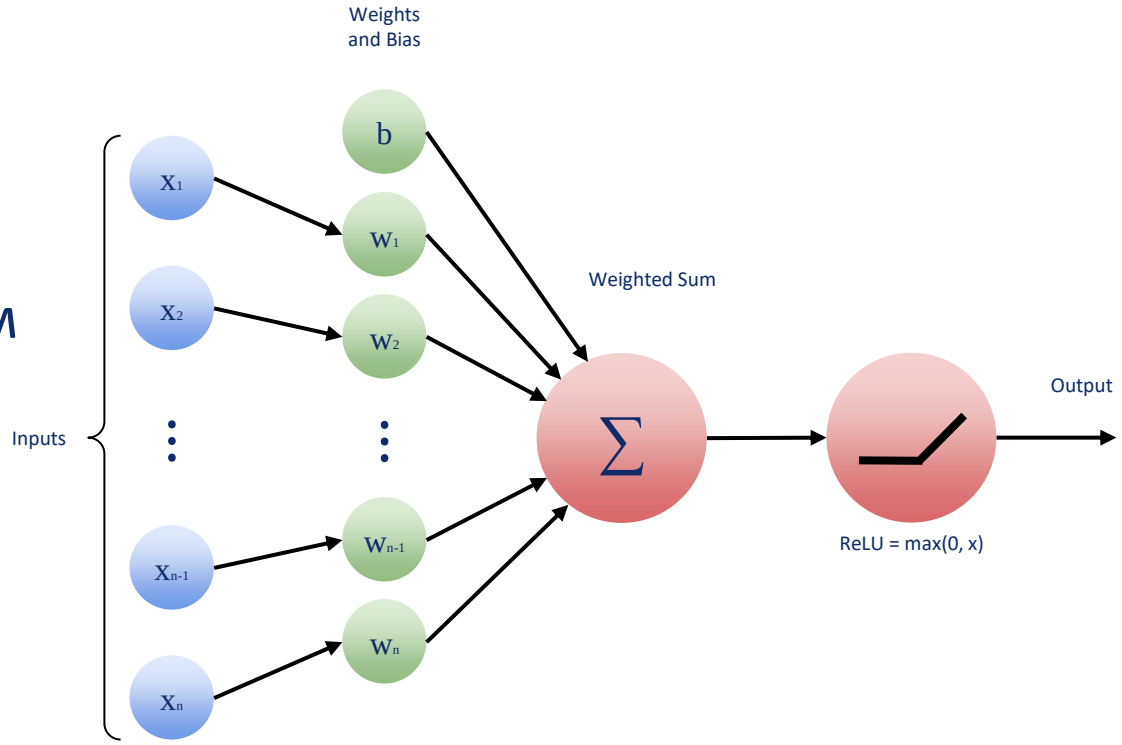
Пример перцептрона с ReLU вместо ступенчатой функции

x1 * w1 + x2 * w2 + x3 * w3 + b
= (3)(3) + (2)(-1) + (1)(-1) + (-2)
= 4
and ReLU(4) = max(0, 4) = 4

x1 * w1 + x2 * w2 + x3 * w3 + b
= (3)(-2) + (2)(1) + (1)(1) + (-2)
= -5
and ReLU(-5) = max(0, -5) = 0

Перцептрон

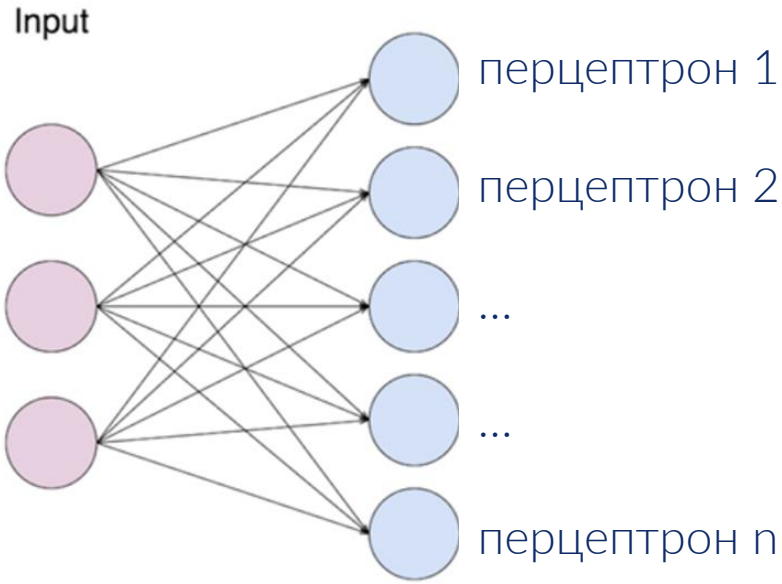
- Давайте немного упростим запись
- Сумму на предыдущем слайде можно переписать как скалярное произведение между вектором веса и входным вектором плюс скаляр смещения



$$f(\bar{x}, \bar{w}, b) = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b) = \mathrm{ReLU}(\bar{w}^\top \bar{x} + b)$$

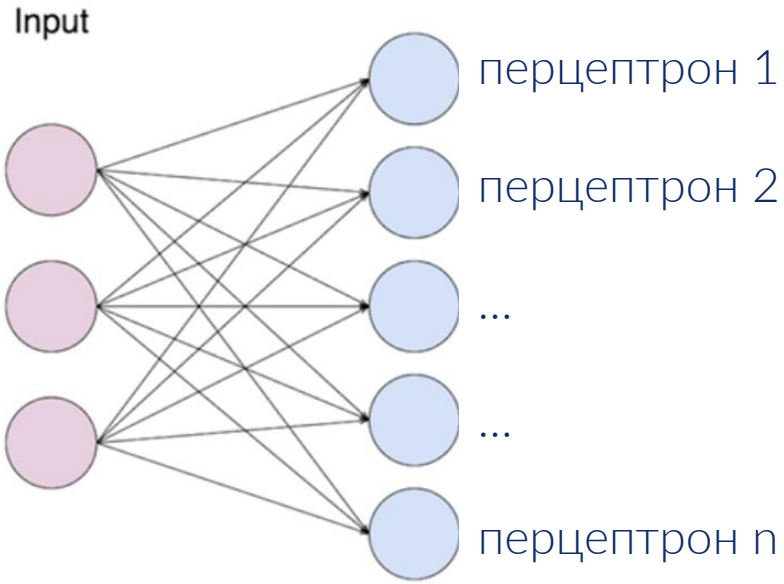
Слой перцептронов

- Что, если у нас есть несколько перцептронов, которые разделяют один и тот же вход?
  - Нейроны в мозге образуют все виды связей
  - Возможно, разные перцептроны можно использовать для извлечения разных видов сигналов из одного и того же входа
- Каждый перцептрон (с его уникальными весами и смещениями) можно сложить в «слой»



Слой перцептронов

- Чтобы вычислить выход слоя, мы можем сложить выход каждого отдельного перцептрона в вектор
- Примечание: ReLU применяется к каждому выходу перцептрона независимо
- Можно рассматривать ReLU как векторную/матричную функцию, которая применяет операцию ReLU поэлементно к каждому компоненту своего входа

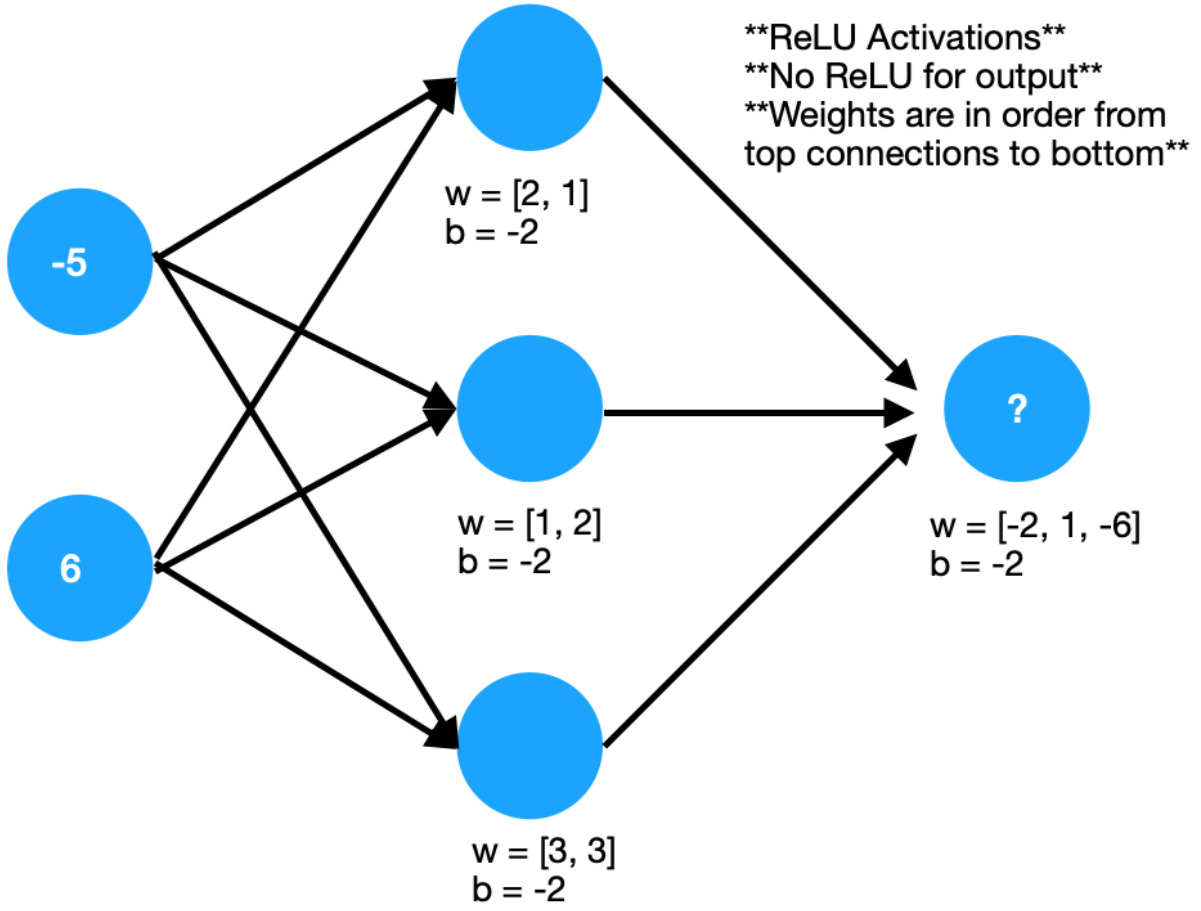


Каждый ряд – уравнение перцептрона для индивидуального нейрона

$$f(\bar{x}, \bar{w}_1, \bar{w}_2, \ldots, \bar{w}_n, b_1, b_2, \ldots, b_n) = \begin{bmatrix} \mathrm{ReLU}\left(\bar{\mathrm{w}}_1^\top \bar{\mathrm{x}} + \mathrm{b}_1\right) \\ \mathrm{ReLU}\left(\bar{w}_2^\top \bar{x} + b_2\right) \\ \vdots \\ \mathrm{ReLU}\left(\bar{w}_n^\top \bar{x} + b_n\right) \end{bmatrix} = \mathrm{ReLU}\left(\begin{bmatrix} \bar{w}_1^\top \bar{x} + b_1 \\ \bar{w}_2^\top \bar{x} + b_2 \\ \vdots \\ \bar{w}_n^\top \bar{x} + b_n \end{bmatrix} \right)$$

Важные замечания по этому примеру:

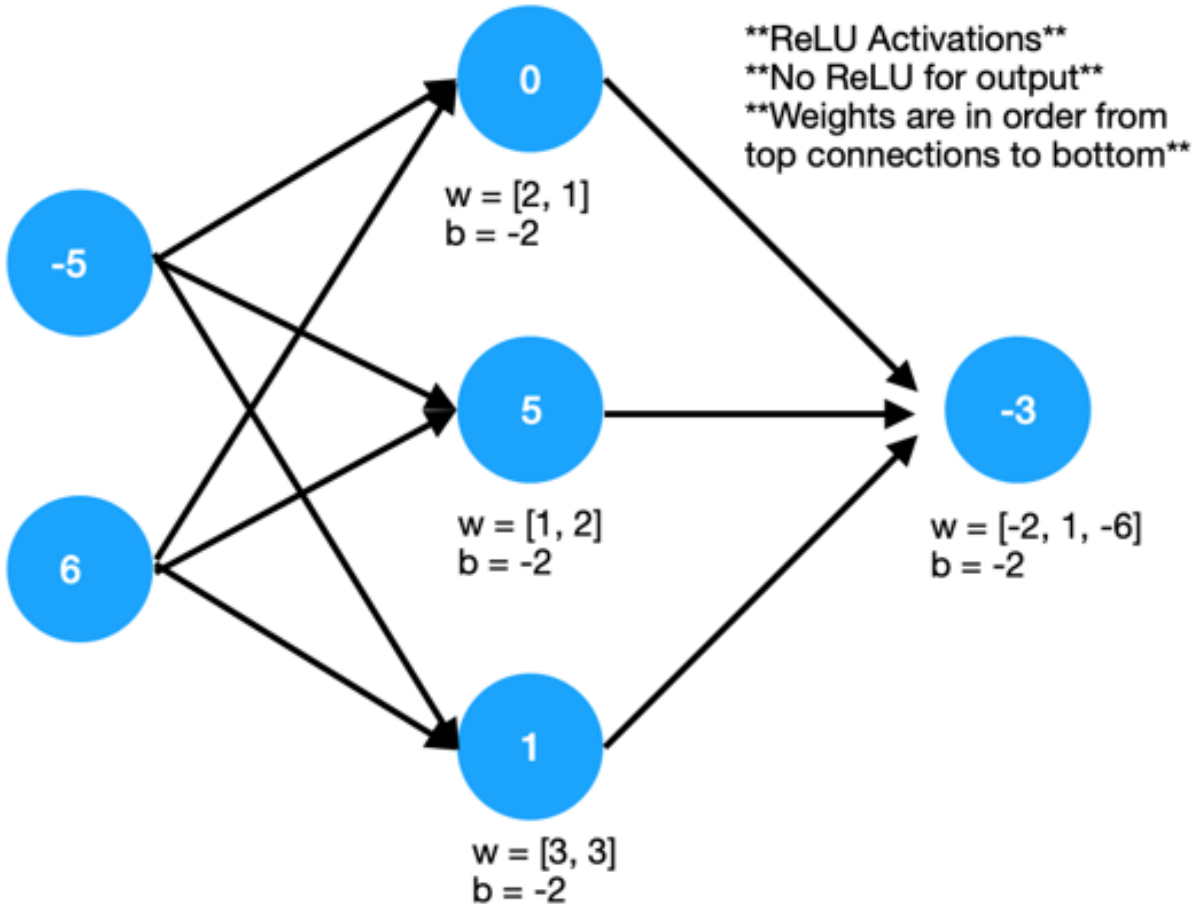
1. Последний слой не имеет ReLU, поэтому его выходной диапазон не ограничен.
2. Перцептроны в середине имеют меньшие веса, так как у них меньше входящих связей



Нейронная сеть для задачи в стиле регрессии (т.е. один скалярный выход в диапазоне (-inf, inf))

Важные замечания по этому примеру:

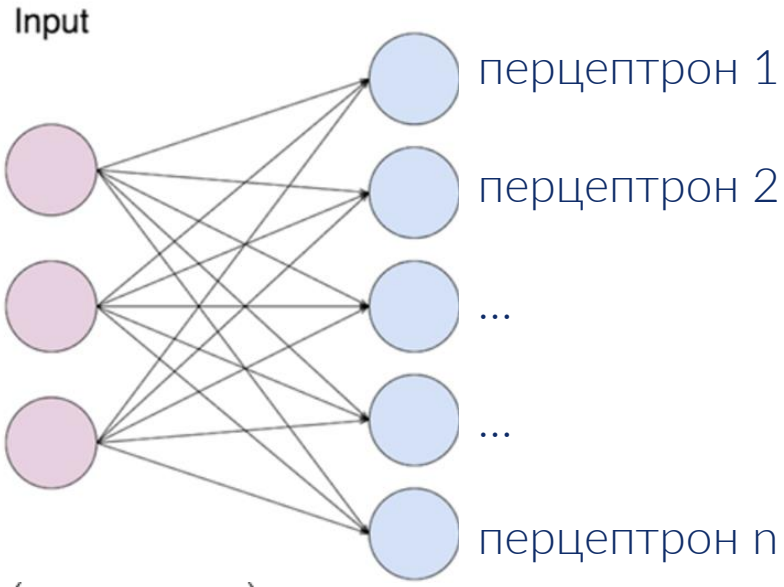
1. Последний слой не имеет ReLU, поэтому его выходной диапазон не ограничен.
2. Перцептроны в середине имеют меньшие веса, так как у них меньше входящих связей



Нейронная сеть для задачи в стиле регрессии (т.е. один скалярный выход в диапазоне (-inf, inf))

Слой перцептронов

- Если мы сложим смещения в вектор, мы сможем переписать все с помощью одного произведения матрицы и вектора и сложения векторов.
- Это самая компактная форма записи слоя.
- Теперь мы можем просто отказаться от представления о сложенных перцептронах, если хотим, и начать думать в терминах целых слоев сразу.



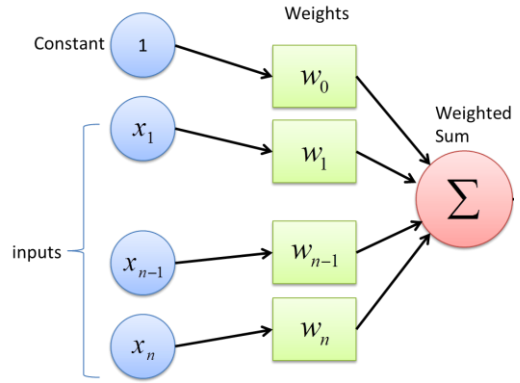
$$f(\bar{x}, \bar{w}_1, \bar{w}_2, \ldots, \bar{w}_n, b_1, b_2, \ldots, b_n) = \mathrm{ReLU}\left(\begin{bmatrix} \bar{w}_1^\top \bar{x} + b_1 \\ \bar{w}_2^\top \bar{x} + b_2 \\ \vdots \\ \bar{w}_n^\top \bar{x} + b_n \end{bmatrix}\right) = \mathrm{ReLU}\left(\begin{bmatrix} \bar{w}_1^\top \bar{x} \\ \bar{w}_2^\top \bar{x} \\ \vdots \\ \bar{w}_n^\top \bar{x} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}\right) = \mathrm{ReLU}\left(\underbrace{\begin{bmatrix} \bar{w}_1^\top \\ \bar{w}_2^\top \\ \vdots \\ \bar{w}_n^\top \end{bmatrix}}_{W} \bar{x} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}}_{\bar{b}}\right) = \mathrm{ReLU}(W\bar{x} + \bar{b})$$

$$f(\bar{x}, W, \bar{b}) = \mathrm{ReLU}(W\bar{x} + \bar{b})$$

Промежуточное резюме

Перцептрон

$$b + w_1x_1 + w_2x_2 + \cdots + w_nx_n = b + \sum_i w_ix_i = b + \begin{bmatrix} w_1 & w_2 & \dots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = b + \overline{w}^\top \overline{x}$$

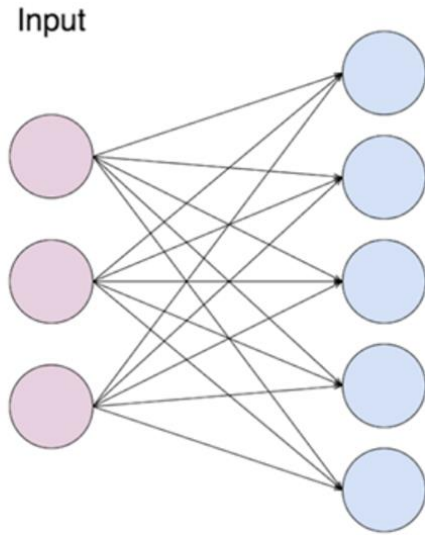


Сборка перцептронов с ReLU

$$f(\bar{x}, \bar{w}_1, \bar{w}_2, \dots, \bar{w}_n, b_1, b_2, \dots, b_n) = \mathrm{ReLU}\left(\begin{bmatrix} \bar{w}_1^\top \bar{x} + b_1 \\ \bar{w}_2^\top \bar{x} + b_2 \\ \vdots \\ \bar{w}_n^\top \bar{x} + b_n \end{bmatrix}\right)$$

Матричная форма

$$\mathrm{ReLU}\left(\begin{bmatrix} \bar{w}_1^\top \bar{x} + b_1 \\ \bar{w}_2^\top \bar{x} + b_2 \\ \vdots \\ \bar{w}_n^\top \bar{x} + b_n \end{bmatrix}\right) = \mathrm{ReLU}\left(W\bar{x} + \bar{b}\right)$$

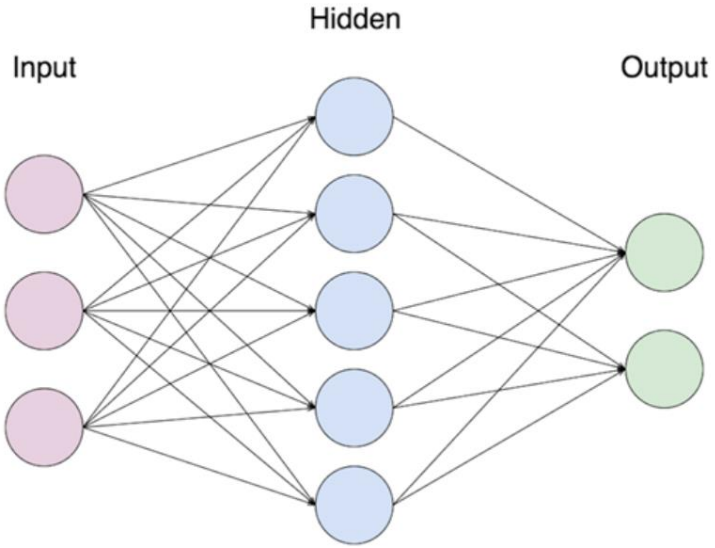


$$f(\overline{x}, W, \overline{b}) = \mathrm{ReLU}(W\overline{x} + \overline{b})$$

Нейронная сеть

- Каждый узел (за исключением столбца входных узлов) — это просто перцептрон
- Каждый слой перцептронов также называется слоем нейронной сети
  - Мы можем выбрать стекирование произвольного количества перцептронов на каждом слое или каскад с произвольным количеством слоев
  - Каждый слой может иметь разное количество перцептронов
  - Средние слои часто называют «скрытыми слоями», поскольку их нельзя интерпретировать непосредственно
- Мы по-прежнему будем использовать функцию ReLU для каждого слоя на данный момент

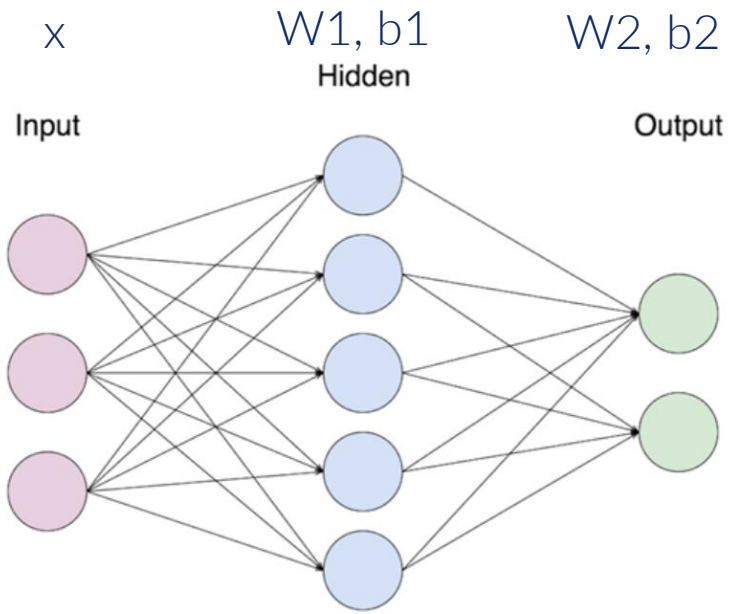
Примечание: мы не активируем ReLU на выходном слое.



Эту модель также называют «многослойным перцептроном» или MLP по понятным причинам.

Каскады слоёв

- Мы можем начать каскадировать эти слои, чтобы сформировать полную нейронную сеть.
- Мы выберем размеры каждой матрицы W_i, чтобы получить слои нужного размера.
  - Ширина W должна быть равна размеру предыдущего слоя.
  - Высота W равна количеству перцептронов, которые мы укладываем для этого слоя.



$$f(\overline{x}, W_1, W_2, \overline{b_1}, \overline{b_2}) = W_2(\mathrm{ReLU}(W_1\overline{x} + \overline{b_1})) + \overline{b_2}$$

Зачем нам ReLU?

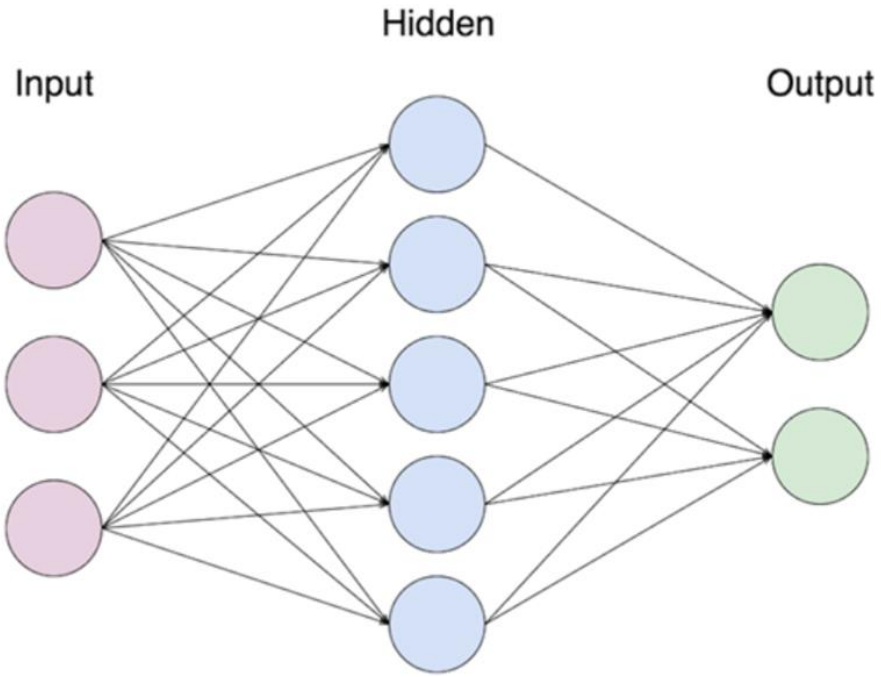
Почему нельзя отбросить ReLU?

$$
\begin{aligned}
f(\bar{x}, W_1, W_2, \bar{b}_1, \bar{b}_2) &= W_2(W_1\bar{x} + \bar{b}_1) + \bar{b}_2 \\
&= W_2W_1\bar{x} + (W_2\bar{b}_1 + \bar{b}_2) \\
&= W_3\bar{x} + \bar{b}_3
\end{aligned}
$$

Без ReLU это выглядит так, будто мы используем только один гигантский линейный слой.

Классификация

- **Напоминание**: это задача, где выход — это вектор, где элемент i — вероятность того, что вход принадлежит классу i
- Поскольку мы имеем дело с вероятностями, нам нужно обеспечить соблюдение двух правил:
  - Выход любого нейрона в последнем слое находится в диапазоне от 0 до 1
  - Все выходы в сумме дают 1
- Выходы сети (зеленые узлы) сейчас могут быть любым значением в диапазоне (-inf, inf)
- Нам нужно выполнить некоторую операцию с ними, которая превратит наши выходы в допустимое распределение вероятностей



Softmax

- **Вам можно (и не нужно) запоминать формулу, только следующие идеи:**
  - Сначала softmax делает все элементы положительными, взяв e^{z_i} для каждой записи z_i Затем мы делим каждую запись на сумму всех экспонированных записей, гарантируя, что сумма всех записей равна 1
- Наибольшая запись до будет наибольшей вероятностью после softmax, имитируя жесткий максимум, будучи при этом хорошо дифференцируемой (потребуется позже)
- Это также удовлетворяет нашу потребность превратить любой вектор чисел в нечто, что можно интерпретировать как распределение вероятностей

Formula

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

σ = softmax
$\vec{z}$ = input vector
e^{z_i} = standard exponential function for input vector
K = number of classes in the multi-class classifier
e^{z_j} = standard exponential function for output vector
e^{z_j} = standard exponential function for output vector

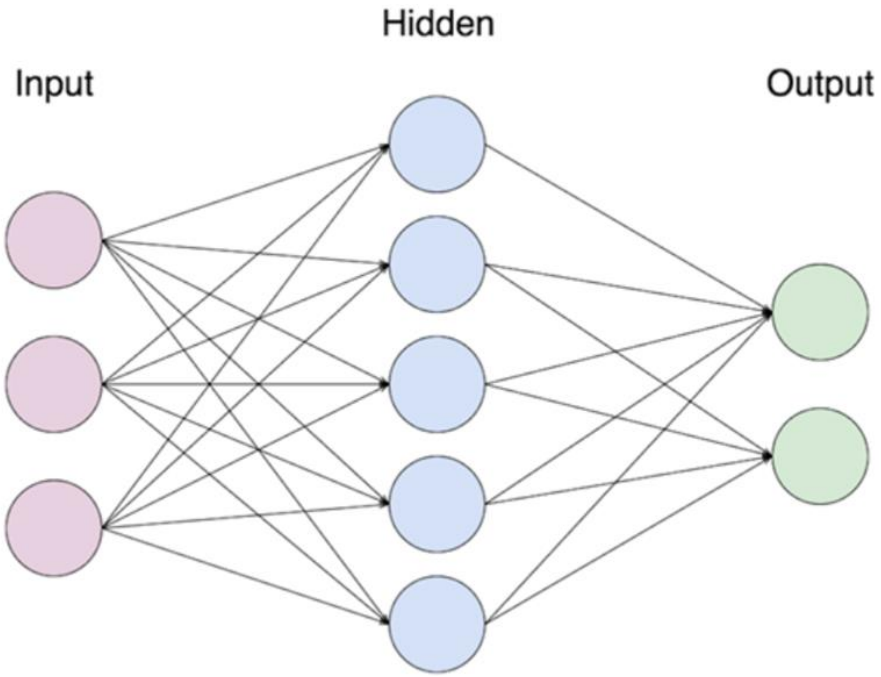


Есть нюанс...

- PyTorch уже имеет встроенный слой softmax в функцию CrossEntropyLoss, поэтому вам никогда не придется явно использовать этот слой при создании сети
  - Неосознанное добавление дополнительного слоя softmax может снизить производительность вашей модели, и эта «скрытая» ошибка уже сделала жизнь многих людей невыносимой...
- Однако все равно полезно знать, что это такое, на случай, если вам когда-нибудь придется использовать другие фреймворки машинного обучения!

Промежуточное резюме

1. Наш модифицированный перцептрон принимает несколько входных значений и выполняет операцию их взвешивания, добавляя смещение, а затем выполняя ReLU
2. Нейронные сети в основном представляют собой стекированные перцептроны, образующие «слой», которые затем каскадируются, а количество стеков (часто называемое «размером» слоя) является произвольным для каждого слоя независимо
3. Если вы хотите выполнить классификацию и вам нужны вероятности – примените softmax



Градиентный спуск

Функция потерь

- Когда мы случайным образом инициализируем параметры нашей модели, она, вероятно, выводит полный мусор
  - Нам нужно найти способ улучшить веса, чтобы они больше не были полным мусором :)
- Нам нужна метрика, для которой мы можем оптимизировать... что-то, что количественно определяет, насколько плохо работает модель. Эти функции «потерь» должны быть
  - Высокими, когда предсказания нашей модели плохие
  - Низкими, когда предсказания нашей модели хорошие
- Для глубокого обучения функции потерь должны быть дифференцируемыми, т. е. вы должны иметь возможность взять производную ваших потерь относительно ваших входов

Пример функции ошибки: MSE

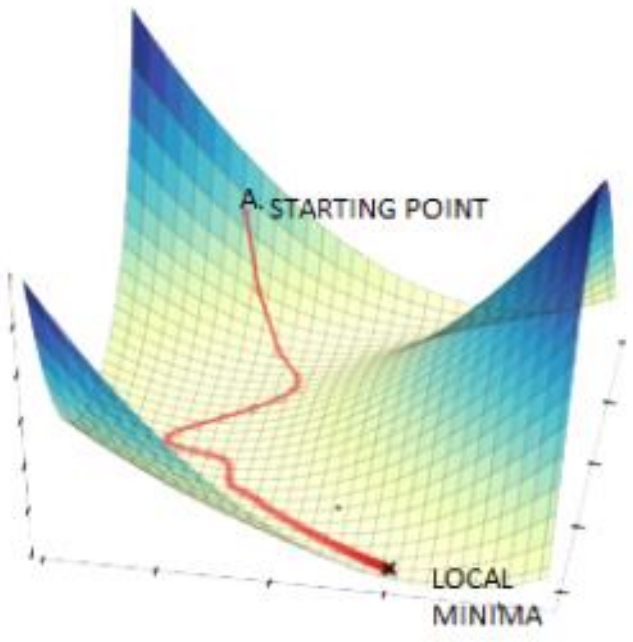
- Классификация или регрессия: предположим, что у нас есть N пар данных (x_i, y_i) и прогноз модели для x_i равен $(y_{pred})_i$
- Можем ли мы просто использовать что-то вроде среднеквадратичной ошибки в качестве нашей функции потерь?
  - MSE возвращает среднее квадратичных разностей между y_i и $(y_{pred})_i$ по всем значениям
  - Эта величина будет низкой, когда прогнозы модели будут достаточно близки к соответствующим истинным меткам для всех (или большинства) из N образцов, но высокой в противном случае
- Будут функции потерь получше, но именно эту легко записать и интерпретировать
- Примечание: MSE выводит скалярное значение и также является дифференцируемой функцией

Примечание: мы можем сложить истинные метки и прогнозы модели для всех обучающих выборок в N-мерные векторы и определить MSE как векторную функцию.

$$\mathrm{MSE}(\bar{y}, \bar{y}_{\mathrm{pred}}) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - (y_{\mathrm{pred}})_i\right)^2$$

Метафора холмов

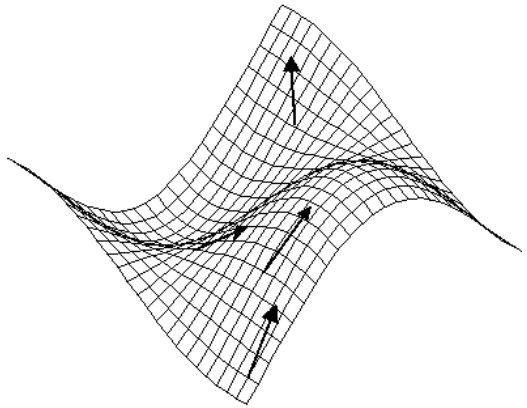
- Если вы находитесь на холме и хотите достичь подножия, но видите только метр вокруг себя, что вы делаете?
  - Мы должны просто следовать по склону холма и надеяться, что он приведет нас вниз, правильно?
- Нам нужно будет сделать пару шагов в направлении вниз, остановиться и пересмотреть наше направление, затем сделать еще несколько шагов и так далее.



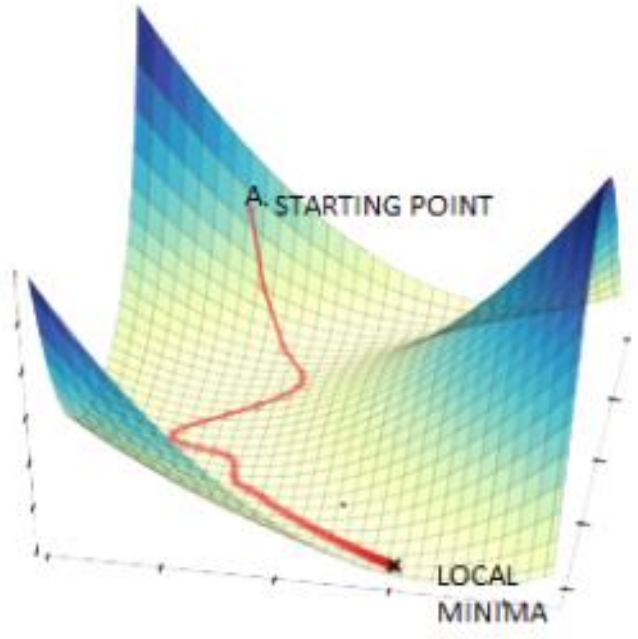
Какое направление самое крутое?

- Предположим, что высота холма задана векторной функцией
- Самый крутой спуск — вдоль отрицательного вектора градиента
  - → Если мы хотим спуститься с холма, следуйте отрицательному градиенту, рассчитанному в нашем текущем положении

$$\nabla f(w_1, w_2, \dots) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \end{bmatrix}$$



Gradient Vectors Shown at Several Points on the Surface of cos(x) sin(y)



Градиентный спуск

- Наши потери являются функцией параметров модели, и мы хотим их минимизировать
- Это то же самое, что спускаться с холма
  - Мы хотим найти, в каком направлении мы можем шагать нашими параметрами, чтобы уменьшить потери нейронной сети (на ее обучающих данных) настолько, насколько это возможно за один шаг
  - Все наши параметры являются скалярами, поэтому мы можем просто взять частные производные (т. е. градиент) наших потерь относительно наших параметров

$$\nabla\text{params}\,\mathrm{L}(\bar{x},\,\bar{y},\,W_1,\,W_2,\,\bar{b}_1,\,\bar{b}_2) = \nabla\text{params}\,\mathrm{L}\Big(\bar{x},\,\bar{y},\,(W_1)_{1,1},\,\ldots,(W_2)_{m,n},\,(\bar{b}_1)_1,\ldots,\,(\bar{b}_2)_m\Big)$$

Скалярные параметры

$$\nabla\text{params}\,\mathbf{L}(\bar{x},\,\bar{y},\,W_1,\,W_2,\,\bar{b}_1,\,\bar{b}_2) = \begin{bmatrix} \frac{\partial \mathrm{L}}{\partial (W_1)_{1,1}} \\ \vdots \\ \frac{\partial \mathrm{L}}{\partial (W_2)_{m,n}} \\ \frac{\partial \mathrm{L}}{\partial (\bar{b}_1)_1} \\ \vdots \\ \frac{\partial \mathrm{L}}{\partial (\bar{b}_2)_m} \end{bmatrix}$$

Матричная форма

Градиентный спуск

- Помните, что нам нужно вычислить: градиент, оцененный в нашей текущей позиции
- Когда вы спускаетесь с холма в тумане, мы хотим знать только склоны холма, на котором мы сейчас стоим
- Поэтому нас интересует только нахождение вектора градиента, оцененного **при наших текущих весах и обучающих примерах**
- С учетом того, что нейронные сети настолько сложны, сложно решить для градиента относительно их параметров аналитически
- Однако, это удивительно легко решить, когда он оценивается при заданном наборе параметров

$$\cancel{\frac{\partial \mathrm{L}}{\partial \theta_i} = 2\theta_i}$$

$$\left.\frac{\partial \mathrm{L}}{\partial \theta_i}\right|_{\theta_i=5} = 10$$

Градиентный спуск

- Чтобы «достичь» параметров в нижней части «холма функции потерь», нам нужно сделать шаг в направлении его самого крутого спуска, т. е. вдоль отрицательного вектора градиента
  - Мы также можем захотеть скорректировать размер нашего шага, чтобы он не был слишком маленьким или слишком большим, и мы делаем это, масштабируя обновление градиента с помощью скалярной лямбды
  - Эта лямбда также называется нашей «скоростью обучения»
- Мы продолжаем делать такие шаги, чтобы продолжать спускаться по этому «холму»

$$\left(\bar{\theta}_{t+1}\right)_i = \left(\bar{\theta}_t\right)_i - \lambda \frac{\partial \mathrm{L}}{\partial \bar{\theta}_i}\bigg|_{(\bar{\theta}_t,\, \mathrm{data})}$$

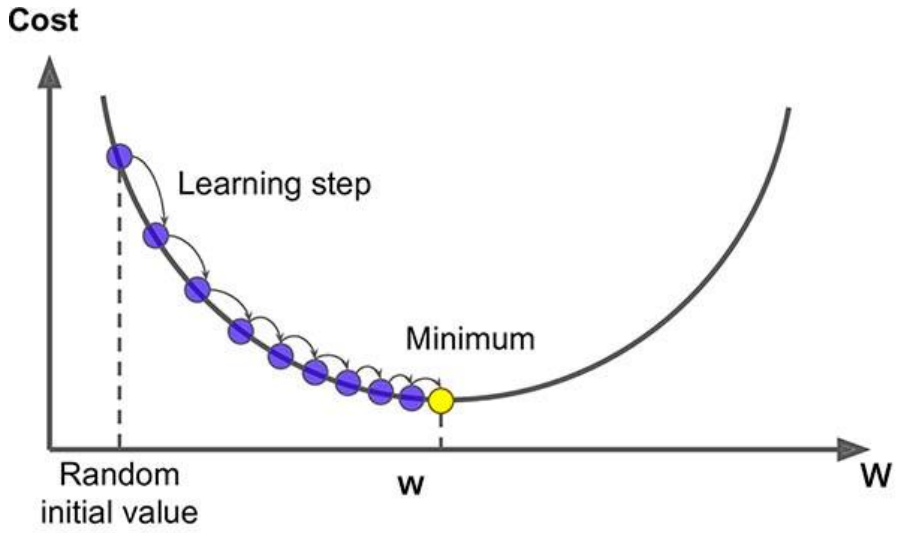
$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \nabla_{\bar{\theta}} \mathrm{L}\bigg|_{(\bar{\theta}_t,\, \mathrm{data})}$$

Ясность обозначений:
Здесь θ представляет все параметры модели в одном векторе, а data — это некоторый обучающий пример (вместе с его меткой, если таковая имеется).

Первое уравнение — это покомпонентное обновление, а второе уравнение — это то же самое обновление в векторной форме.

Схождение градиентного спуска

- Вы заметите, что по мере приближения к подножию холма частные производные становятся меньше, и мы в конечном итоге замедляются до полной остановки (предполагая, что скорость нашего обучения не слишком велика)
- Что происходит, когда размер нашего шага слишком велик?
  - Мы просто перескакиваем через минимум вместо того, чтобы замедлиться, чтобы достичь его
  - Это вызывает нестабильность обучения
- Что происходит, когда размер нашего шага слишком мал?
  - Наши шаги будут настолько малы, что потребуется целая вечность, чтобы сойтись



Настоящий (батчевый) градиентный спуск

$$\mathrm{MSE}\left(\bar{y},\, \bar{y}_{\mathrm{pred}}\right) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \left(y_{\mathrm{pred}}\right)_i\right)^2$$

- Помните, что MSE усредняет квадратичную ошибку по всем обучающим образцам
  - В конце концов, мы хотим, чтобы наши потери уменьшились на всем обучающем наборе, а не только на одном примере
- Математический факт: это эквивалентно усреднению градиентов по всем отдельным обучающим примерам
  - В действительности, программное обеспечение для автоматической дифференциации само справится с любыми такими вычислениями, так что вам не придется думать об этом

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \nabla_{\bar{\theta}} \left(\frac{1}{N}\sum_{i=1}^{N} L\left(x_i,\, y_i, \bar{\theta}_t\right) \right) \iff \bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{1}{N}\sum_{i=1}^{N} \nabla_{\bar{\theta}} L\left(x_i,\, y_i,\, \bar{\theta}_t\right)$$

Мини-батчевый градиентный спуск

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{1}{B} \sum_{i=1}^{B} \nabla_{\bar{\theta}} L\left(x_i, y_i, \bar{\theta}_t\right)$$

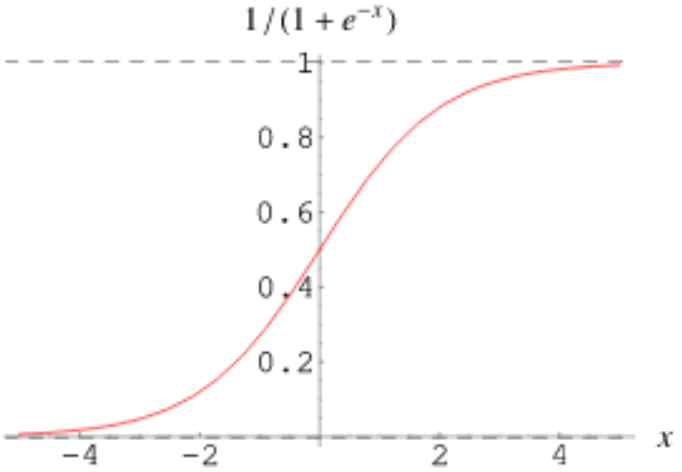
- Почему поиск истинного градиента во всем наборе данных может быть неидеальным?
  - Вычислительно неэффективно, а также легче переобучается
- Как мы можем приближать?
  - Вместо этого мы можем взять «мини-пакеты» данных, которые являются просто фрагментами обучающего набора данных
    - Этот «размер пакета» или «размер мини-пакета» — это гиперпараметр, который вам нужно настроить
  - Опять же, наши потери — это среднее значение всех потерь по всем обучающим примерам в мини-пакете
  - Мы надеемся, что это приблизит истинный градиент
  - Математика получается такой же, просто усредняем градиенты для всех примеров в нашем пакете
- Меньшие пакеты могут помочь предотвратить переобучение, позволяя вам только оценить истинное направление — случайность в этом процессе в конечном итоге помогает!

Больше строительных блоков нейронных сетей

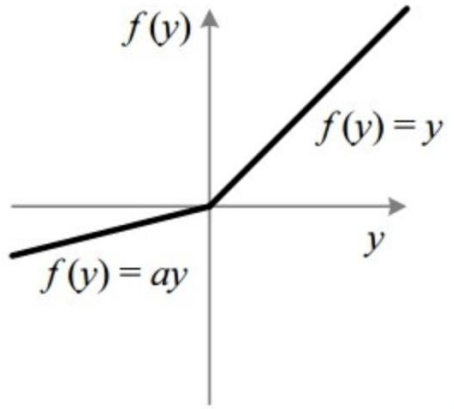
Функции активации

- Нужно ли нам использовать ReLU?
  - Нет, есть много других вариантов, нам просто нужно что-то нелинейное
  - Вспомните, что происходит, когда у нас нет нелинейности!
- Примеры:
  - Сигмоида: она также известна как логистическая функция и выдает значения в диапазоне (0, 1) LeakyReLU: ReLU, за исключением значений меньше нуля, у нас есть очень маленький наклон, а не просто ноль
  - GeLU: версия ReLU, которая используется в трансформерах (что является популярной архитектурой нейронных сетей)
  - Гиперболический тангенс: похож на сигмоиду, но диапазон [-1, 1]
- Мы на самом деле просто используем ReLU, потому что ее вычисление дешево и, похоже, она достаточно хорошо работает для большого количества приложений

Сигмоида



Leaky ReLU



Функции потерь

- Помимо MSE?
  - Да, есть много разных метрик, которые вы можете захотеть оптимизировать, если они дифференцируемы относительно выходных данных модели
  - Что-то вроде перекрестной энтропии обычно будет предпочтительным для классификации
  - Начните с «binary cross entropy» в свободное время, это довольно полезно знать
- Если вы хотите оптимизировать для двух вещей, вы можете просто создать вторую функцию потерь и добавить ее к любой существующей функции потерь, которую вы используете
- Математический факт: когда вы добавляете 2 функции потерь, градиент объединенной функции потерь является суммой градиентов функций потерь независимо друг от друга
- Программное обеспечение для Autodiff снова выполнит эту математику за вас в любом случае, но полезно знать
- Вы можете масштабировать каждый член потерь с помощью постоянного веса, чтобы подчеркнуть его влияние меньше или больше

Регуляризация весов / Уменьшение весов

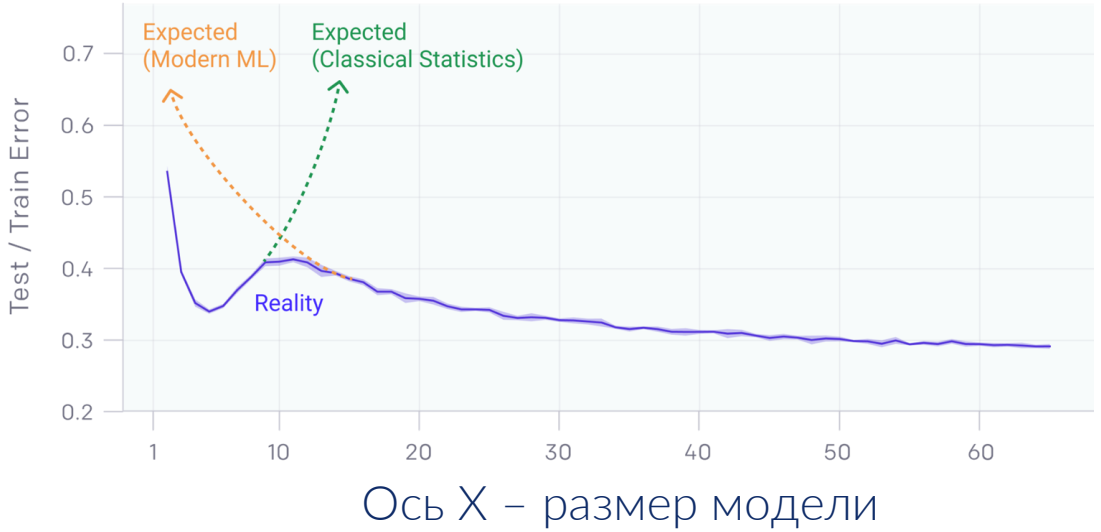
- Наши веса могут быть настолько огромными, насколько они хотят... желательно ли это?
- Если весам модели позволить вырасти до огромных размеров, то она может начать предсказывать дикие, неустойчивые, безумные границы решений
  - Большие веса делают так, что небольшие изменения на входе все еще могут привести к большим изменениям на выходе
  - Может быть, мы можем попытаться ограничить веса?
- Мы можем добавить член в нашу функцию потерь, который является суммой всех весов в квадрате
  - Это называется регуляризацией L2 и наказывает сеть за большие веса
  - Мы масштабируем этот член потерь на некоторую константу (обозначенную лямбдой ниже), чтобы контролировать, насколько мы хотим его усилить
- Иногда уменьшение веса может быть очень полезным, поэтому хорошо иметь этот инструмент в своем наборе инструментов

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} w_i^2$$

Двойной глубокий спуск

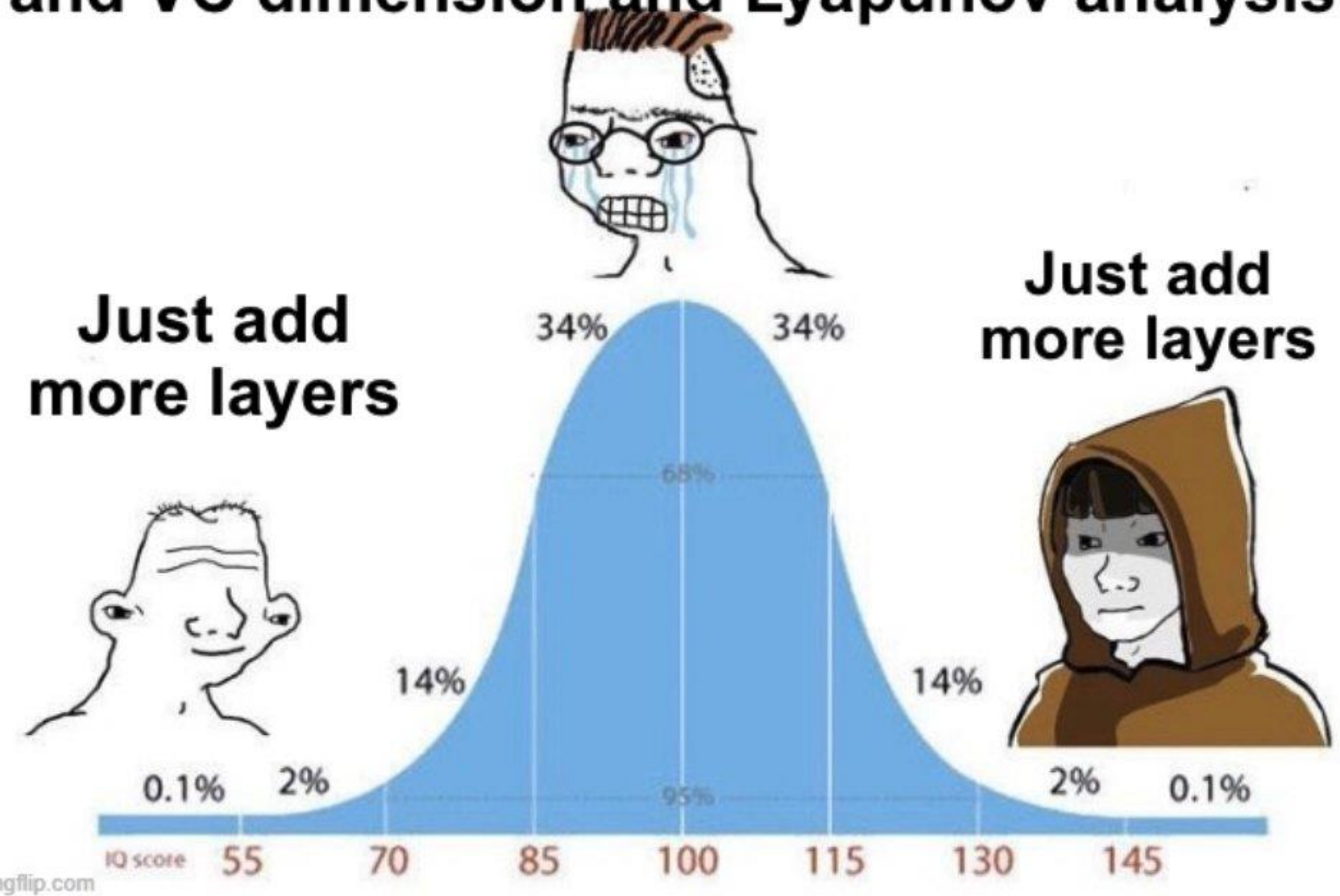
Почему глубокое обучение так распространено?

- Когда вы начинаете увеличивать размер модели, вы можете начать переобучать ее, что приведет к ухудшению производительности во время тестирования на новых данных



- *Но* когда вы асимптотически добавляете больше слоев и обучаетесь дольше, вы можете вполне увидеть явление «глубокого двойного спуска» для определенных классов моделей
- Вы увидите, что точность вашего теста снизится, когда вы начнете перепараметризацию, но по мере продолжения вы заметите, что точность обучения модели начнет увеличиваться во второй раз
- Это очень странное (но полезное) поведение, которое на самом деле не объясняется классической теорией

Nooooo you need convergence proofs and VC dimension and Lyapunov analysis



Полезные выводы

Выводы

$$f(\overline{x}, W_1, W_2, \overline{b_1}, \overline{b_2}) = W_2(\mathrm{ReLU}(W_1\overline{x} + \overline{b_1})) + \overline{b_2}$$

- Нейронная сеть — это каскад сложенных персептронов
  - Мы можем выбрать количество слоев, функции активации и функцию потерь
- Мы можем оптимизировать это с помощью градиентного спуска
  - Мы берем частную производную потерь (скаляр) на пакете обучающих примеров по каждому из параметров
  - Мы спускаемся вниз по склону в направлении наибольшего изменения
  - Мы можем настроить скорость обучения (размер шага), размер пакета

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{1}{B} \sum_{i=1}^{B} \nabla_{\bar{\theta}} L(x_i, y_i, \bar{\theta}_t)$$

Выводы

- Ваша работа как инженера МО:
  - Выберите количество (а затем и тип) слоев и размеры слоев, которые будут хорошо работать, функции активации и любые другие детали архитектуры модели
  - Выберите функцию потерь или придумайте новую (при условии, что это скалярная функция)
  - Выберите, как вы будете оптимизировать нейронную сеть / использовать градиентный спуск (это размер вашего пакета, скорость обучения и т. д.). Вы даже можете выбрать изменение скорости обучения с течением времени (обычно это убывающая скорость обучения)!
- Выбор лучшей конфигурации вашей сети будет методом проб и ошибок, поэтому убедитесь, что у вас есть набор данных только для настройки этих гиперпараметров
  - Помните наборы для валидации?

Дополнительные ресурсы

- Deep Learning видеоконтент
  - 3Blue1Brown для дополнительных иллюстраций: https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- Поиграться с нейронными сетями:
  - https://playground.tensorflow.org/
- Почитать и послушать
  - Глубокое обучение на пальцах
  - МФТИ DLS (поток Осень 2026 как раз скоро начнётся)

Содержание

1. Предупреждение
2. Обзор математики
3. Обратное распространение
4. Современные техники
5. Оптимизаторы
6. Нормализация
7. Ансамбли
8. Дропаут
9. Skip Connections
10. PyTorch

Предупреждение

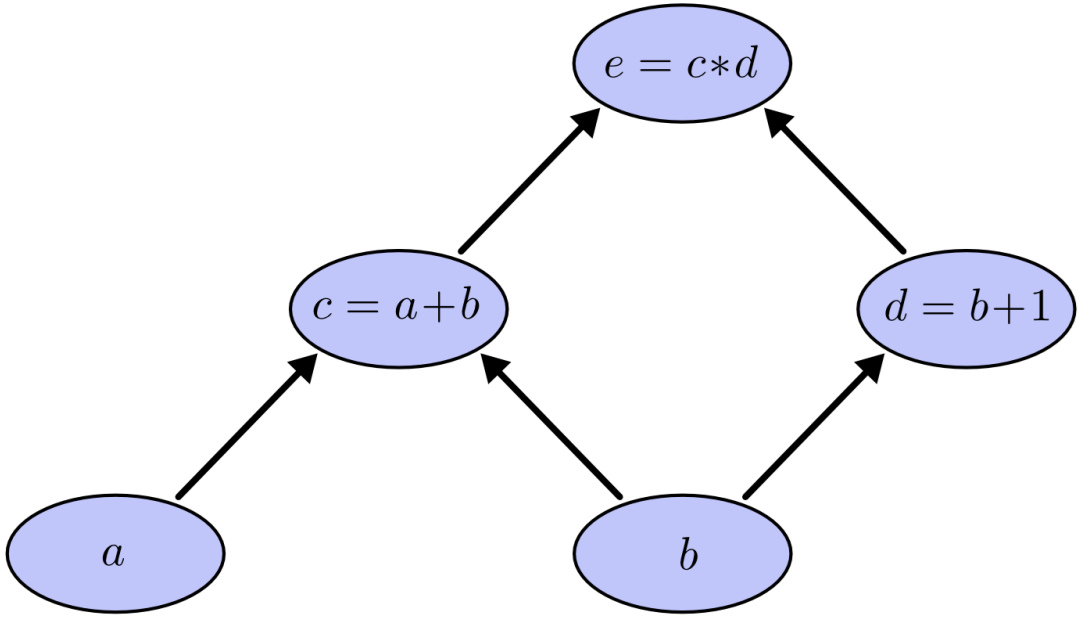
Предупреждение

- В первой части лекции мы обсудим алгоритм обратного распространения ошибки.
  - Если вы заблудились - ничего страшного. Главное - получилте общее представление о том, **что такое вычислительный граф, что делает алгоритм обратного распространения и зачем мы его используем**.
  - Последняя половина лекции, однако, будет иметь **решающее** значение для современных систем глубокого обучения.
    - Здесь будет много математики. Если вы не понимаете математику, ничего страшного, просто **обратите внимание на пункты с надписью «вывод» и выделенный жирным шрифтом текст** - они помогут вам понять, как использовать эти инструменты.

Математическая база

Вычислительные графы

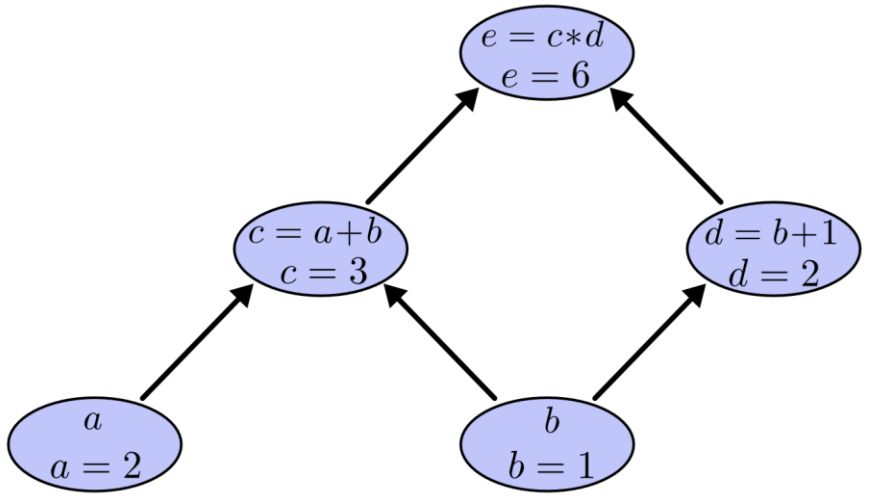
- Допустим, у нас есть некоторая функция e(c, d), но c и d - функции других переменных. У нас есть c(a, b) и d(b).
- Мы можем записать зависимость этих функций друг от друга в виде дерева.
- Мы называем его **вычислительным графом**, потому что он говорит нам, как вычислить конечное значение e из листовых узлов (входов) a и b.
- **Каждый узел в этом дереве является функцией** от входящих узлов

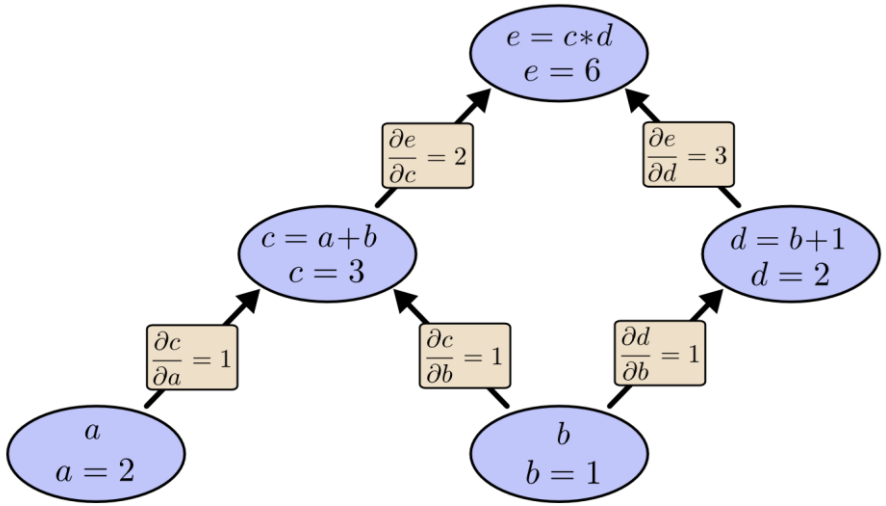


Вычислительные графы и цепное правило

- Если мы хотим вычислить производные входа относительно выхода, нам нужно использовать правило цепи для множества переменных
  - Просуммируйте все **уникальные** пути от входа к выходу
  - Для каждого пути перемножьте все частные производные каждого выходного узла по отношению к соответствующему входному узлу

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c}\frac{\partial c}{\partial b} + \frac{\partial e}{\partial d}\frac{\partial d}{\partial b}$$

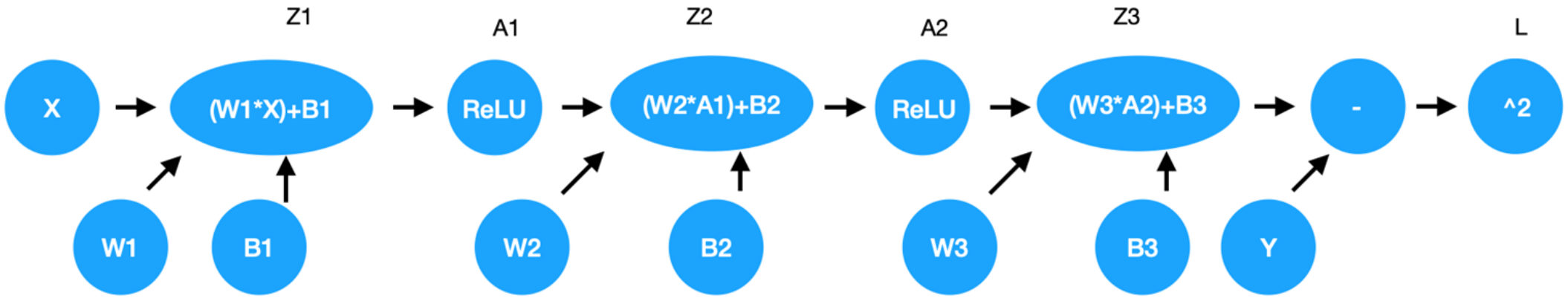




Обратное распространение ошибки

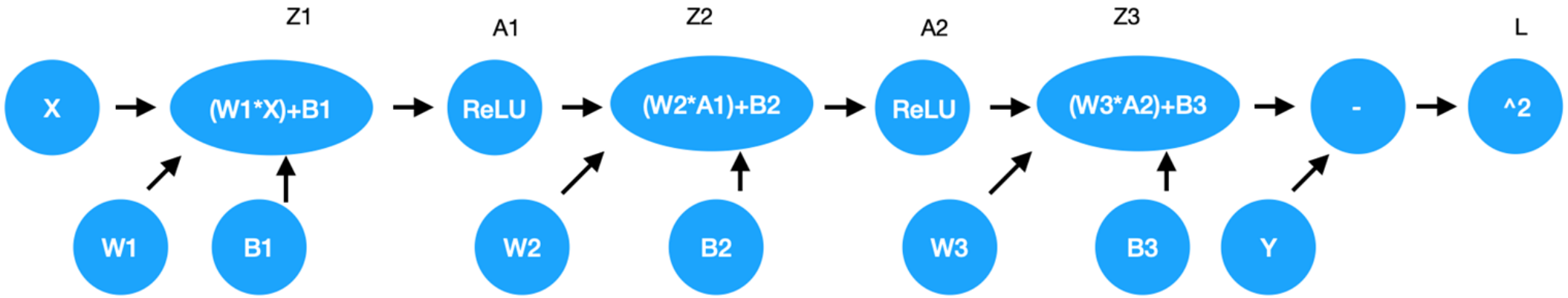
Обратное распространение ошибки

- Вот пример графа вычислений с функцией потерь MSE в игрушечной нейронной сети на одном обучающем примере
  - Эта нейронная сеть имеет только один нейрон на слой, делая входы и выходы скалярными.
- Наша задача с помощью градиентного спуска вычислить частную производную выхода по отношению к w1, w2, w3, b1, b2 и b3... но мы не хотим делать в 6 раз больше вычислений... как мы можем это сделать?



Обратное распространение ошибки

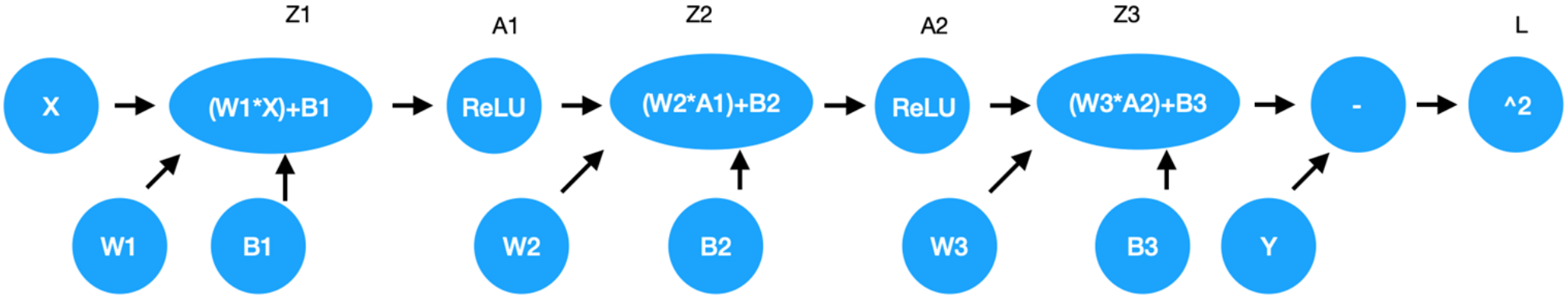
- Мы можем выписать цепное правило для всех этих случаев и посмотреть, есть ли где-нибудь место, которое мы можем оптимизировать и сэкономить на вычислениях.
- Примечание: Мы будем выписывать много частных производных... **каждая из них оценивается для текущего обучающего примера и текущих параметров**, просто не хватило места, чтобы это написать



- В этом примере x, y, b1, b2, b3, w1, w2, w3 - скаляры.
- Не зацикливайтесь на значении каждого члена, это неважно.

Примечание: при прохождении вперед мы вычисляем и сохраняем такие вещи, как частное от z_3 по отношению к a_3, чтобы использовать его здесь позже

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial w_3} \quad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial w_2} \quad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

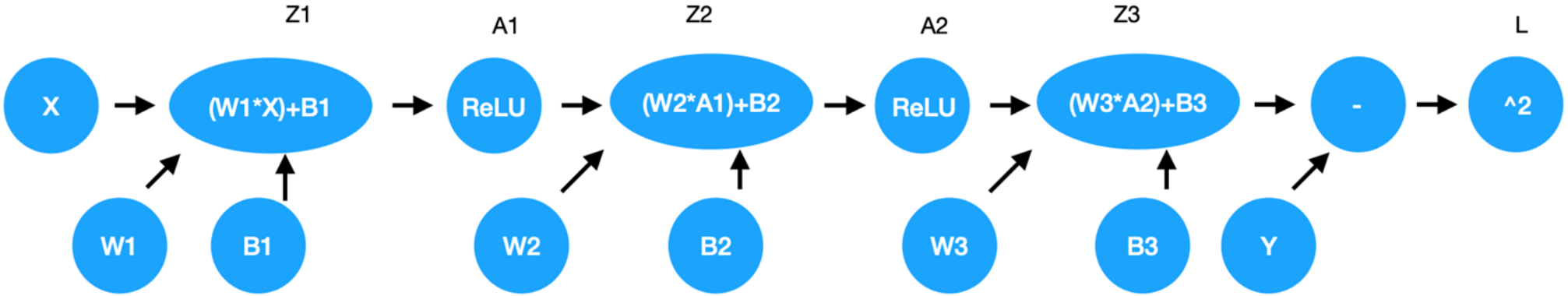


Пример графа вычислений скаляров

- В этом примере x, y, b1, b2, b3, w1, w2, w3 - скаляры.

Мы видим, что вычисляли эти значения несколько раз

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial w_3} \quad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial w_2} \quad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

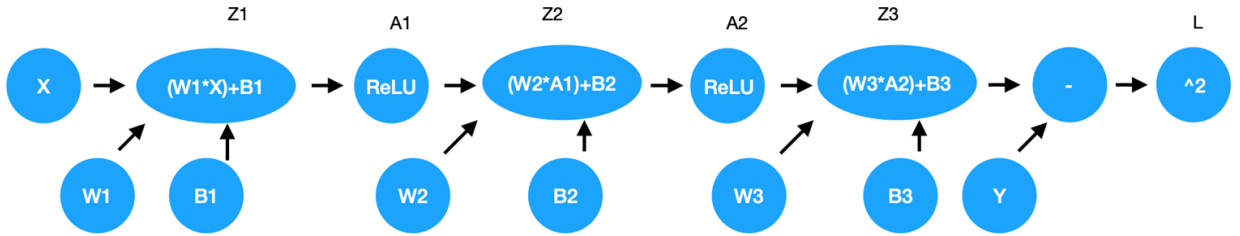


Пример графа вычислений скаляров

Обратное распространение ошибки

- Вместо того чтобы снова вычислять эти значения с помощью многократного умножения, давайте просто сохраним и повторно используем их
  - Это избавляет глубокие нейронные сети от множества лишних вычислений.
- Мы просто будем работать **от конца сети к ее передней части**, кэшируя значения, которые нам нужны, по мере продвижения вперед
- Примечание: все части здесь оцениваются для текущих данных и параметров.

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial w_3} \qquad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial w_2} \qquad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

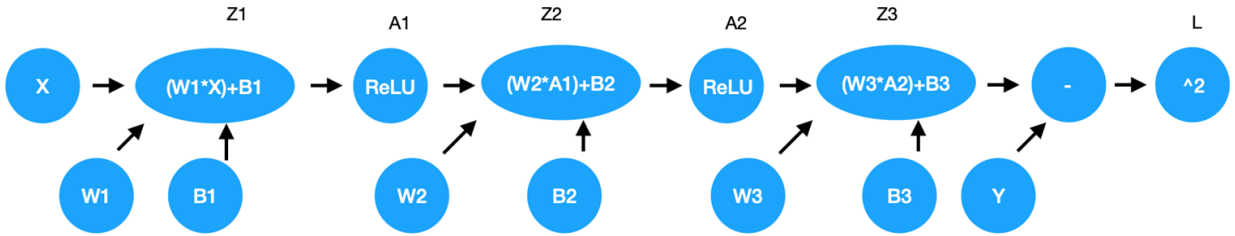


1. Сначала мы вычисляем обновление для W3, кэшируя **красное**.
2. Затем мы используем **красное** для вычисления значения **синего** перед вычислением обновления для W2
3. Эта модель использования последнего вычисления для экономии избыточных умножений при следующем обновлении продолжается

Обратное распространение ошибки

- Теперь представьте, что W_i - весовые матрицы, B_i - векторы смещения, X - входной вектор, и замените любое скалярное умножение матричным умножением.
  - Критически важно, что наша потеря L по-прежнему скалярная.
- Вам не нужно понимать, как выглядит производная L по Zi, просто представьте, что она существует - мы можем реализовать **ту же идею кэширования из скалярного случая и для реальной нейронной сети**.
  - Кэширование избавляет нас от выполнения избыточных матричных умножений, которые могут быть очень дорогими

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial w_3} \qquad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial w_2} \qquad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$



Выводы

- **Главное**, что вы должны вынести из обратного распространения, это то, что это быстрый метод получения всех частных производных, необходимых для градиентного спуска, удаляя лишние (матричные) умножения.
  - Для этого мы работаем от конца вычислительного графа к началу, кэшируя все вычисления, использованные при вычислении предыдущих частных производных
  - Работая от конца графа к его передней части, мы можем быстро и эффективно обрабатывать гораздо более сложные вычислительные графы
- Современное программное обеспечение для автодифференцирования, например **pytorch**, будет следить за графом и вычислять наши градиенты с помощью обратного распространения.
  - Он может обрабатывать произвольно большие вычислительные графы (наш игрушечный пример был очень простым по сравнению с реальными системами глубокого обучения)

Современное глубокое обучение

Оптимизация

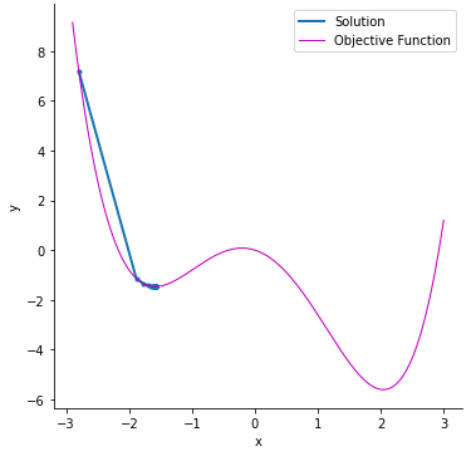
- Можем ли мы добиться большего, чем ванильный градиентный спуск? Да
- Мы расскажем о трех оптимизаторах: Momentum, RMSProp и Adam.
- Не поймёте математику - ок, просто убедитесь, что вы понимаете интуицию/**вывод** из каждого метода.

Вот правило обновления обычного GD для справки

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \nabla_{\bar{\theta}} \mathrm{L} \Big|_{(\bar{\theta}_t, \mathrm{data})}$$

Момент (импульс)

- Нам бы не хотелось, чтобы небольшой случайный локальный оптимум или случайные плоские точки остановили наш спуск.
    - Вдохновляйтесь реальным мячом, катящимся вниз по склону, который будет иметь импульс.
    - Вывод: Обновляем параметры с помощью средневзвешенного значения прошлых градиентов
- Технически это обновление больше не является настоящим градиентом, но оно хорошо работает для избежания локальных оптимумов



Снижение предыдущего средневзвешенного значения

Добавьте текущий градиент к нашему средневзвешенному значению, уменьшите масштаб

$$\bar{g}_{t+1} = \beta * \bar{g}_t + (1-\beta) * \nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\, \mathrm{data}}$$

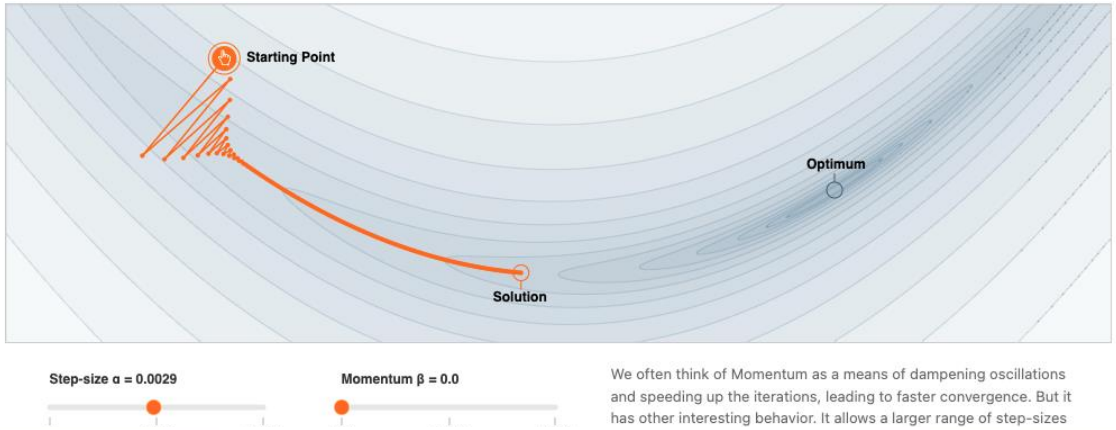
$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \bar{g}_{t+1}$$

Выполните шаг обновления градиента с помощью нашего средневзвешенного значения

- g - средневзвешенное значение предыдущих градиентов
- t - шаг по времени / t-й шаг градиента
- beta - некоторый гиперпараметр в диапазоне [0, 1], который мы выбираем, он управляет силой взвешенного среднего

Момент: визуализация

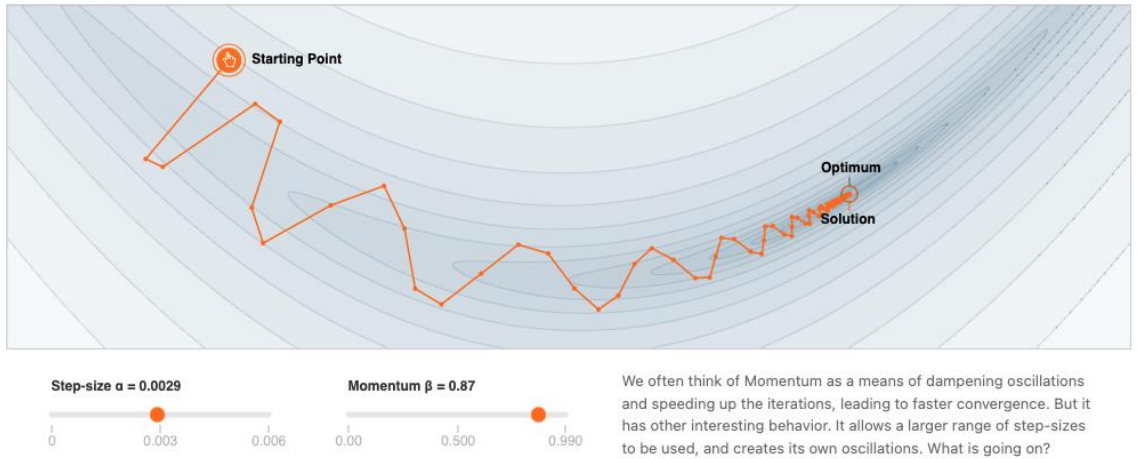
Без момента



$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \nabla_{\bar{\theta}} \mathrm{L} \Big|_{(\bar{\theta}_t,\, \mathrm{data})}$$

С моментом

(представьте, что вы уронили шарик на старте, и как он будет спускаться)



$$\bar{g}_{t+1} = \beta * \bar{g}_t + (1-\beta) * \nabla_{\bar{\theta}} \mathrm{L} \Big|_{\bar{\theta}_t,\, \mathrm{data}}$$

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \bar{g}_{t+1}$$

RMSProp

- Вместо того чтобы хранить средневзвешенное значение градиентов, мы будем хранить средневзвешенное значение компонентов квадратичного градиента.
- Мы собираемся разделить наше обновление истинного градиента на квадратный корень из этого средневзвешенного квадрата градиентов
  - Выводы/интуиция - на следующем слайде
- Первое уравнение в итоге выглядит очень похоже на уравнение импульса, поскольку мы просто вычисляем средневзвешенное значение (хотя и другой величины)

Средневзвешенное значение компонентов квадратичного градиента, аналогично импульсу. Обратите внимание, что g^2 - это имя переменной, ссылка на отслеживание квадратичных градиентов вместо обычных градиентов

$$\left(\bar{g}^2\right)_{t+1} = \beta * \left(\bar{g}^2\right)_t + (1-\beta) * \left(\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\, \mathrm{data}}\right)^2$$

Возведите в квадрат наше градиентное обновление, масштабируйте его и добавьте к нашему старому средневзвешенному, которое было уменьшено

Делим наше обновление на квадратный корень из средневзвешенного значения

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\, \mathrm{data}}}{\sqrt{\left(\bar{g}^2\right)_{t+1} + \epsilon}}$$

epsilon: какое-то очень маленькое значение, чтобы убедиться, что мы не делим на ноль

RMSProp

- Случай 1: В прошлом градиенты были очень маленькими.
  - Наше скользящее среднее квадратов градиентов будет еще меньше.
  - Квадратный корень из этого скользящего среднего будет очень маленьким числом, и деление на него должно увеличить размер финального обновления градиента
- Случай 2: Наши градиенты были очень большими в прошлом
  - Скользящее среднее квадратов градиентов будет огромным
  - Квадратный корень из этого скользящего среднего будет очень большим числом, и деление на него должно уменьшить размер финального шага обновления

$$\left(\bar{g}^2\right)_{t+1} = \beta * \left(\bar{g}^2\right)_t + (1-\beta) * \left(\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}\right)^2$$

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}}{\sqrt{\left(\bar{g}^2\right)_{t+1} + \epsilon}}$$

Вывод: это помогает бороться с проблемой, что градиенты могут быть разного размера, из-за чего мы либо застреваем на маленьких градиентах, либо проваливаемся мимо цели с большими градиентами. RMSProp гарантирует, что наши шаги никогда не будут слишком большими или слишком маленькими!

Adam

- Adam - лучший оптимизатор - и точка. Просто используйте Adam с гиперпараметрами по умолчанию
- Мы просто объединим импульс и RMSProp
  - Мы сохраним 2 скользящих средних: одну для градиентов и одну для квадратичных градиентов
- **Вывод: Это комбинация импульса и RMSProp, дающая преимущества обоих, и она работает из коробки (параметры по умолчанию) лучше, чем почти все остальное.**

$$\bar{g}_{t+1} = \beta * \bar{g}_t + (1-\beta) * \nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}$$

$$\left(\bar{g}^2\right)_{t+1} = \beta * \left(\bar{g}^2\right)_t + (1-\beta) * \left(\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}\right)^2$$

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{\bar{g}_{t+1}}{\sqrt{\left(\bar{g}^2\right)_{t+1} + \epsilon}}$$

Adam

- Adam - лучший оптимизатор - и точка. Просто используйте Adam с гиперпараметрами по умолчанию
- Мы просто объединим импульс и RMSProp
  - Мы сохраним 2 скользящих средних: одну для градиентов и одну для квадратичных градиентов
- **Вывод: Это комбинация импульса и RMSProp, дающая преимущества обоих, и она работает из коробки (параметры по умолчанию) лучше, чем почти все остальное.**

$$\bar{g}_{t+1} = \beta * \bar{g}_t + (1-\beta) * \nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}$$

$$\left(\bar{g}^2\right)_{t+1} = \beta * \left(\bar{g}^2\right)_t + (1-\beta) * \left(\nabla_{\bar{\theta}} \mathrm{L}\Big|_{\bar{\theta}_t,\,\mathrm{data}}\right)^2$$

$$\bar{\theta}_{t+1} = \bar{\theta}_t - \lambda \frac{\bar{g}_{t+1}}{\sqrt{\left(\bar{g}^2\right)_{t+1} + \epsilon}}$$

Самопроверка

Вопросики

1. Какова роль оптимизатора в глубоком обучении? Меняет ли он оптимальный набор весов для модели?
2. Перечислите основные преимущества Момента, RMSProp и Adam.

Самопроверка

Ответики

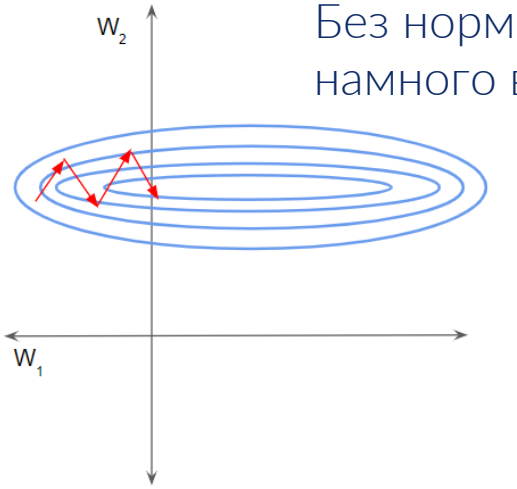
1. Оптимизатор определяет лучшее решение, которое мы способны найти, но не влияет на сами лучшие веса.
2. Момент не позволяет нам застрять в локальных минимумах, RMSProp не позволяет нашим градиентам быть слишком маленькими или слишком большими, а Adam обладает преимуществами обоих методов.

Содержание

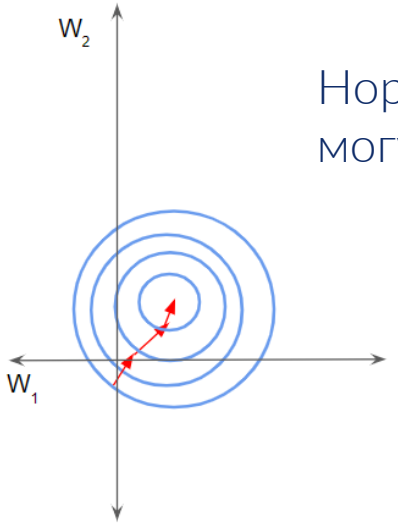
1. Предупреждение
2. Обзор математики
3. Обратное распространение
4. Современные техники
5. Оптимизаторы
6. Нормализация
7. Ансамбли
8. Дропаут
9. Skip Connections
10. PyTorch

Нормализация

- Нормализованные данные позволяют быстрее обучаться, поскольку наши поверхности функции потерь выглядят гораздо более «нормальными».
  - Когда наши данные нормализованы, веса первого слоя имеют одинаковый порядок величины, поэтому наши шаги градиента также имеют примерно одинаковый порядок величины во всех направлениях
- Мы не можем контролировать, остаются ли наши активации нормализованными в сети... как мы можем убедиться, что наши активации нормализуются так же, как и наши входные данные?



Без нормализации: W2 всегда намного важнее, чем W1



Нормализация: оба признака могут играть равную роль

Батч-нормализация (BatchNorm)

- Для батча активаций нейронов после слоя нормализуйте выход **каждого нейрона независимо (от других нейронов**) по всему батчу.
- Затем мы изменяем масштаб этих активаций с помощью некоторых изученных параметров, позволяя сети научиться изменять вес своих собственных характеристик, если она сочтет это полезным.
- **Вывод: Мы предоставляем сети более явную структуру для обучения нормализованным активациям, поскольку нормализованные признаки имеют более приятные градиенты**
- Мы обычно просто вставляем batchnorm либо до, либо после любых активаций не финального слоя, это работает очень хорошо!

$$\overline{a}' = \frac{\overline{a} - mean(\overline{a})}{std.dev.(\overline{a}) + \epsilon}$$

$$BNoutput = \overline{a}' * \overline{\gamma} + \overline{\beta}$$

a - батч активаций в нашей сети
a' - нормализованные активации
γ - вектор выученных параметров
β - вектор выученных параметров
ϵ - небольшое значение, чтобы не делить на ноль
Примечание: символ * в уравнении вывода означает поэлементное умножение

Батч-нормализация: примечания

- Примечание: во время тестирования, когда мы работаем с единичными примерами, вместо вычисления среднего и среднеквадратичного отклонения мы используем средневзвешенное значение наших последних средних и среднеквадратичных отклонений, полученных во время обучения.
- Напоминание: γ и β - это обучаемые параметры, то есть мы добавляем их в список того, что мы обновляем в градиентном спуске

$$\overline{a}' = \frac{\overline{a} - mean(\overline{a})}{std.dev.(\overline{a}) + \epsilon}$$

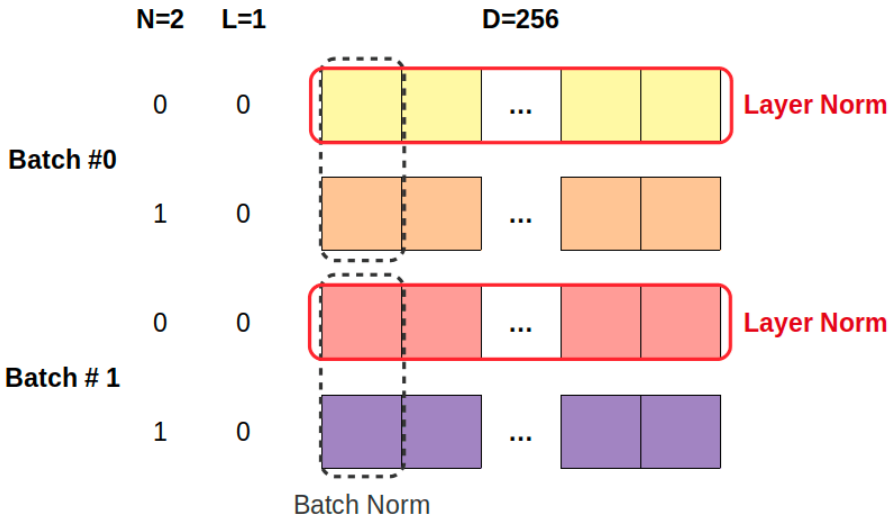
$$BNoutput = \overline{a}' * \overline{\gamma} + \overline{\beta}$$

Батч-нормализация: самопроверка

- Делает ли batchnorm нашу модель более «выразительной» (т.е. позволяет ли она обучать или аппроксимировать новые функции, которые мы не смогли бы сделать раньше).
- Ответ: НЕТ
  - Теоретически мы могли бы регулировать веса и смещения линейного слоя, чтобы получить тот же выход без использования batchnorm, но цель batchnorm не в том, чтобы получить разные выходы (batchnorm не делает нашу модель более выразительной).
  - Смысл в том, чтобы получить более красивые градиенты для наших весов и смещений, что и делается, позволяя сети обучаться нормализованным активациям (или выбирать больший вес определенных активаций, если это выгодно)

LayerNorm

- Точно так же, как batchnorm, только вместо нормализации по статистике батча мы нормализуем по признакам каждой отдельной тренировки независимо.
- Мы по-прежнему можем использовать параметры для изменения масштаба и смещения данных так же, как и в случае с batchnorm.
- Существует множество способов нормализации активаций, цель которых - обеспечить хорошие градиенты



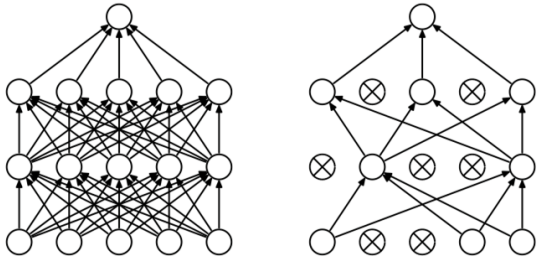
Нормализация: самопроверка

- Мы знаем, что при батчевом градиентном спуске обычно наши градиенты представляют собой среднее арифметическое градиентов обучающих примеров. Верно ли это для layernorm и batchnorm?
- Ответ: НЕТ
  - Layernorm: каждый обучающий пример обрабатывается независимо, поэтому батчевый градиентный спуск - это то же самое, что усреднение отдельных градиентов.
  - Batchnorm: значение активации после batchnorm зависит от статистики партии, и если разбить партию (батч) на более мелкие подпартии или рассматривать обучающие примеры по отдельности, то значения в прямом проходе изменятся
  - При использовании batchnorm градиенты не будут одинаковыми, если вы накапливаете их, беря среднее значение градиентов отдельных примеров

Ансамбли (Ensembling)

- Наша модель делает все, что может... Есть ли способ выжать из нее больше производительности?
- Сделайте много разных моделей (параметры каждый раз инициализируются случайным образом) и посмотрите, что думает вся группа моделей.
  - Мудрость группы
  - Можно усреднить их предсказания для задач регрессии или принять решение большинством голосов для задач классификации
  - Можно использовать множество различных конфигураций моделей и объединить их в ансамбль.
- Часто это может дать лучшую производительность, но, очевидно, требует гораздо больше вычислений, так как приходится обучать много моделей вместо одной, а также требует гораздо больше времени для оценки во время тестирования.
- Чтобы каждая модель не училась одному и тому же: опустите случайную часть обучающего набора данных для обучения каждой модели

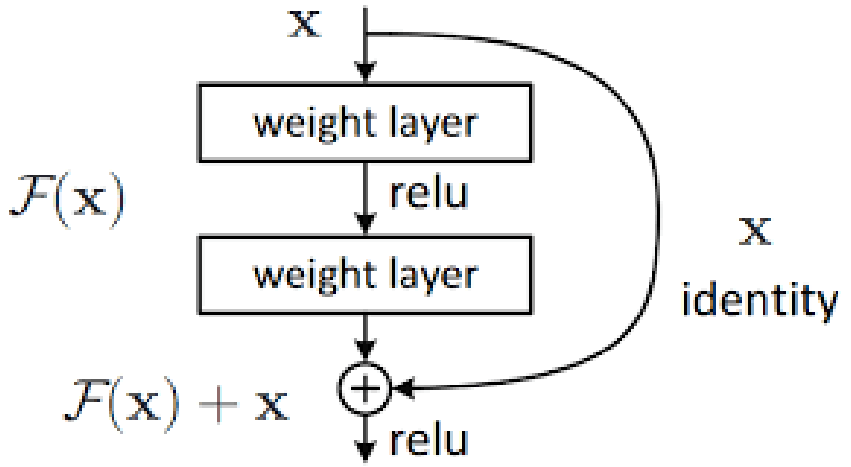
Дропаут (Dropout)


(a) Standard Neural Net (b) After applying dropout.

- Переобучение является проблемой
- Случайно обнуляйте новый набор нейронов в каждой партии.
- Это означает, что сеть должна научиться использовать различные комбинации нейронов.
- Придется обучать одну и ту же функцию несколькими способами, поскольку каждый нейрон не может полагаться на то, что предыдущий нейрон не будет обнулен.
- Это относится к концепции ансамбля.
  - **Вывод: поскольку нам приходится учить одну и ту же вещь несколькими способами, мы получаем эффект «мудрости группы», который давал нам ансамбль.**
- Во время тестирования мы ничего не обнуляем.
  - Нам просто нужно убедиться, что все наши активации уменьшены на нужную величину, иначе сумма значений, поступающих в каждый нейрон, будет намного больше, чем во время обучения
- Вывод: отсев оказывает **регуляризирующее воздействие**, делая более сложным полагаться на конкретные признаки (или чрезмерно приспосабливаться к ним)

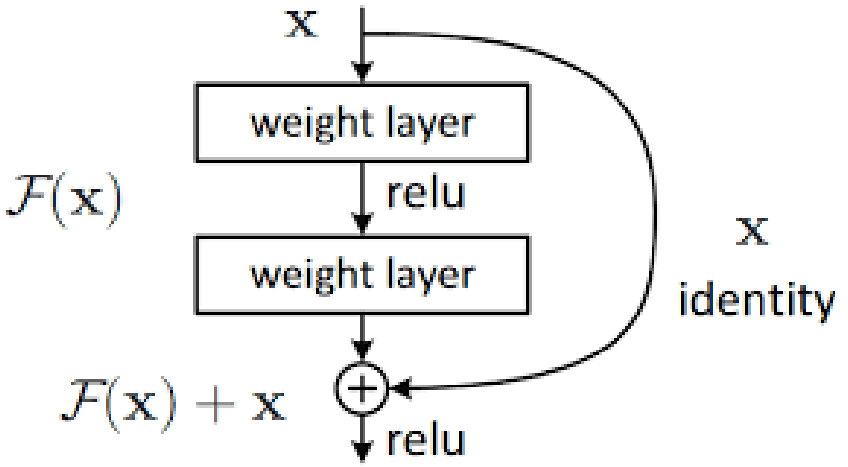
Skip Connections

- Мы просто добавляем активации предыдущего слоя к активациям текущего слоя.
  - Обычно выход каждого слоя (до активации) равен F(x), где F - наш линейный слой с функцией активации.
  - Вместо этого теперь мы будем выводить F(x) + x перед активацией.
- Вы можете пропустить произвольные слои, прежде чем снова добавить активацию, просто убедитесь, что добавили до того, как прошли через функцию активации.
  - Эти участки сети с пропущенными соединениями обычно называют «блоками».



Skip Connections

- Скажем, на полпути в обычной сети активации достаточно информативны, чтобы хорошо классифицировать входы, но в выбранной нами сети после этого остается еще несколько слоев (потенциально добавляющих еще больше шума, если мы не будем тщательно выбирать веса).
- Оказывается, можно сделать так, чтобы наш блок выплевывал именно то, что он принял (просто установите все веса в ноль), это называется функцией тождества
  - Если наши веса равны нулю, то есть F(x) = 0, то на выходе всего блока получается 0 + x = x



Вывод: теоретически это позволяет нам строить сети произвольной глубины, поскольку блоки теперь могут легко выучить функцию тождественности или очень маленькие обновления.

Резюме

- Все это лишь дополнения к набору инструментов глубокого обучения, которые часто используются в современном глубоком обучении, вот основные выводы для каждого из них
- Adam: лучшая оптимизация из коробки, чем что-либо еще
  - Берет преимущества momentum и RMSProp и объединяет их.
- Batchnorm: вы можете добавить его после любого нефинального слоя для лучшего обучения
  - Обеспечивает лучшее поведение градиентов, позволяя сети иметь нормализованные активации
- Ансамбль: Если у вас есть вычислительные мощности, это может дать вам большую производительность на вашем наборе данных
  - «Мудрость группы»
- Отсев: Бюджетный ансамбль
- Усиливает регуляризацию, заставляя каждый слой по-разному изучать одно и то же, используя различные характеристики предыдущего слоя, «мудрость группы» здесь все еще применима
- Пропустить соединения: БОЛЬШЕ СЛОЕВ
- Позволяет идти глубже, поскольку обучение слоя идентификации является тривиальным, а обучение небольших обновлений предыдущей активации - менее сложным.

PyTorch

Обратное распространение - это тяжело...

- Backprop трудно реализовать, но он необходим для того, чтобы глубокое обучение стало возможным.
  - Поэтому нам нужно что-то, что будет делать это за нас автоматически.
- Pytorch автоматически создаст для нас произвольные вычислительные графы и выполнит обратное распространение для них автоматически, так что нам не придется возиться с математикой.
  - Слава ему

PyTorch: с какой стороны браться

- Если вы понимаете Numpy, то PyTorch будет прост
  - Для пользователя PyTorch выглядит и ведет себя как numpy.
  - Вместо массивов np в Torch есть так называемые «тензоры», которые действуют точно так же.
    - За исключением того, что они генерируют вычислительный граф в фоновом режиме по ходу работы.
- На более высоком уровне PyTorch также будет иметь некоторые встроенные функции и классы для таких вещей, как активации, слои и т. д.
- Чего PyTorch не умеет:
  - Символическое дифференцирование
- Что может PyTorch:
  - Брать частные производные от одного значения (возможно, loss...) по некоторому параметру, оцененному для текущего значения параметра

Введение/Демонстрации

- Torch на самом деле выглядит как Numpy на самом низком уровне.
  - Вы можете видеть, что он выглядит практически так же, только вместо массивов у нас есть штуки, называемые тензорами.
- Мы можем выполнять все обычные операции с массивами, и они ведут себя точно так же.
  - Мы можем складывать, вычитать, возводить в квадрат.
  - Мы можем изменять форму и т. д.
- Мы можем узнать размер тензора, проверив атрибут .shape

```
>>> import torch
>>> a = torch.tensor([[1,2],[3,4]])
>>> a
tensor([[1, 2],
        [3, 4]])
>>> a + 2
tensor([[3, 4],
        [5, 6]])
>>> a ** 2
tensor([[ 1,  4],
        [ 9, 16]])
>>> a.reshape(4, 1)
tensor([[1],
        [2],
        [3],
        [4]])
>>> a.reshape(1, -1)
tensor([[1, 2, 3, 4]])
>>> a.shape
torch.Size([2, 2])
>>> torch.flatten(a)
tensor([1, 2, 3, 4])
>>> torch.flatten(a).shape
torch.Size([4])
>>>
```

Введение/Демонстрации

Вот наша функция и символьная производная.
PyTorch не знает символьных производных!

$$f(x) = x^2$$
$$\frac{\partial f}{\partial x} = 2x$$

$$f(x) = x^2$$
$$f(5) = 25$$
$$\left.\frac{\partial f}{\partial x}\right|_{x=5} = 10$$

Вот что знает и вычисляет PyTorch

```
>>> import torch
>>> input = torch.tensor([5], dtype = float)
>>> input.requires_grad = True
>>> def f(x):
...     return x ** 2
...
>>> output = f(input)
>>> output.backward()
>>> input.grad
tensor([10.], dtype=torch.float64)
>>>
```

Определите новый тензор (в данном случае считайте его параметром) с текущим значением = 5. Мы убедимся, что это float, что необходимо для дифференцирования

Установив requires_grad = True, мы поручаем torch сохранить все, что может понадобиться для включения этой переменной в вычислительный граф

Определите просто некоторую обычную функцию

Оценим функцию на нашем тензоре

Вызов функции .backward() для скалярного тензора позволит Pytorch использовать вычислительный граф для вычисления частных производных этого значения по отношению ко всем параметрам, требующим градиента, которые находятся в нашем вычислительном графе, с оценкой по текущему значению параметра

Мы можем просмотреть атрибут .grad тензора, чтобы увидеть частную производную нашего выхода по отношению к нашему входному тензору, оцененному по текущему значению, которое мы инициализировали на 5

Содержание

1. Глубокое обучение - это обучение представлений
2. Обучение переносом (Transfer Learning)
3. Обучение с учителем и без учителя

Что мы пытаемся выучить?

Признаки

Что такое «признак»

- Рассмотрим классическую задачу машинного обучения:
  - У вас есть входные данные x и некоторая функция $\mathbf{\Phi}(x)$, которая возвращает любые релевантные признаки из x.
    - Например, если x - это дом, а y - его продажная цена, то $\mathbf{\Phi}(x)$ может быть вектором с такой информацией, как возраст дома, количество комнат, наличие или отсутствие подвала и т.д.
  - Вы передаете эти характеристики в модель $f_\theta(\mathbf{\Phi}(x))$ с параметрами θ, чтобы предсказать метку y.
  - **Задача машинного обучения – «выучить» θ на основе обучающего набора пар (x, y).**

House	House Age	Number of Bathrooms	Number of Bedrooms	Size (sq. feet)	Floors	Basement?	Garage?	Backyard?	Pool?
1	12 months	4	4	2500	2	Yes	Yes	No	No
2	30 months	2	3	2000	1	No	Yes	Yes	Yes

Что такое «признак»

- Рассмотрим классическую задачу машинного обучения:
  - У вас есть входные данные x и некоторая функция $\mathbf{\Phi}(x)$, которая возвращает любые релевантные признаки из x.
    - Например, если x - это дом, а y - его продажная цена, то $\mathbf{\Phi}(x)$ может быть вектором с такой информацией, как возраст дома, количество комнат, наличие или отсутствие подвала и т.д.
  - Вы передаете эти характеристики в модель $f_\theta(\mathbf{\Phi}(x))$ с параметрами θ, чтобы предсказать метку y.
  - **Задача машинного обучения – «выучить» θ на основе обучающего набора пар (x, y).**



Извлечение признаков
$\mathbf{x \rightarrow \Phi(x)}$

Алгоритм МО
$\mathbf{\Phi(x) \rightarrow y}$
$\mathbf{f_\theta}$

Альтернативный взгляд на глубокое обучение

- В ранние времена экстракторы признаков программировались вручную, т. е. специалисты по ML вручную выбирали, какие признаки они будут вводить в выбранную ими модель.○
  - Если на вход подается изображение, то это может быть информация о краях и углах, наличие определенных форм, узоров или цветов и т. д. - все это различные представления одного и того же изображения.



Подобранные вручную признаки

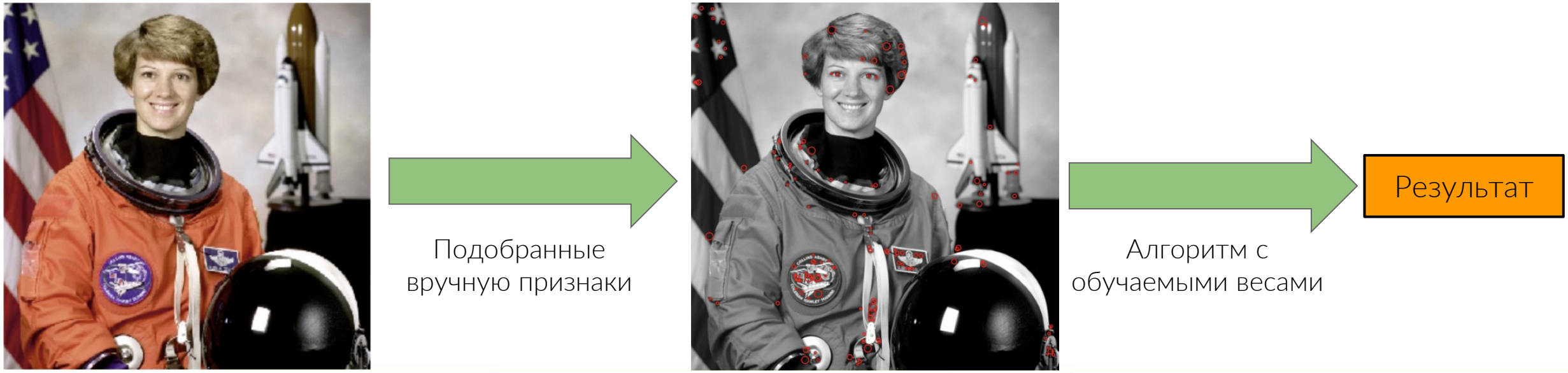


Алгоритм с обучаемыми весами

Результат

Альтернативный взгляд на глубокое обучение

- Однако это компромисс! Модель обучается своим параметрам на основе данных, но мы все еще вручную программируем экстракторы признаков. Можем ли мы сделать весь процесс обучаемым от начала до конца?
  - Да! Именно здесь на помощь приходит глубокое обучение!
  - Обучайте и экстрактор признаков - изучайте ВСЕ ШАГИ в конвейере!

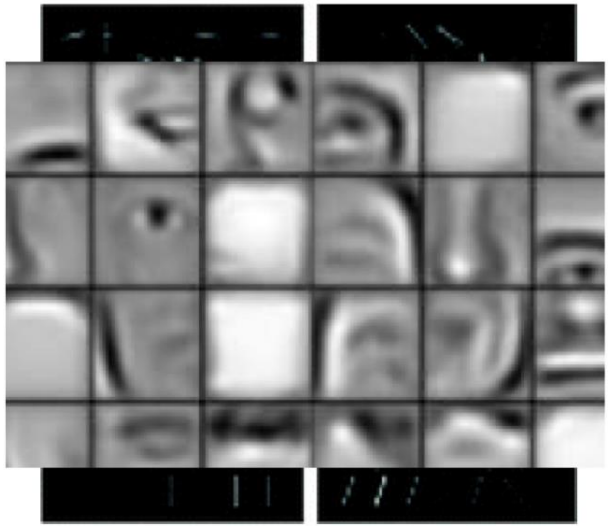


Альтернативный взгляд на глубокое обучение

- Однако это компромисс! Модель обучается своим параметрам на основе данных, но мы все еще вручную программируем экстракторы признаков. Можем ли мы сделать весь процесс обучаемым от начала до конца?
  - Да! Именно здесь на помощь приходит глубокое обучение!
  - Обучайте и экстрактор признаков - изучайте ВСЕ ШАГИ в конвейере!



Подобранные вручную признаки



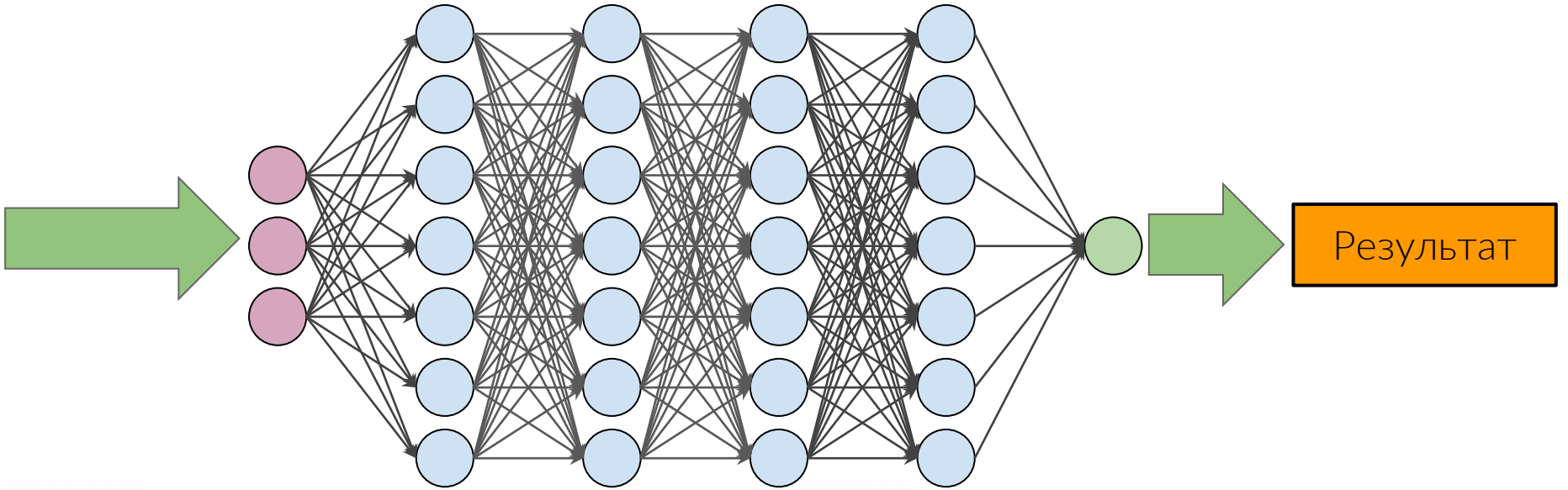
Алгоритм с обучаемыми весами

Результат

Альтернативный взгляд на глубокое обучение

- Однако это компромисс! Модель обучается своим параметрам на основе данных, но мы все еще вручную программируем экстракторы признаков. Можем ли мы сделать весь процесс обучаемым от начала до конца?
  - Да! Именно здесь на помощь приходит глубокое обучение!
  - Обучайте и экстрактор признаков - изучайте ВСЕ ШАГИ в конвейере!

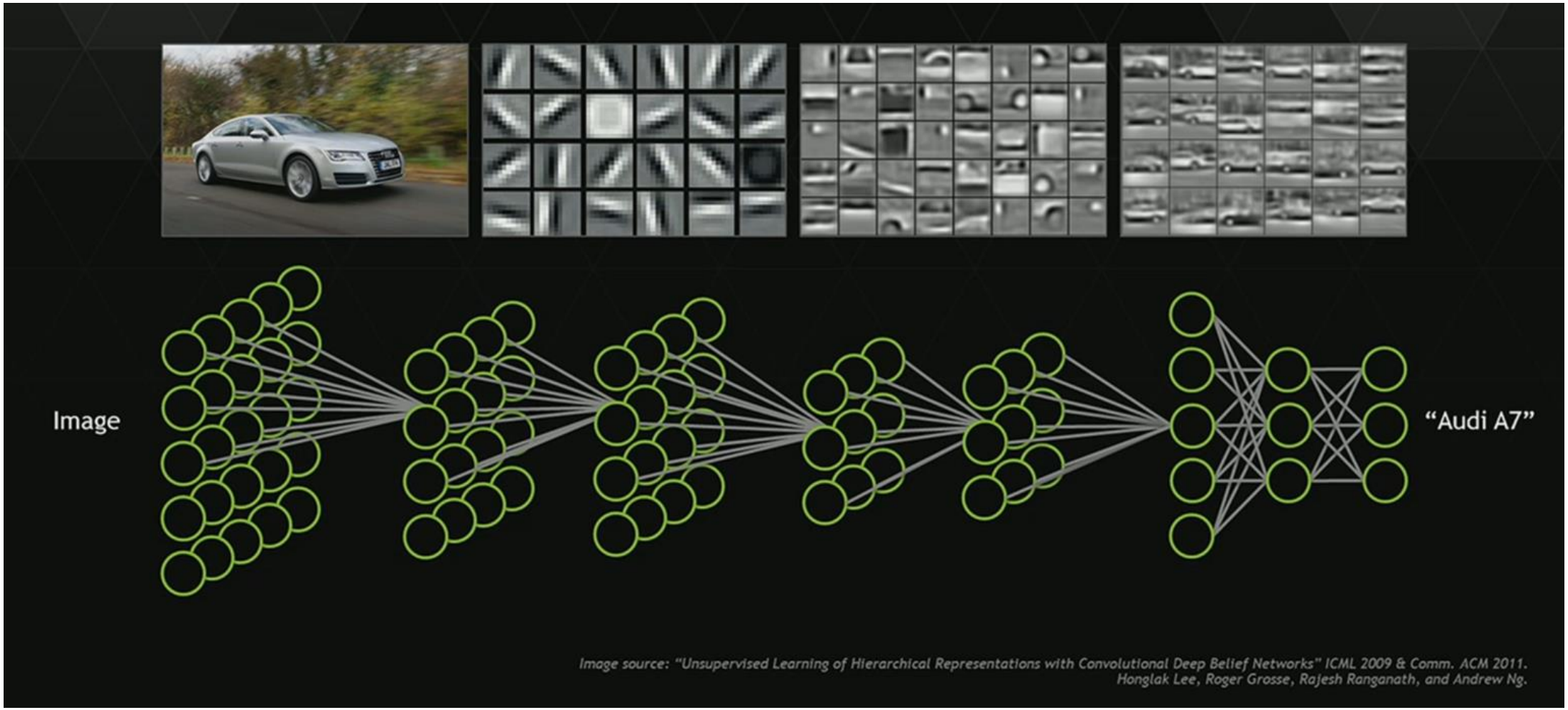




Глубокое обучение - это обучение представлениям

- Глубокое обучение позволяет модели обучаться **«хорошим» представлениям** непосредственно из данных!
  - Основная идея заключается в том, чтобы передать весь контроль модели и позволить ей научиться всему, что она считает важным для решения поставленной задачи.
  - Признаки являются синонимом представлений в ML
- Выход каждого слоя нейронной сети - это **выученное представление**, поэтому глубокое обучение можно рассматривать как процесс выучивания представлений.
  - Мы называем эти представления иерархическими, т. е. более поздние представления зависят от более ранних и являются более абстрактными - **глубина совершенствует представления**

Глубокое обучение - это обучение представлениям



Обучение переносом (Transfer Learning)

Обучение сети с нуля

- Обучение хорошей модели с нуля требует
  - Время
  - Вычислительные мощности
  - Данные для обучения - чем больше, тем лучше.
    - Модели значительно выигрывают от большого количества данных - особенно в компьютерном зрении, несколько тысяч примеров обычно не подходят.
  - Деньги на все вышеперечисленное







Пример

- Обучение YOLOv8s с нуля
  - Время – 500 эпох * 20 минут = 10000 минут = 166,6 часов = 6,9 суток.
  - Вычислительные мощности – допустим, RTX4090 с 24Гб VRAM
  - Данные для обучения - чем больше, тем лучше.
    - Допустим, COCO 2017 – 164К изображений, 25Гб
  - Счета за электричество 😟







Пример

- Обучение LLAMA3 400B
  - Время – примерно 8 миллионов GPU*ч
  - Вычислительные мощности – кластер с 24000 GPU H100
  - Данные для обучения – 15 триллионов токенов + 10 миллионов размеченных примеров
    - Для простоты представим, что каждый токен – слово.
    - Грубая оценка - 174 623 Гб
  - Более 720 миллионов долларов США (только цена железа)







Мотивация обучения переносом

- При обучении с нуля параметры модели инициализируются случайным образом и обновляются с помощью какого-либо алгоритма оптимизации, например SGD или Adam.
- Таким образом, если необходимо решить две разные задачи, независимо от того, насколько они похожи, этот процесс будет повторяться отдельно оба раза.
- Однако так ли это необходимо? Рассмотрим случай, когда мы учимся чему-то новому: всегда ли мы начинаем с нуля?



Мотивация обучения переносом

- Давайте применим эту идею к глубоким нейронным сетям - рассмотрим обучение отдельных моделей для классификации кошек/собак и распознавания лиц.
  - Все они должны будут выучить подходящий набор экстракторов признаков для своих входов. Поскольку **представления иерархичны**:
    - Низкоуровневые (ранние) слои могут изучать экстракторы признаков для очень конкретных деталей, таких как края, углы, формы, узоры, цвета и т. д.
    - Высокоуровневые (более глубокие) слои могут обучать экстракторы признаков для абстрактных понятий, таких как ментальные модели кошек/собак или человеческих лиц.
- Примечание: эти низкоуровневые признаки достаточно общие, чтобы их можно было извлекать из любых изображений для любых задач, а представления/признаки начинают существенно различаться для разных задач только на более поздних слоях.
- Не беспокойтесь пока о том, как именно выглядят эти экстракторы признаков / слои нейронной сети... мы обсудим это в следующий раз



Layer 1



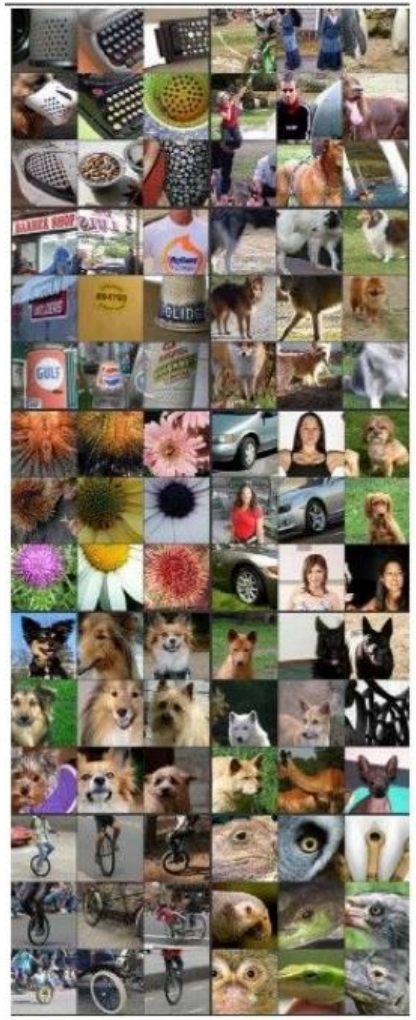
Layer 2



Layer 3



Layer 4



Layer 5

Возьмем гипотетическую модель, обученную на входных изображениях – какую информацию и/или визуальный контент ищет каждый слой?

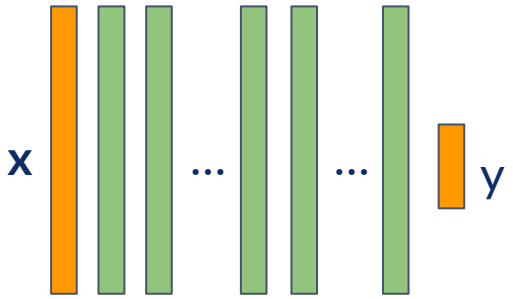
Мотивация обучения переносом

Как мы можем передать «знания, полученные» одной сетью, другой? Чем мы на самом деле «делимся» между двумя сетями? Или, скорее, чем мы можем поделиться между ними?

Подсказка: нейронные сети - это стопки слоев.

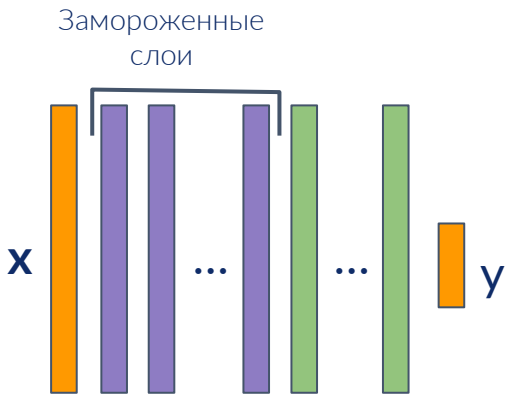
Замораживаем слои

- Возьмите **предварительно обученную (pretrained)** сеть из задачи 1



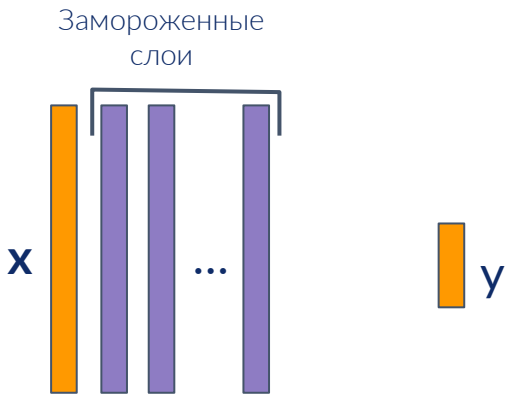
Замораживаем слои

- Возьмите **предварительно обученную (pretrained)** сеть из задачи 1
- **Заморозьте** некоторые из начальных слоев (т.е. отключите обновление градиента для них) и рассматривайте их как фиксированный экстрактор признаков - возьмите активации этих слоев в качестве некоторого промежуточного представления входных данных



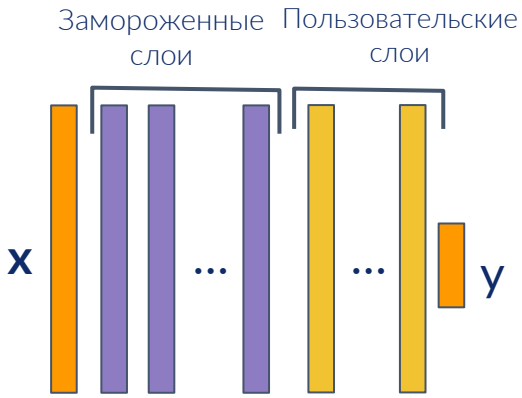
Замораживаем слои

- Возьмите **предварительно обученную (pretrained)** сеть из задачи 1
- **Заморозьте** некоторые из начальных слоев (т.е. отключите обновление градиента для них) и рассматривайте их как фиксированный экстрактор признаков - возьмите активации этих слоев в качестве некоторого промежуточного представления входных данных
- Оставшиеся незамороженные (возможно, нисколько) последующие слои выбросьте.



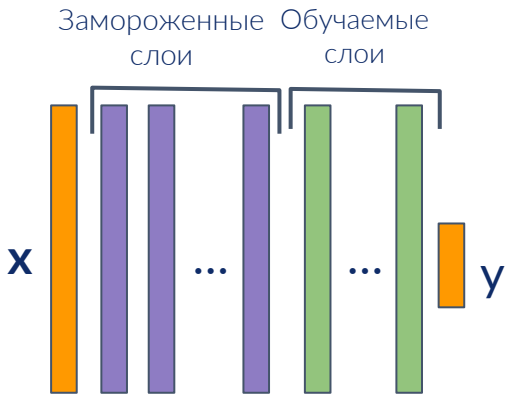
Замораживаем слои

- Возьмите **предварительно обученную (pretrained)** сеть из задачи 1
- **Заморозьте** некоторые из начальных слоев (т.е. отключите обновление градиента для них) и рассматривайте их как фиксированный экстрактор признаков - возьмите активации этих слоев в качестве некоторого промежуточного представления входных данных
- Оставшиеся незамороженные (возможно, нисколько) последующие слои выбросьте.
- По желанию, присоедините к замороженным слоям новую сеть (часто всего несколько линейных слоев, которые инициализируются случайным образом), которая не заморожена.



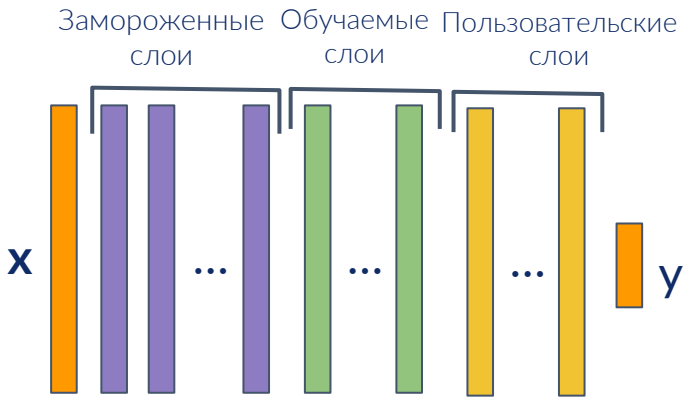
Тонкая настройка (fine-tuning) слоев

- Вместо того чтобы отбрасывать незамороженные слои **предварительно обученной** сети, настройте их, т. е. **обучите** их дальше (начиная с тех же самых предварительно обученных весов из задачи 1) на новых данных для задачи 2.



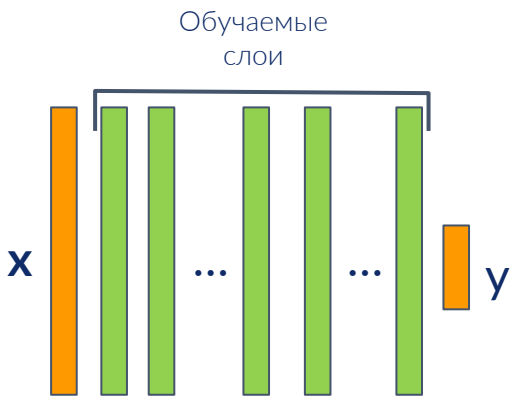
Тонкая настройка (fine-tuning) слоев

- Вместо того чтобы отбрасывать незамороженные слои **предварительно обученной** сети, настройте их, т. е. **обучите** их дальше (начиная с тех же самых предварительно обученных весов из задачи 1) на новых данных для задачи 2.
- Также можно одновременно обучать любые (необязательные) дополнительные выходные слои (опять же, как правило, это просто случайно инициализированные линейные слои) поверх исходных предварительно обученных слоев сети по мере необходимости!



Тонкая настройка всей сети

- В некоторых случаях может оказаться целесообразным провести тонкую настройку всей предварительно обученной сети, а не некоторого подмножества слоев. Это можно рассматривать как инициализацию параметров сети параметрами предварительно обученной сети, а не обычной случайной инициализацией. Другими словами, хорошая предварительно обученная сеть дает вам «фору» в процессе обучения.



Тонкая настройка vs заморозка

- Выбор количества замороженных, тонко настраиваемых и пользовательских слоев во многом зависит от конкретной задачи. Тем не менее, вот что предлагает CS231N (курс DL в Стэнфорде) для четырех распространенных сценариев, в которых трансферное обучение обычно применимо:

	Большой набор данных задачи 2	Малый набор данных задачи 2
Набор данных задачи 2 аналогичен набору данных задачи 1	Следует провести точную настройку всей сети.	Нет необходимости в тонкой настройке. Заморозьте большинство начальных слоев и обучите линейный классификатор поверх них.
Набор данных задачи 2 отличается от набора данных задачи 1	Следует ли проводить тонкую настройку всей сети.	Не настраивать. Заморозьте некоторые начальные слои и обучите на них пользовательскую сеть.

Небольшое отступление о терминологии глубокого обучения

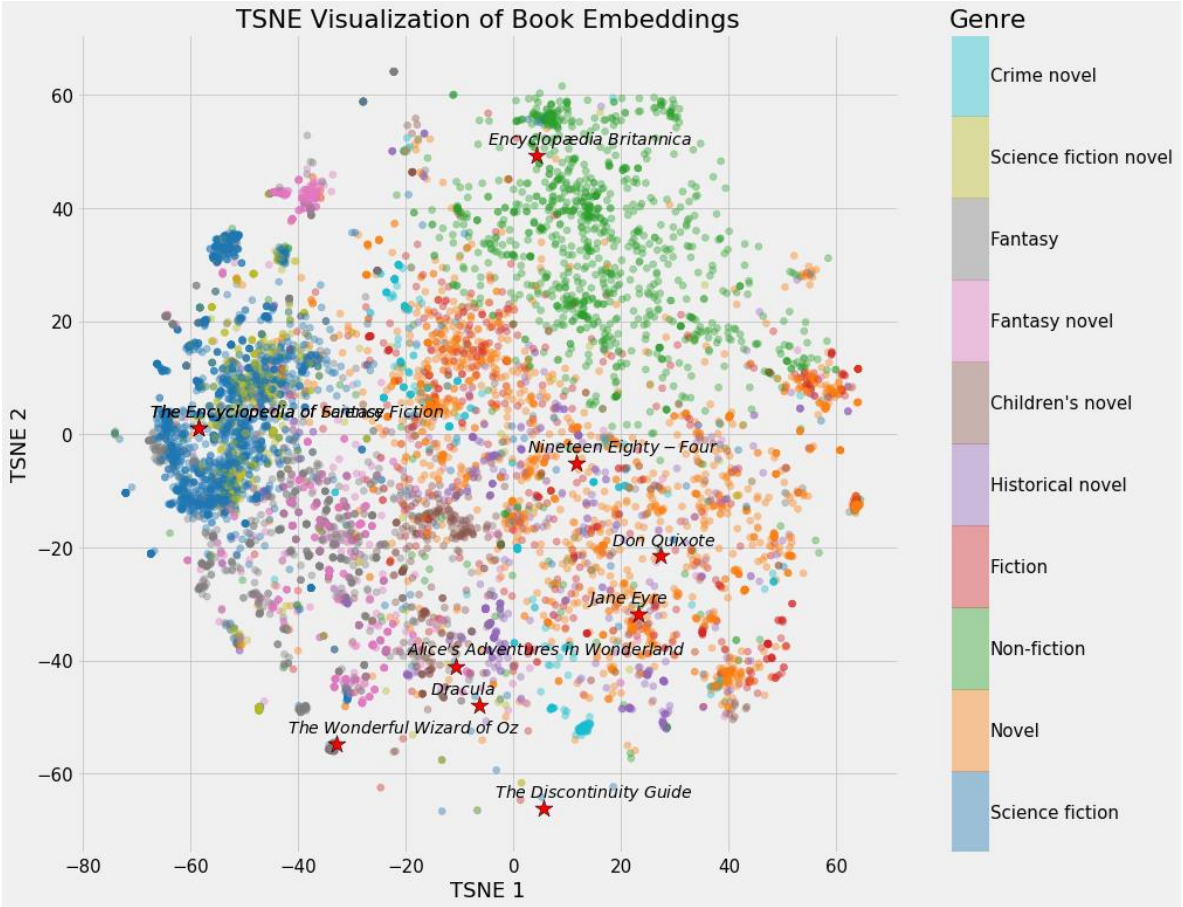
Шоты (shots)

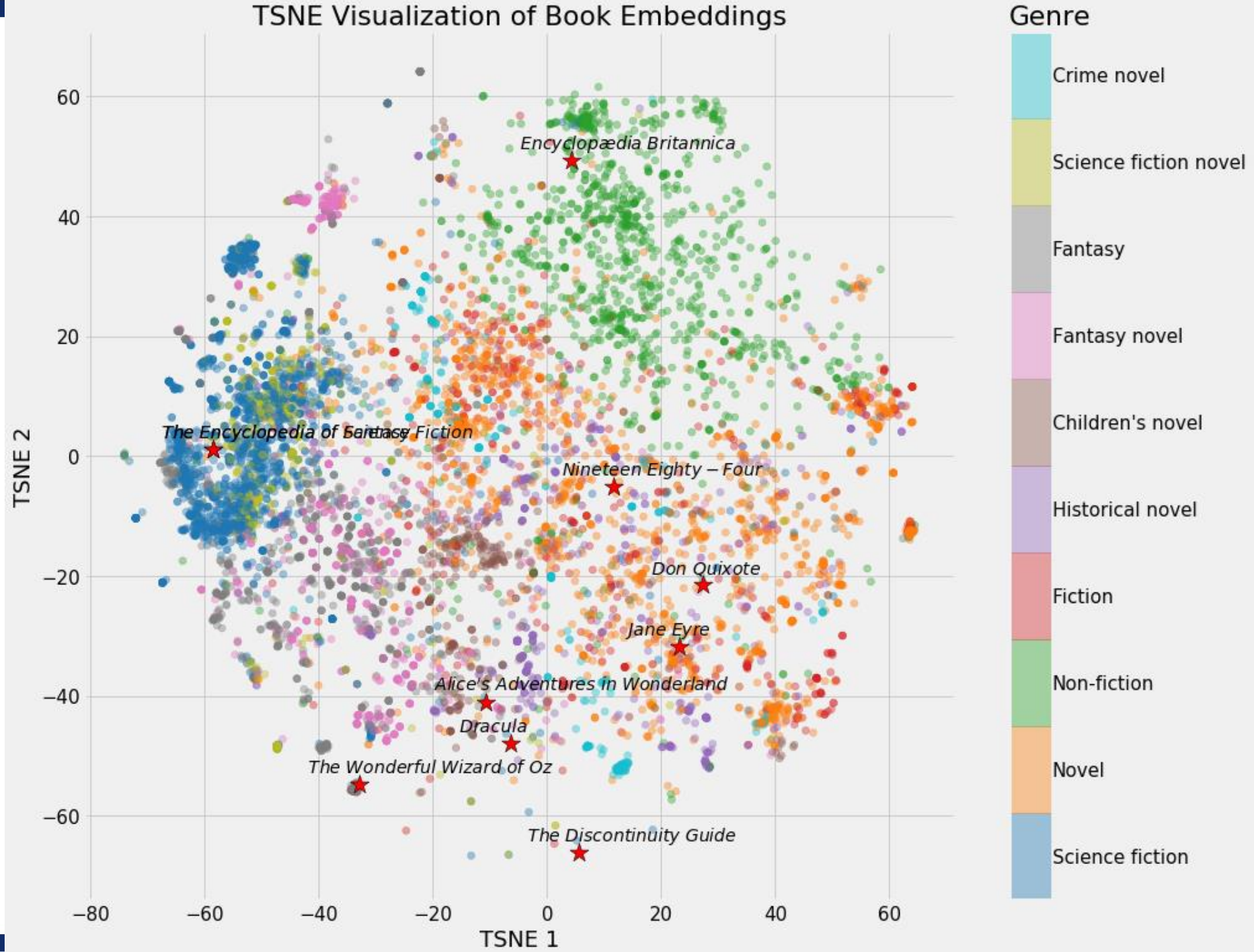
- Шоты - количество новых примеров, которые вы показываете предварительно обученной сети во время обучения с переносом.
- Few shot: показать сети небольшое количество примеров
- k-shot: показать сети ровно k примеров (скажем, k помеченных точек данных)
- 1-shot: показать сети только один пример
- 0-shot: не показывать сети никаких новых примеров.
- Использование замороженных экстракторов признаков является примером адаптации 0-shot - один из примеров это LLM

Эмбеддинги (embeddings)

Русское название – вектор встроенных признаков.

- Встраивание - это вывод карты из некоторого (дискретного или непрерывного) пространства в другое (непрерывное) пространство.
  - Например, из пространства пикселей изображения, текстовых лексем или звуковых волн в пространство d-мерных векторов.
  - Эта карта дает вкрапления изображения, текста или звука соответственно.
- Другими словами, эмбеддинги - это просто еще один способ представления сложных нечисловых данных с помощью чисел!

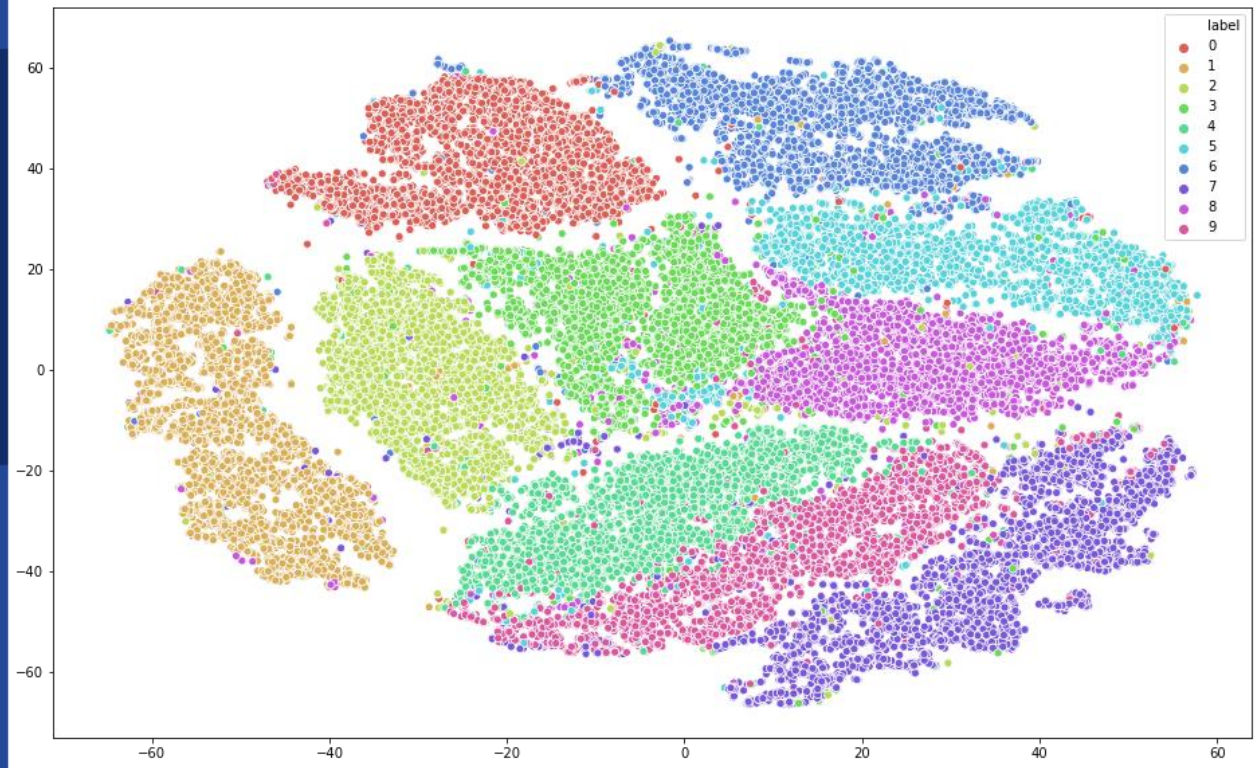


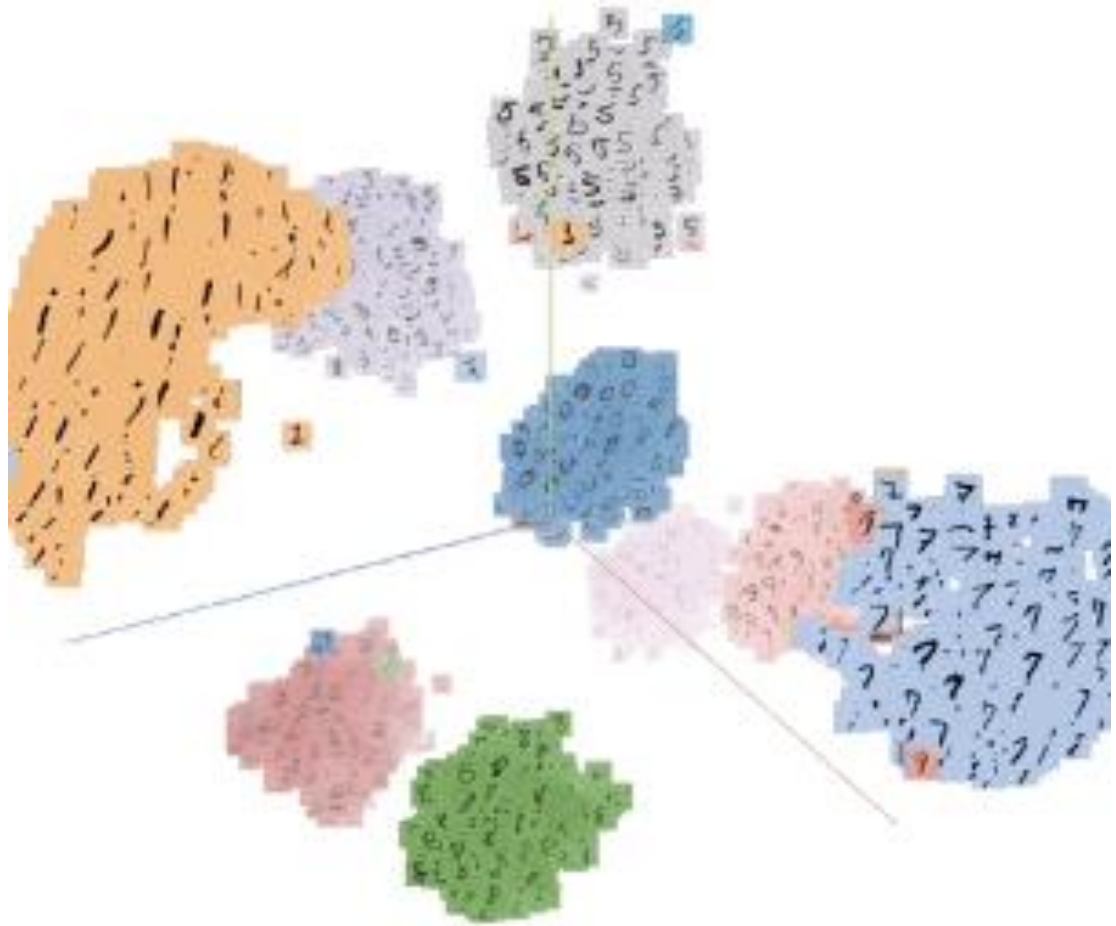

TSNE Visualization of Book Embeddings
TSNE 1
TSNE 2
Genre
Crime novel
Science fiction novel
Fantasy
Fantasy novel
Children's novel
Historical novel
Fiction
Non-fiction
Novel
Science fiction
Encyclopædia Britannica
The Encyclopedia of Fantasy
Nineteen Eighty – Four
Don Quixote
Jane Eyre
Alice's Adventures in Wonderland
Dracula
The Wonderful Wizard of Oz
The Discontinuity Guide

Выученные эмбеддинги

- Как получить эти эмбеддинги?
  - Как и во всем, что касается глубокого обучения, давайте их выучим!
- Нейронные сети уже учат свои параметры с помощью градиентной оптимизации - вполне естественно использовать ее для генерации вкраплений данных.
- Или же, вместо того чтобы обучать новую сеть с нуля, просто возьмите выход какого-нибудь промежуточного или конечного слоя существующей нейронной сети.
  - Напомним, что предварительно обученные модели с замороженными слоями можно рассматривать как экстракторы признаков.
  - Теперь эти признаки могут функционировать как предварительно обученные вкрапления!

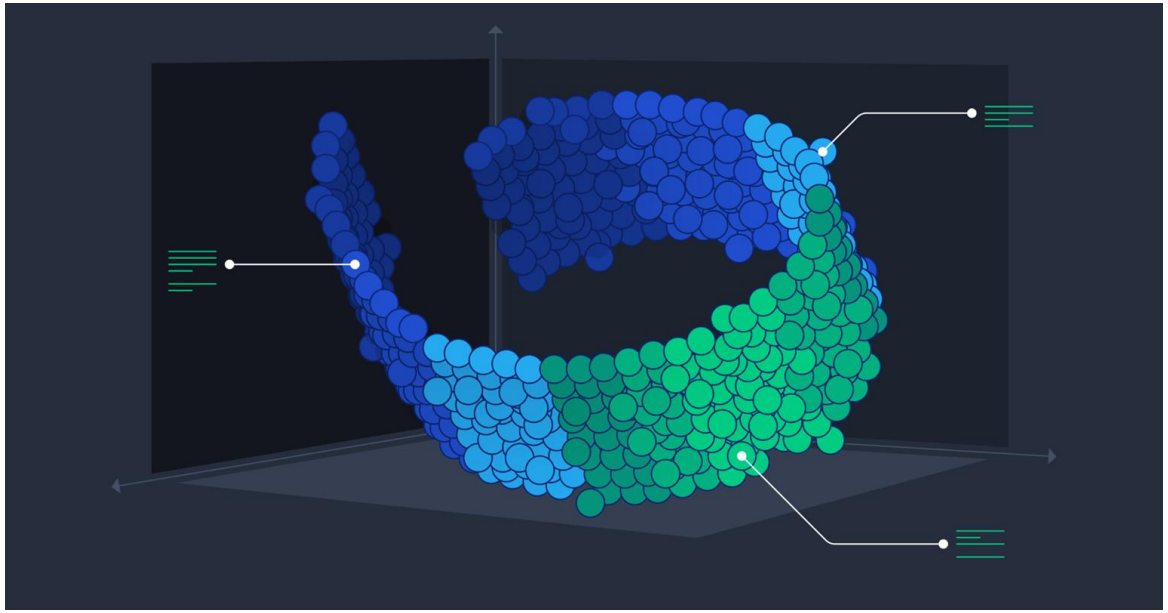
Векторные представления = Сжатые представления = Встраиваемые представления = Скрытые представления = Эмбеддинги


label
0
1
2
3
4
5
6
7
8
9
60
40
20
0
−20
−40
−60
−60
−40
−20
0
20
40
60



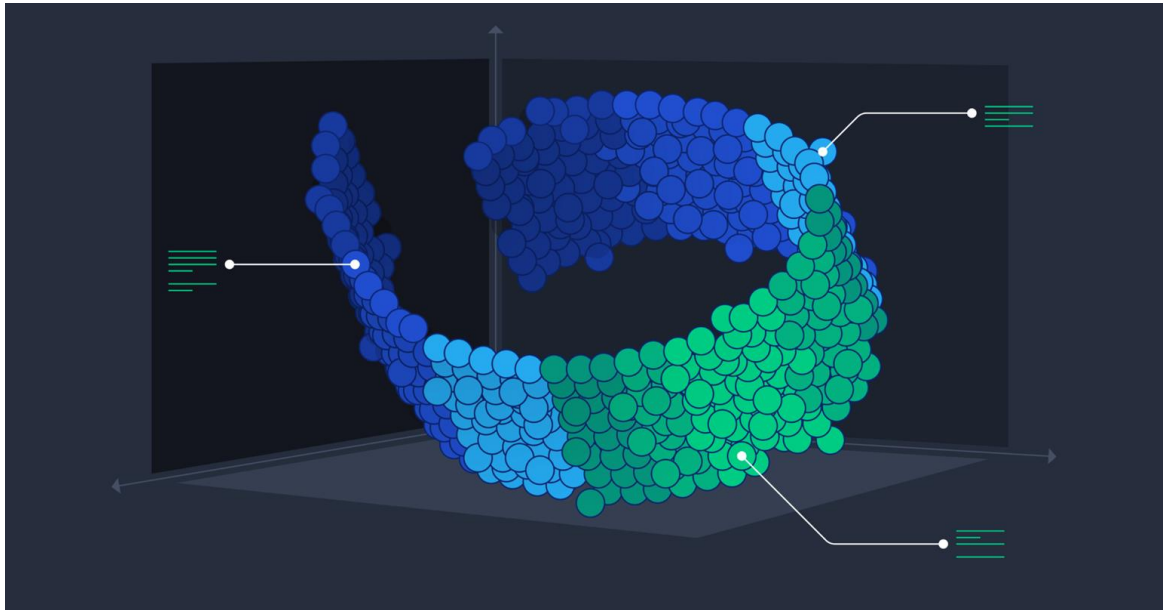
Скрытые (латентные) пространства

- Данные с большой размерностью могут иметь внутреннюю низкоразмерную структуру, с которой предпочтительнее работать.
- Эта низкоразмерная структура может быть отражена в **латентном пространстве** признаков, которое призвано закодировать всю значимую информацию, необходимую для представления исходных данных большой размерности.
- В частности, говорят, что низкоразмерное латентное пространство **встроено** в входное пространство данных большой размерности.



Почему важны латентные пространства?

- Латентные пространства содержат сжатые представления (иногда также называемые латентными) исходных входных данных.
  Латентные векторы -
  - Это вкрапления, которые имеют тенденцию быть ближе друг к другу для предметов, семантически «похожих» друг на друга в их исходном высокоразмерном пространстве
  - Это очень важное понятие в теории генеративного моделирования, и мы будем возвращаться к нему при обсуждении таких моделей, как VAE, GAN, LDM и т. д.



Векторные представления = Сжатые представления = Встраиваемые представления = Скрытые представления = Эмбеддинги

Назад к обучению с переносом

Почему полезны сети с предварительным обучением?

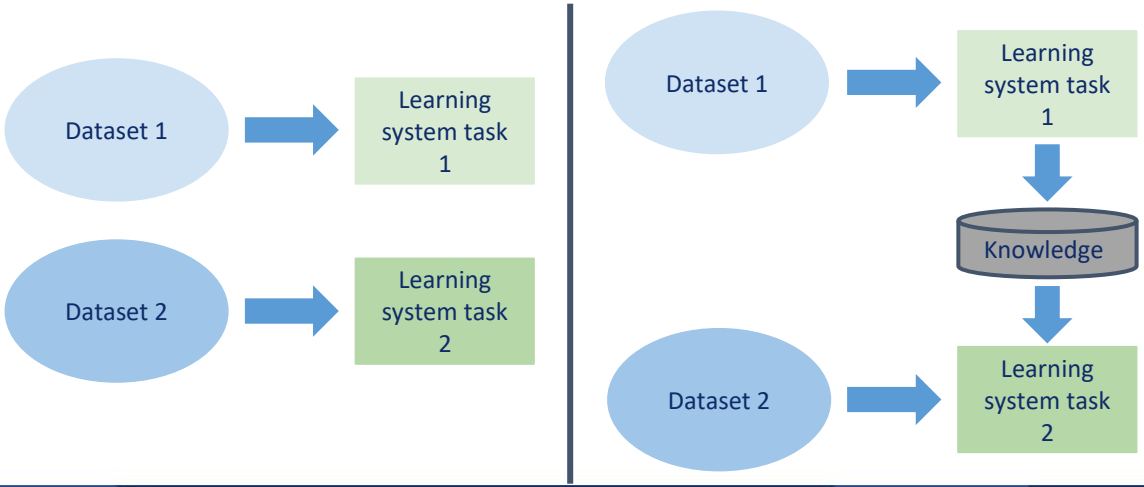
- Самые полезные сети, прошедшие предварительное обучение, как правило, обучаются на массивных наборах данных: много разнообразных данных ⇒ лучшие и обобщенные представления.
  - Они особенно полезны, когда используются для задач с малым количеством данных, где нет достаточного количества данных для того, чтобы модель самостоятельно выработала эти представления.
  - Кто-то уже проделал за нас тяжелую работу по обучению на больших наборах данных... давайте просто опираться на это, чтобы улучшить наши собственные решения.
- Популярные сети с предварительным обучением доступны в готовом виде. Например,
  - Веса моделей зрения, обученных на ImageNet/COCO, можно найти в Интернете
  - LLM - это предварительно обученные сети, и некоторые из них (например, LLaMa от Meta) даже имеют открытый исходный код!
  - Различные вкрапления word2vec или GLoVe доступны через определенные библиотеки Python

Почему полезны сети с предварительным обучением?

- Векторные представления могут быть предварительно вычислены и сохранены в памяти вместо исходных высокоразмерных данных.
  - Это позволяет освободить много места.
    - Изображение размером 256 x 256 x 3 содержит 196 608 пикселей. Предположим, что один пиксель занимает 1 байт памяти - таким образом, это изображение фактически занимает 192 кБ пространства.
    - Что, если мы сможем получить 2048-мерное встраивание этого изображения? Каждый элемент вектора встраивания будет представлять собой 4-байтовое число с плавающей точкой - такое встраивание займет всего 8192 байта = 8 кБ пространства.
    - Это почти на 96 % сокращает потребность в памяти для хранения этого набора данных!
  - Более низкоразмерные векторы (которые уже захватывают большинство низкоуровневых характеристик исходных данных) также могут ускорить обучение - подумайте о времени вычислений при прохождении ~200k-мерного изображения через большую нейронную сеть по сравнению с прохождением ~2k-мерного вкрапления в маленькую нейронную сеть.

Короче, обучение переносом

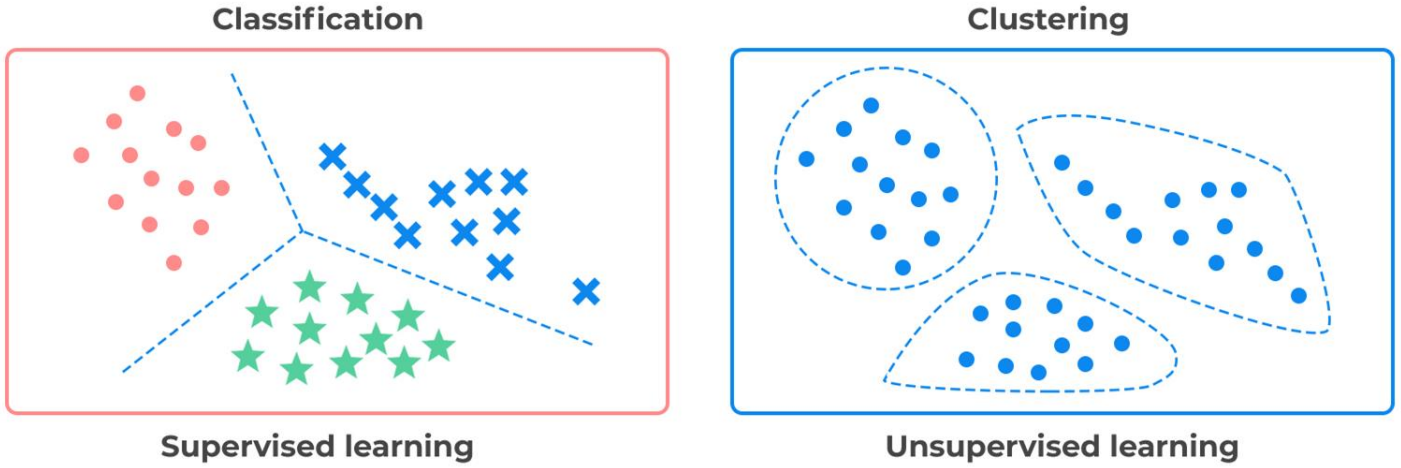
- Обучение с переносом в двух словах: использование чужих наработок, которые уже проделали тяжелую часть работы по обучению больших моделей на огромных наборах данных, для помощи своей собственной модели.
- Хотя большинство примеров сегодня было посвящено CV, в целом все можно применить к любому типу нейронных сетей. На самом деле, обучение с переносом играет огромную роль в NLP!



Обучение без учителя (Unsupervised learning)

Обучение с учителем vs без учителя

- Обучение под наблюдением - процесс обучения проходит под влиянием меток.
  - Например: классификация, регрессия, обнаружение объектов и т. д.
- Неконтролируемое обучение - процесс обучения не получает контроля от меток
  - Например: уменьшение размерности (PCA, t-SNE), кластеризация, генеративное моделирование и т. д.

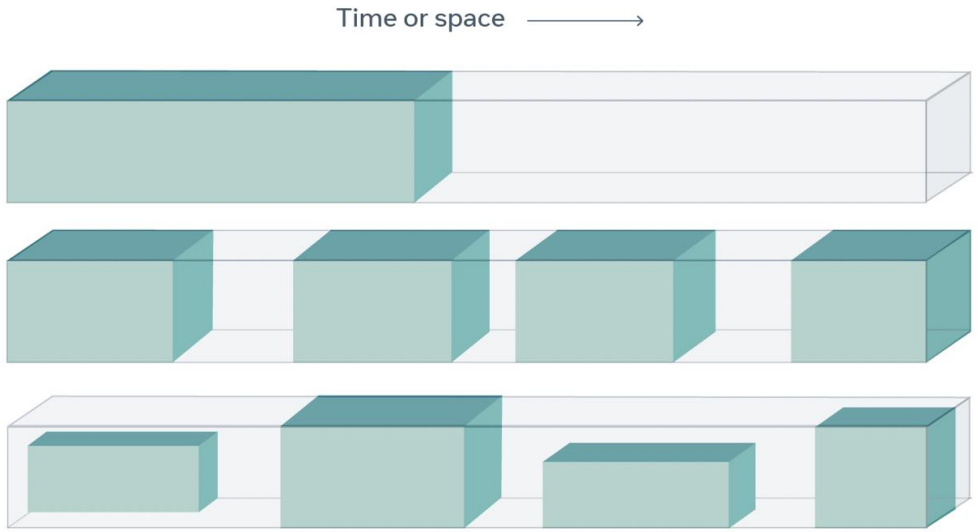


Обучение представлениям без учителя

- Модели, обученные на больших наборах меченых данных, в целом приобретают довольно полезные характеристики, но можем ли мы также изучать их на совершенно немаркированных данных?
- Представьте себе гигантскую коллекцию изображений, текстов, аудиофайлов и т. д. из Интернета без необходимости вручную маркировать их. Подумайте о последствиях этого:
  - В ImageNet всего миллион изображений, но в Интернете их миллиарды.
  - Наборы курируемых текстовых данных (например, английская Википедия) могут содержать миллиарды лексем, но в Интернете существуют триллионы слов.
- Кроме того, маркировка данных для хорошего контроля может быть чрезвычайно сложной, утомительной, дорогой и трудоемкой.
  - Например, медицинские данные, юридические данные и т. д.

Самостоятельное обучение (Self-Supervised Learning)

- Самоконтролируемое обучение - ML-модель сначала генерирует метки из исходных данных, а затем обучается контролируемым образом.
- «Общая техника самоконтролируемого обучения заключается в предсказании любой ненаблюдаемой или скрытой части входных данных по любой наблюдаемой или скрытой части входных данных».

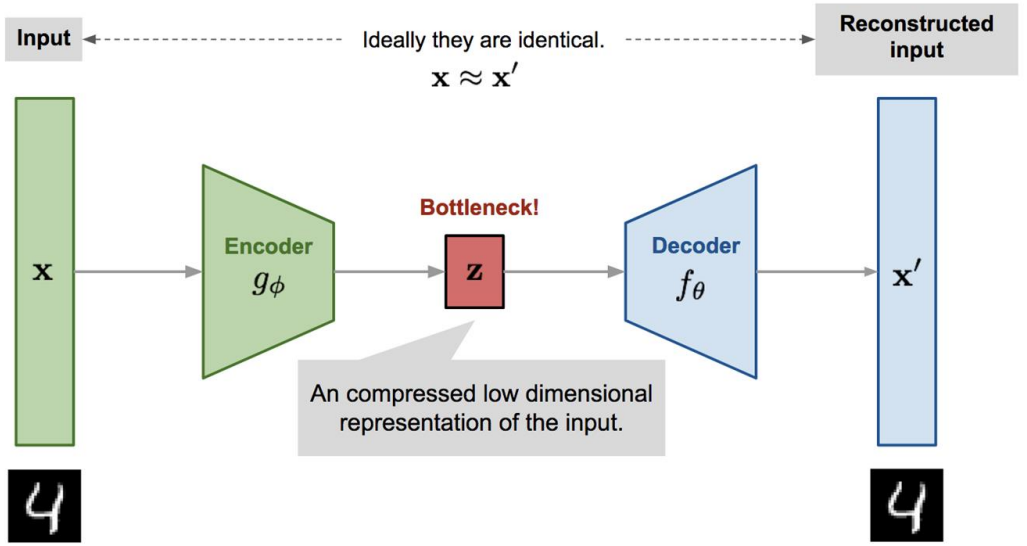


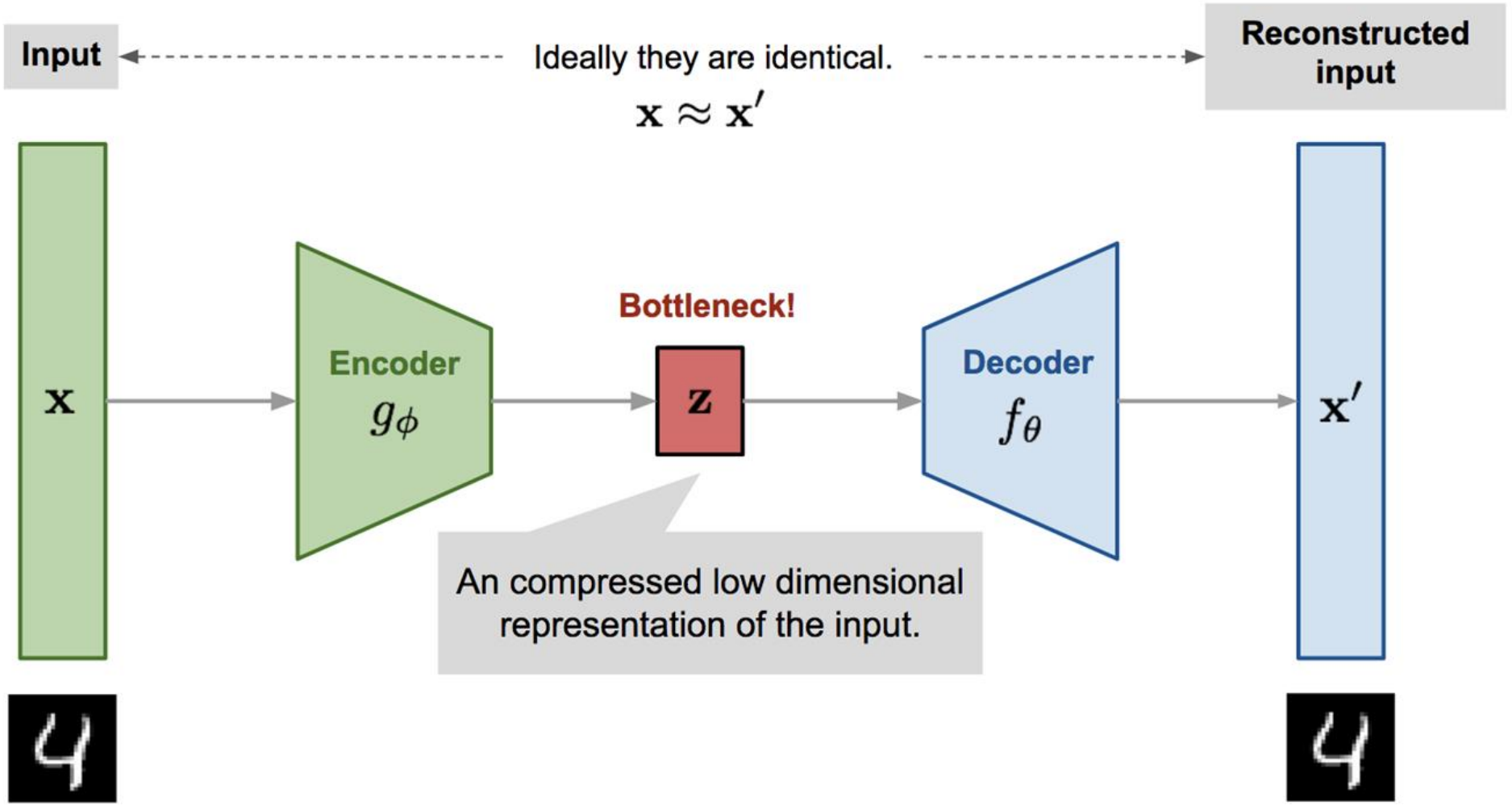
Некоторые термины

- Вспомните, как мы до сих пор говорили о «задаче 1» и «задаче 2» при обсуждении трансферного обучения. Оказывается, у них обоих есть специальное название:
  - **Предварительная задача** - задача, на которой сначала обучается самоподдерживающаяся предварительно обученная сеть
  - **Нисходящие задачи** - задачи, в которых предварительно обученные представления используются каким-либо образом. Некоторые примеры в различных областях ML включают:
    - Компьютерное зрение - классификация, обнаружение объектов, сегментация и т. д.
    - NLP - классификация, перевод, обобщение, ответы на вопросы и т. д.
    - RL - онлайновая тонкая настройка без модели и т. д.

Автоэнкодер (самая простая модель без учителя)

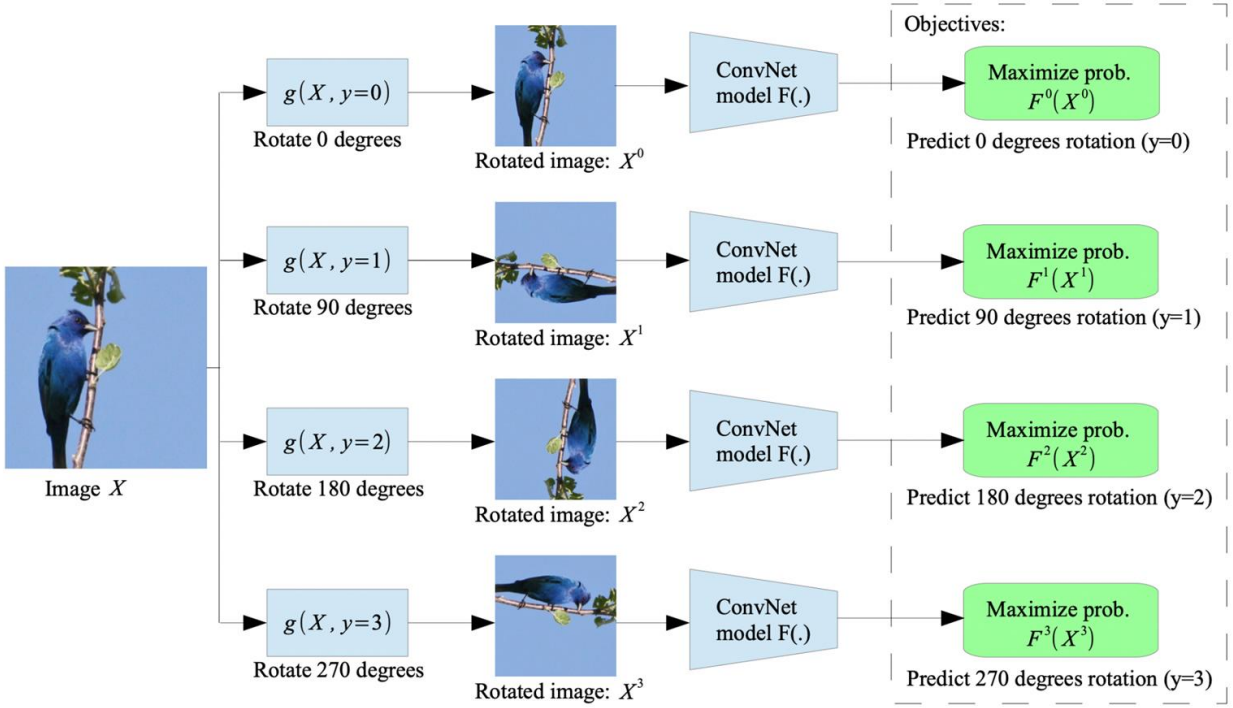
- Предварительная задача - выучить «сжатое» представление входного сигнала (это не обязательно должно быть изображение, но в примере ниже для наглядности оно одно).
- **Узким местом** является скрытый слой с очень малым количеством нейронов - если вход можно восстановить по выходу узкого слоя, значит, он захватил достаточно «важной» информации об исходном входе в виде меньшего вектора



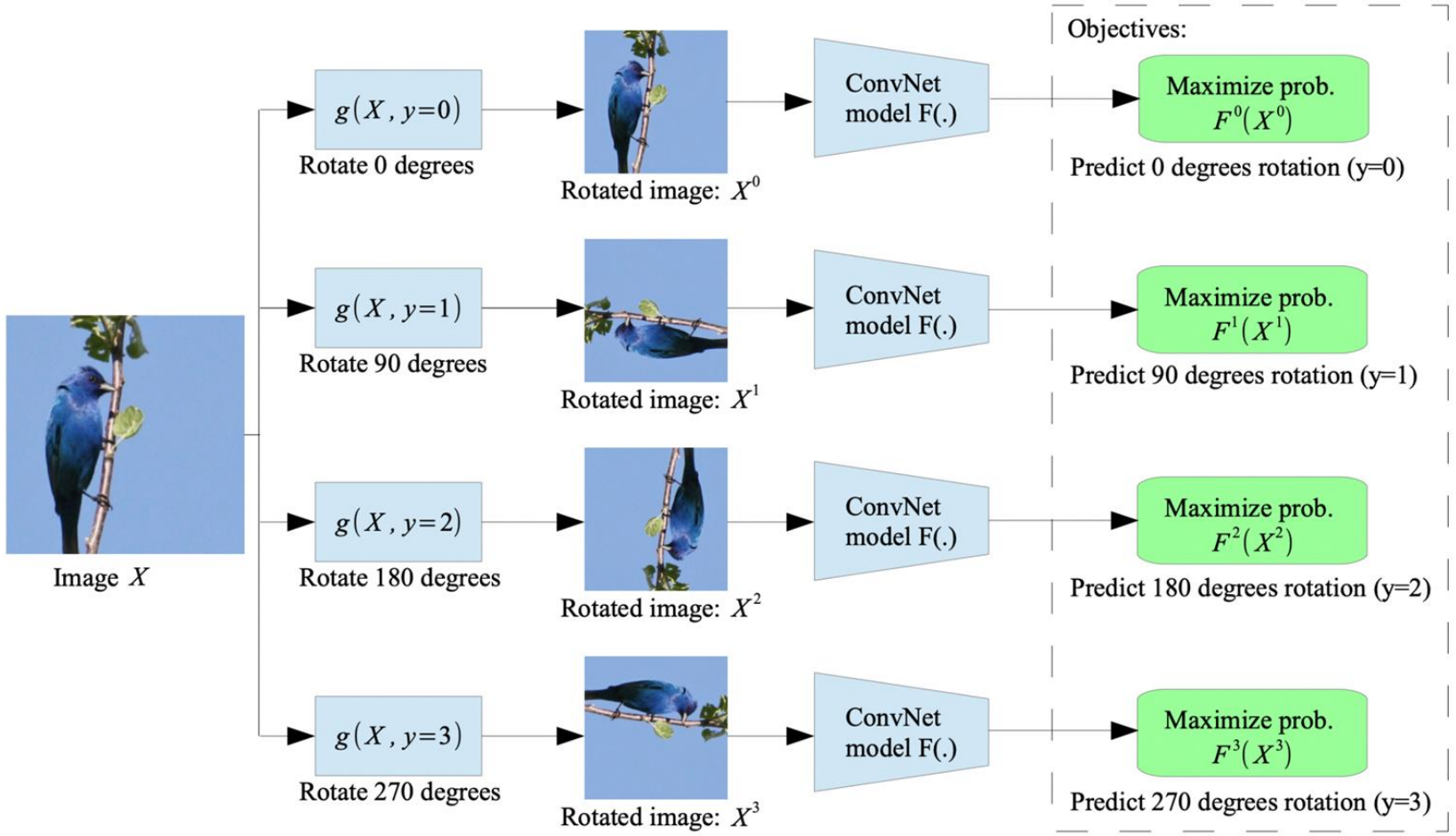

Input
Ideally they are identical.
$\mathbf{x} \approx \mathbf{x}'$
Reconstructed input
$\mathbf{x}$
Encoder
g_ϕ
Bottleneck!
$\mathbf{z}$
Decoder
f_θ
$\mathbf{x}'$
An compressed low dimensional representation of the input.

Rotnet

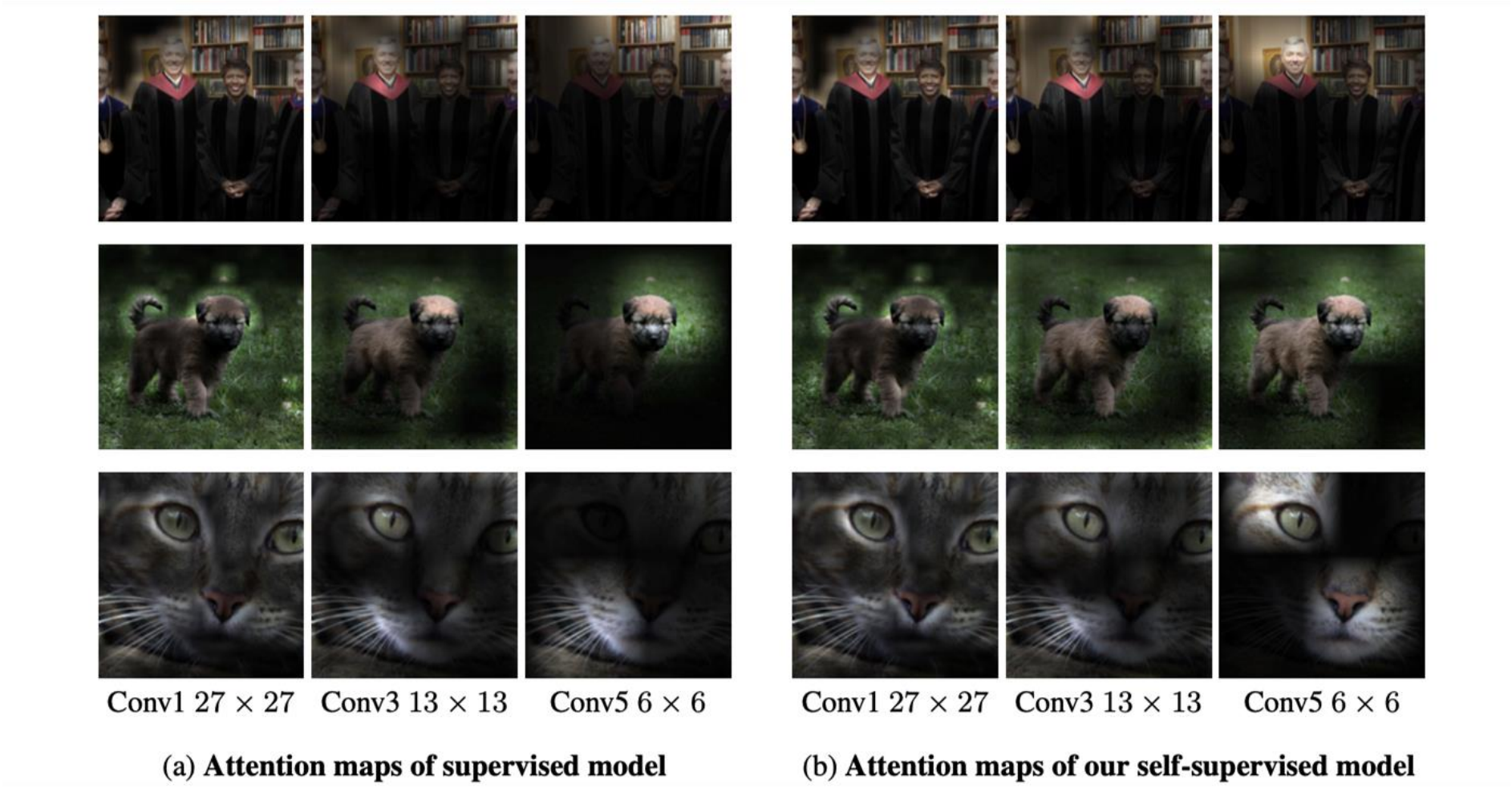
- Задача предтекста - предсказать угол поворота повернутого изображения
  - 4-сторонняя классификация между 0, 90, 180 и 270 градусами
- Модель изучает взаимосвязи между высокоуровневыми характеристиками изображения (например, местоположение, тип и положение объектов), а не только низкоуровневые паттерны



Rotnet


Image X
$g(X, y=0)$
Rotate 0 degrees
Rotated image: X^0
$g(X, y=1)$
Rotate 90 degrees
Rotated image: X^1
$g(X, y=2)$
Rotate 180 degrees
Rotated image: X^2
$g(X, y=3)$
Rotate 270 degrees
Rotated image: X^3
ConvNet model F(.)
ConvNet model F(.)
ConvNet model F(.)
ConvNet model F(.)
Objectives:
Maximize prob. $F^0(X^0)$
Predict 0 degrees rotation (y=0)
Maximize prob. $F^1(X^1)$
Predict 90 degrees rotation (y=1)
Maximize prob. $F^2(X^2)$
Predict 180 degrees rotation (y=2)
Maximize prob. $F^3(X^3)$
Predict 270 degrees rotation (y=3)

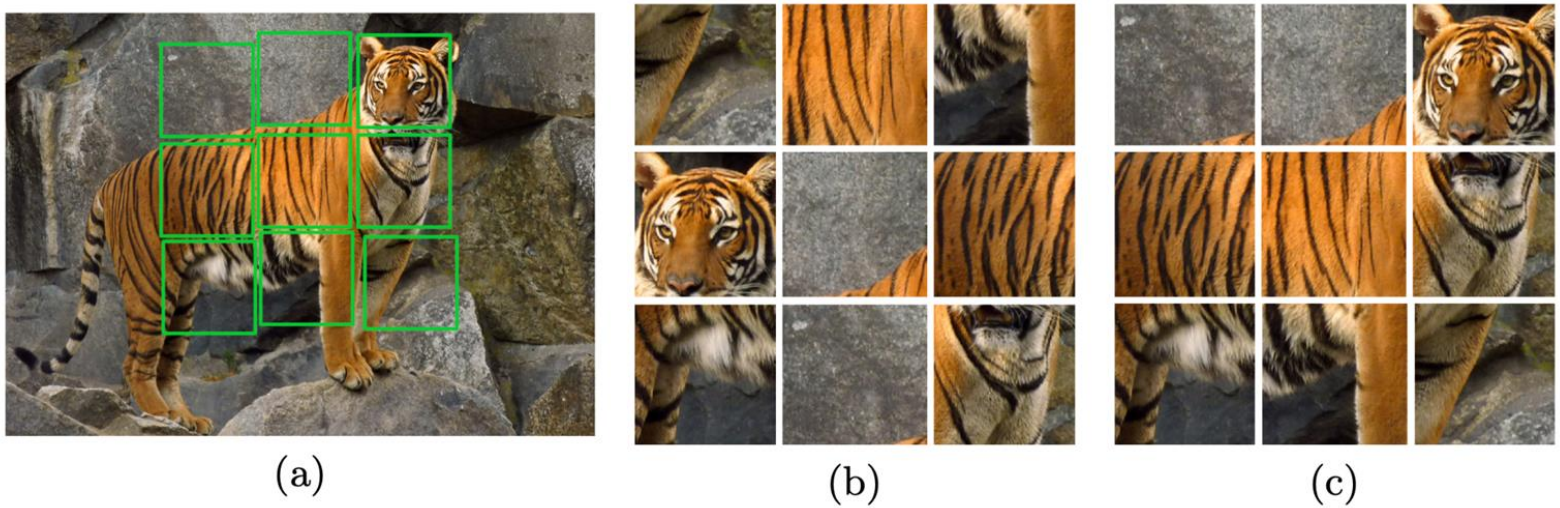
Rotnet



(a) **Attention maps of supervised model** (b) **Attention maps of our self-supervised model**

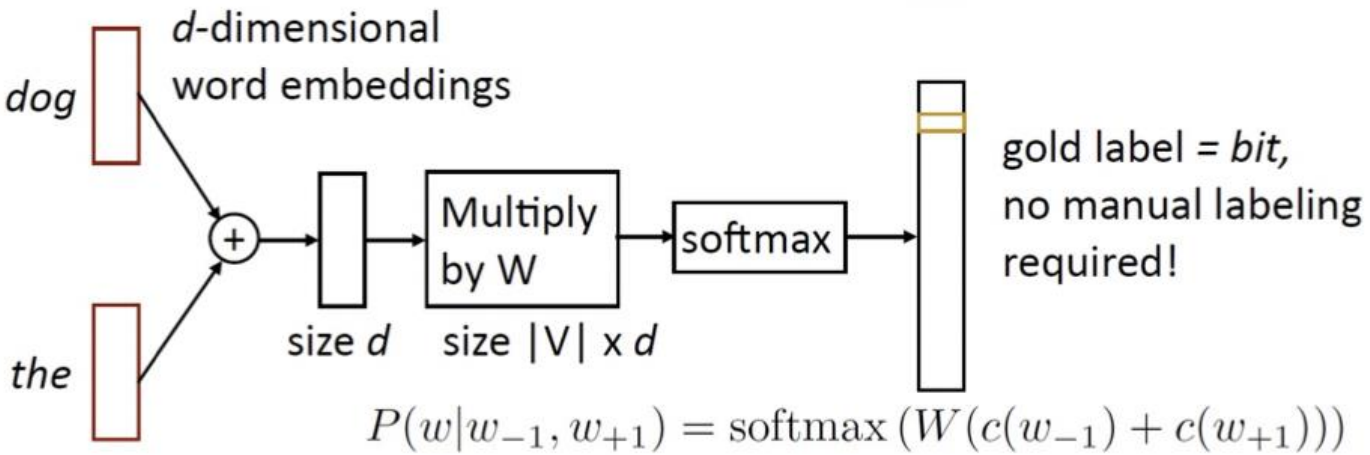
Jigsaw

- Предварительная задача - выбрать 9 патчей из квадратной сетки 3 на 3, перетасовать их, а затем предсказать их исходный порядок
  - Порядок перестановки выбирается из заранее определенного набора, а не из всех 9! перестановок
- Модель учится тому, что изображения состоят из «частей», которые связаны друг с другом!



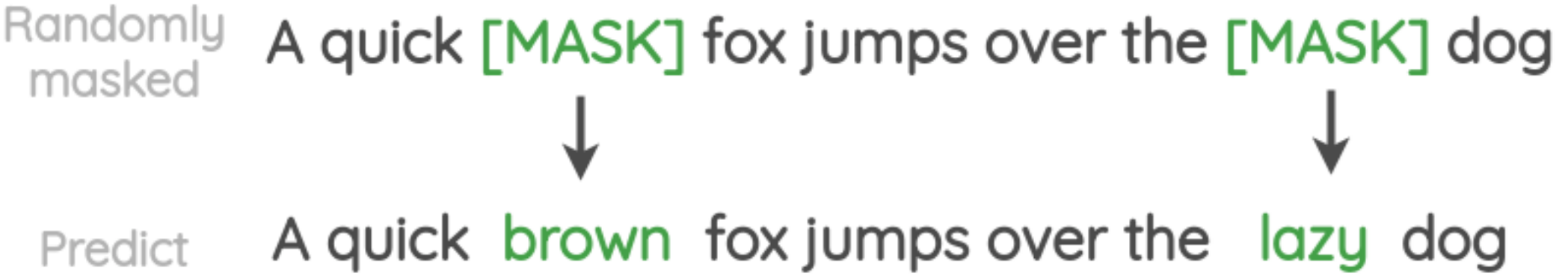
Word2Vec

- Способы генерации эмбеддингов, отображающих дискретные слова в векторы
- Несколько способов обучения эмбеддингов word2vec, два из которых включают:
  - Continuous Bag of Words - предсказание слова из окружающих (контекстных) слов.
    - Например: предсказать слово bit в фразе «The dog *bit* the man» на основе слов dog и the.
  - Skip-Gram - предсказание окружающего контекста по слову
    - Например: предсказать *dog* and *bit* в фразе «The *dog* bit *the* man» из bit



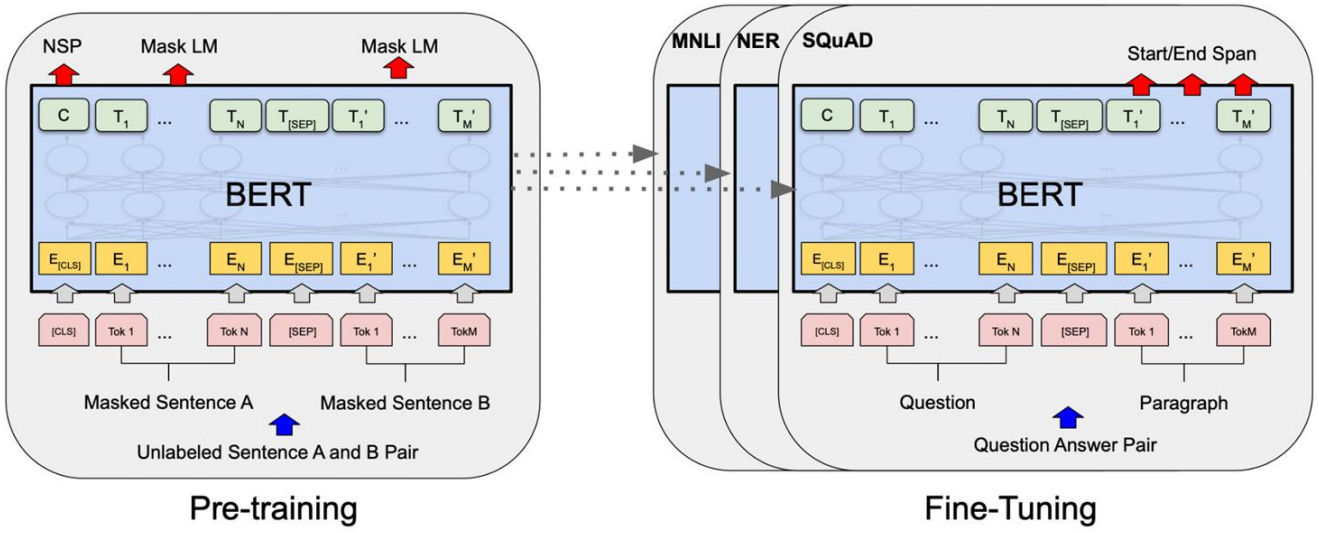
Маскированные языковые модели

- Обычные подходы word2vec предполагают предсказание слов только на основе непосредственно окружающего их контекста. А что, если мы будем учитывать *все предложение, в котором находится слово*?
- Более того, что если предсказать несколько слов в этом предложении одновременно?
- Мы говорим, что эти слова были *замаскированы*, и предварительная задача состоит в том, чтобы *разоблачить(unmask)* их, используя оставшиеся слова в качестве контекста.



BERT (2018)

- Входные токены (слова) маскируются случайным образом с вероятностью 15% - модель должна их предсказать
  - Получаем представления на уровне слов
- Пары предложений передаются вместе - модель должна предсказать порядок предложений
  - Получает представления на уровне предложений



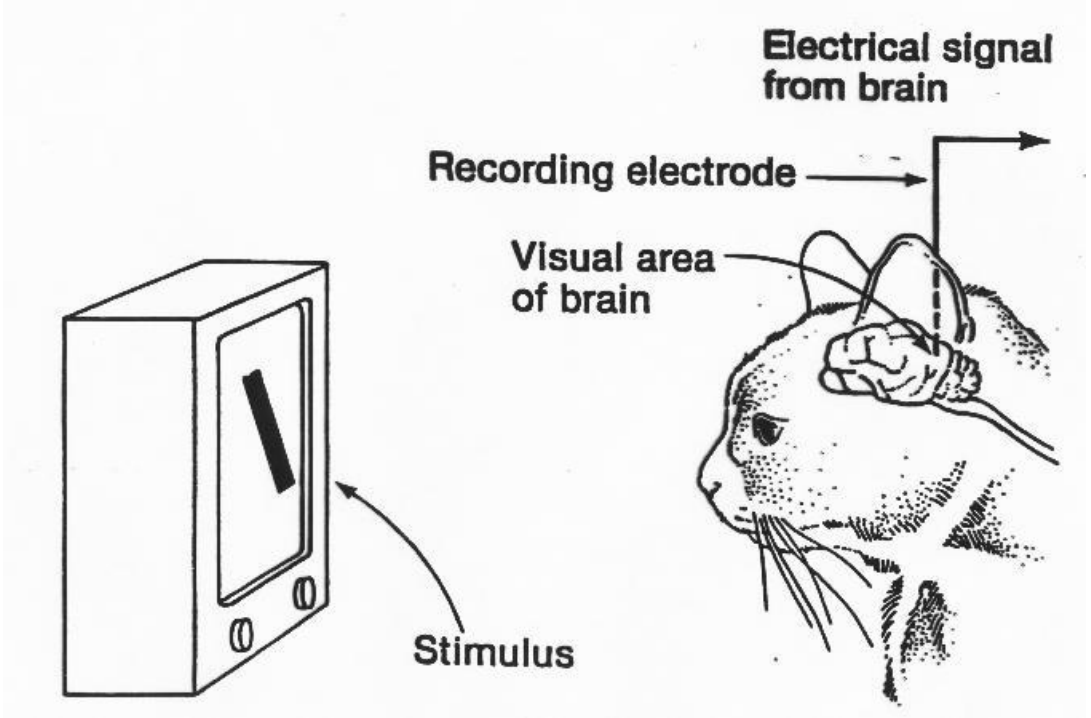
Содержание

1. Что такое компьютерное зрение (опять)
2. Представление изображений
3. Проблемы с MLP
4. Механика свёртки
5. Стэкинг свёрточных слоев
6. Построение CNN
7. Интуиция CNN
8. Обучение CNN

Краткий обзор современного CV

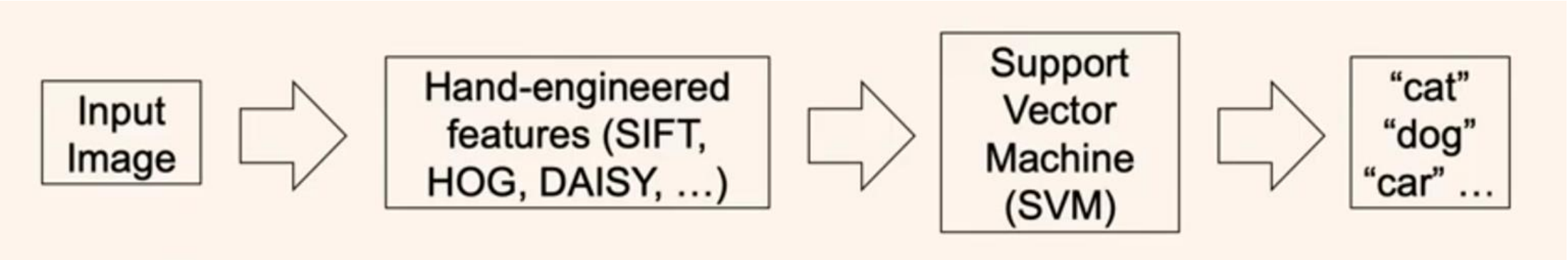
Краткая история КЗ

- 1959
  - Дэвид Хьюбел и Торстен Визель начали проводить эксперименты на зрительной коре кошек.
  - Обнаружили, что зрительная кора кошек обрабатывает изображения, сначала анализируя простые структуры, такие как края.



Эволюция CV

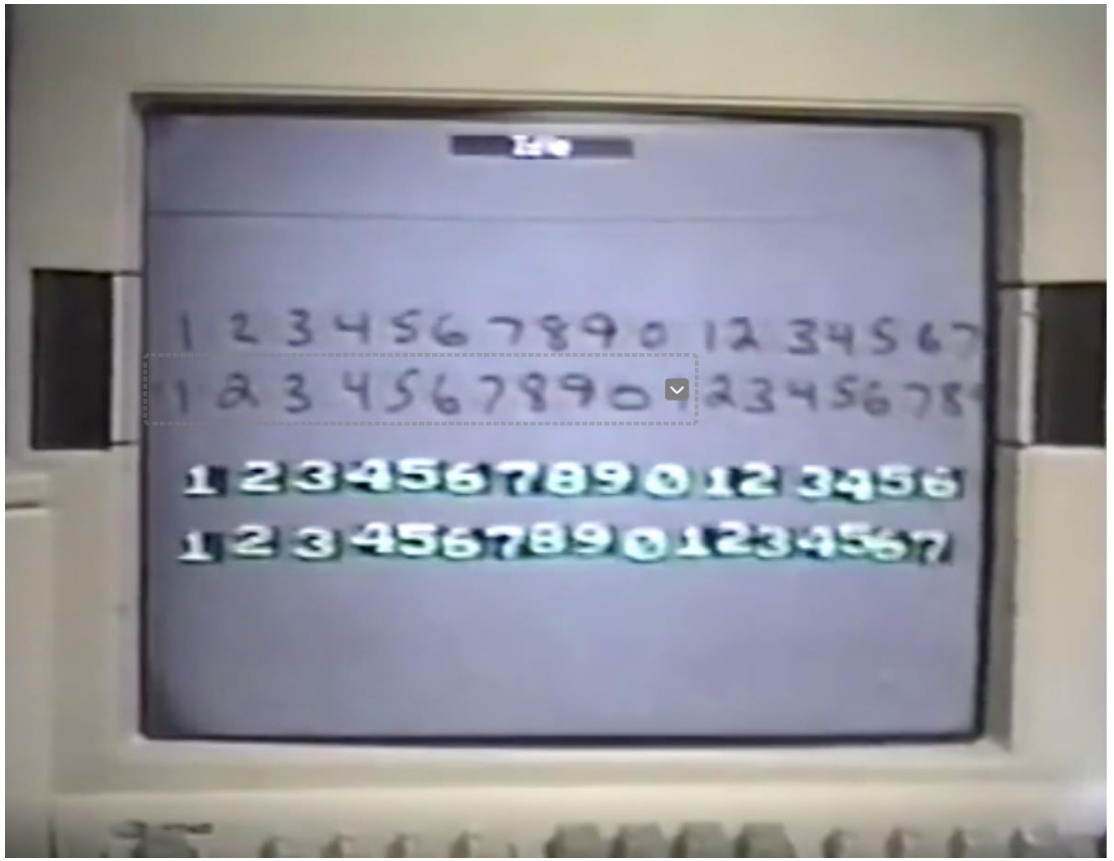
- Обнаружение объектов до 2012 года:



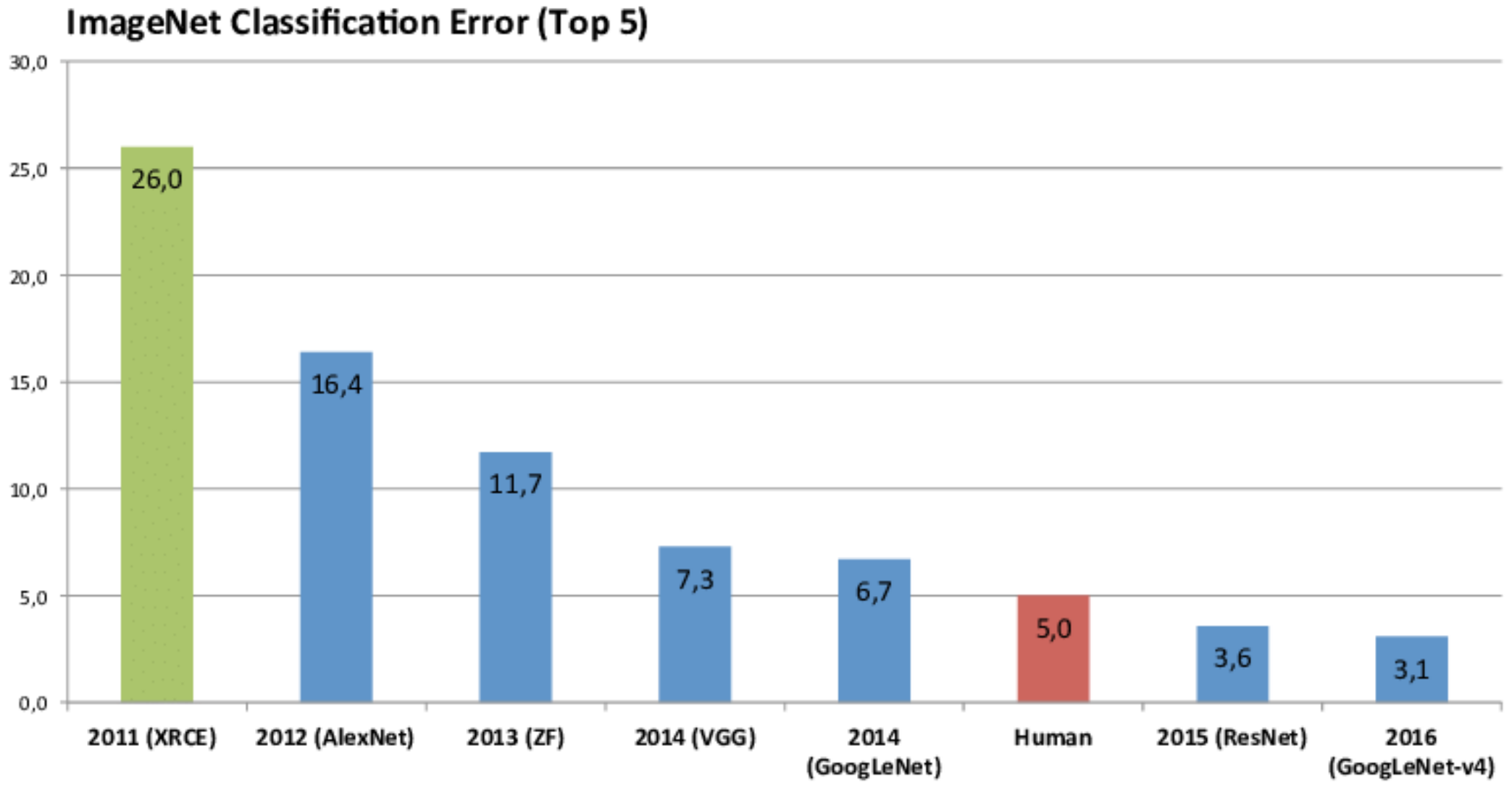
- Затем появилось глубокое обучение...
  - Больше никакого извлечения признаков вручную!
  - Используйте большую нейронную сеть для изучения важных характеристик.
  - Глубокое обучение открыло путь к значительному ускорению прогресса в области компьютерного зрения

Эволюция CV

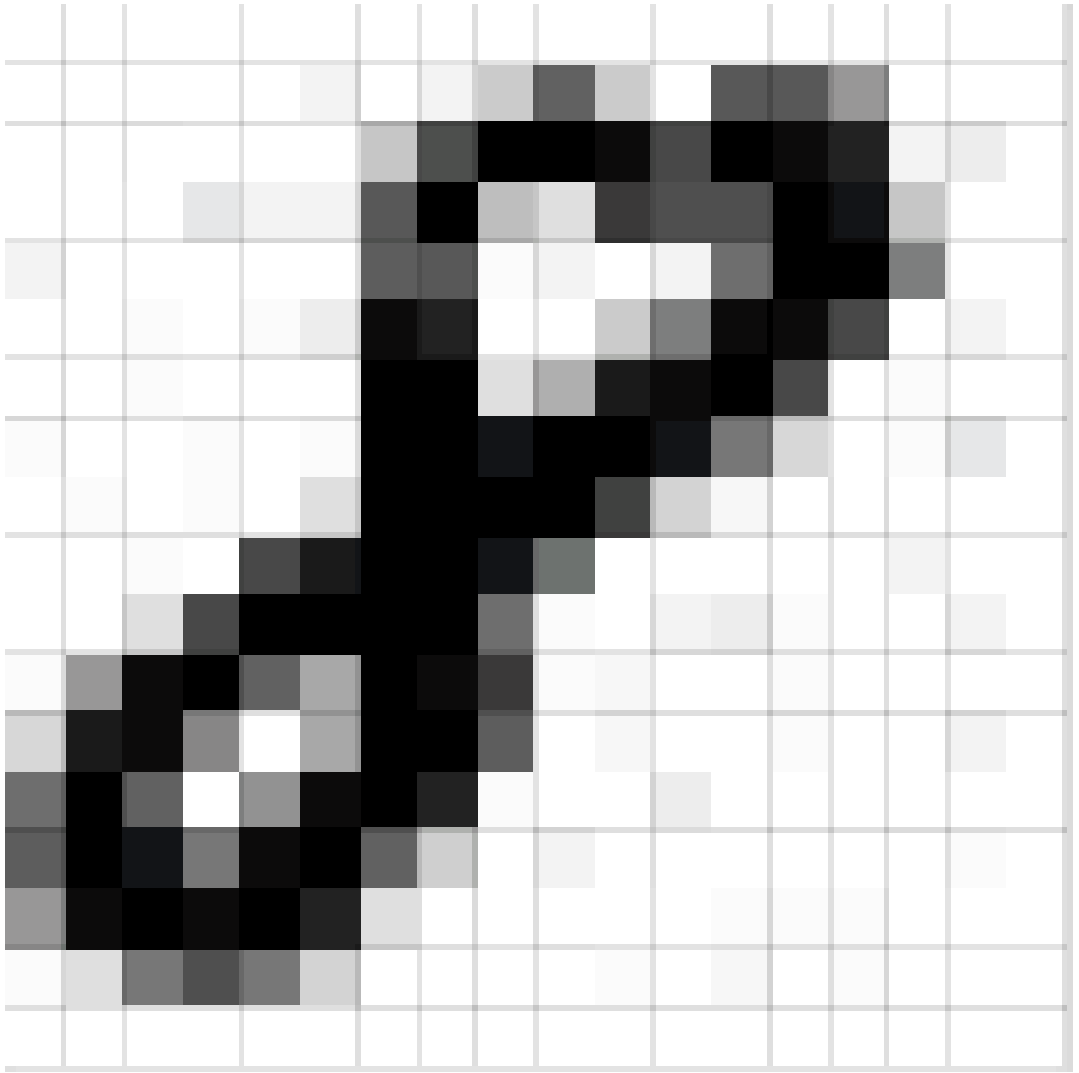
- Конволюционные нейронные сети (CNN)
  - Алгоритм глубокого обучения, используемый для анализа изображений
  - Изобретен Янном Лекуном (LeNet - 5) в 1990-х годах.
- Почему произошел внезапный взрыв после 2012 года?
  - AlexNet (коэффициент ошибок 16,4%!!)
  - Графические процессоры Nvidia
  - Более мощные компьютеры
  - Больше доступа к данным

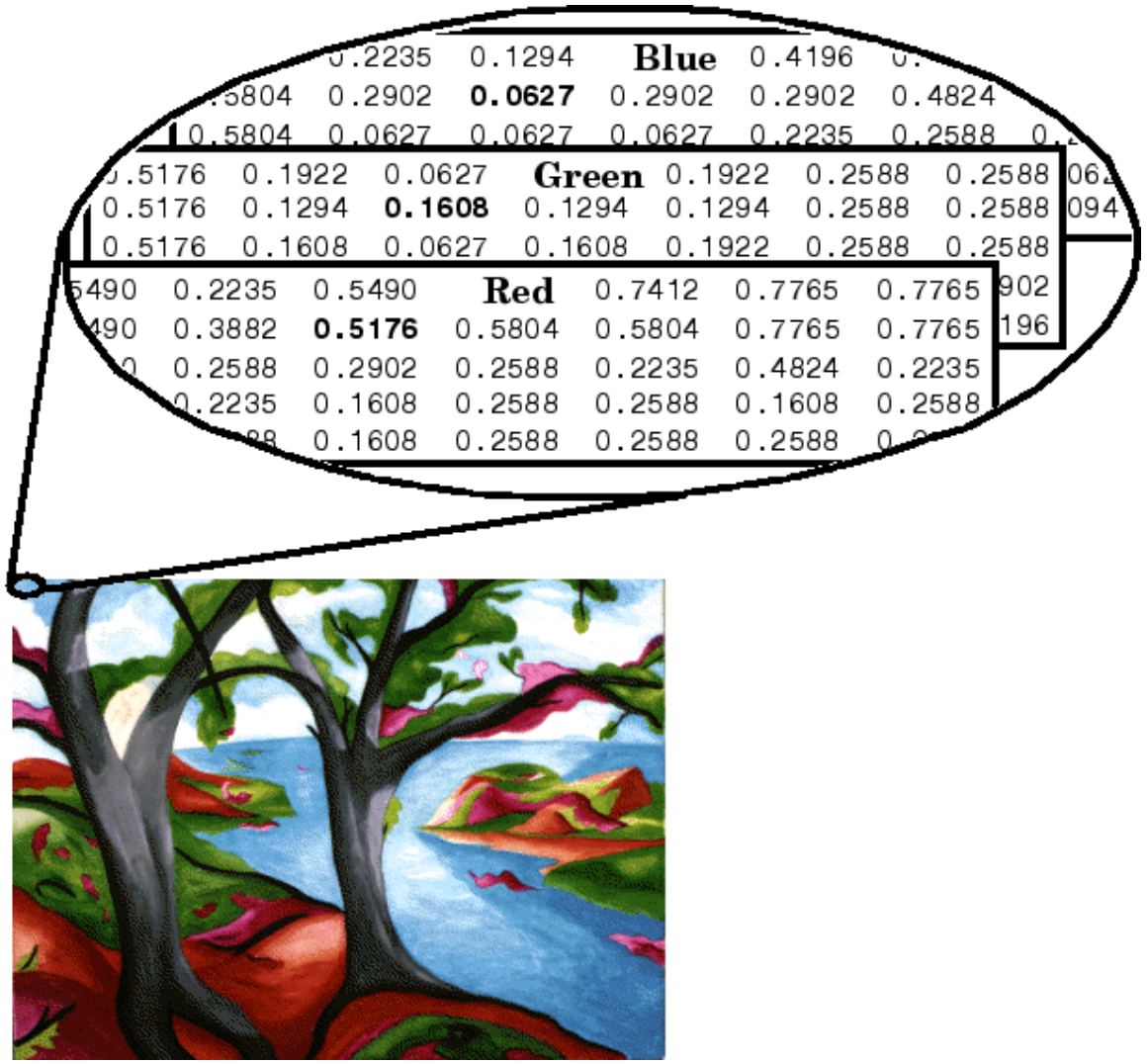


https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf


ImageNet Classification Error (Top 5)
30,0
25,0
20,0
15,0
10,0
5,0
0,0
26,0
16,4
11,7
7,3
6,7
5,0
3,6
3,1
2011 (XRCE)
2012 (AlexNet)
2013 (ZF)
2014 (VGG)
2014 (GoogLeNet)
Human
2015 (ResNet)
2016 (GoogLeNet-v4)

Изображения как данные



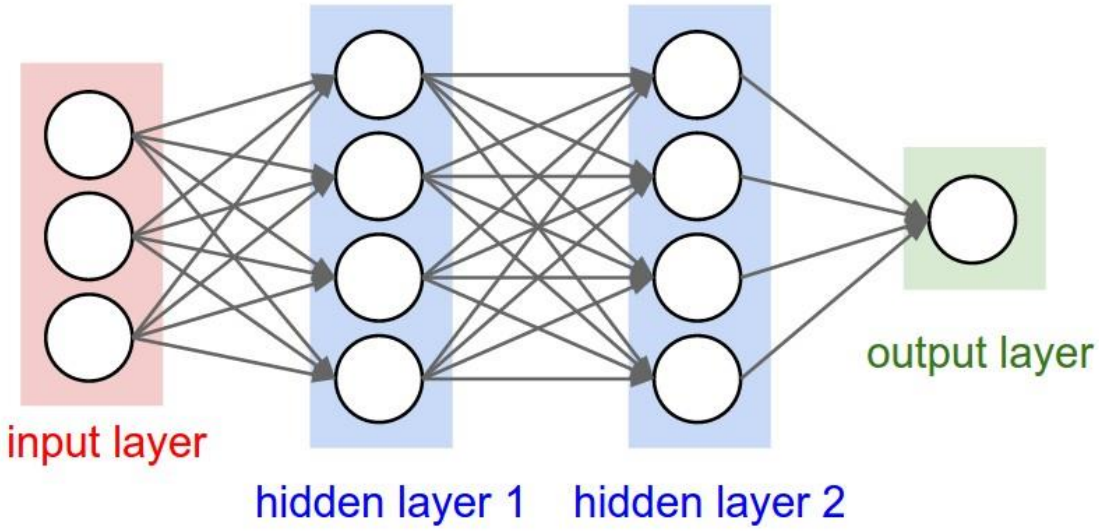

0.2235 0.1294 Blue 0.4196
.5804 0.2902 0.0627 0.2902 0.2902 0.4824
0.5804 0.0627 0.0627 0.0627 0.2235 0.2588
.5176 0.1922 0.0627 Green 0.1922 0.2588 0.2588
0.5176 0.1294 0.1608 0.1294 0.1294 0.2588 0.2588
0.5176 0.1608 0.0627 0.1608 0.1922 0.2588 0.2588
5490 0.2235 0.5490 Red 0.7412 0.7765 0.7765
490 0.3882 0.5176 0.5804 0.5804 0.7765 0.7765
0.2588 0.2902 0.2588 0.2235 0.4824 0.2235
0.2235 0.1608 0.2588 0.2588 0.1608 0.2588
0.1608 0.2588 0.2588 0.2588

Уходим от MLP

Почему стандартные плотные сети (MLP) не идеальны для классификации изображений?

Классификация с ИНС

- Рассмотрим изображение размером 200 x 200 x 3.
  - Это RGB-изображение с высотой и шириной 200 пикселей.
  - Мы можем представить его вектором 200x200x3 = 120 000 элементов.
- Сколько параметров нам нужно для MLP с одним полностью связанным скрытым слоем из 10 единиц?



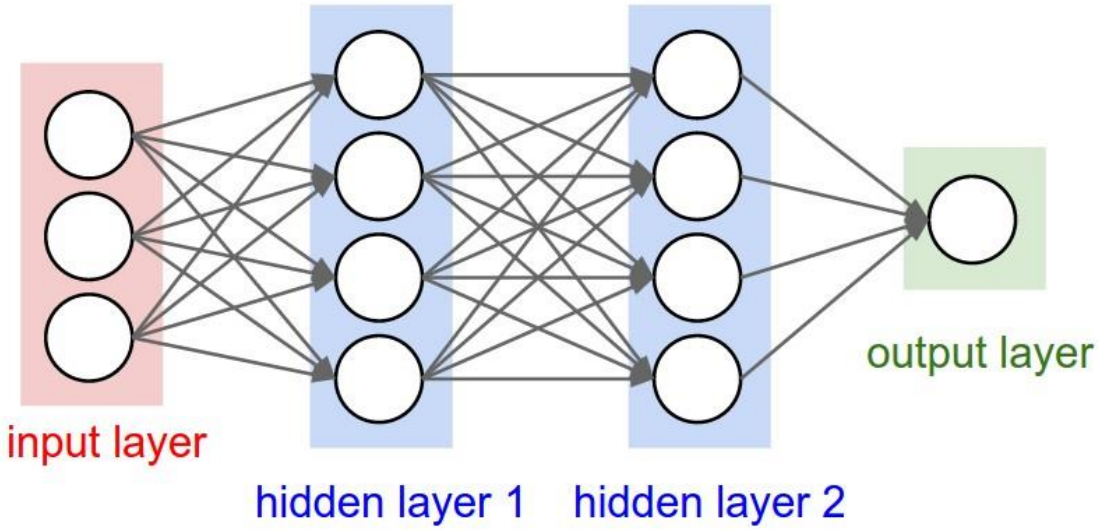
Полностью связанные слои: y = Wx + b

Классификация с ИНС

- Рассмотрим изображение размером 200 x 200 x 3.
  - Это RGB-изображение с высотой и шириной 200 пикселей.
  - Мы можем представить его вектором 200x200x3 = 120 000 элементов.
- Сколько параметров нам нужно для MLP с одним полностью связанным скрытым слоем из 10 единиц?

200 * 200 * 3 * 10 + 10 = 1,200,010

СЛИШКОМ МНОГО!!!



Полностью соединенные слои: y = Wx + b

Каждый пиксель - это индивидуальный входной «признак» сети. Почему это не имеет смысла?

Как интуитивно работает классификация изображений? Как убедить человека в том, что изображение содержит объект X

Признаки в изображении





Локальные окрестности изображения

- Основная идея заключается в том, чтобы оперировать локальными областями изображения, а не отдельными пикселями или далекими друг от друга пикселями.
- Плотный шаблон нейросетевой модели не особо подходит для распознавания таких участков!



Соседние пиксели связаны сильнее, чем удалённые.

Объекты сложены из малых частей.

Классическое CV тоже этим пользовалось!



Canny Edge Detector - очень известный экстрактор признаков, разработанный профессором Беркли Джоном Канни!

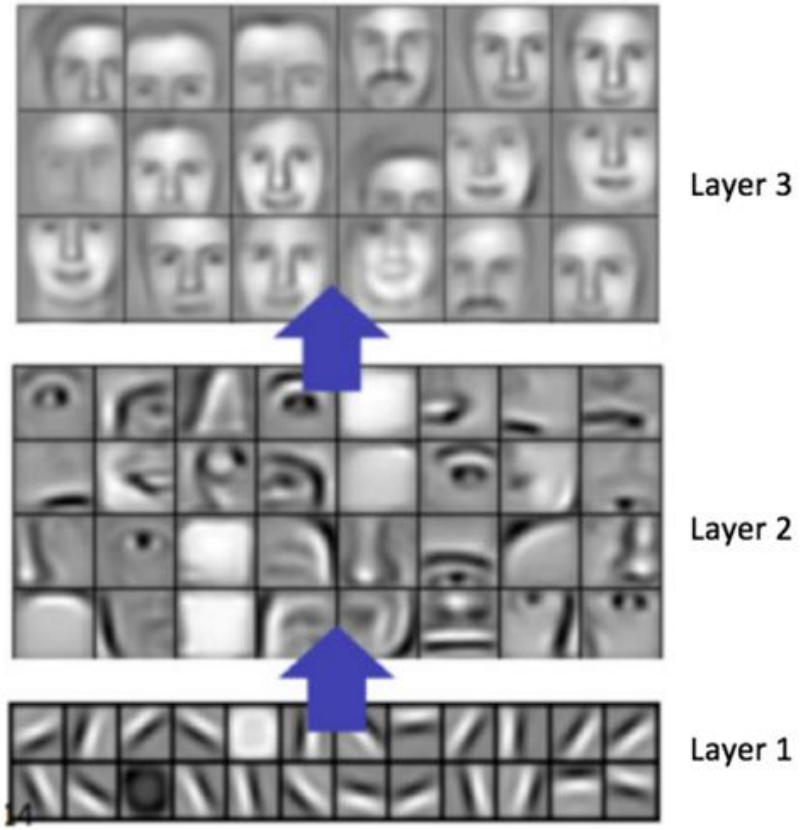
https://ieeexplore.ieee.org/document/4767851

Вывод: нельзя извлечь полезную информацию, рассматривая отдельные пиксели

Признаки

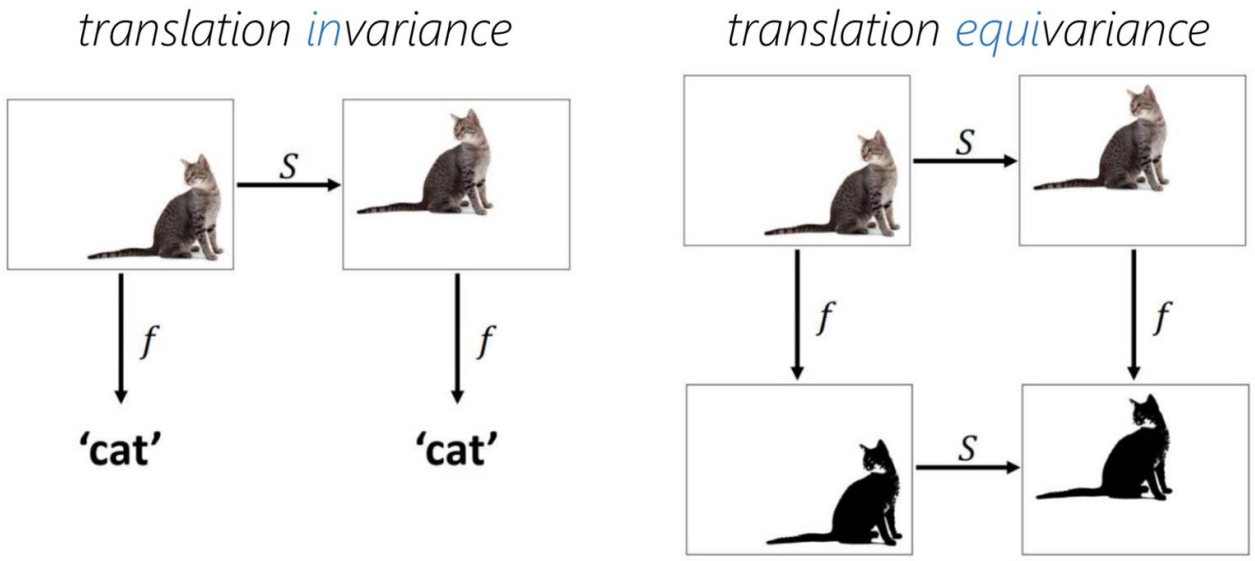
- Напомним, что глубокое обучение — это процесс извлечения ***иерархических*** представлений из входных данных. Как это выглядит в случае изображений?

1. Обучение распознаванию краев, текстур и цветов из необработанных пикселей в первом слое.
2. Использование краев для распознавания простых форм и узоров в промежуточных слоях.
3. Объединение форм и узоров для распознавания абстрактных характеристик более высокого уровня, таких как формы лица, в более высоких слоях.

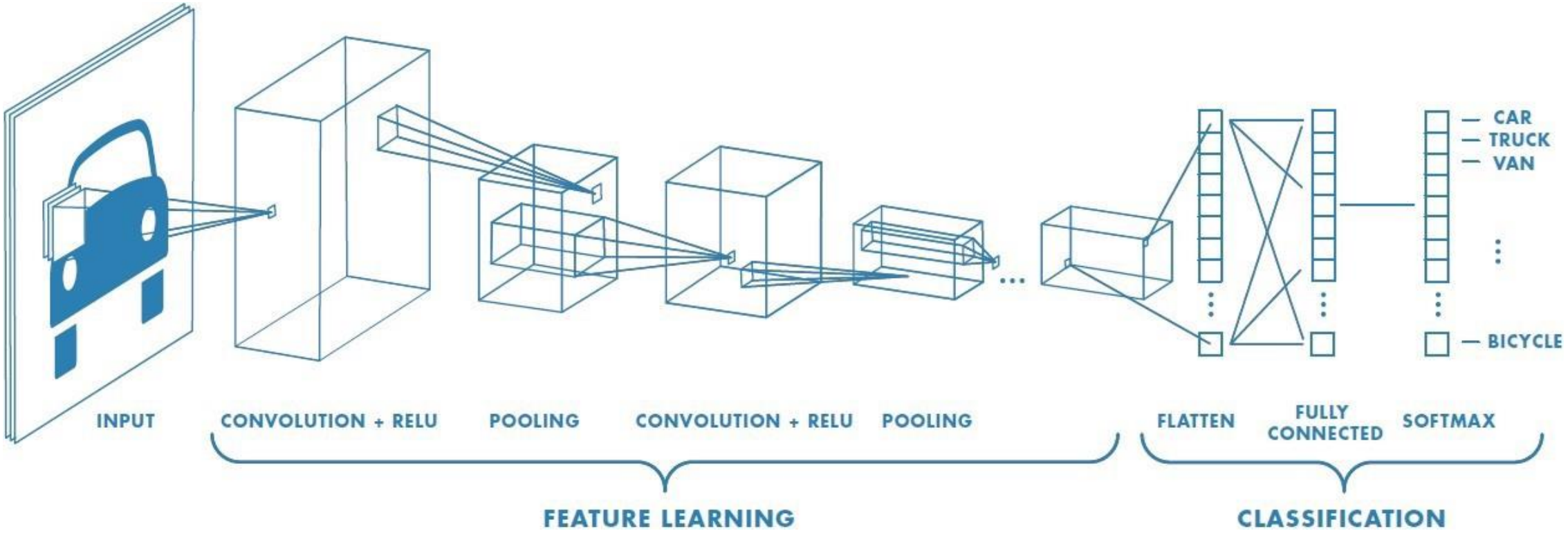


Другие особенности

- Эквивариантность по отношению к смещению: при переводе один и тот же набор пикселей должен иметь одинаковое представление (выход меняется сообразно смещению).
- Инвариантность по отношению к смещению: семантическое значение не изменяется в результате смещения (выход не меняется вне зависимости от смещения).



Решение: CNN



Сжимайте изображения и извлекайте только то, что имеет значение

Свёртки

Высокоуровневый концепт

*пока не беспокойтесь о реализации

Изначальная идея

- Вместо того чтобы обрабатывать все изображение сразу, обрабатывайте его фрагменты!
  - Позволяет сети обращать внимание на локальные области изображения
- Обрабатывайте каждый патч с помощью слоя, параметризованного одинаковыми весами и смещением - это также называется совместным использованием весов:
  - Гарантирует, что одни и те же участки будут выдавать одинаковые «представления».
  - Сохраняет трансляционную эквивариантность и инвариантность

Фильтры + свёртки

1	0	1
0	1	0
1	0	1

Фильтр весов

Терминология: также называется ядром

$1_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$	0	0
$0_{\times 0}$	$1_{\times 1}$	$1_{\times 0}$	1	0
$0_{\times 1}$	$0_{\times 0}$	$1_{\times 1}$	1	1
0	0	1	1	0
0	1	1	0	0

Image

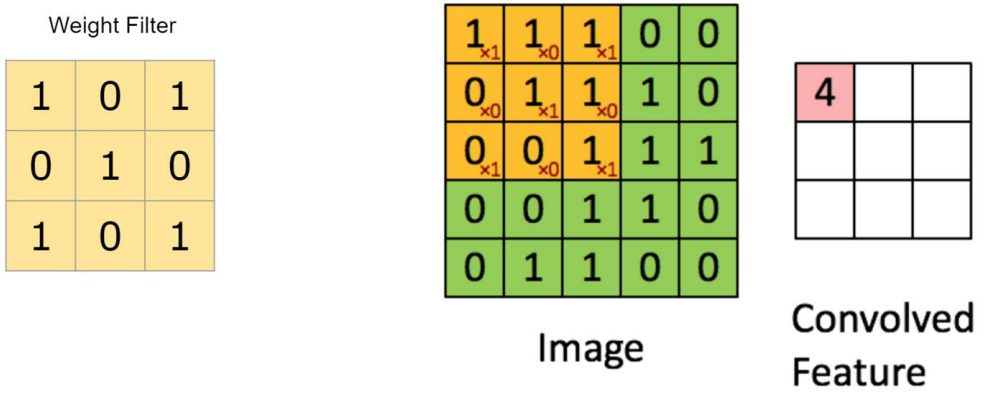
4		

Convolved Feature

Фильтры 2D

Как выполнить свертывание:

1. Сдвиньте фильтр по ширине и высоте на определенную величину (stride).
2. Вычислите точечные произведения между элементами фильтра и входных данных в любой позиции.



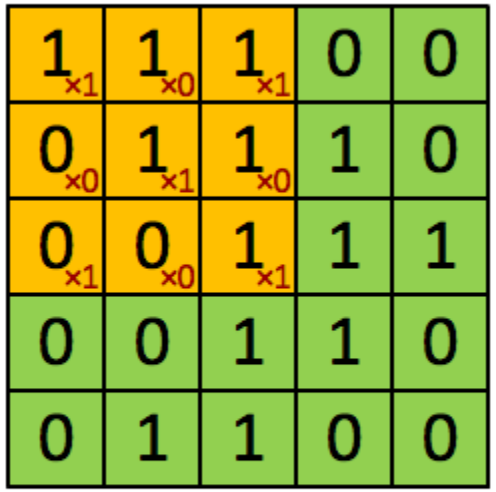
Примечание: После свертки применяется один член смещения на фильтр, только не в наших примерах.

Фильтры 2D

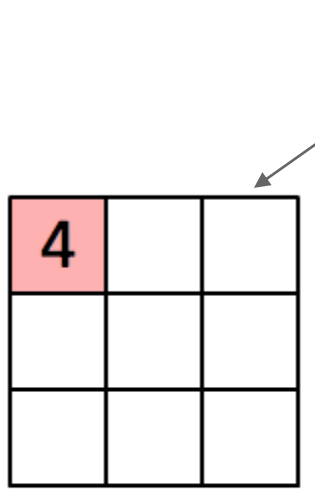
Терминология: вот эта выходная матрица называется карта активаций

1	0	1
0	1	0
1	0	1

Веса фильтра



Image



Convolved Feature

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

Что это делает? Есть идеи?

1	0	-1
1	0	-1
1	0	-1

=

?			

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0			

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$

0	?		

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$

0	30		

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0	30	?	

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$

0	30	30	

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0	30	30	?

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$

0	30	30	0

Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

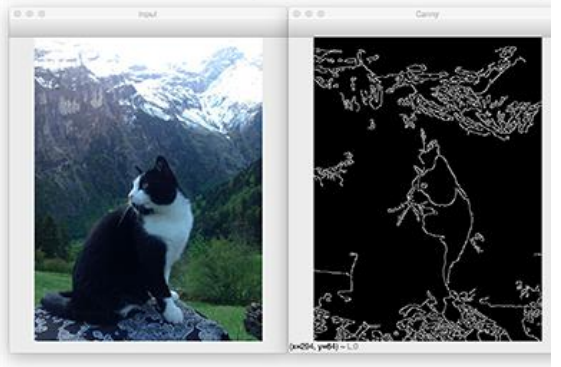
Пример

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

$=$



0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Свёртки: самопроверка

1. Как выглядит детектор горизонтального края?

2. Каков выход одного и того же входного сигнала с горизонтальным детектором краев?

3. Что это ^ говорит нам о выходе некоторого «детектора <вставьте форму здесь>», основанного на свертке?

Свёртки: самопроверка

1	1	1
0	0	0
-1	-1	-1

1. Как выглядит детектор горизонтального края?

То же, но повёрнуто на 90 градусов

1. Каков выход одного и того же входного сигнала с горизонтальным детектором краев?

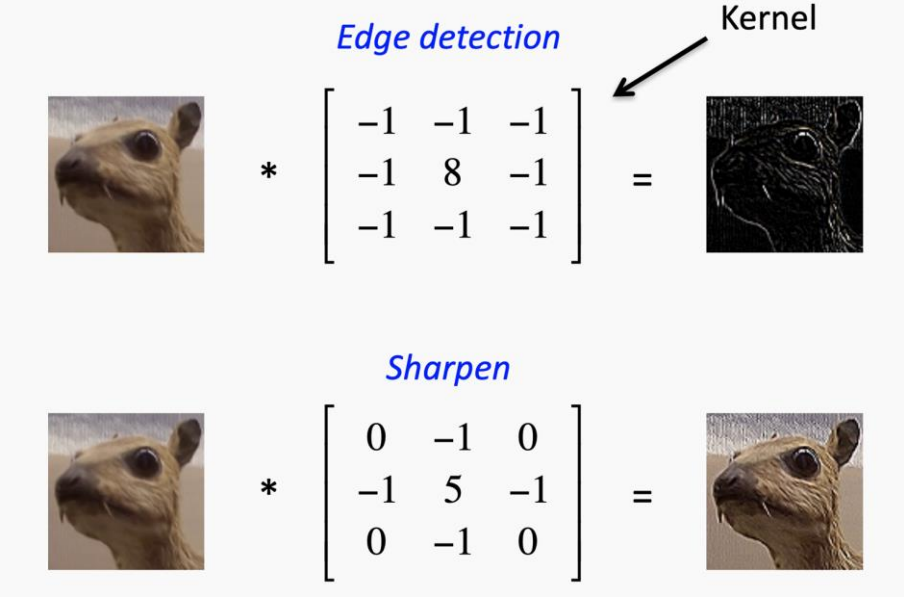
Сплошные нули

1. Что это ^ говорит нам о выходе некоторого «детектора <вставьте форму здесь>», основанного на свертке?

Результат свертки ядра в любом месте будет высоким, если присутствует признак, который ядро должно было обнаружить (или что-то подобное), и низким, если его нет. В данном случае горизонтальных линий не было, поэтому наше ядро горизонтальных линий везде выдает ноль

Цель свёрток

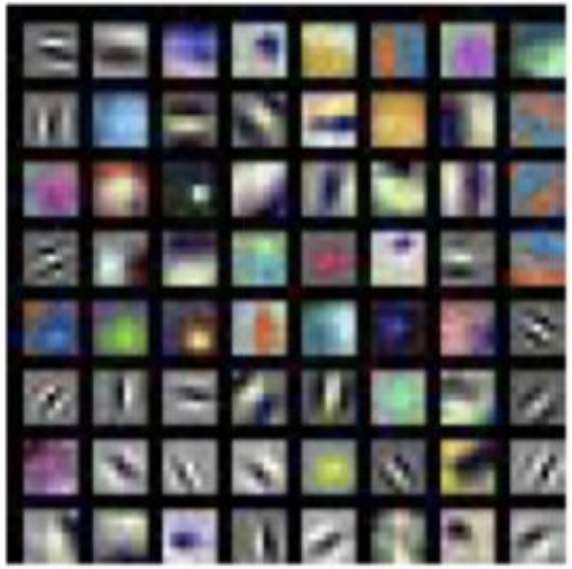
- Различные фильтры могут быть использованы для извлечения различных характеристик (признаков) изображения, таких как определение краев и размытие.





И где здесь глубокое обучение?

Как и в случае с плотными полносвязными слоями, мы можем просто выучить эти фильтры!



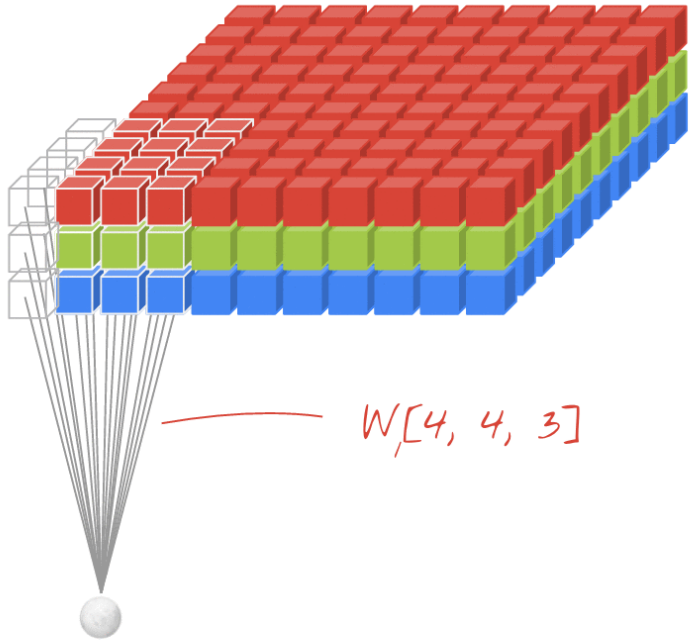
ResNet-18:
64 x 3 x 7 x 7



ResNet-101:
64 x 3 x 7 x 7

Фильтры 3D

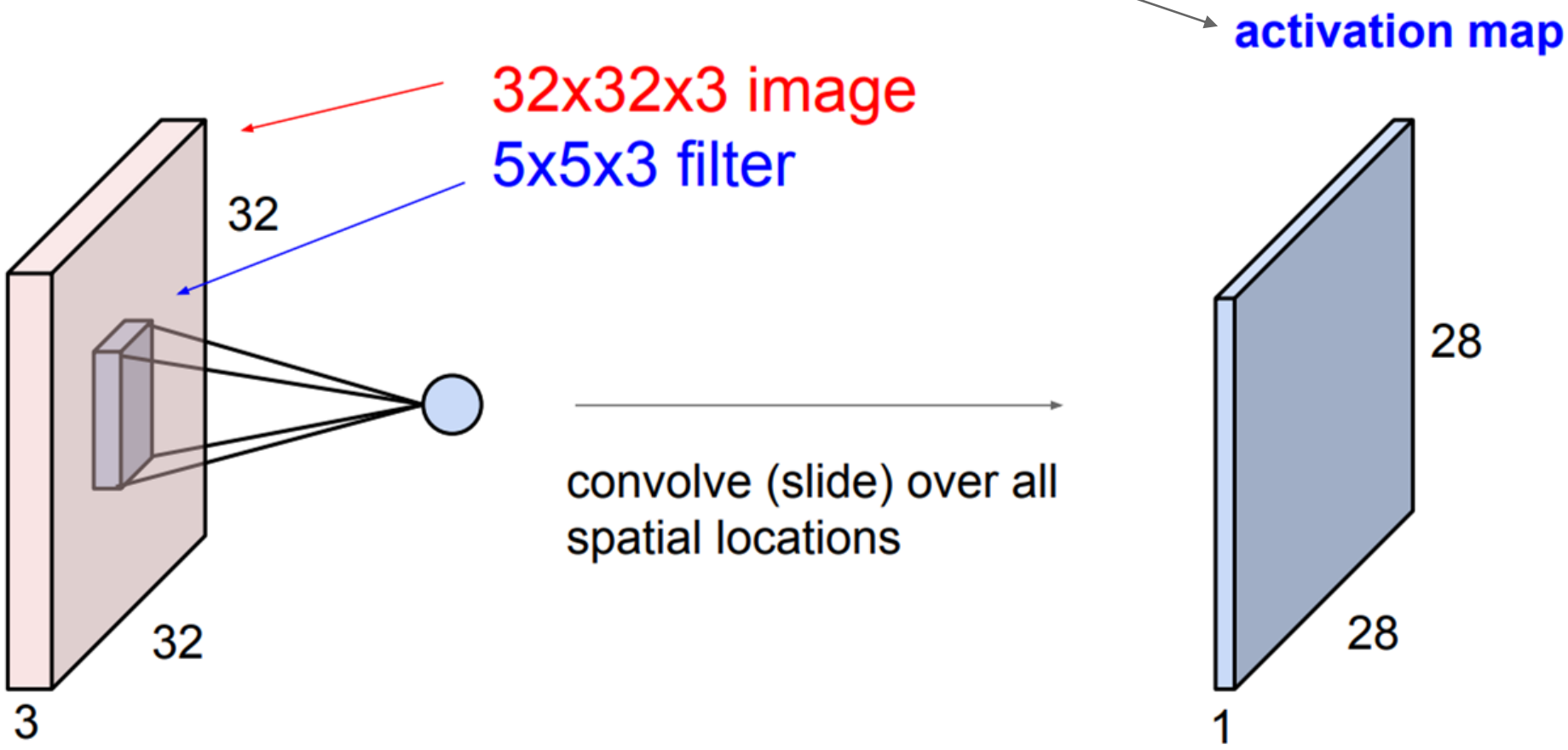
- Шаги:
  - Вычислите точечное произведение для каждого канала **(так же, как и в 2D)**
  - Сумма по каждому каналу
- **Примечание:** глубина фильтра всегда равна глубине входного изображения



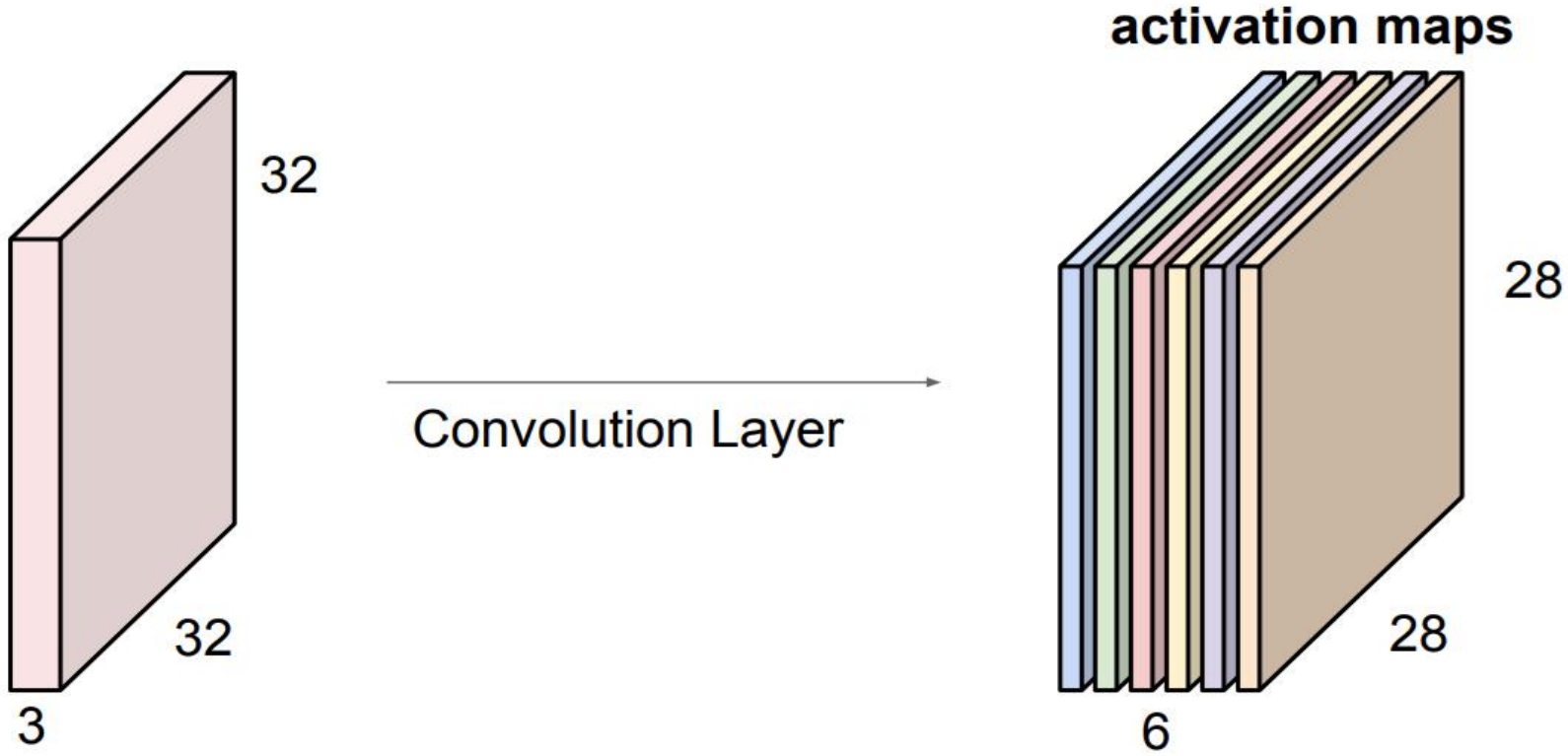
⚠ W_1 и W_2 это **разные** 4 x 4 x 3 **фильтры**

Свёрточные слои

Также называются «карты признаков»



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

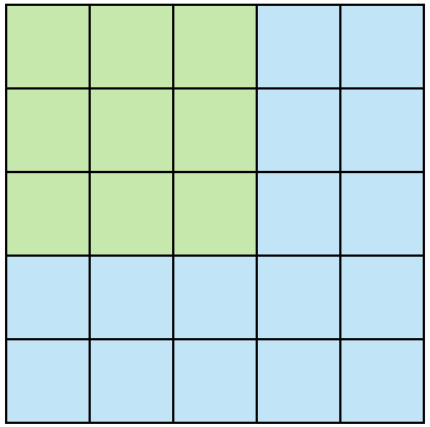


We stack these up to get a "new image" of size 28x28x6!

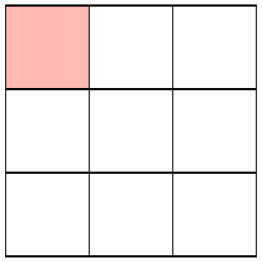
Шаг (stride)

Количество пикселей, на которое нужно сдвинуть фильтр (как по горизонтали, так и по вертикали):

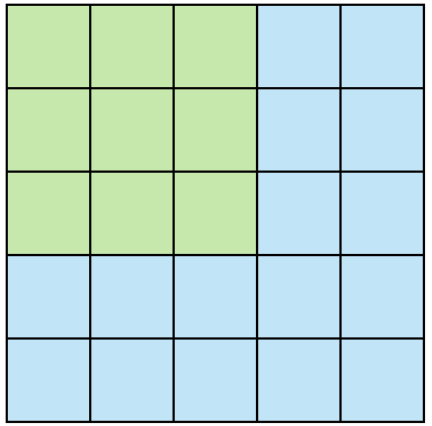
1. При шаге 1 фильтр будет сдвигаться на каждый пиксель
2. При значении stride 2 фильтр будет сдвигаться на каждые 2 пикселя



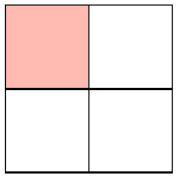
Stride 1



Feature Map



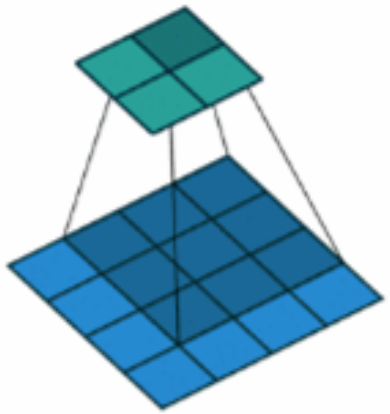
Stride 2

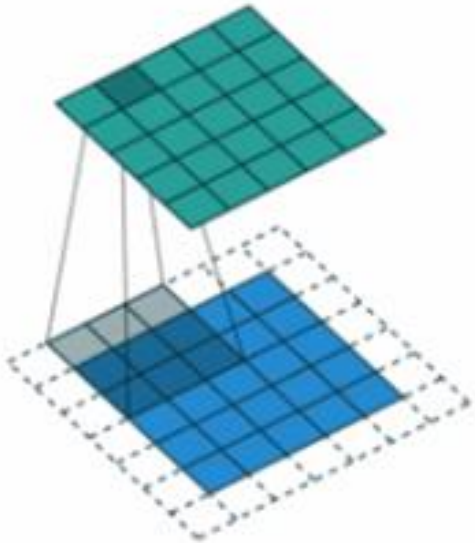


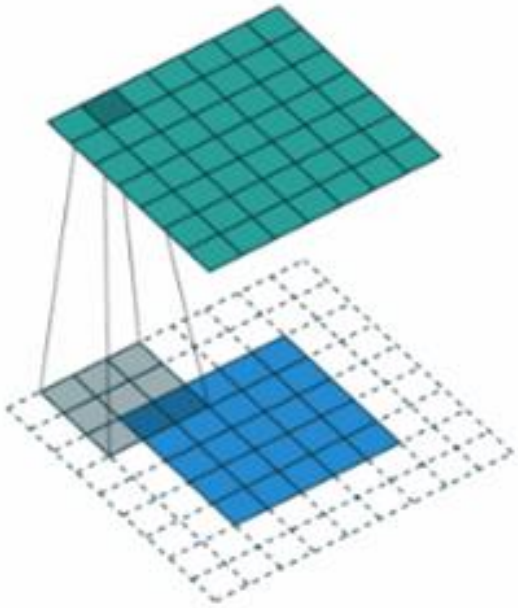
Feature Map

Отступ (padding)

В результате преобразования изображения с помощью фильтра получается блок с меньшей высотой и шириной - а что, если нам нужны прежние высота и ширина?

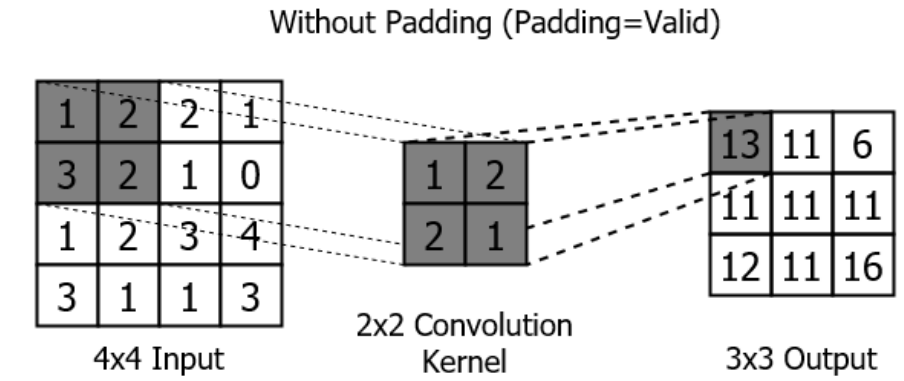


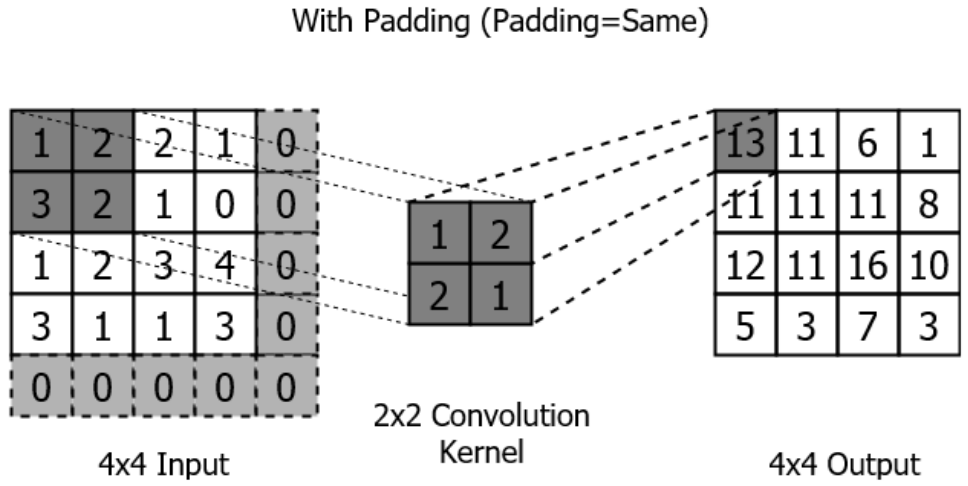




Same vs Valid отступ

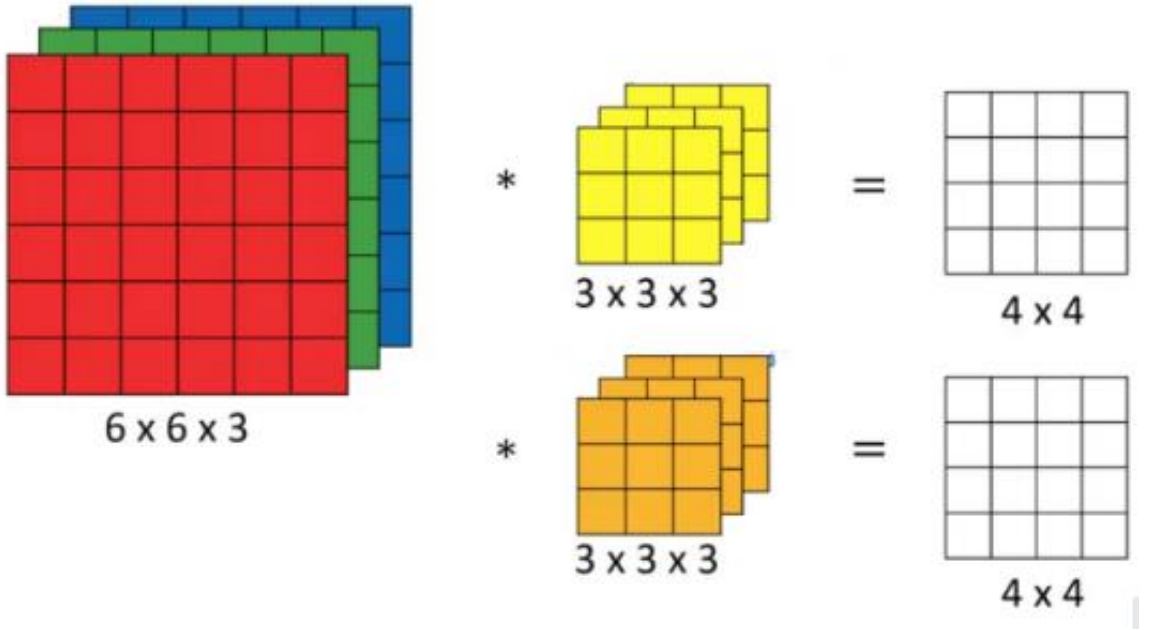
- Same вставка: вставка 0 (или какого-то другого одного значения) для сохранения пространственных размеров вывода.
- Valid вставка: без отступа





Выходные размеры

- ceil[(W - F + 2P) / S] + 1
- W: размерность входного сигнала
- F: размер ядра / фильтра
- P: размер прокладки
- S: полосы
- ceil - это функция округления вверх
  - ceil[3.5] = ceil[4] = 4
- Для рисунка справа:
- Предположим, что отступа нет и страйд равен 1
- W' = ceil[(6 - 3 + 2 * 0) / 1] + 1 = 4
- H' = ceil[(6 - 3 + 2 * 0) / 1] + 1 = 4

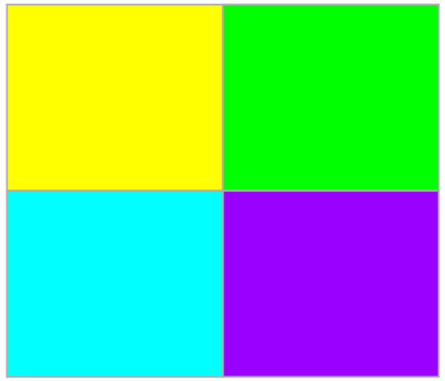


Обратное распространение ошибки в свёртках

- Как обычный алгоритм backprop (см. предыдущие лекции)
- Производная ошибки по конкретному весу = сумма производных по каждому выходу

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}

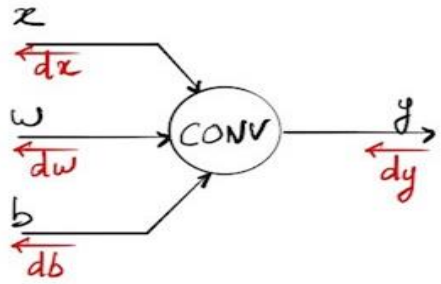
Входные значения



Производные на карте признаков

∂h_{11}	∂h_{12}
∂h_{21}	∂h_{22}

Производные выхода



$$\partial W_{11} = X_{11}\partial h_{11} + X_{12}\partial h_{12} + X_{21}\partial h_{21} + X_{22}\partial h_{22}$$

$$\partial W_{12} = X_{12}\partial h_{11} + X_{13}\partial h_{12} + X_{22}\partial h_{21} + X_{23}\partial h_{22}$$

$$\partial W_{21} = X_{21}\partial h_{11} + X_{22}\partial h_{12} + X_{31}\partial h_{21} + X_{32}\partial h_{22}$$

$$\partial W_{22} = X_{22}\partial h_{11} + X_{23}\partial h_{12} + X_{32}\partial h_{21} + X_{33}\partial h_{22}$$

Определение конволюционного слоя в PyTorch

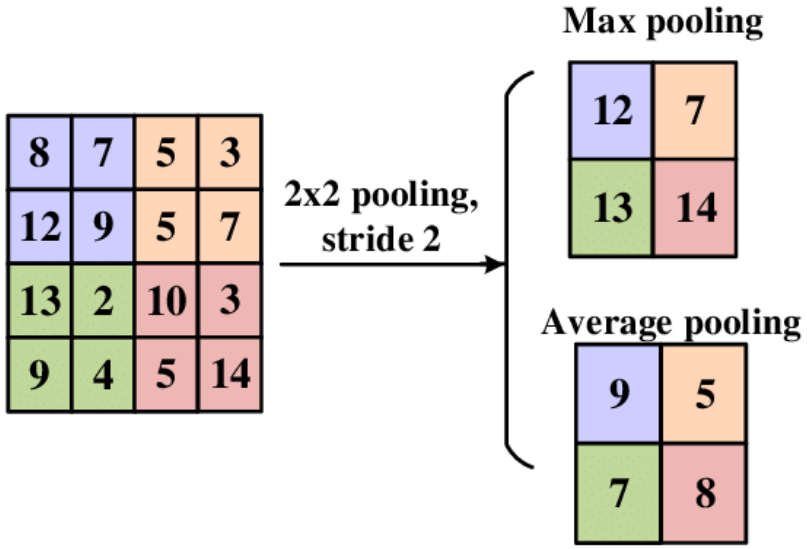
```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
      dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) [SOURCE]
```

https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

Другие операции

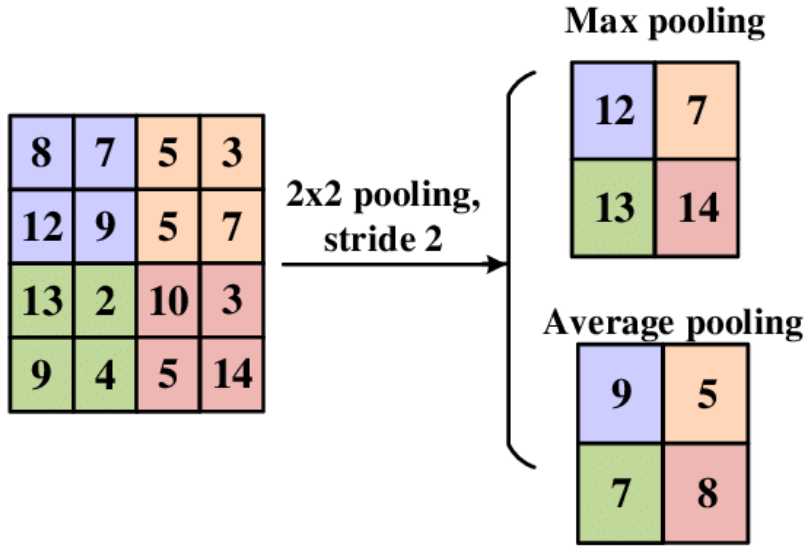
Pooling Layers

- Уменьшает размер выходного сигнала
- Применяется к каждому каналу независимо
- Соседние признаки могут быть похожими
- Не удаляет слишком много информации
- Максимальное объединение берет максимум
- Среднее объединение берет среднее значение



Pooling Layers

- При максимальном объединении каковы частные производные правого верхнего выхода по отношению к подсеточке входов 2x2 в правом верхнем углу?

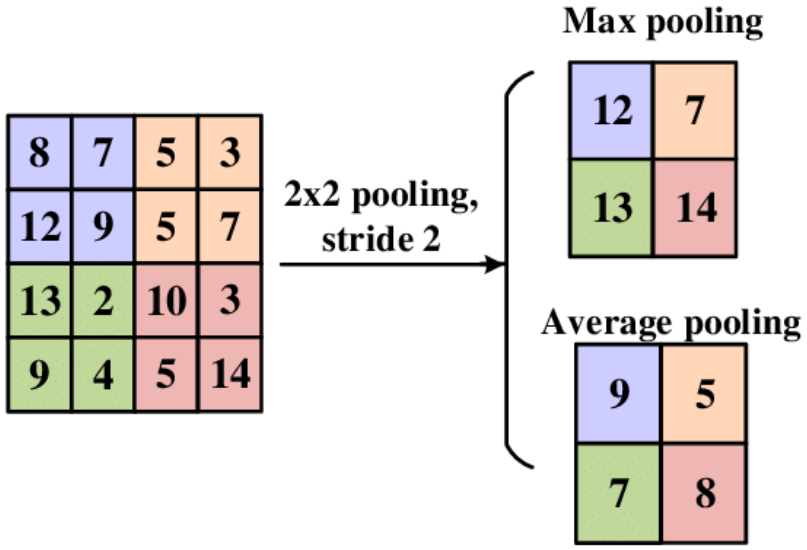


Pooling Layers

- При MaxPool, каковы частные производные правого верхнего выхода по отношению к подмножеству входов 2x2 в правом верхнем углу?

0	0
0	1

- В следующий слой поступает информация только от этой семёрки в правом верхнем сегменте, поэтому только она будет получать градиентный поток во время обратного распространения ошибки.



Резюме

Резюме

Интуиция

- На изображении локальные области содержат больше информации, чем отдельные пиксели.
- Конволюция - это операция, которая сама по себе может фильтровать различные вещи, такие как края, и вы можете использовать несколько фильтров для создания карт признаков.
  - Меньшее количество параметров, чем у линейного слоя, позволяет эффективно извлекать признаки
  - Гиперпараметры включают stride, padding, размер ядра
- Операция свертки представляет собой «скользящее окно» и является пространственно инвариантной.

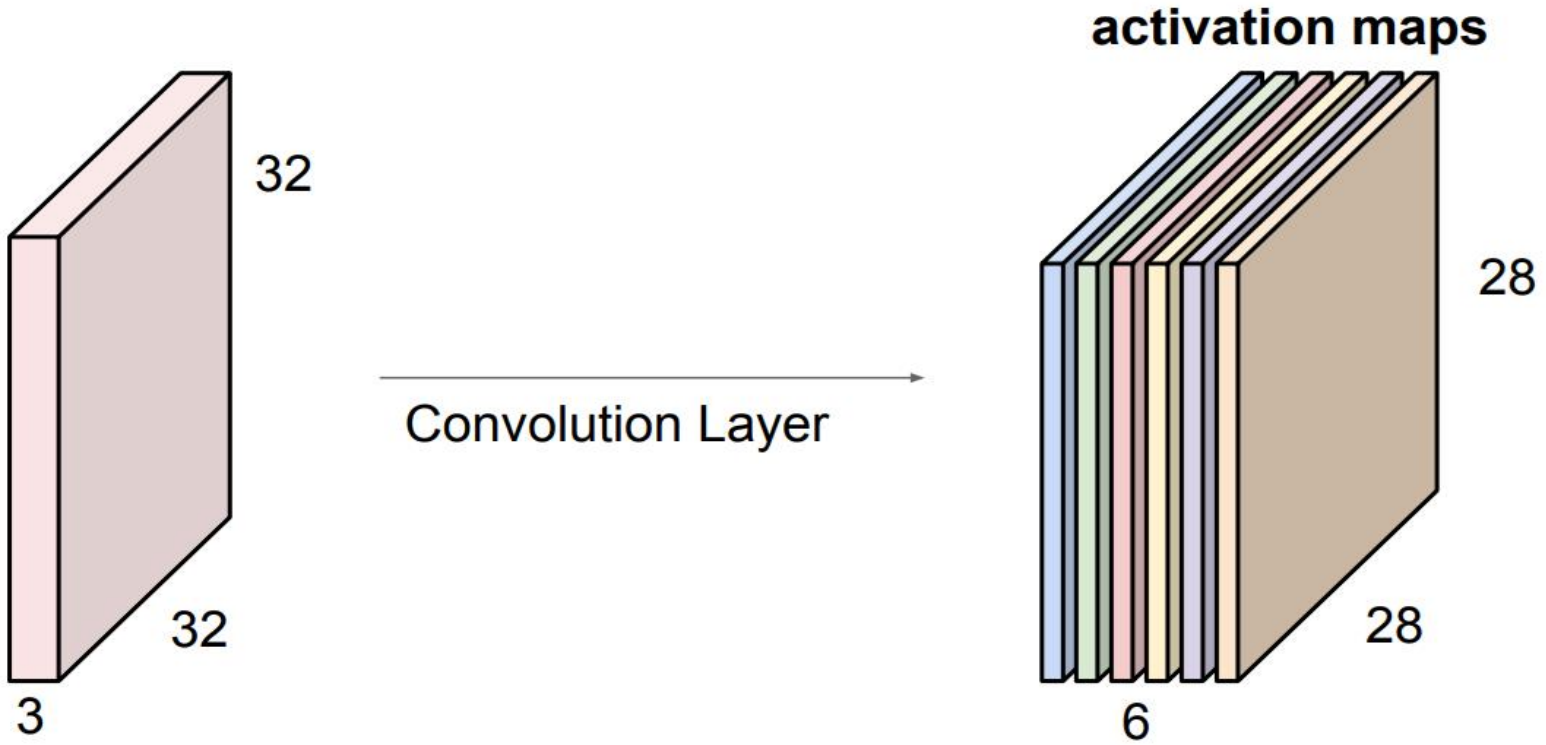
Инструменты для вашего инструментария:

- Слой Conv2D
- Pooling layers
- Транспонированная свёртка

Стэкинг свёрток

Pooling Layers

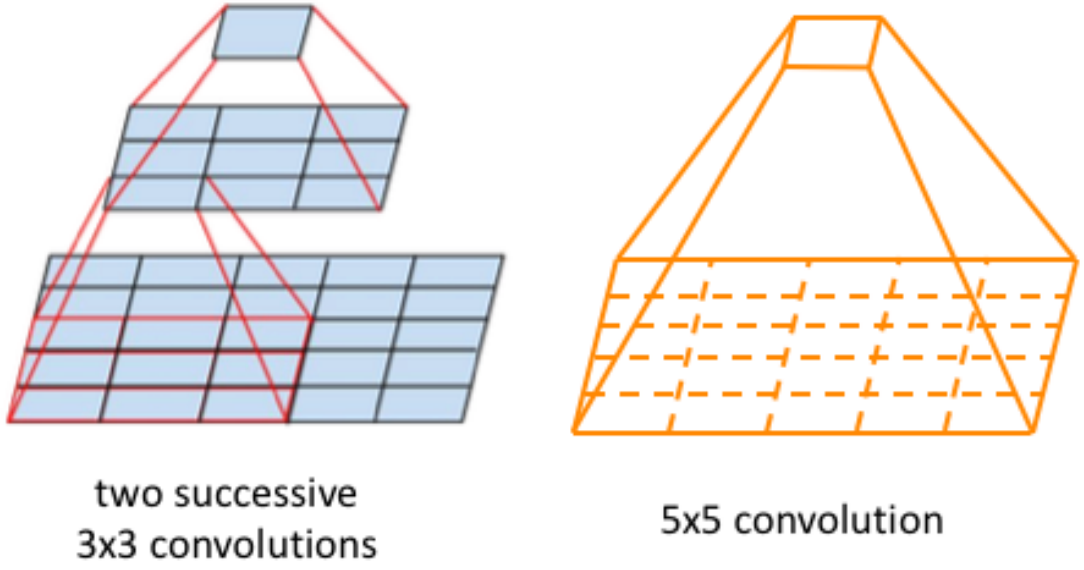
Допустим, если у нас 6 фильтров 5*5, мы получим 6 отдельных карт активаций:



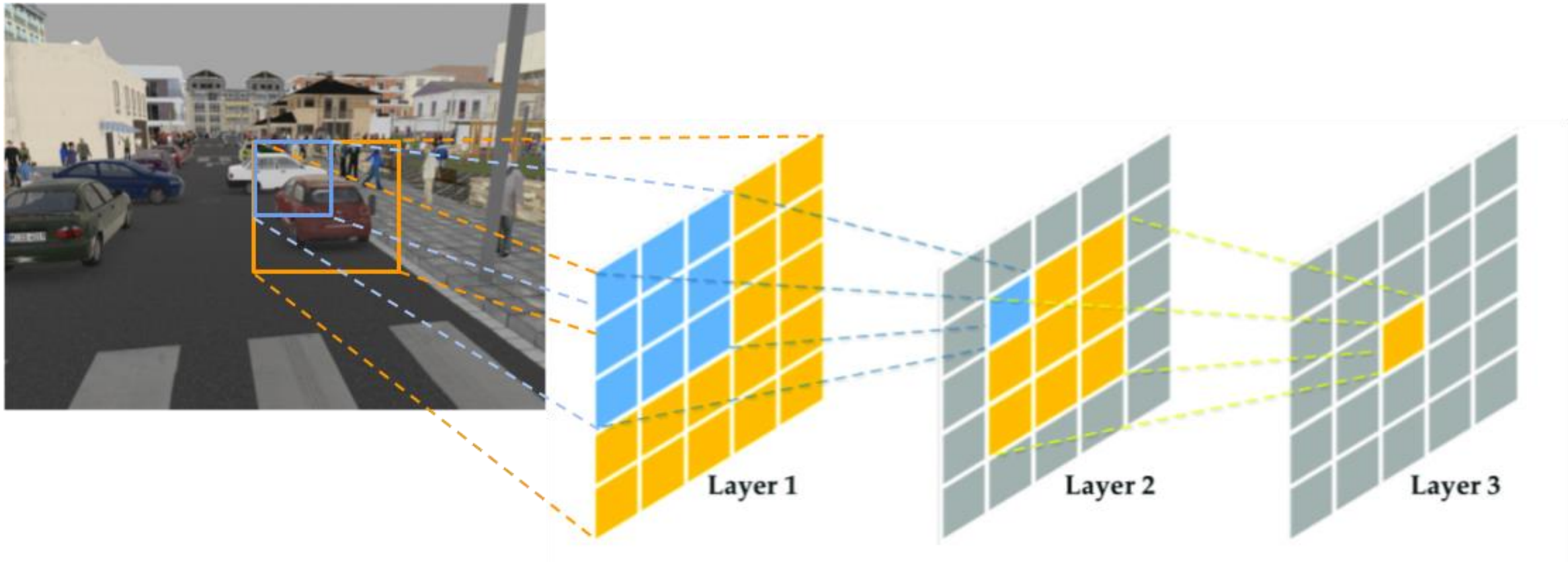
Мы стэкаем их, чтобы получить «изображение» размером 28*28*6

Окно восприятия/пятно контакта

Регион входных данных, влияющий на каждый элемент конкретной карты активаций.



Окно восприятия/пятно контакта



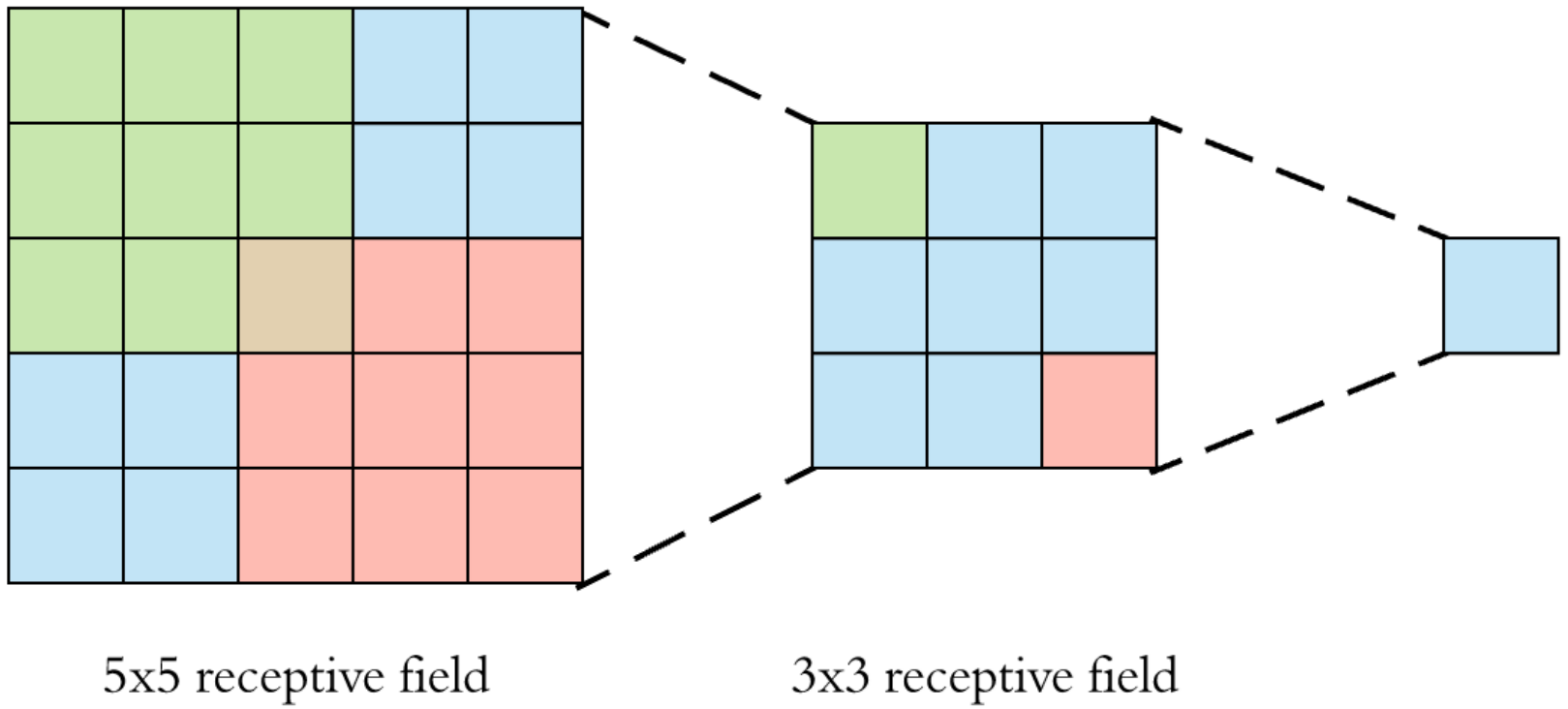
Окно восприятия/пятно контакта

2 свёртки 3*3 – то же, что 1 свёртка 5*5?

Нет! В 2 свёртках

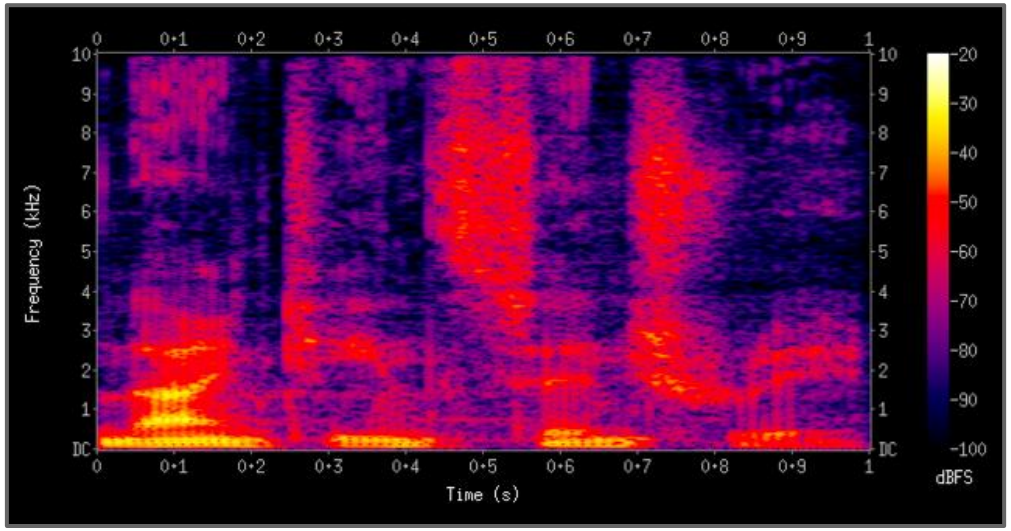
3*3 меньше параметров

и больше нелинейности.



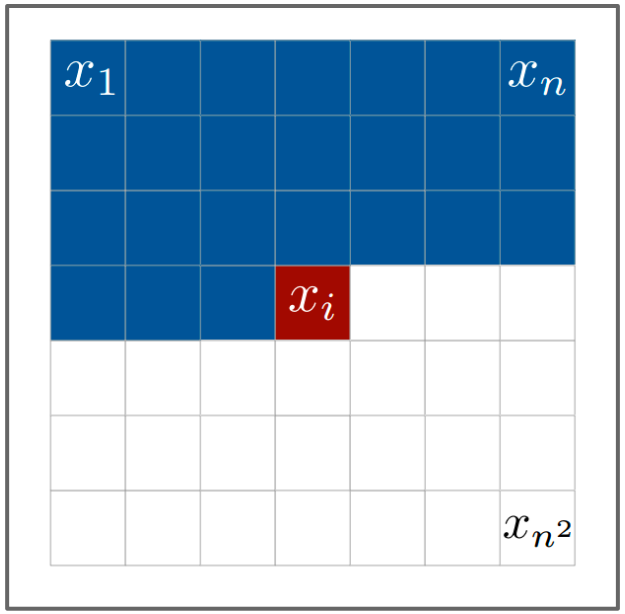
Моделирование последовательностей - круто 

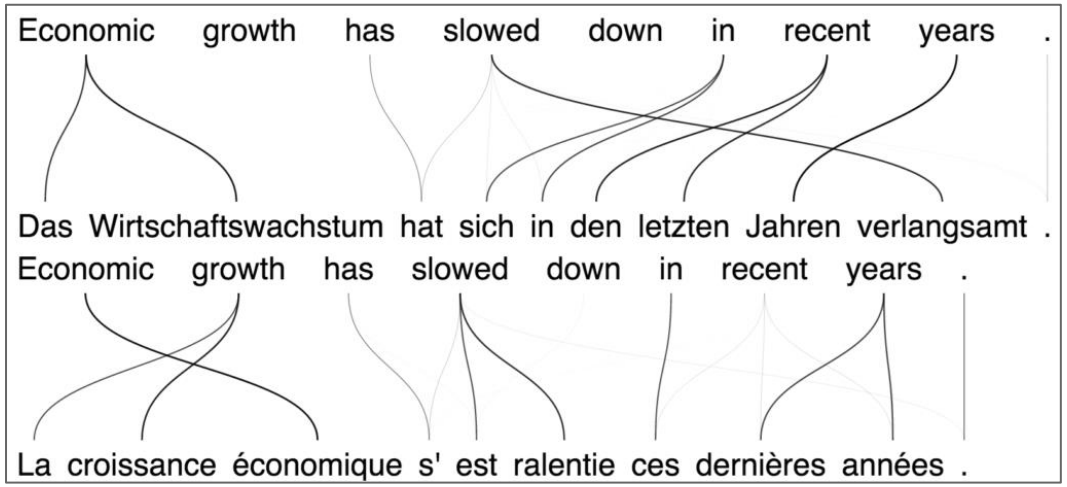
Почему моделирование последовательностей – круто?





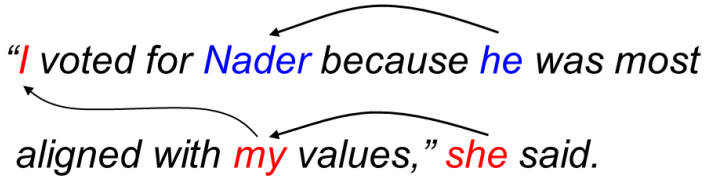
$$p(y|x_0, x_1, \ldots x_t)$$



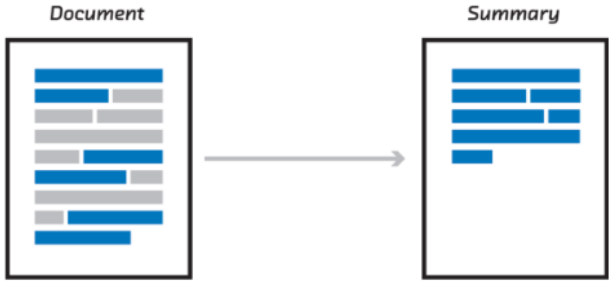




Большие языковые модели



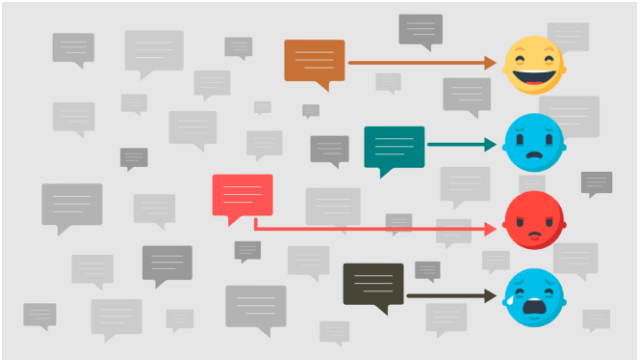
Разрешение кореференции



Обобщение текста



Ответы на вопросы



Анализ настроения



Чат-боты/голосовые помощники



Машинный перевод

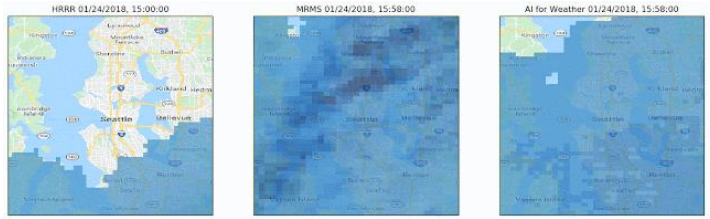
Содержание

1. Последовательности данных
2. Цели моделирования последовательностей
3. RNN и LSTM
4. Эмбеддинги
5. Позиционные эмбеддинги

Последовательности данных

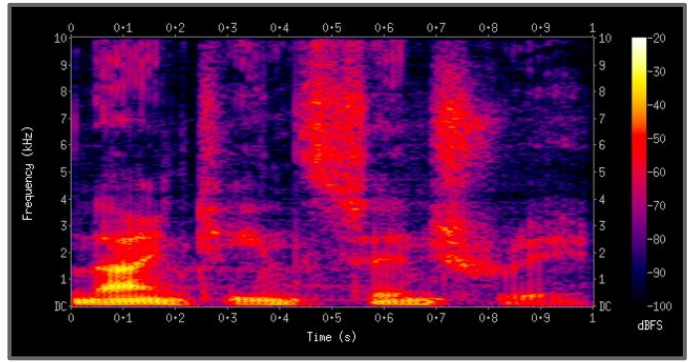
Временные ряды

Временные ряды данных - это данные, записанные через дискретные временные интервалы, где для каждого интервала имеется свой фрагмент данных.

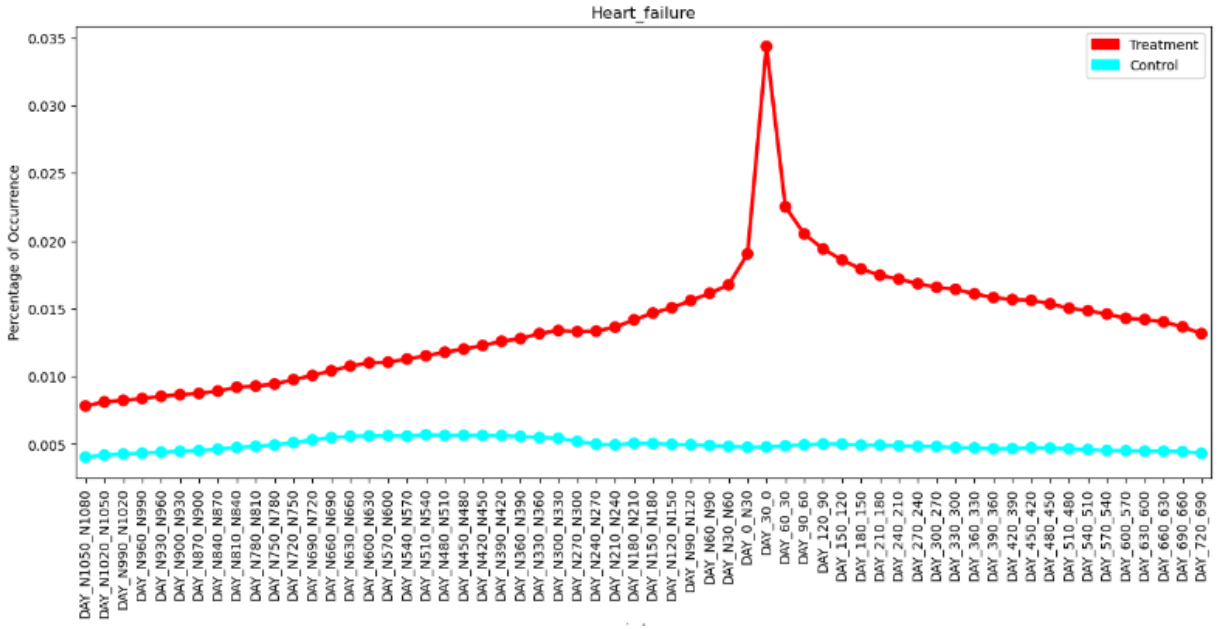


Примеры:

- Погода:
  - Температура может быть записана во времени - ваша последовательность может быть температурой в разные часы дня в течение нескольких дней.
- Музыка / Аудио:
  - Звук - это просто волна, которую можно измерять много раз в секунду.
  - Для некоторого разумно большого временного среза (скажем, для каждой секунды) просмотрите все частоты, регистрируемые в нем, и запишите «силу» (громкость) каждой «корзины».



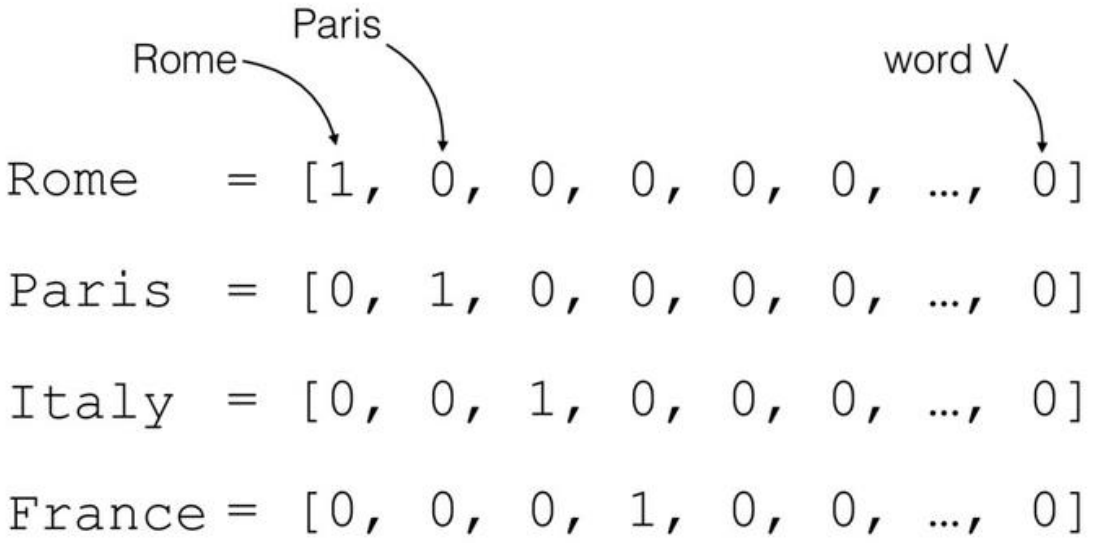
Более изысканные временные ряды



- Пример: Данные о состоянии здоровья
  - Симптомы и данные о состоянии здоровья пациентов могут быть измерены в виде временных рядов, например, с интервалом в месяц.
- Практическое представление:
  - Для некоторого разумно большого временного интервала (например, каждый месяц) запишите частоту диагнозов и т. д., зарегистрированных у пациентов.
  - Теперь у нас есть вектор, представляющий частоту заболеваний/состояний здоровья на данном временном срезе

Тексты

- Самая распространенная форма моделирования последовательностей
  - Существует огромное количество текстовых данных - например, Интернет!
- Каждое слово («лексема») в простейшей форме представлено one-hot вектором.
  - Размер каждого вектора равен размеру нашего словаря (плюс дополнительная метка для нераспознанного слова, которого нет в нашем словаре)



- Эти векторы могут быть огромными, так как во многих языках много слов... Нам нужны способы сжать наше представление текста из one-hot до чего-то более разумного.
  - Подробнее об этом в конце лекции

Цели моделирования последовательностей

1. Данные произвольной длины

- В разных местах мы можем иметь разную продолжительность регистрации температуры или облачного покрова.
- У разных пациентов может быть разная продолжительность записи данных о здоровье, например, 12 или 24 месяца,
- Текст может быть длинным, как «Война и мир», или коротким, как хайку.

Мы должны быть уверены, что наши модели могут работать с произвольными длинами входных последовательностей!

2. Парадигмы моделей последовательностей

- Многие к одному:
  - Моделирование «Многие к одному»: множество данных → один результат
  - Классификация настроений: Взять последовательность текста, вывести оценку настроения для этого предложения (злой, счастливый и т. д.)

Нам нужно поддерживать выходные последовательности переменной длины, учитывая вышеуказанные парадигмы!

Почему «внимание»?

Высокоуровневая интерпретация

- Внимание позволяет моделям "фокусироваться" на важных частях входных данных
- Когда мы делаем предсказания о чем-то (например, идентифицируем объекта), нам может понадобиться лишь небольшое подмножество заданных признаков
  - Не все признаки одинаково важны!

Рассмотрение случая NLP

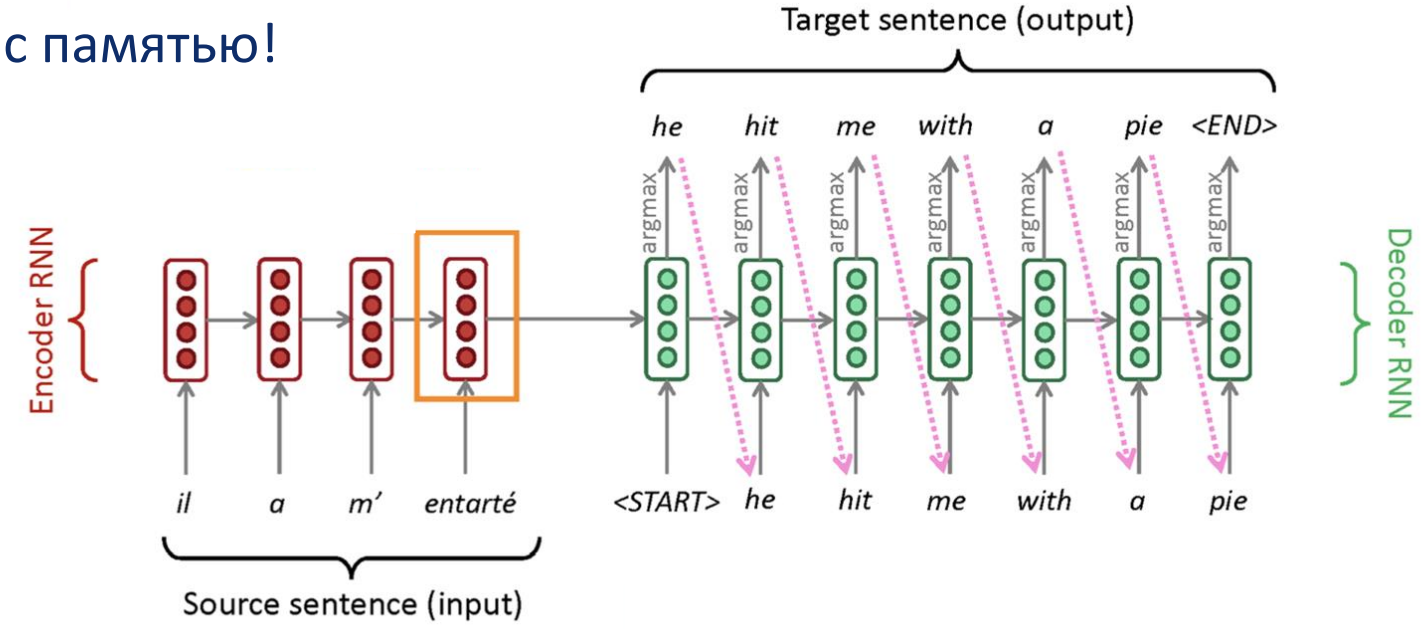
Обзор моделей последовательностей

- RNN обрабатывают последовательности шаг за шагом (например, слово за словом), принимая на вход предыдущее скрытое состояние в дополнение к текущей лексеме
- Ванильные RNN испытывают трудности с длинными последовательностями из-за исчезающих/исчезающих градиентов → используйте LSTMs

Рассмотрение случая NLP

Задача: машинный перевод

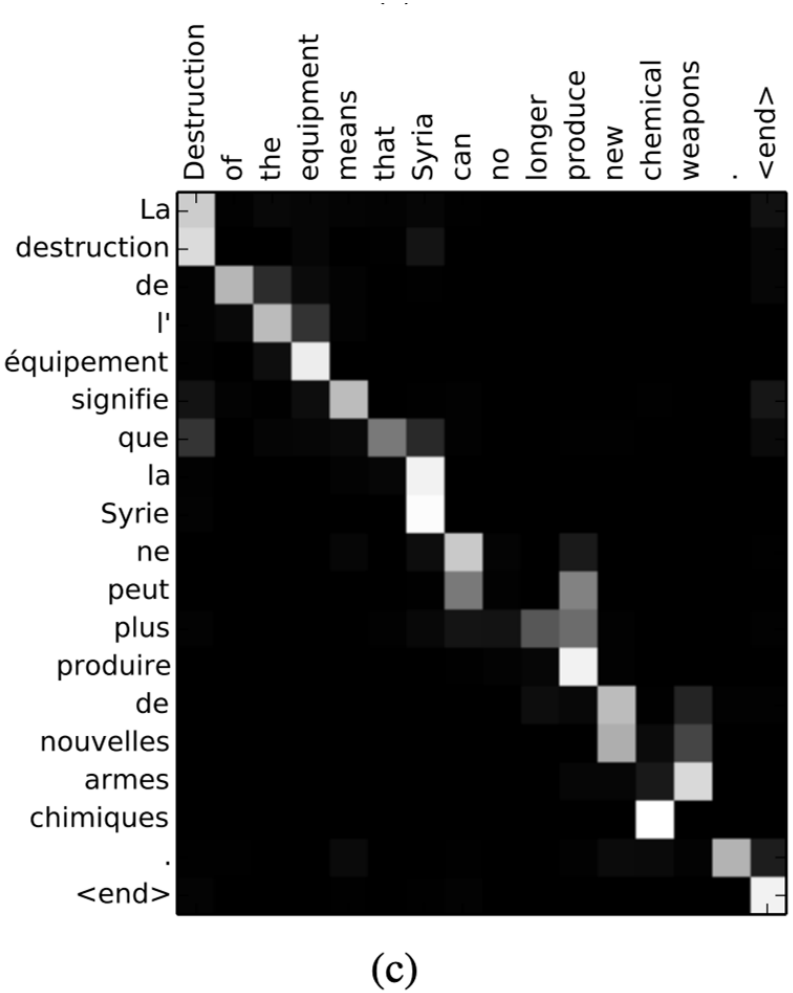
- Модели Seq2seq
  - Кодируют последовательности как вектор фиксированной длины, затем декодируют в другую последовательность
  - По-прежнему имеет проблемы с памятью!
  - Кодирование длинных последовательностей в вектор фиксированной длины



Рассмотрение случая NLP

Допустим, мы хотим перевести с французского на английский.

Обратите внимание: имеет место хорошее соответствие между словами обоих языков, даже если они не 1 к 1



Рассмотрение случая NLP

Что, если мы обратимся непосредственно к входной последовательности?

- Вместо того чтобы пытаться получить информацию только из вектора фиксированной длины, мы также будем искать во входных представлениях наиболее релевантные входные токены!

Как это реализовать?

Рассмотрение случая NLP

Подход 1: жесткое внимание

- Предсказать, какие индексы являются релевантными при попытке декодирования на данном временном шаге → использовать эти входы
- Проблема: сложно (хотя и не невозможно) реализовать обратный проход.

Рассмотрение случая NLP

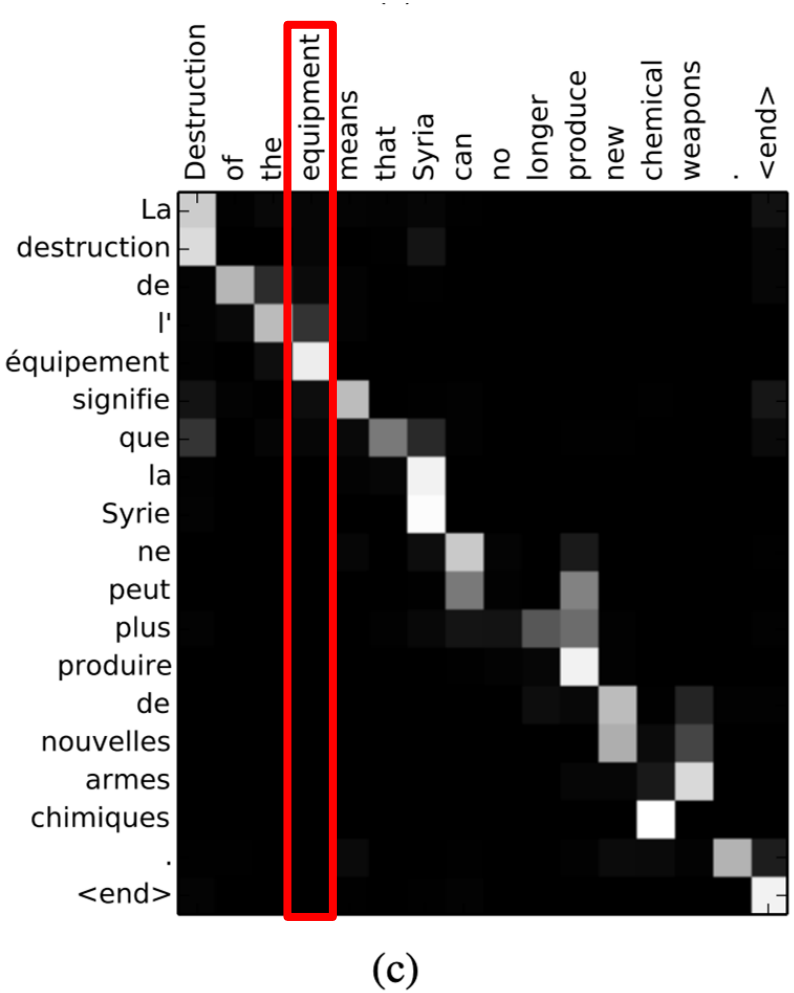
Подход 2: мягкое внимание

- Сгенерируйте набор мягких весов (например, с помощью softmax) для входных лексем, которые указывают на их важность
- Выполните взвешенную сумму
  - Мысль: ожидаемая значимость наиболее важных характеристик
- Преимущество: дифференцируемость и интуитивное объединение представлений нескольких значимых признаков

Рассмотрение случая NLP

При переводе «destruction» **какие входные токены являются наиболее релевантными**?

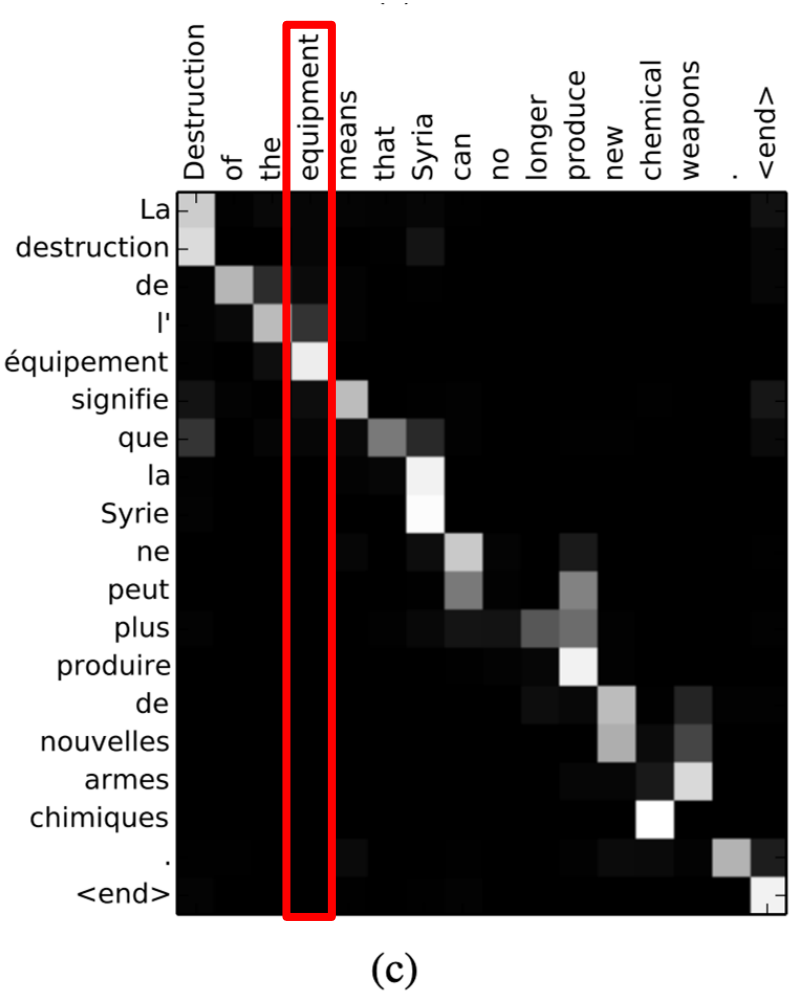
- Мы хотим сослаться на «La destruction».
- Весовые коэффициенты внимания могут быть следующими
  - [~0.4, ~0.5, ~0, ~0, ~0, ~0...]
- Теперь модель использует объединенные векторы признаков «La» и «destruction».



Рассмотрение случая NLP

Посмотрев «Destruction of the», **какой следующий токен будет наиболее актуальным**?

- Мы хотим сослаться на «équipment».
- Спрогнозируйте распределение релевантности признаков:
  - [~0, ~0, ~0, ~0, ~0.99, ~0…]
- То же, что и раньше: находим мягкие веса признаков и выдаем взвешенную сумму в модель



Мягкое внимание

Мягкое внимание

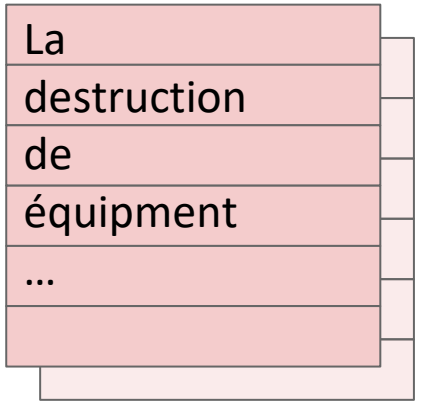
Мысль: дифференцируемый метод поиска в базе данных

У нас есть база данных интересующих нас признаков (например, французское предложение).

И мы хотим использовать подмножество этих признаков для предсказания (например, декодирования английского перевода).

Мягкое внимание

Database of features:



Мягкое внимание

Database of features:

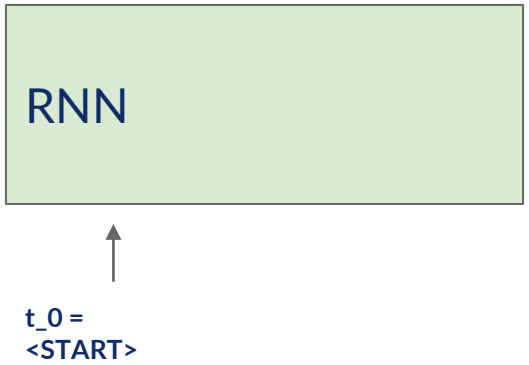
La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…

Мягкое внимание

Database of features:

La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…

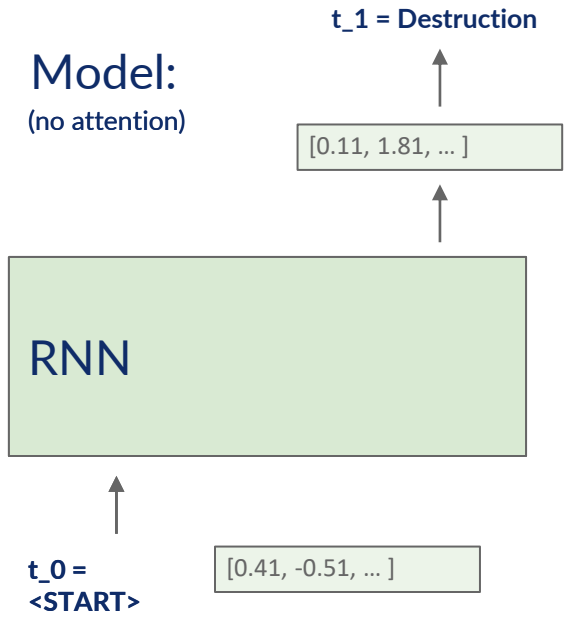
(no attention)



Мягкое внимание

Database of features:

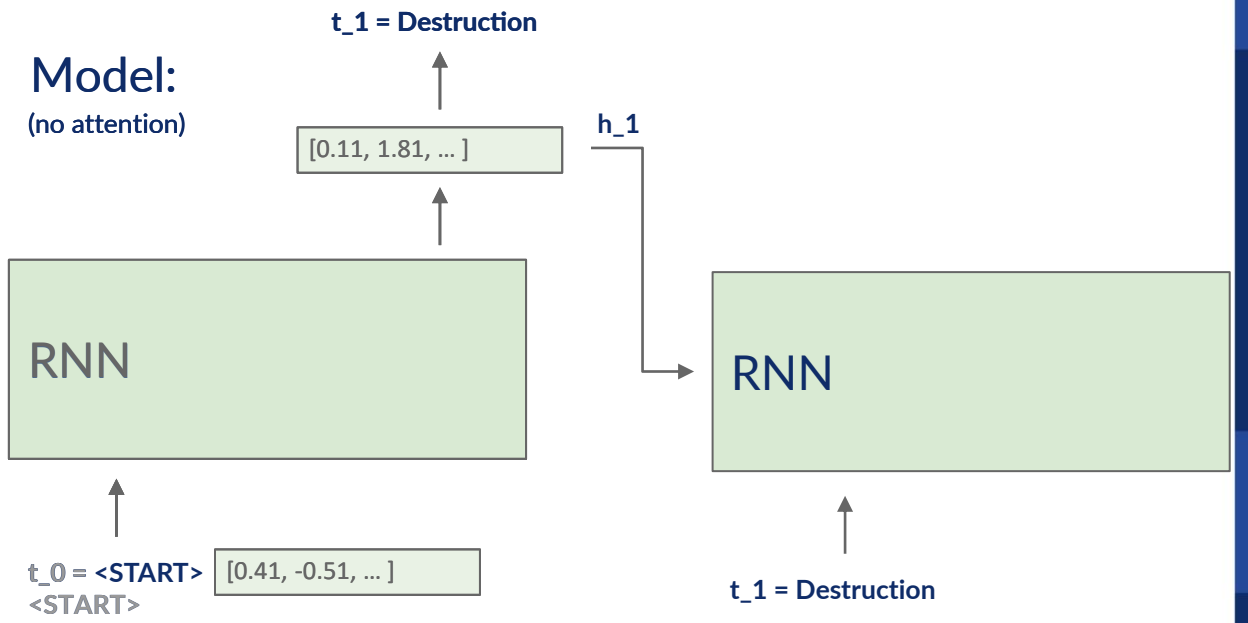
La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…



Мягкое внимание

Database of features:

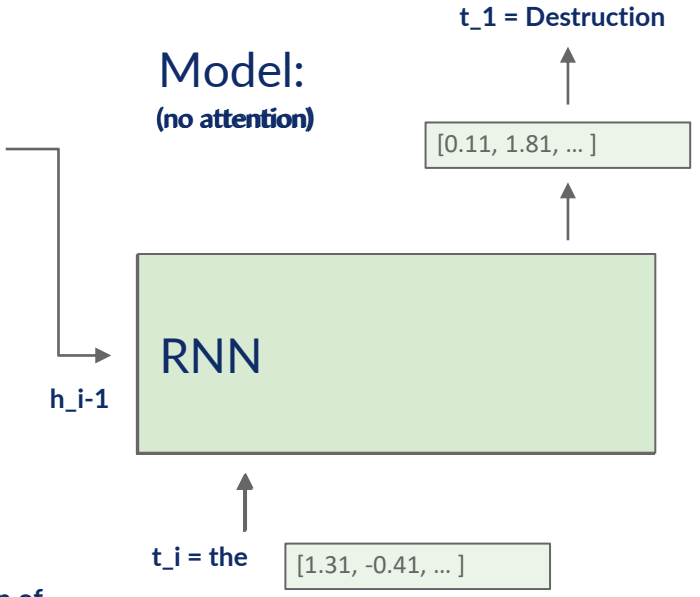
La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…



Мягкое внимание

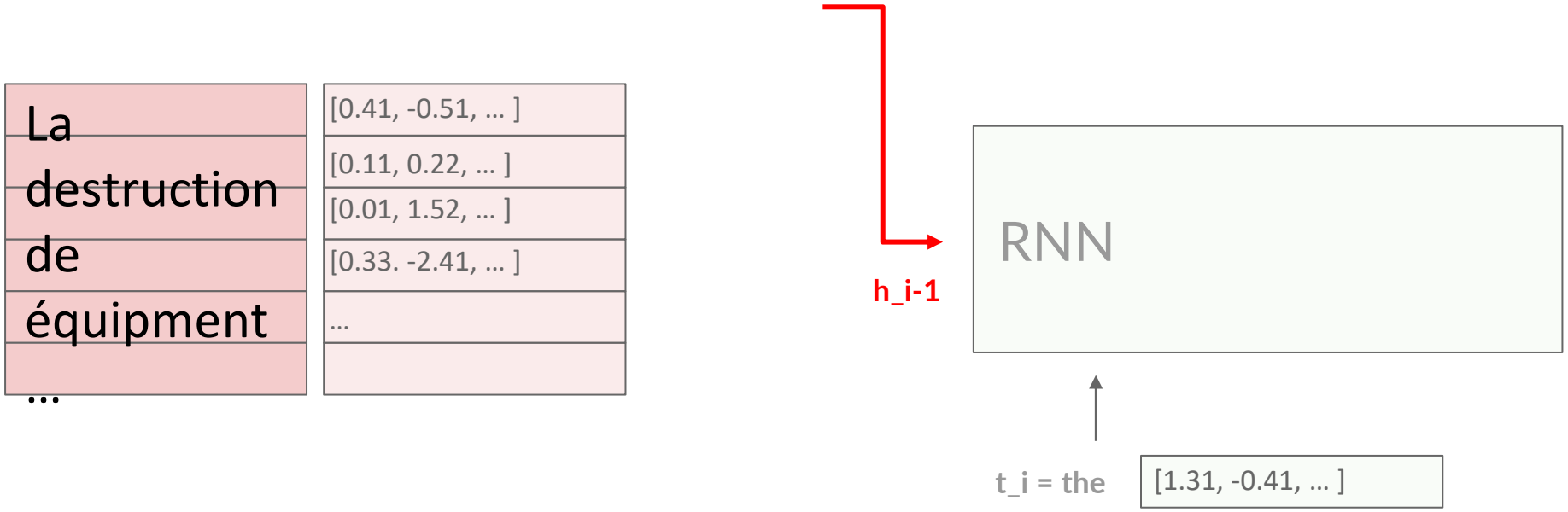
Database of features:

La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…

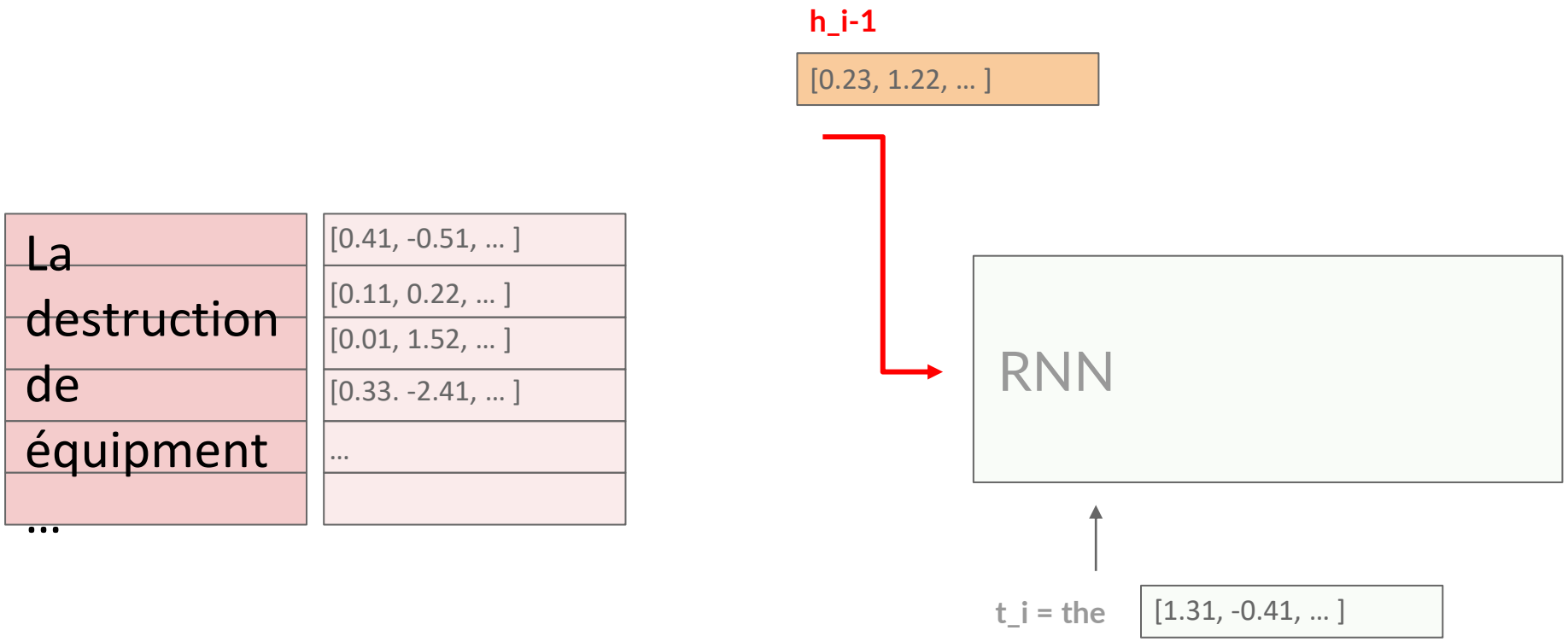


Уже предсказано «Destruction of the». Целевое слово: «equipment»

Soft attention



Soft attention



MACHINE LEARNING
@ BERKELEY

Soft attention

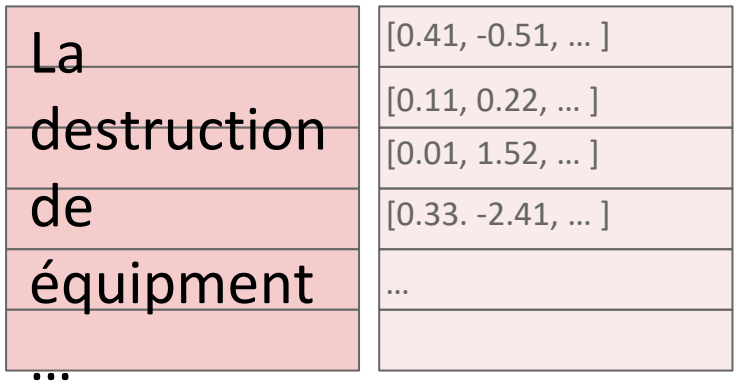
h_i-1

[0.23, 1.22, …]

La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…

MACHINE LEARNING
@ BERKELEY

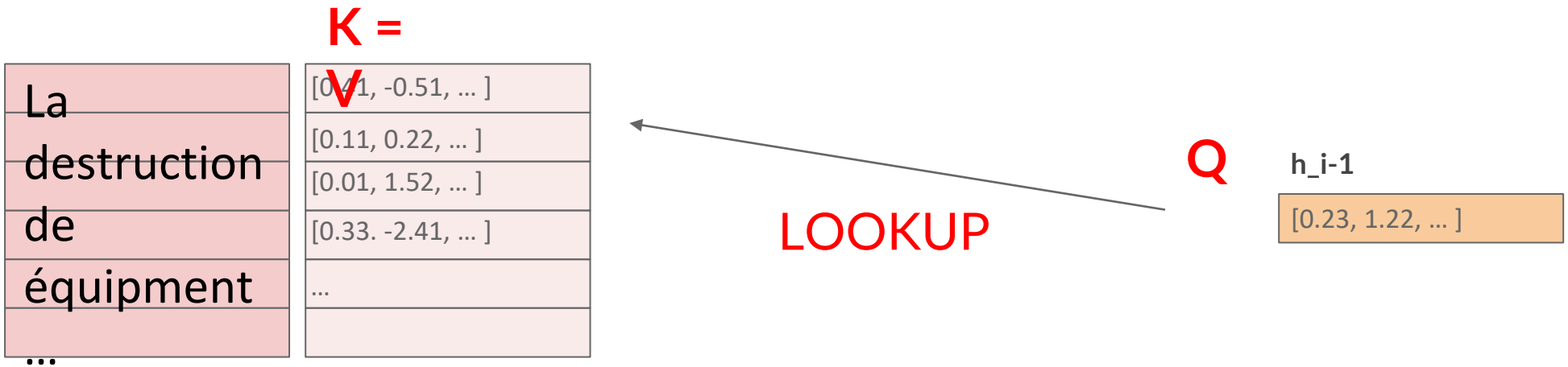
Soft attention



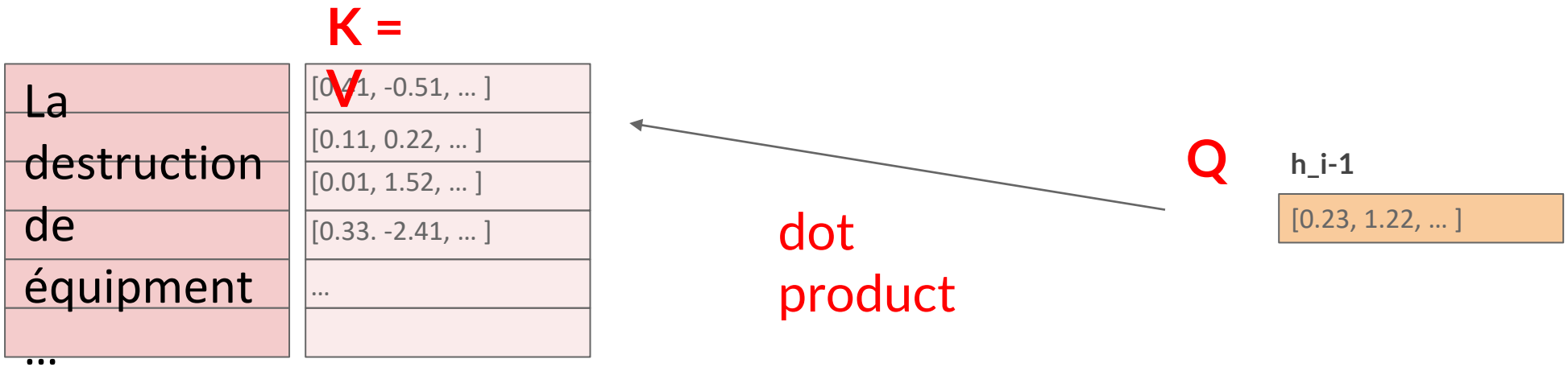
h_i-1

[0.23, 1.22, …]

Soft attention

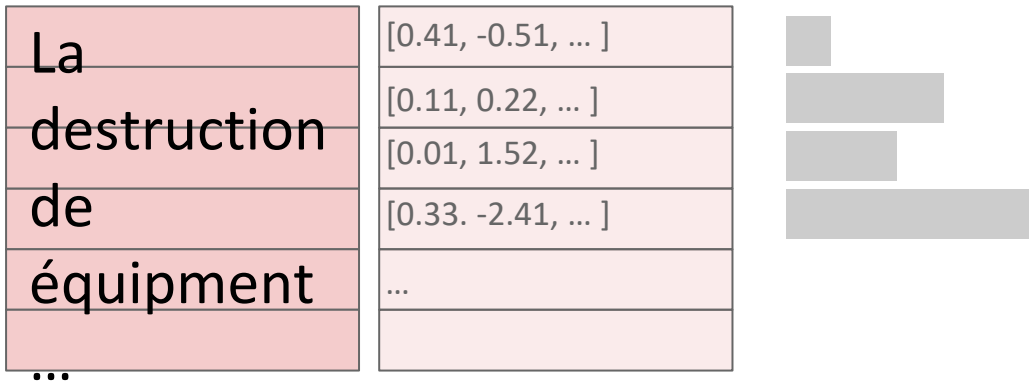


Soft attention



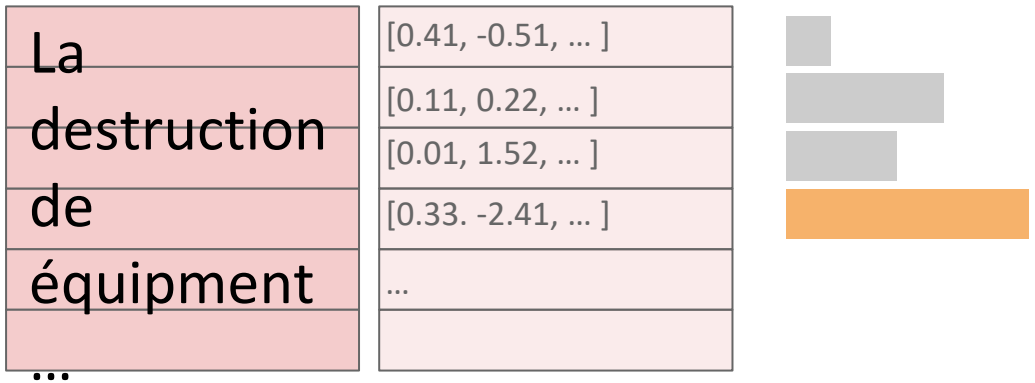
MACHINE LEARNING
@ BERKELEY

Soft attention



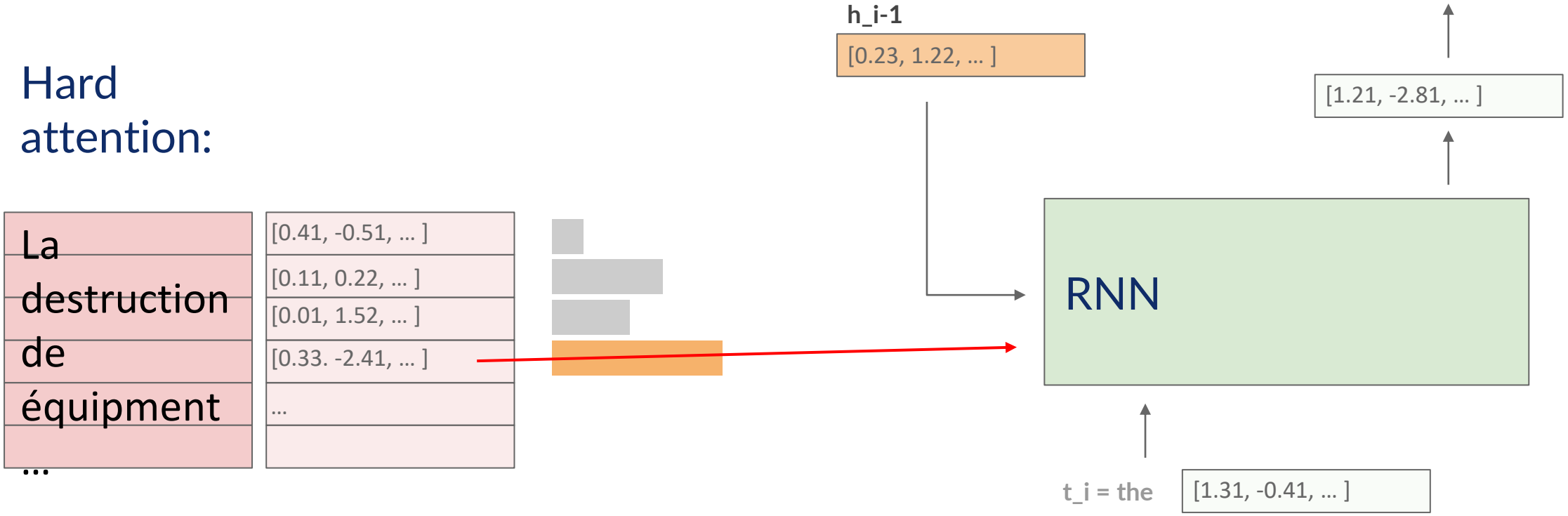
MACHINE LEARNING
@ BERKELEY

Soft attention

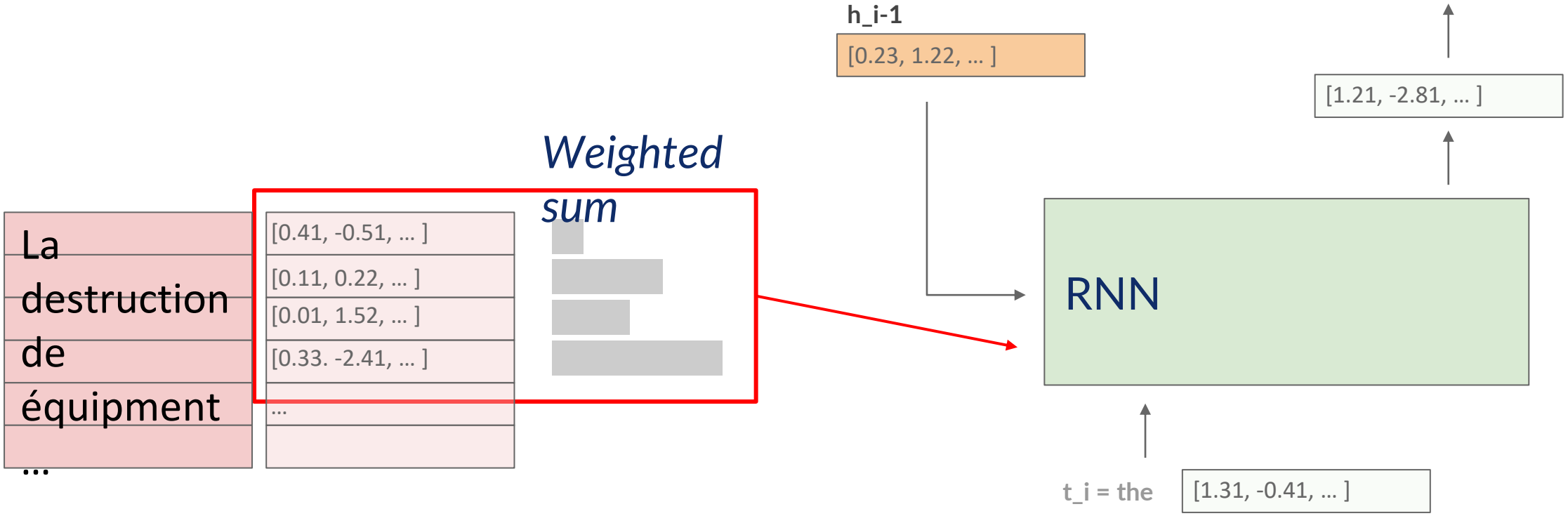


~~Soft~~ attention

Hard attention:



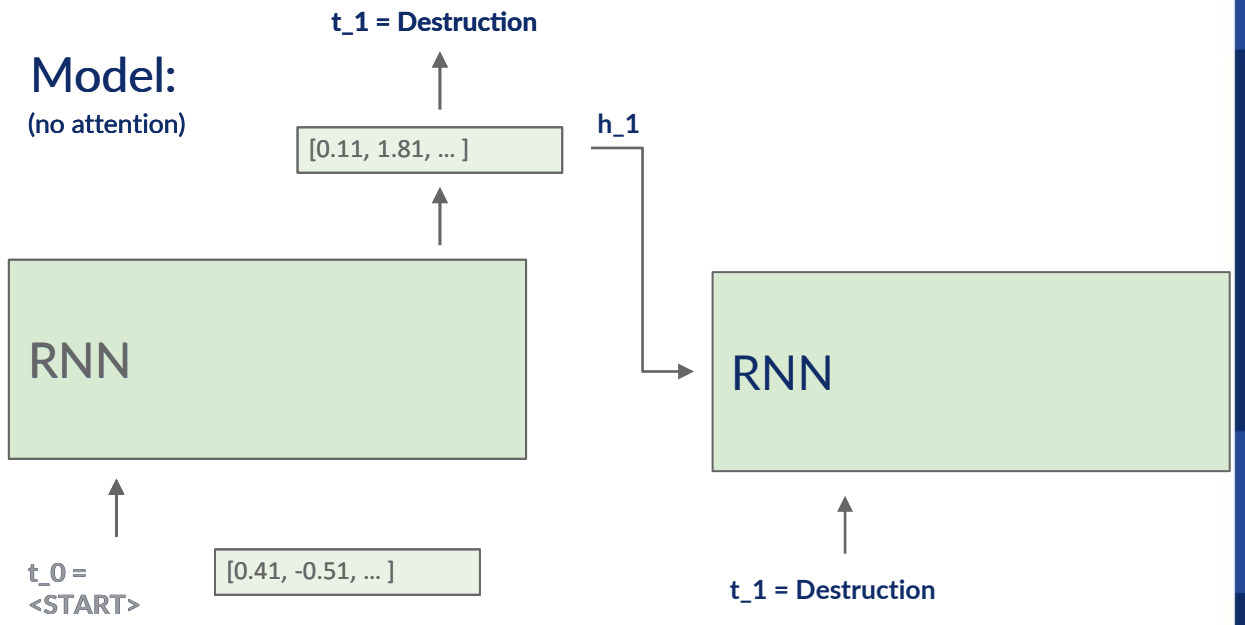
Soft attention



Мягкое внимание

Database of features:

La	[0.41, -0.51, …]
destruction	[0.11, 0.22, …]
de	[0.01, 1.52, …]
équipment	[0.33. -2.41, …]
…	…



Мягкое внимание

Мысль: дифференцируемый метод поиска в базе данных

У нас есть база данных интересующих нас признаков (например, французское предложение).

И мы хотим использовать подмножество этих признаков для предсказания (например, декодирования английского перевода).

В предыдущем примере мы выполнили две операции:

- **Поиск** с помощью простого точечного произведения
  - Подумайте: как сопоставить заданный «запрос» базы данных с набором «ключей» базы данных.
- **Извлечение** с помощью взвешенной суммы
  - Подумайте: объединение «значений» из выбранных «ключей»

Мягкое внимание

Поиск: Нужна метрика для определения **относительной важности / сходства / родства** (как мы узнаем, что ищем «équipment», основываясь на текущем состоянии декодера)

Name	Alignment score function	Citation
Content-base attention	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \text{cosine}[\boldsymbol{s}_t, \boldsymbol{h}_i]$	Graves2014
Additive(*)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\boldsymbol{s}_t; \boldsymbol{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \boldsymbol{s}_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \mathbf{W}_a \boldsymbol{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \boldsymbol{h}_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \frac{\boldsymbol{s}_t^\top \boldsymbol{h}_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

s_i - скрытое состояние декодера (запрос)

h_i - скрытое состояние шифратора (ключи).

Для каждого s_i мы находим его релевантность для каждого h_i

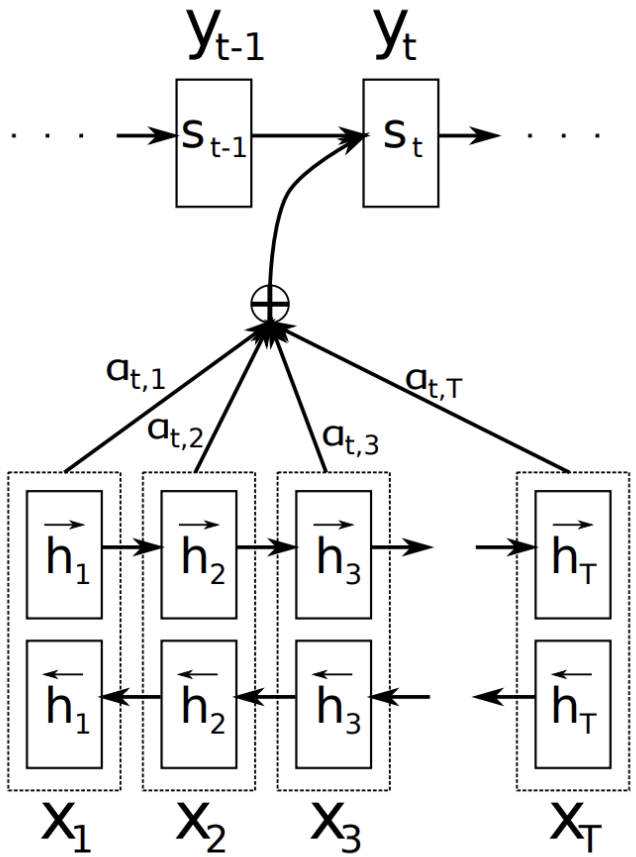
Оценка проходит через softmax для получения весов

Мягкое внимание

Извлечение: Взвешенная сумма представлений признаков (например, представления «la destruction»).

Примечание: В других случаях использования мягкого внимания (например, в различных архитектурах), вы иногда просто объединяете каждое значение, а не берете их взвешенную сумму

Реальная модель



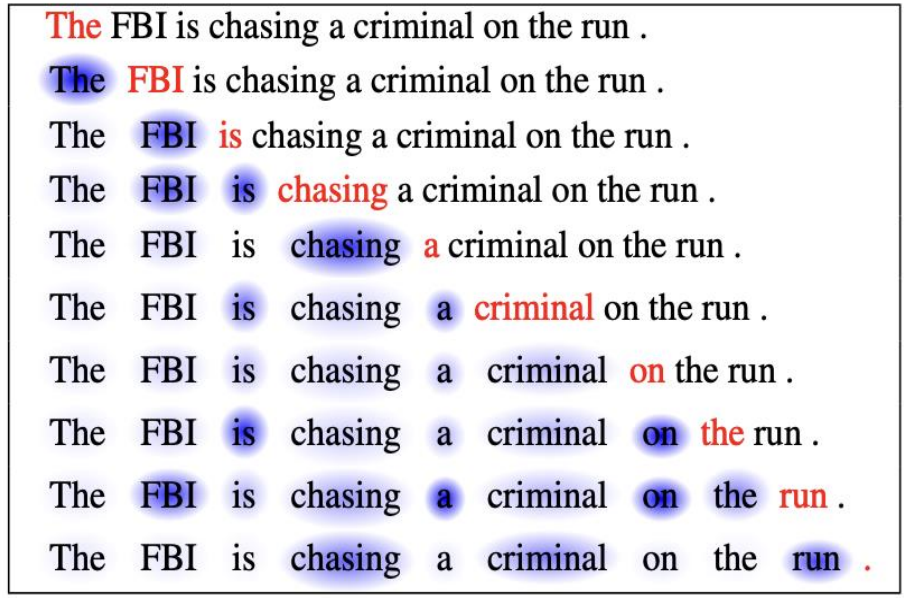
(Бахданау, 2014) Модель для NMT

- Первая работа, в которой используется известный нам сегодня механизм внимания
- Похожа на предыдущий пример, за исключением того, что мы используем аддитивное внимание вместо внимания точечного произведения
- Также используется двунаправленная LSTM

Самовнимание

Кратко возвращаясь к NLP: «Сети долгой краткосрочной памяти для машинного чтения»

- Допустим, вы занимаетесь языковым моделированием.
- Предсказание следующего слова на основе предыдущих слов.
- Идея: слова в предложении имеют связи между собой.
- При декодировании с помощью RNN используется дополнительная информация о предыдущих лексемах в предложении.
  - До: поиск лексем из отдельного набора признаков
  - Self-attention: поиск лексем из того же набора признаков



Трансформеры

Проблемы с RNN

- Последовательный - для вычисления t_{i-1} требуется t_{i}.
  - Не очень хорошо поддается распараллеливанию
- Плохо отражает длинные последовательности
  - Количество вычислительных шагов, необходимых для получения зависимости между t_i и t_{i+d}, линейно зависит от d
  - т. е. для перехода от t_i к t_{i+d} вам нужно запустить RNN d раз, и за это время вы можете забыть t_i

“Attention is all you need”

aka *Vaswani et al.*

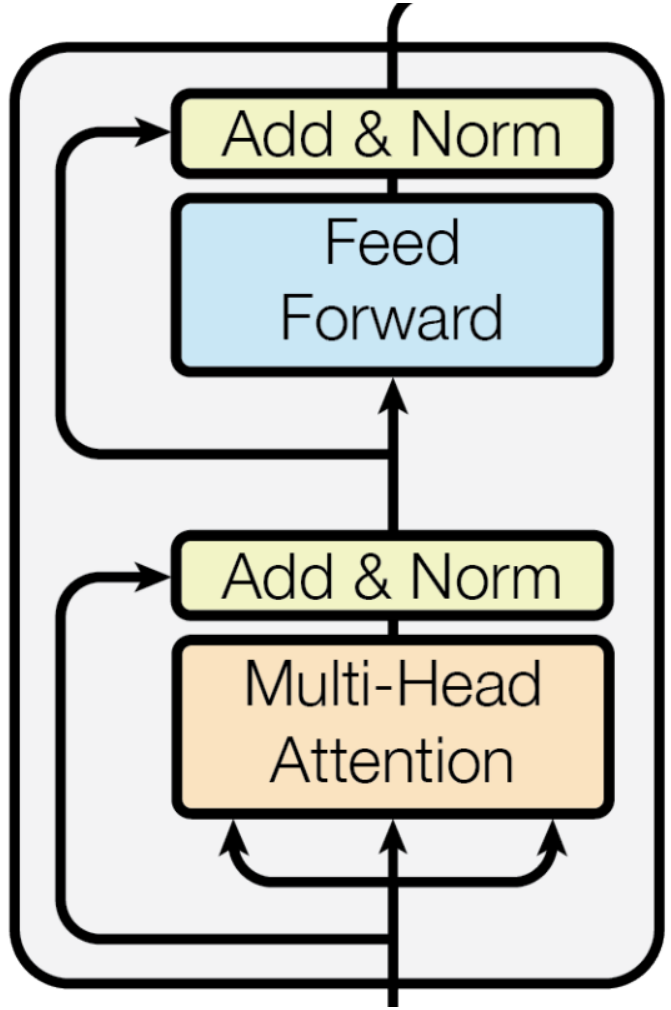
Что, если мы вообще избавимся от рекуррентности?

- И просто использовать самовнимание для моделирования последовательностей
  - (или групп лексем в целом)

Слой трансформера

Справа: один слой трансформатора в модели трансформера

Входные данные аналогичны предыдущим: эмбеддинги последовательности токенов + позиционные эмбеддинги (объясняется позже)



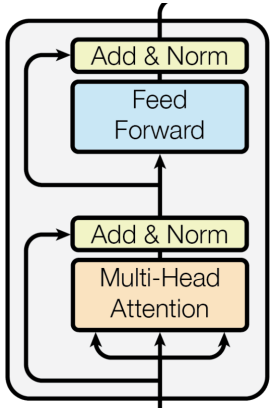
MACHINE LEARNING
@ BERKELEY

Overview

The big red cat .

E E E E

Input representation



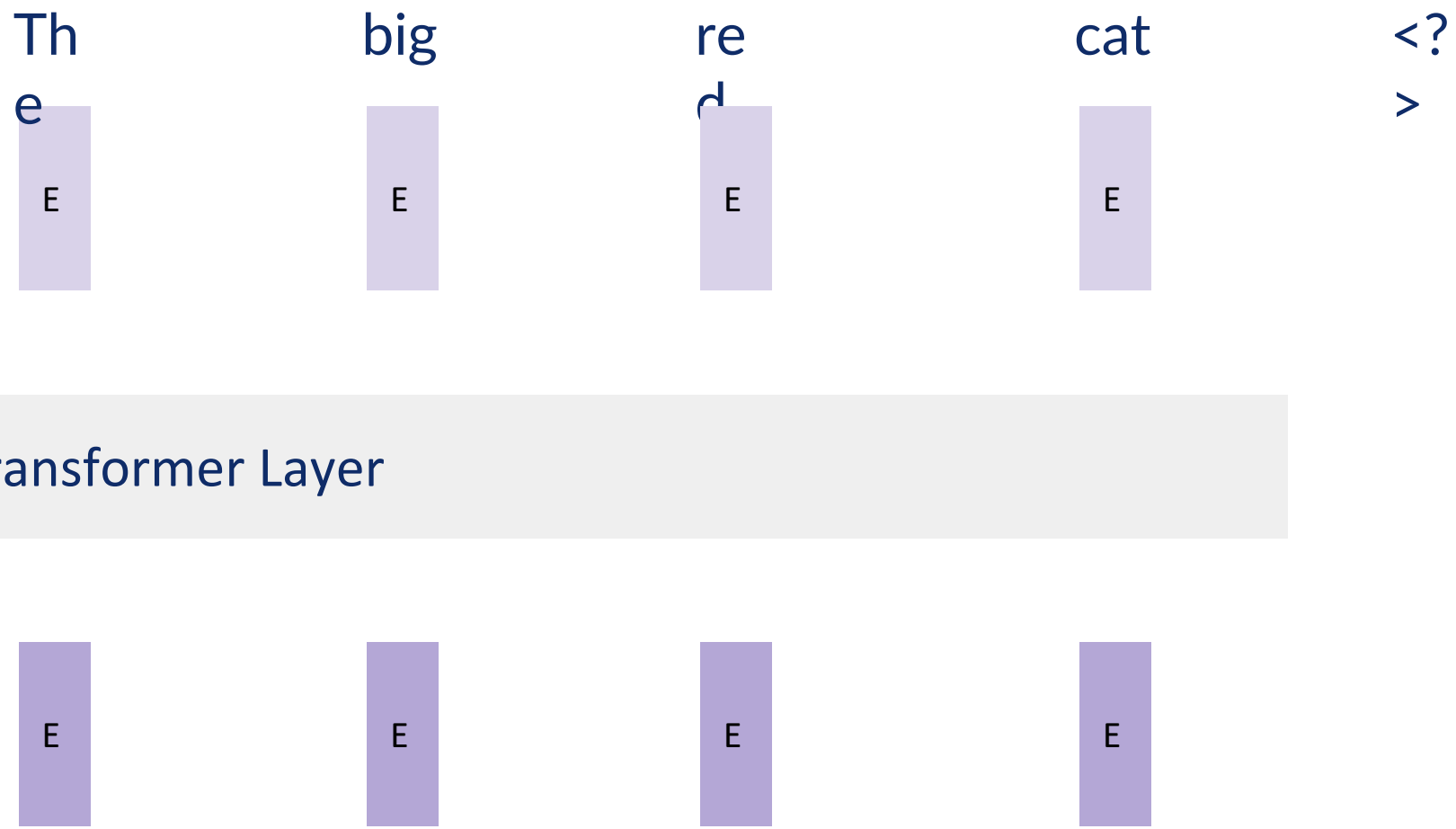
Transformer Layer

E E E E

Output representation

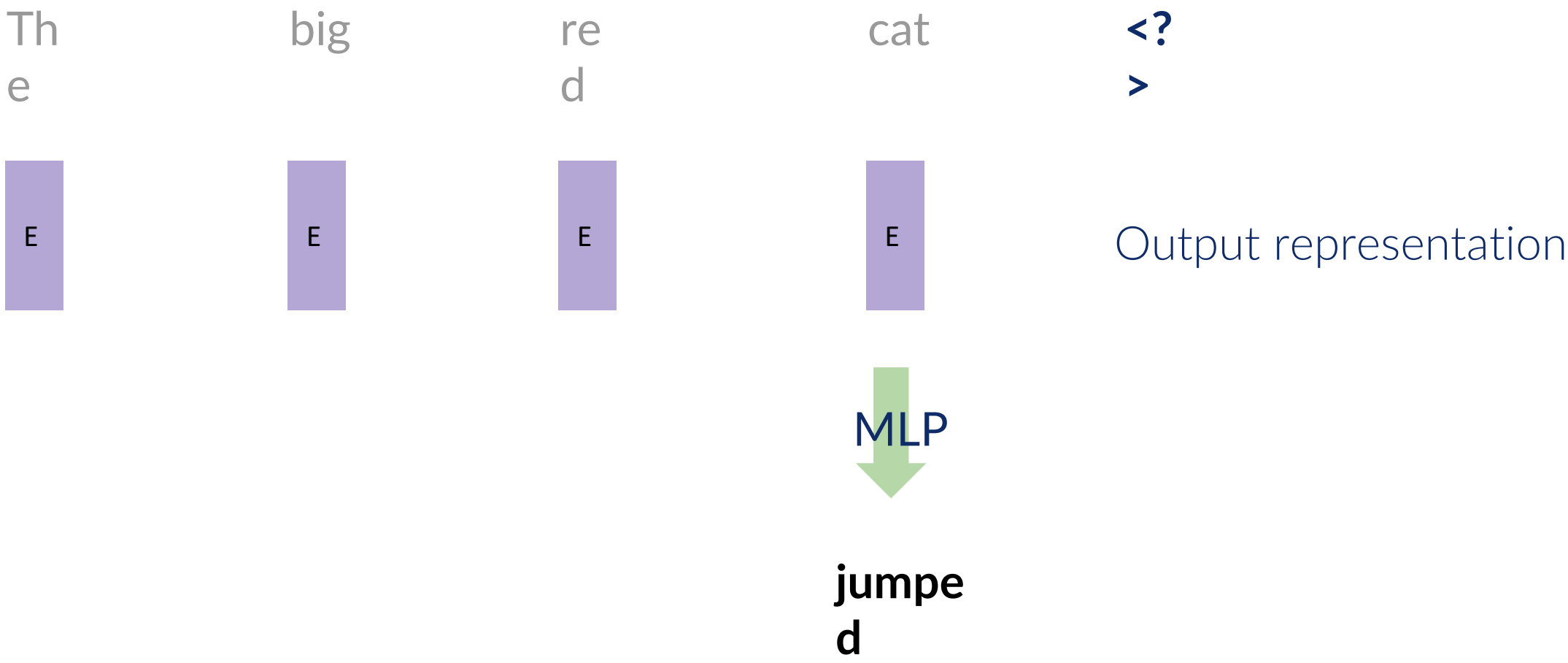
MACHINE LEARNING
@ BERKELEY

Example: Next-Token Prediction



MACHINE LEARNING
@ BERKELEY

Example: Next-Token Prediction

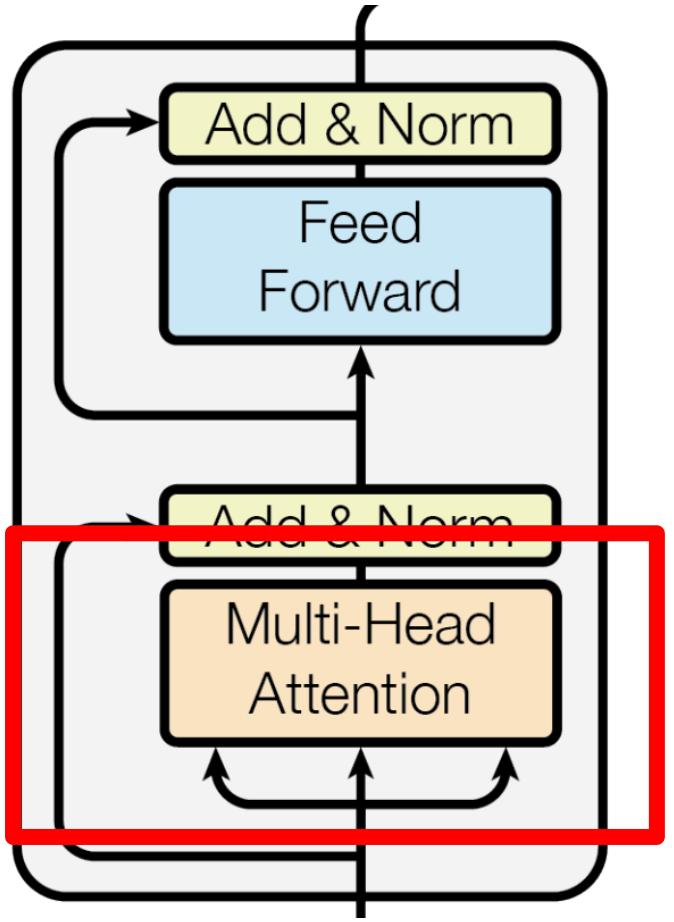


Multi-Head Self-Attention

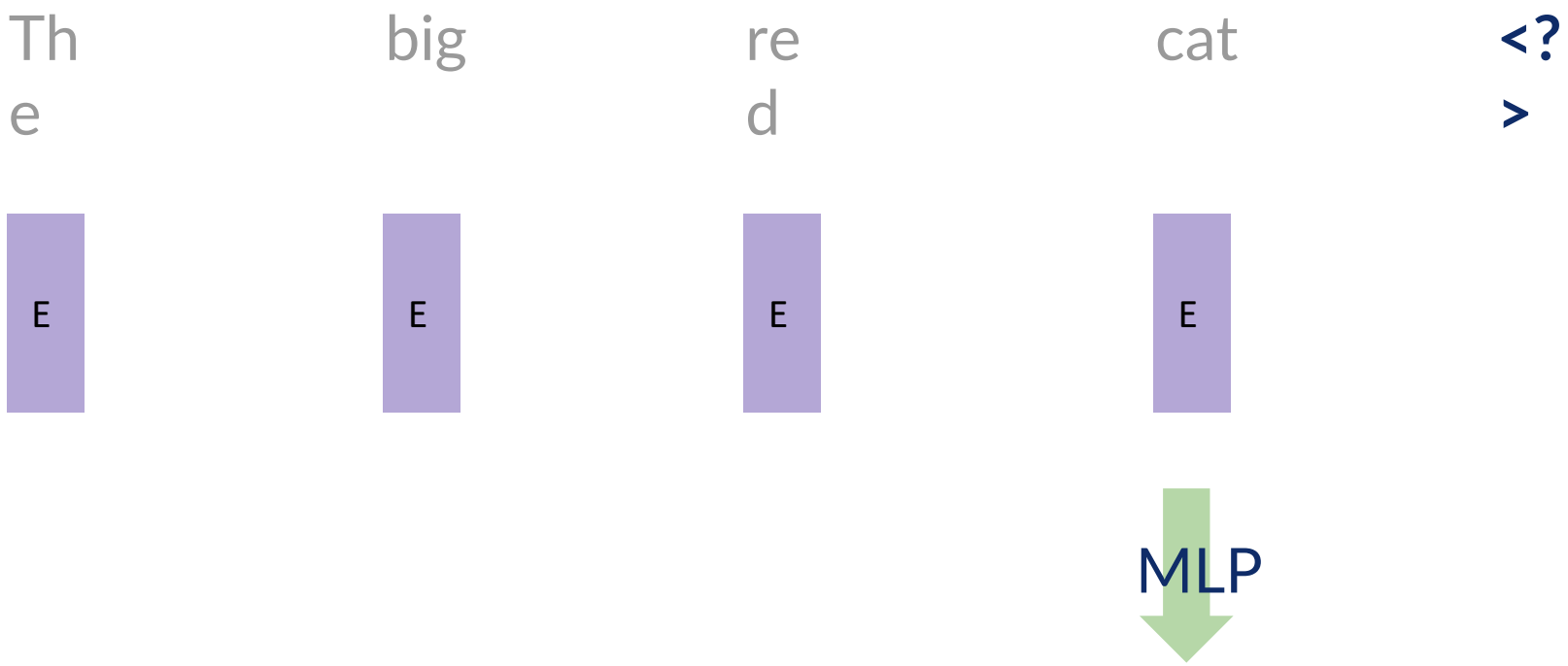
At a high level: for each token representation,

- **Augment its representation based on its relationship to other tokens**
- Self-attention is all about token to token interactions

We will get to the “multi-head” part soon…



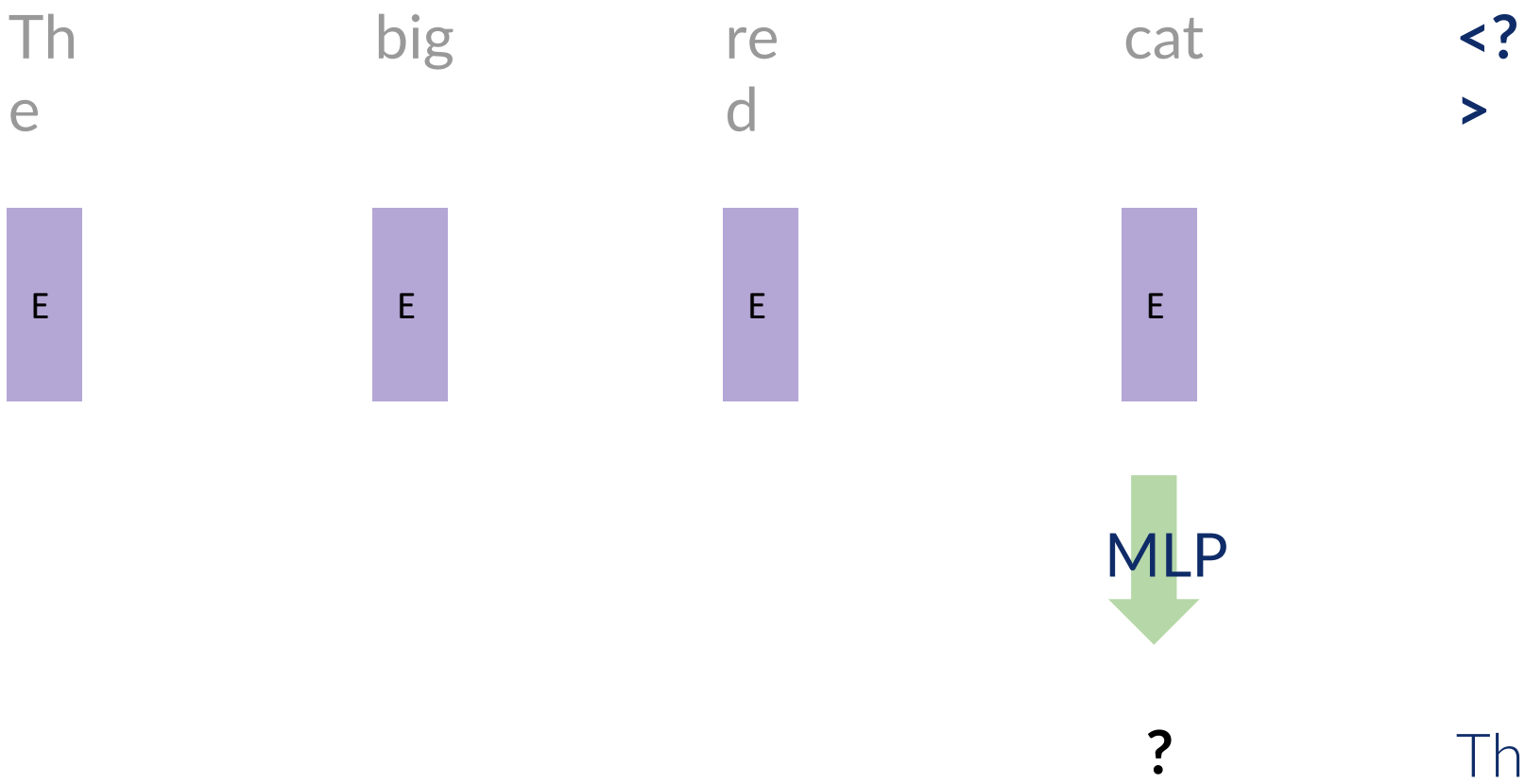
Token-Token Relationships



Can we predict a period here?

warning: illustrative example to justify self-attention

Token-Token Relationships

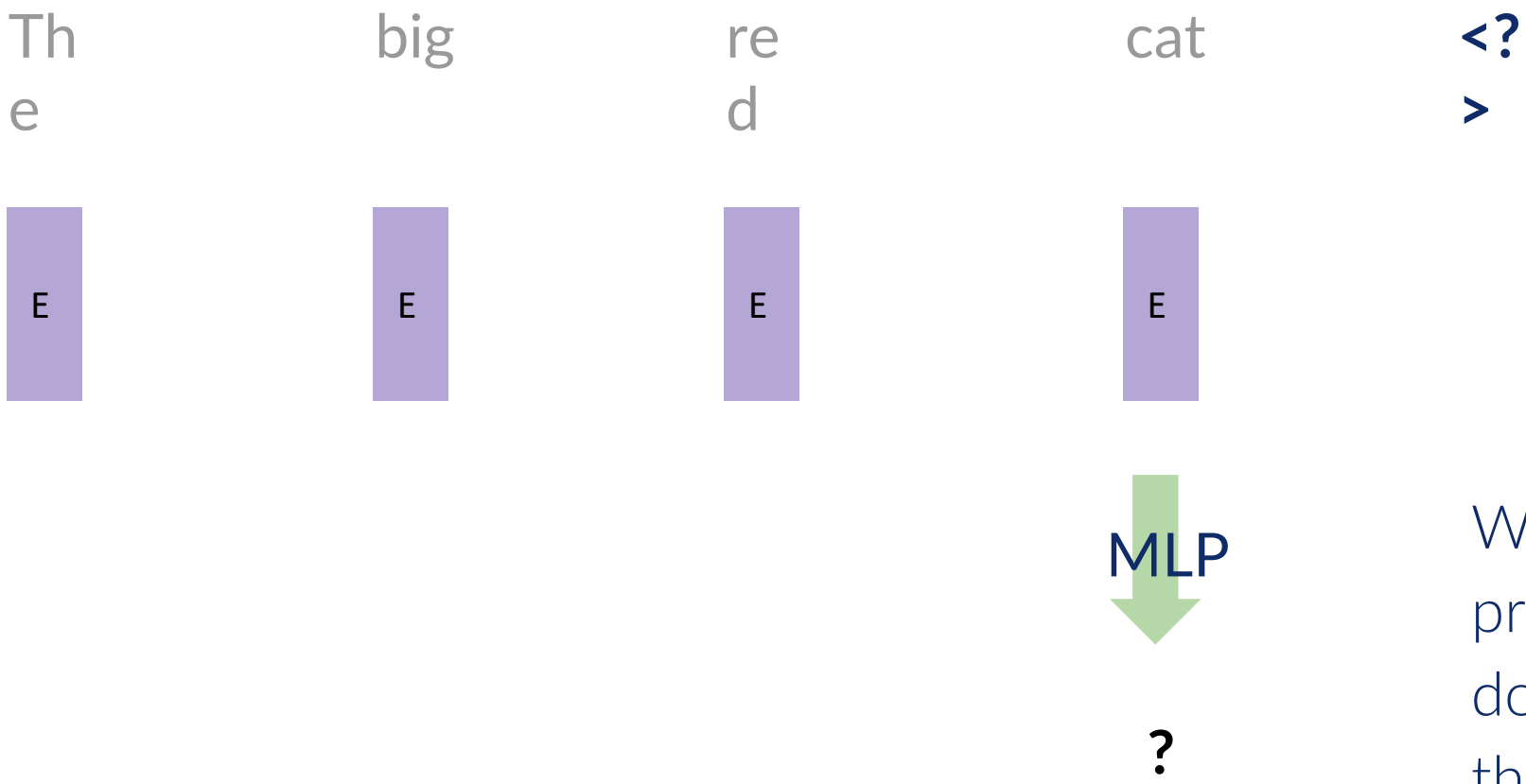


warning: illustrative example to justify self-attention

The sentence isn't complete! Might need a verb.

MACHINE LEARNING
@ BERKELEY

Token-Token Relationships

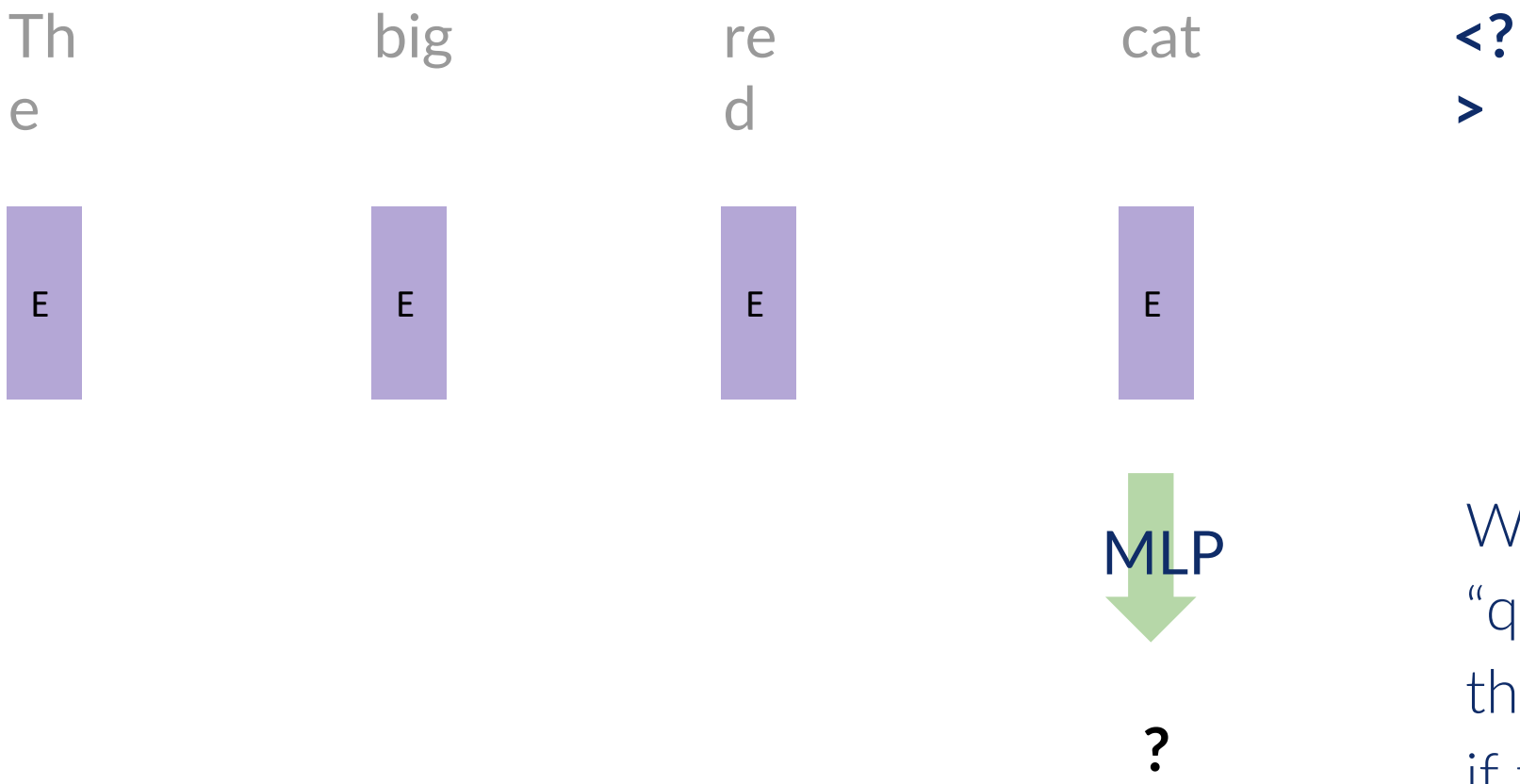


When we do prediction here, how does the token “know” that there aren’t any verbs previously?

warning: illustrative example to justify self-attention

MACHINE LEARNING
@ BERKELEY

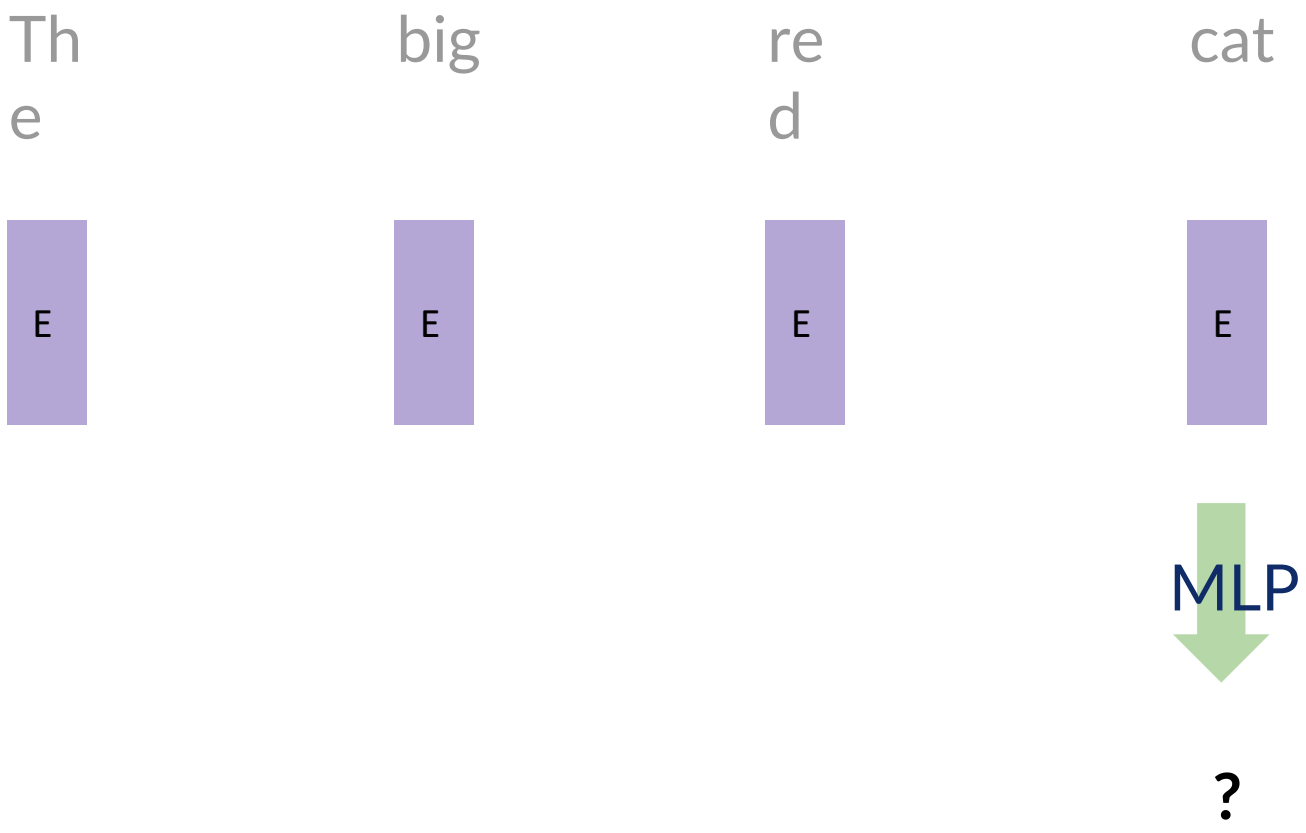
Token-Token Relationships



We want this token to "query" other tokens in the sequence to check if there's a verb.

warning: illustrative example to justify self-attention

Token-Token Relationships



In reality, attention weights tend to be much harder to interpret :(

Also repeat this many times!

warning: illustrative example to justify self-attention

Takeaway: self-

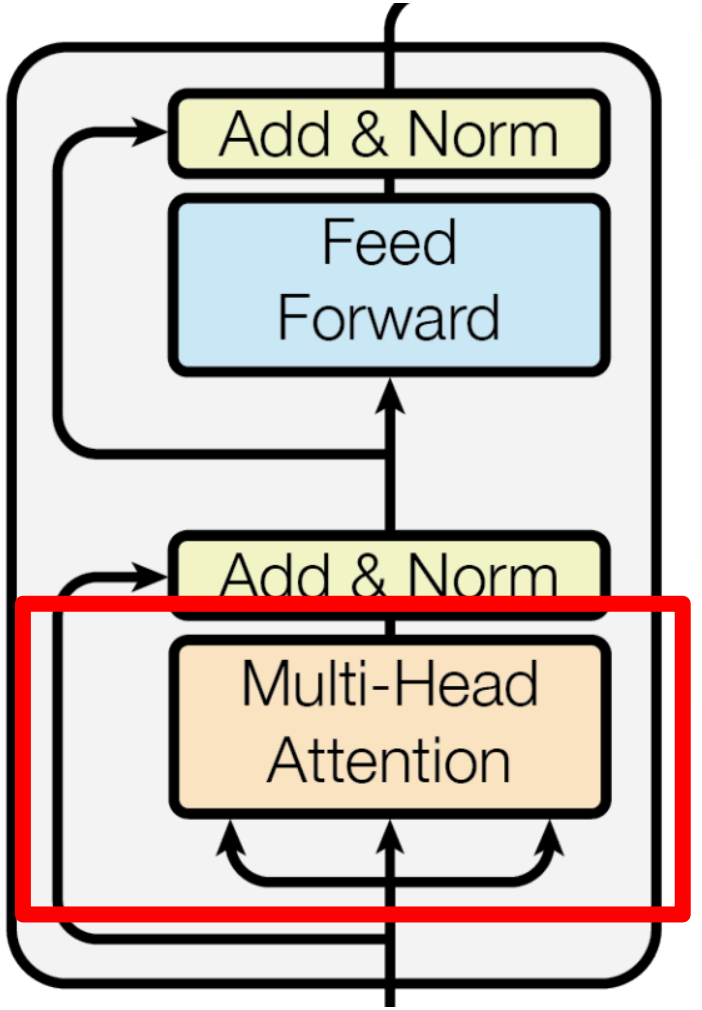
Многоголовое самовнимание

Мысль: поиск в базе данных

Дайте каждому токену представление KEY, QUERY и VALUE

Для каждого токена:

- Мы используем его QUERY, чтобы найти наиболее релевантный токен(ы) в последовательности на основе их соответствующих KEY.
  - Рассчитываем важность с помощью точечного произведения + softmax
- Выбираем и комбинируем VALUE(ы) релевантных лексем.
  - Взвешенная сумма на основе важности
- -> становится новым представлением этой лексемы



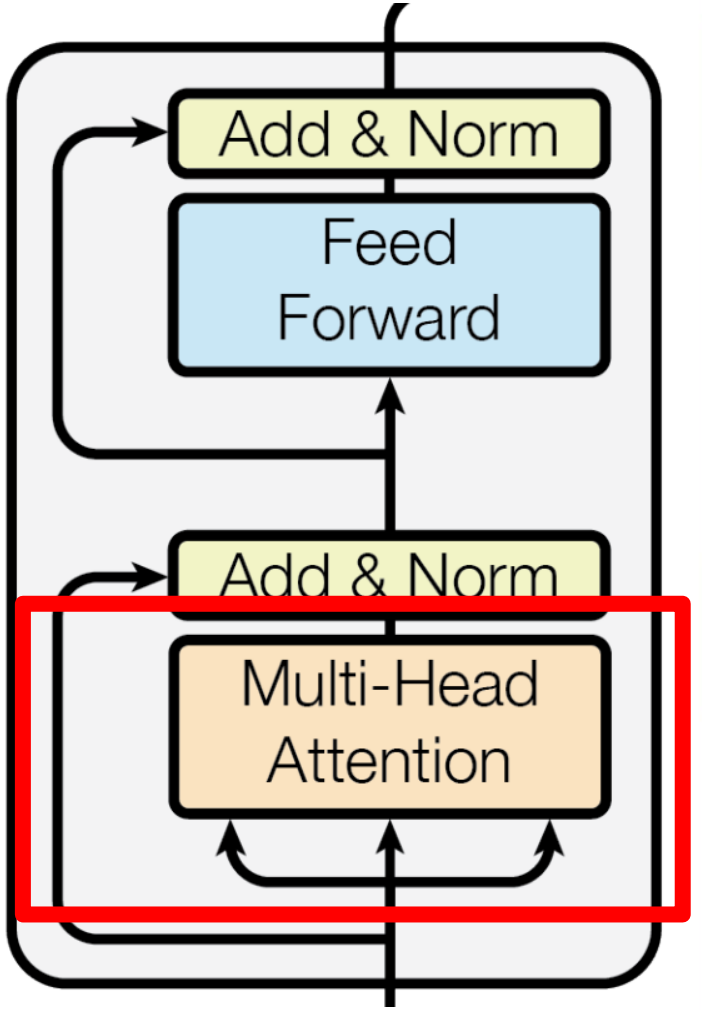
Многоголовое самовнимание

Мысль: поиск в базе данных

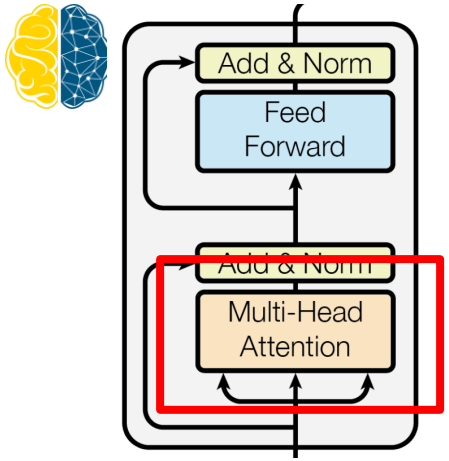
Дайте каждому токену представление KEY, QUERY и VALUE

Эти представления KEY, QUERY и VALUE являются линейными проекциями исходных представлений токенов:

- Это означает, что вектор вложения лексем умножается на некоторые матрицы W_K, W_Q и W_V соответственно.
  - Это обучаемые параметры
- Это просто линейный слой без члена смещения (хотя некоторые архитектуры могут решить включить его)



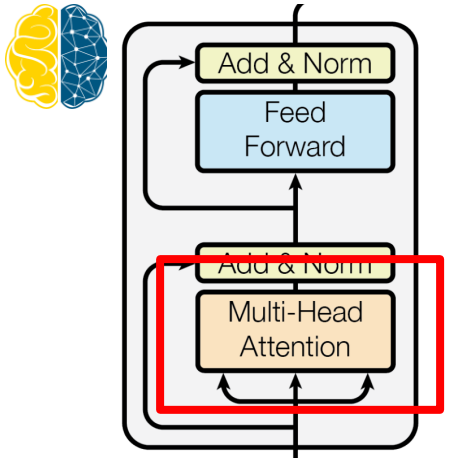
Attention Block



The	big	red	cat .
E	E	E	E

Input representation

Attention Block



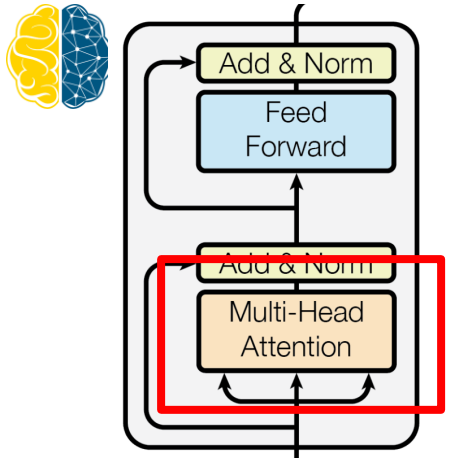
The big red cat.

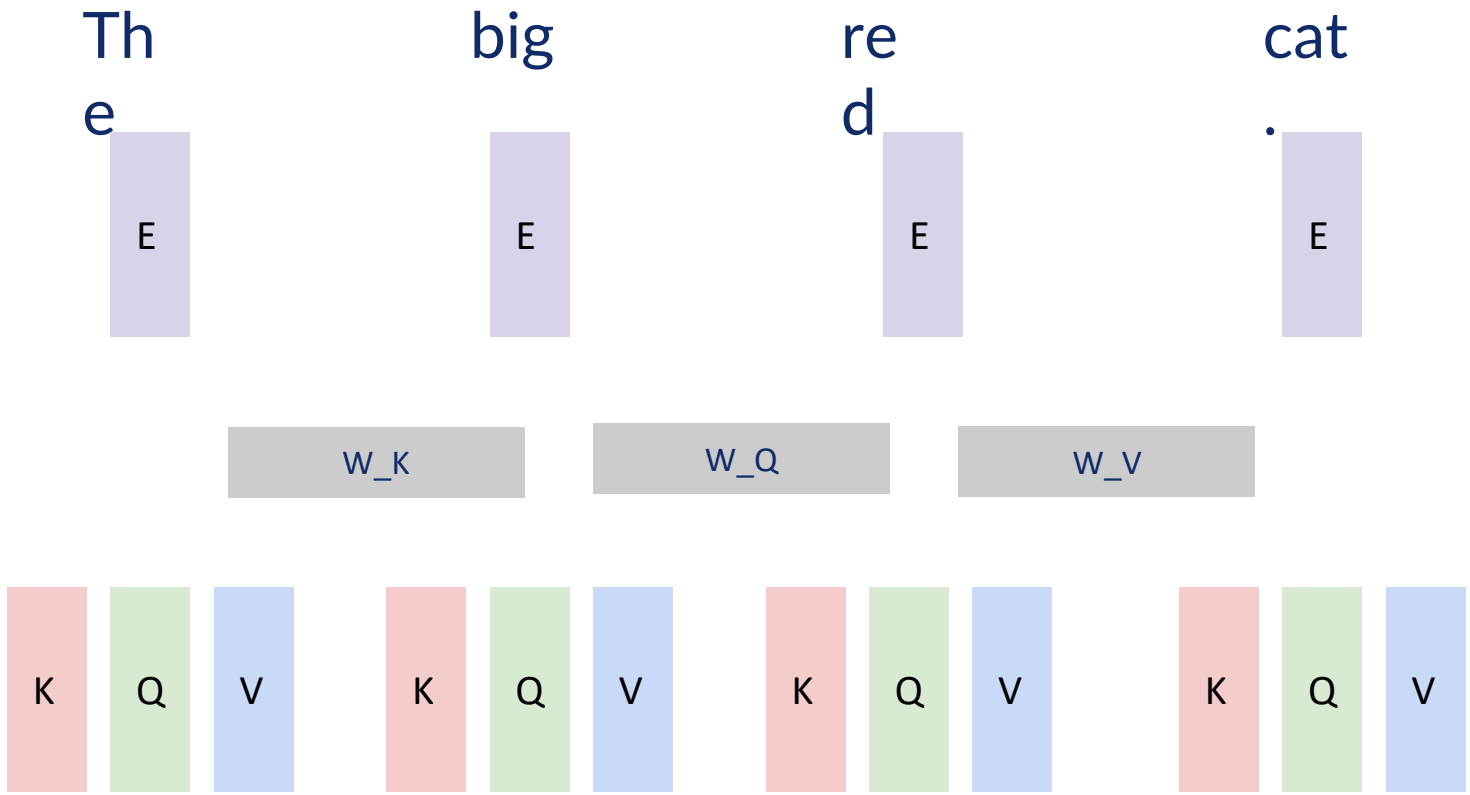
E E E E

W_K W_Q W_V

Apply linear layer to get K Q V from input embeddings

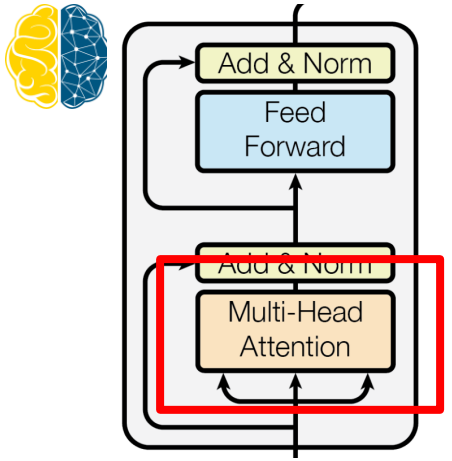
Attention Block





Apply linear layer to get K Q V from input embeddings

Attention Block



The big red cat .

E

K Q V

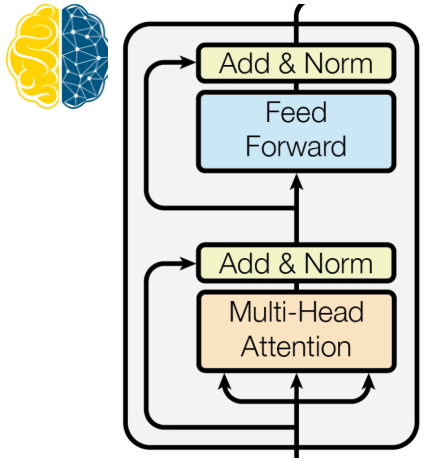
K V

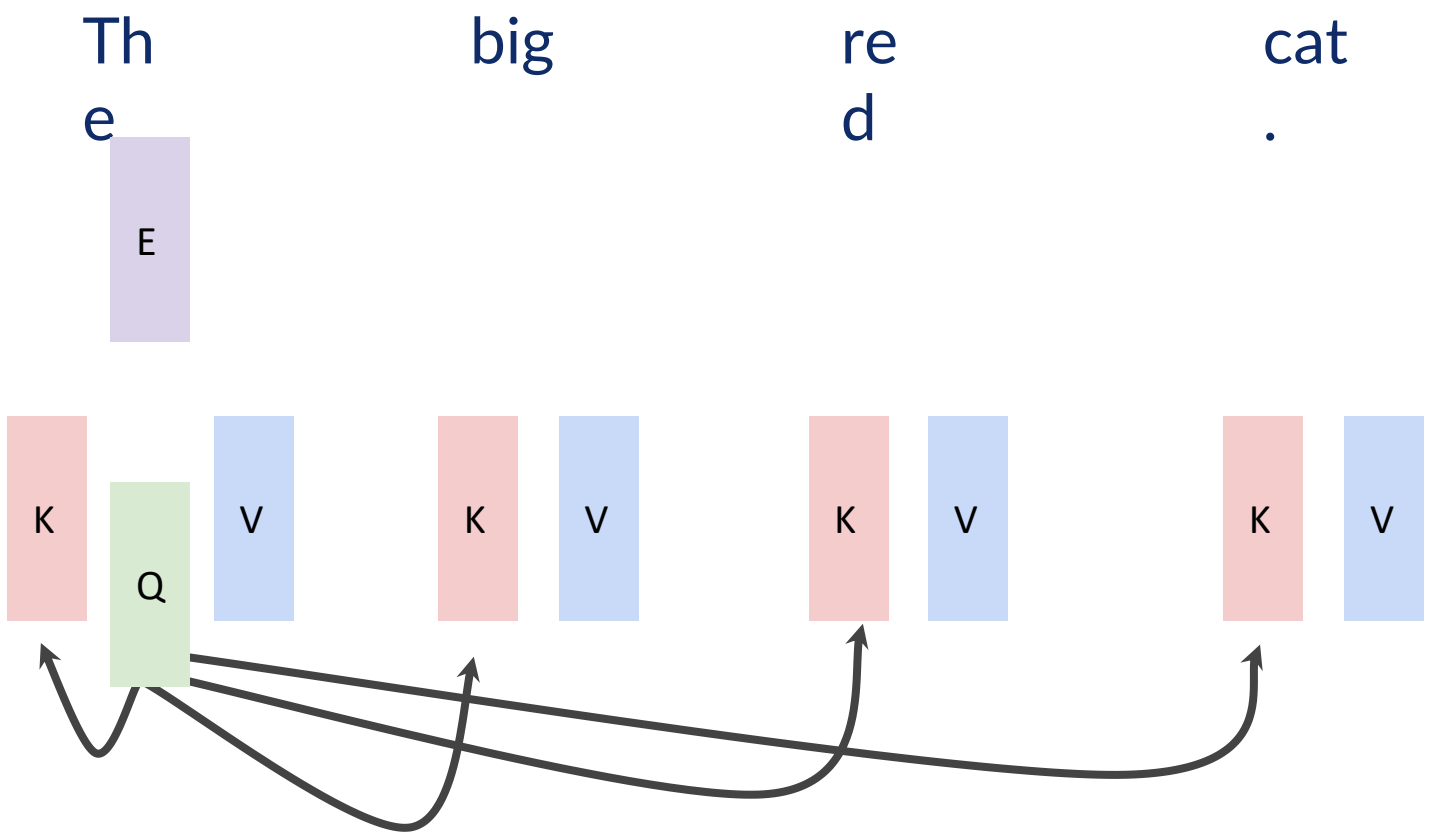
K V

K V

Let's just look at how to get the output representation of "The"

Attention Block



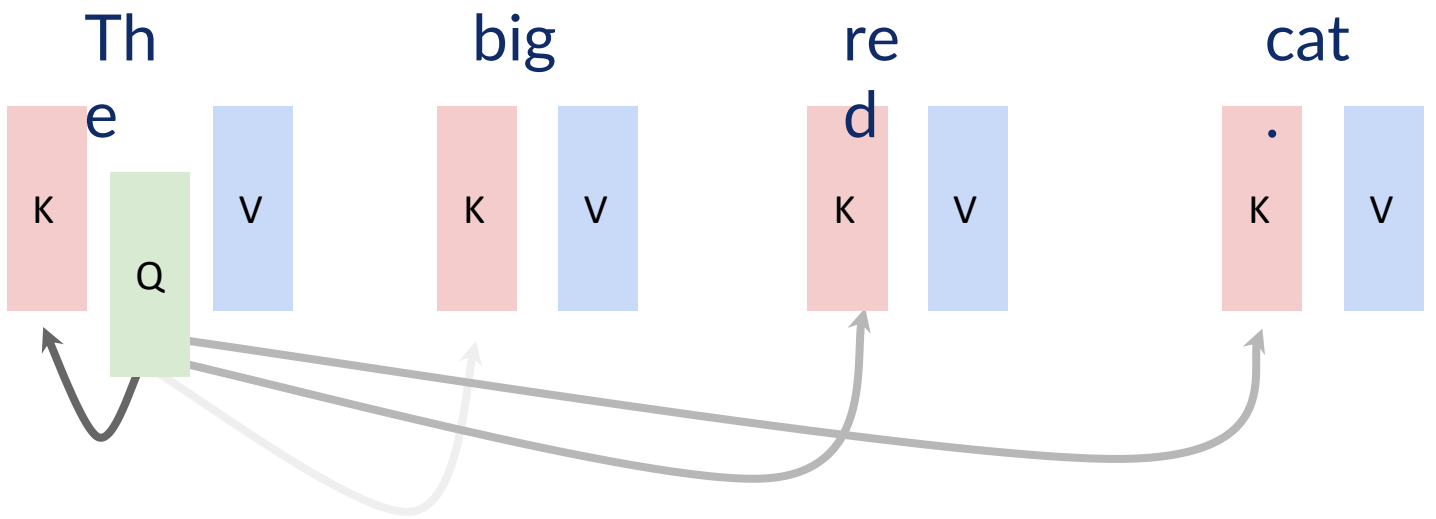


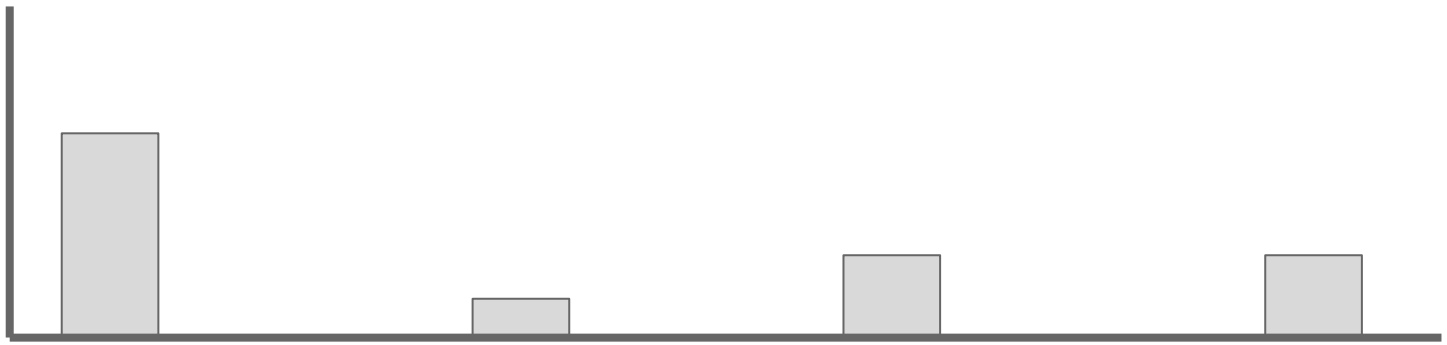
Dot product between Q and Ks

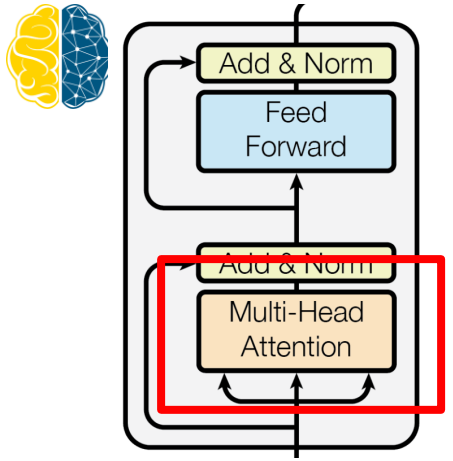
Find the most relevant token based on Q

NOTICE: Each token directly attends to every other token -> connections between arbitrarily long inputs is O(1)!

Attention Block

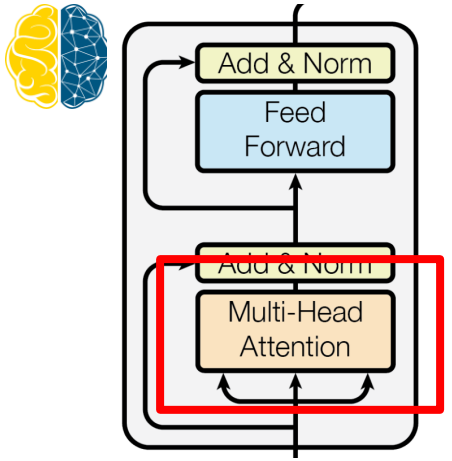


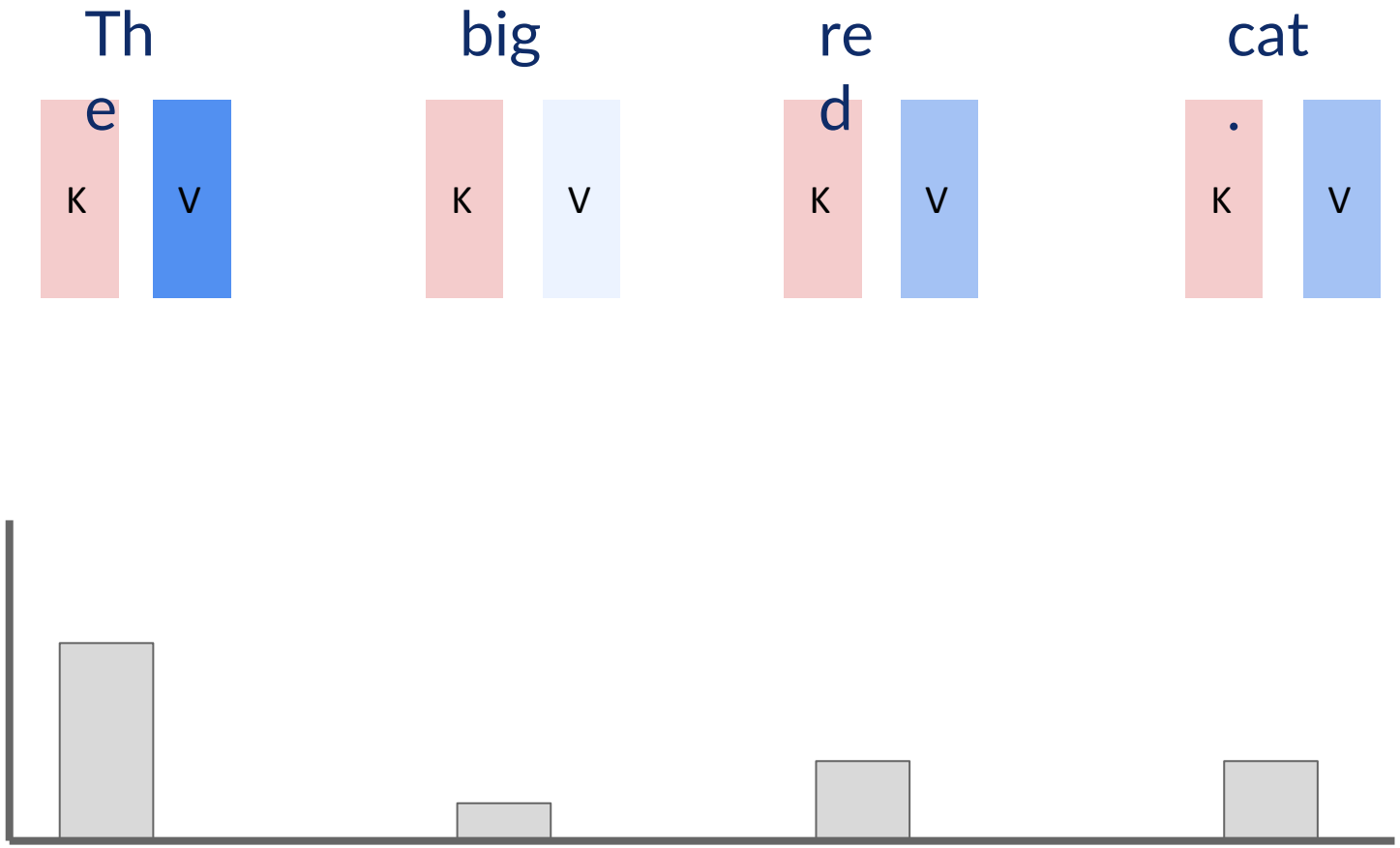




Calculate softmax distribution based on dot product values

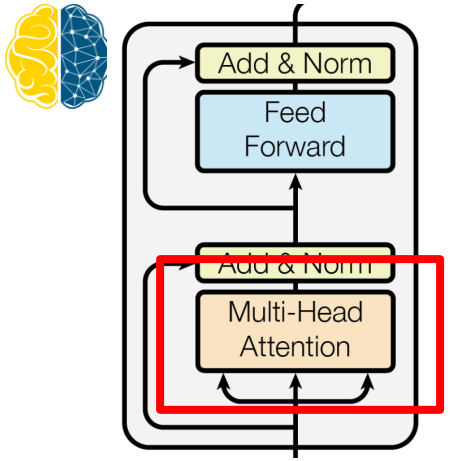
Attention Block

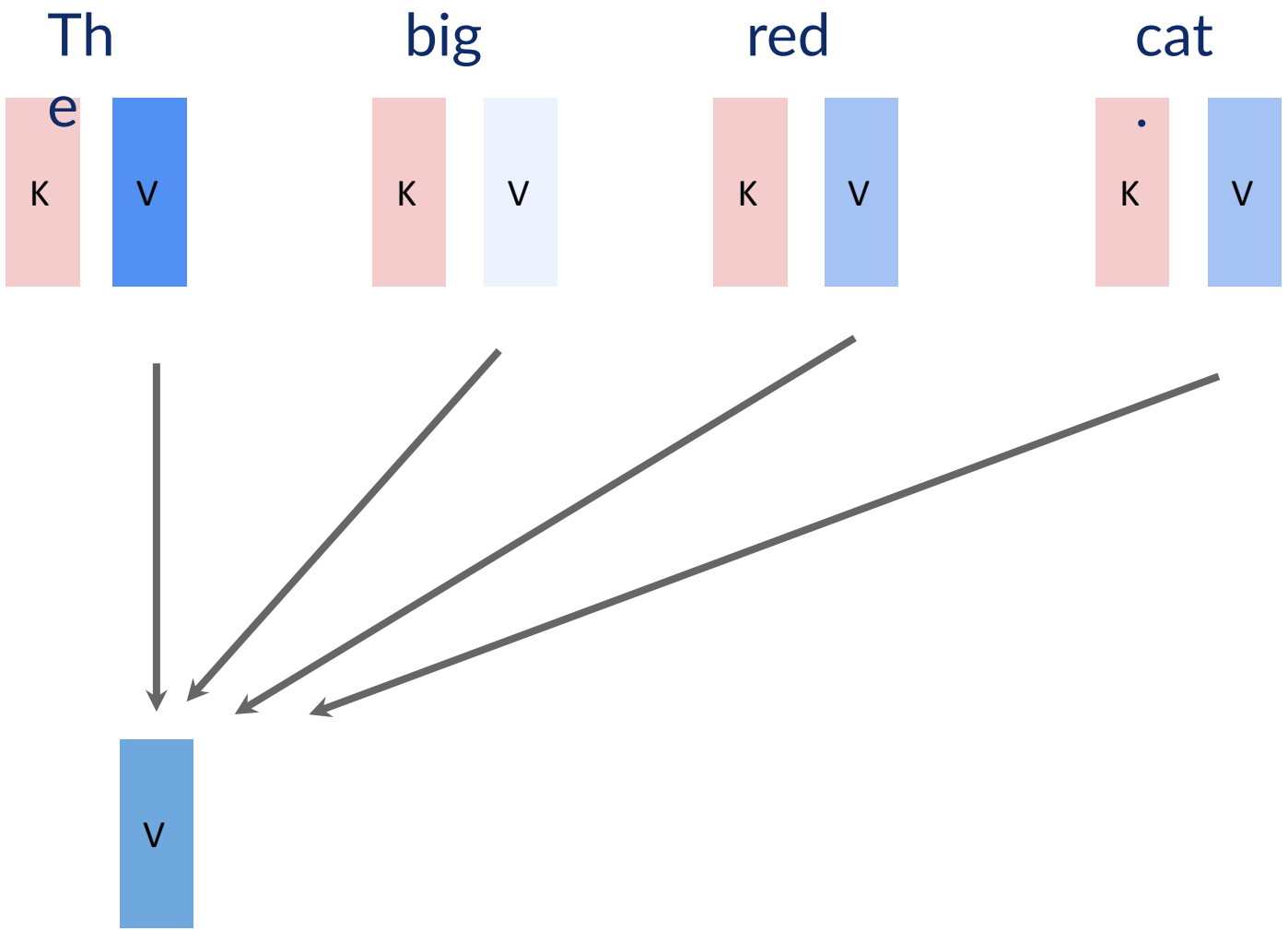




Reweigh Vs based on softmax distribution

Attention Block





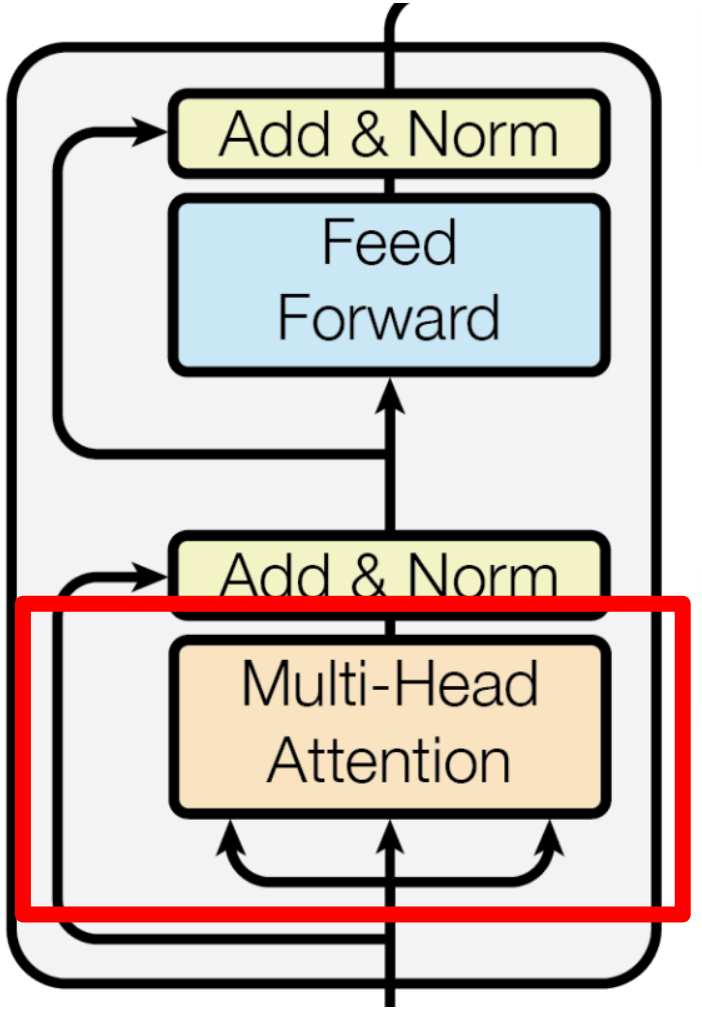
Sum reweighed Vs

Масштабированное внимание

Масштабируйте скалярные произведения в 1/sqrt(d_k) раз.

d_k - размерность эбединга ключа.

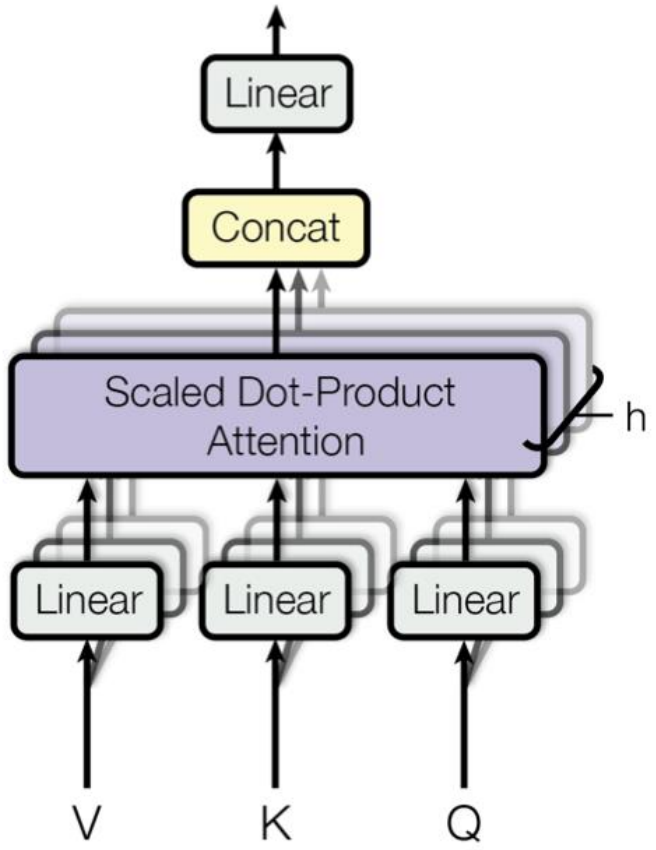
Помогает, поскольку значения точечных произведений увеличиваются с ростом размерности → некоторые компоненты softmax будут иметь очень маленькие градиенты



Многоголовое внимание

Проецируйте каждый триплет (K, Q, V) вниз h раз с помощью различных проекционных матриц

- h - количество «голов».
- Параллельно обработайте каждую «голову» независимо, переконкатенируйте все значения h и повторно спроецируйте их, чтобы получить окончательное вложение

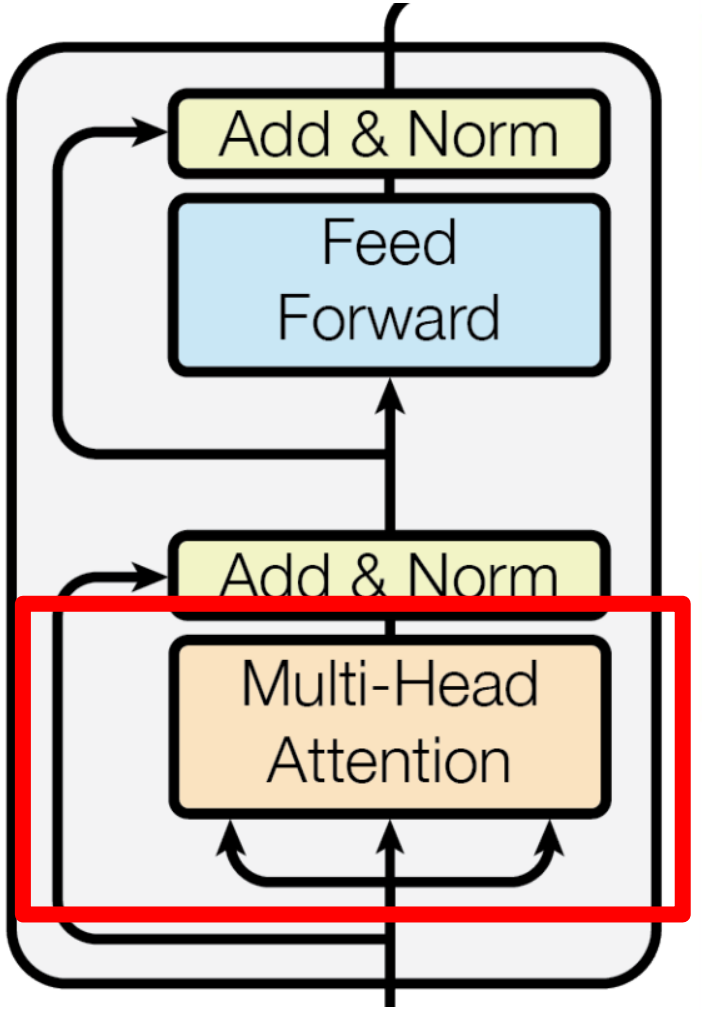


Многоголовое внимание

Проецируйте каждый триплет (K, Q, V) вниз h раз с помощью различных проекционных матриц

- h - количество «голов».
- Параллельно обработайте каждую «голову» независимо, переконкатенируйте все значения h и повторно спроецируйте их, чтобы получить окончательное вложение

Несколько голов позволяют моделировать различные представления одного и того же токена (поскольку он может присутствовать на разных наборах токенов) и рассматривать их все вместе



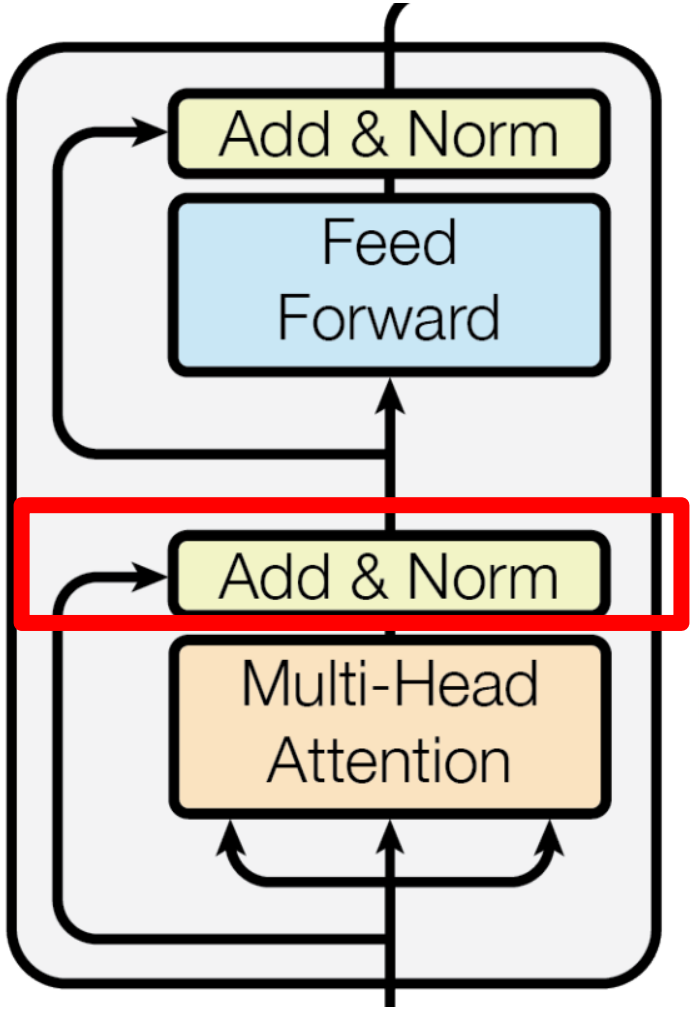
Add и Norm

Пропускающие соединения (add)

- Для устранения исчезающих градиентов в моделях с большим количеством слоев
- Вспомните мотивацию для ResNets

Нормализация слоев (Norm)

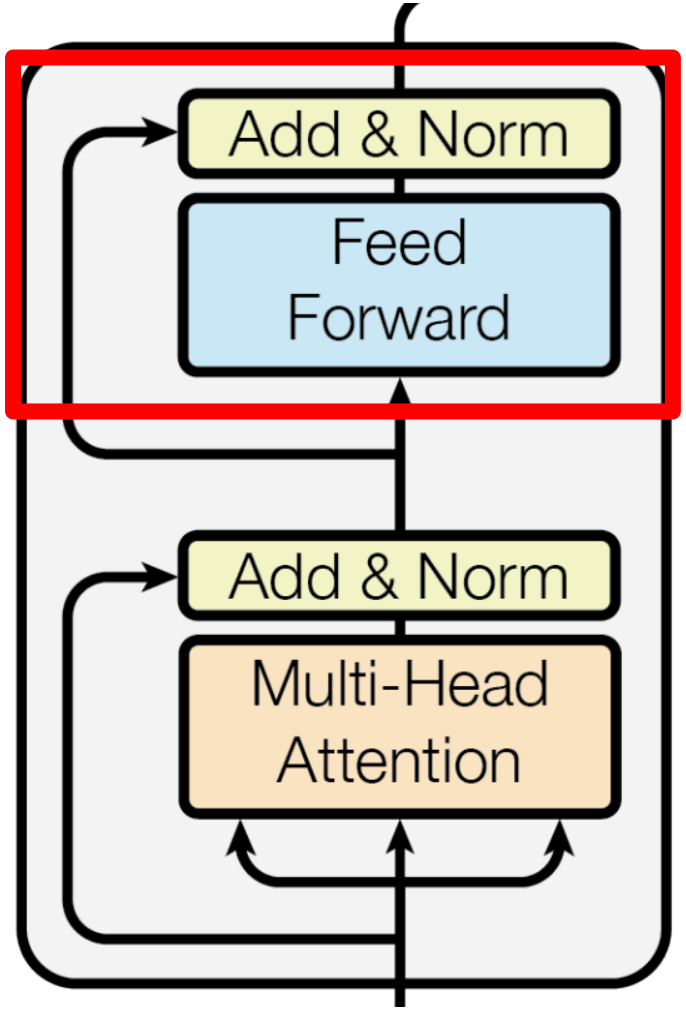
- Пакетная нормализация, но статистика вычисляется для каждой выборки, относительно параметров в каждом слое



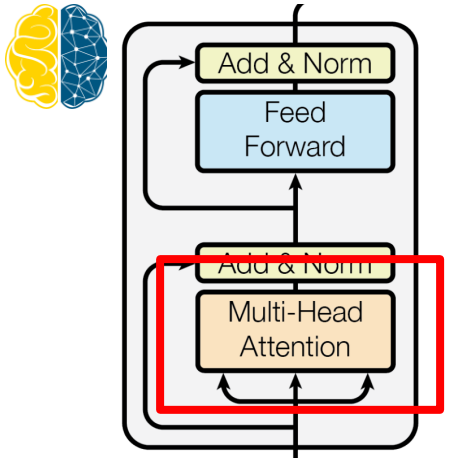
Полносвязный блок

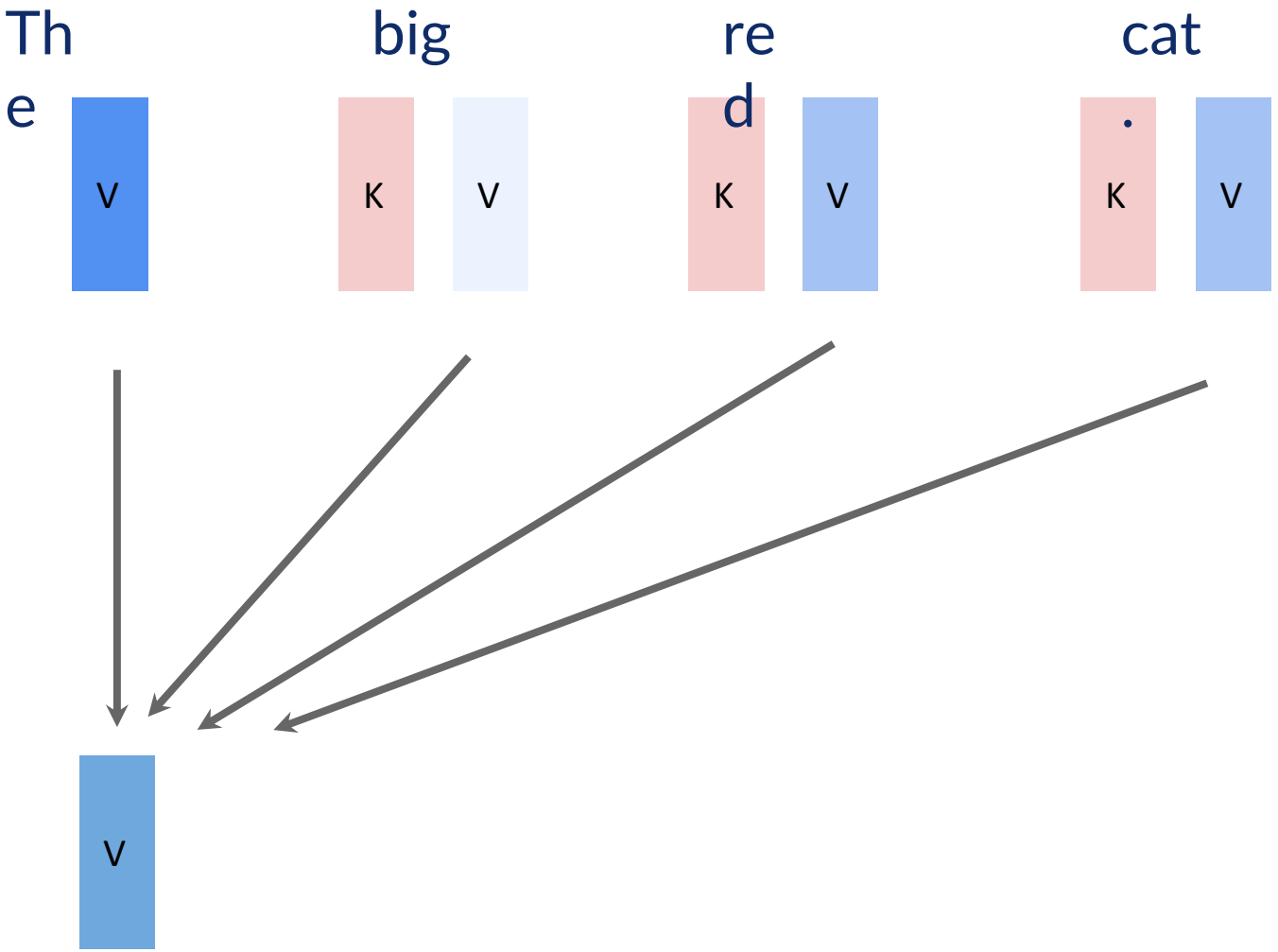
Позиционные сети с прямой передачей данных

- Двухслойные MLP (веса распределены в зависимости от положения), применяемые (индивидуально) к каждому вкраплению лексем
- Внимание - это просто «перекомбинирование» входных векторов
  - MLP добавляет нелинейности
  - MLP добавляет линейные слои для «смешивания» значений каналов

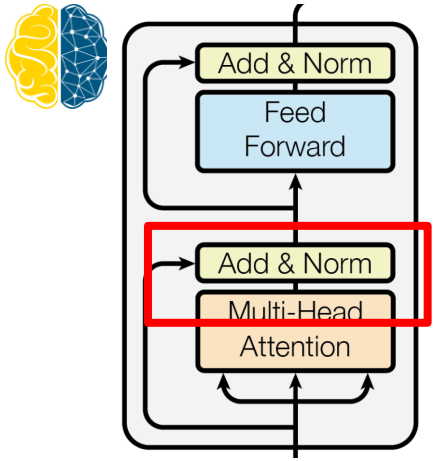


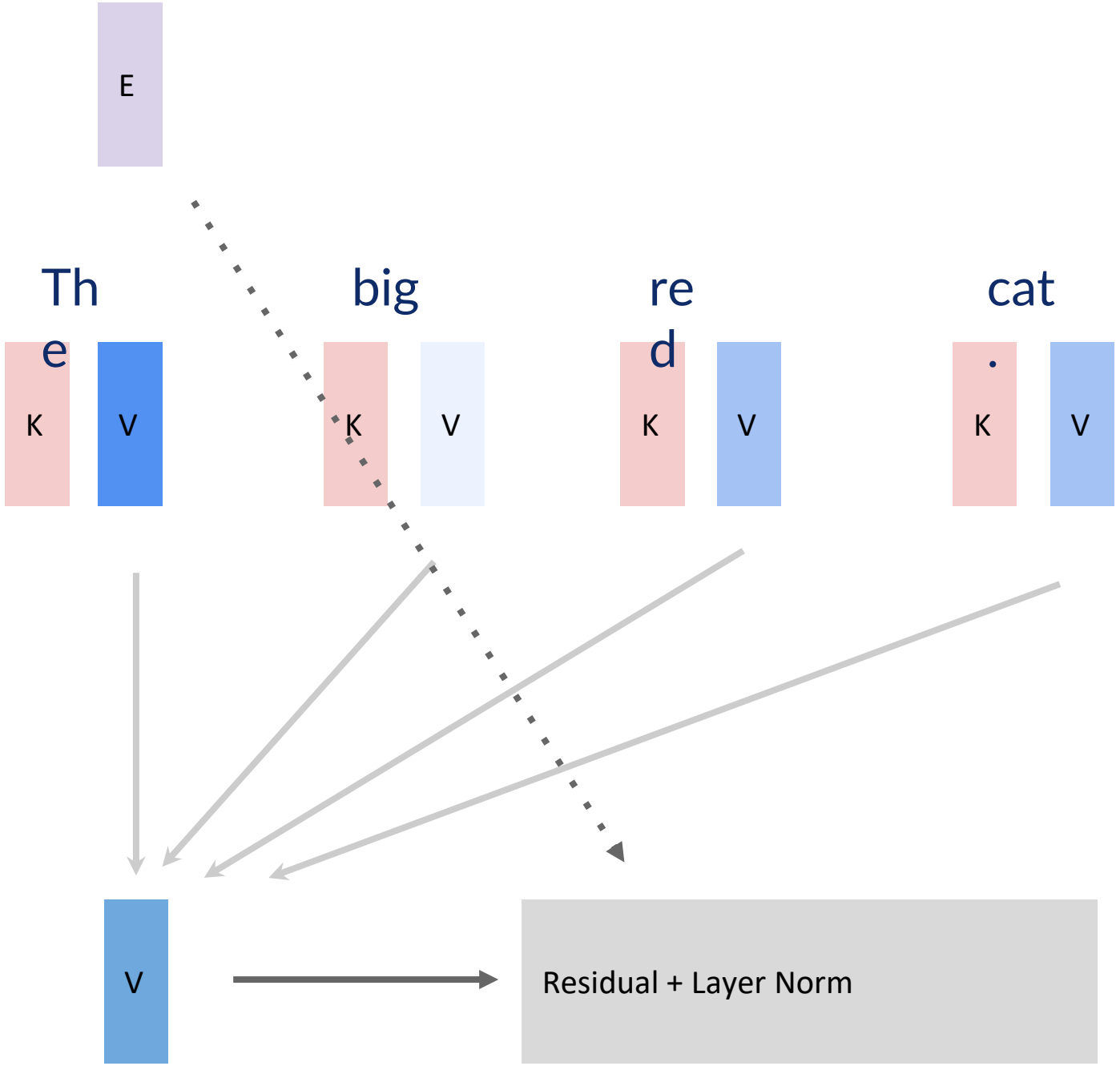
Attention Block





Sum reweighed Vs

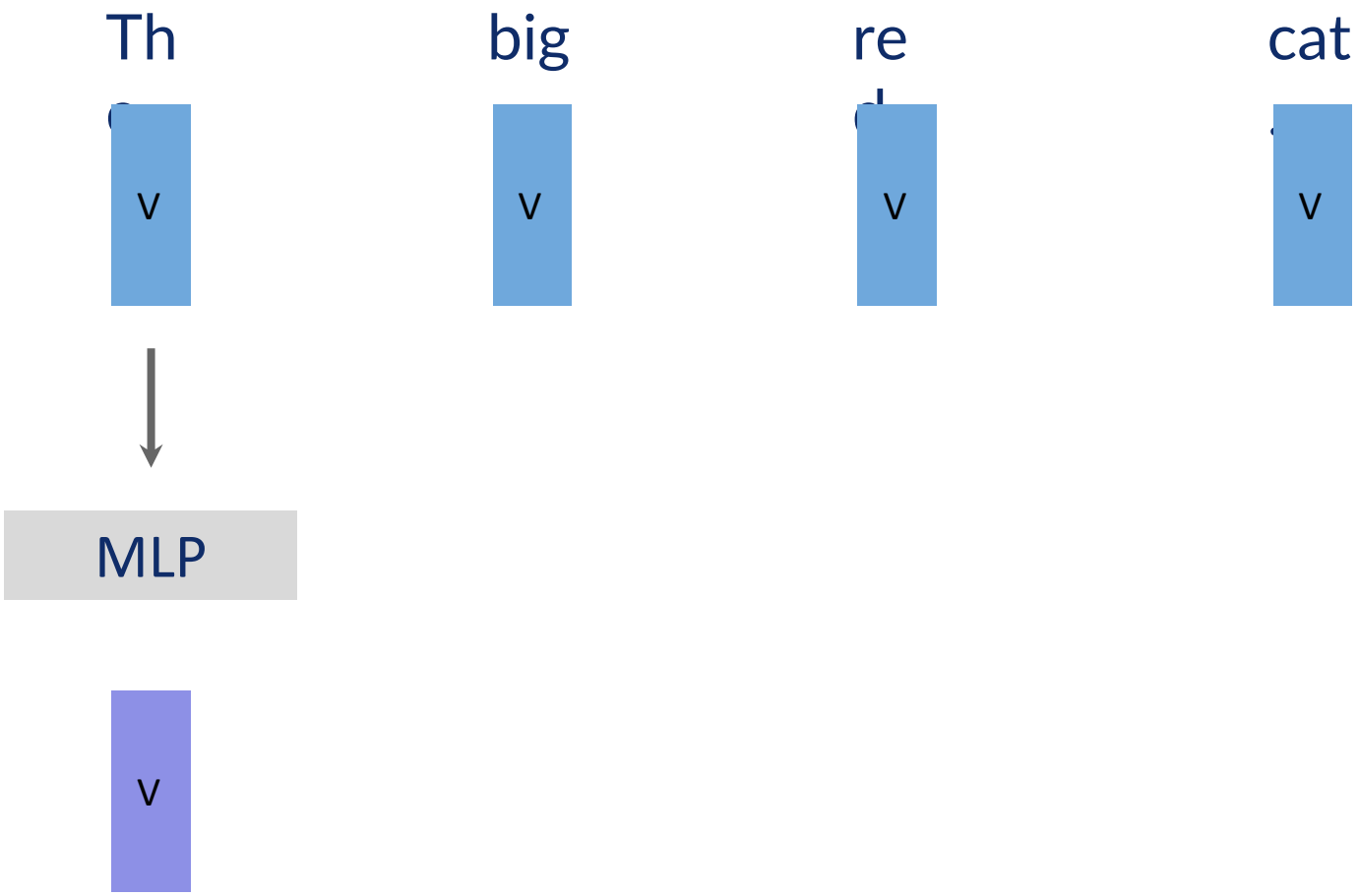


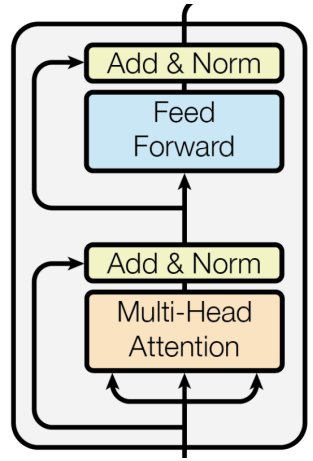


Combine with input embeds (Residual)

Layer Normalization

Attention Block

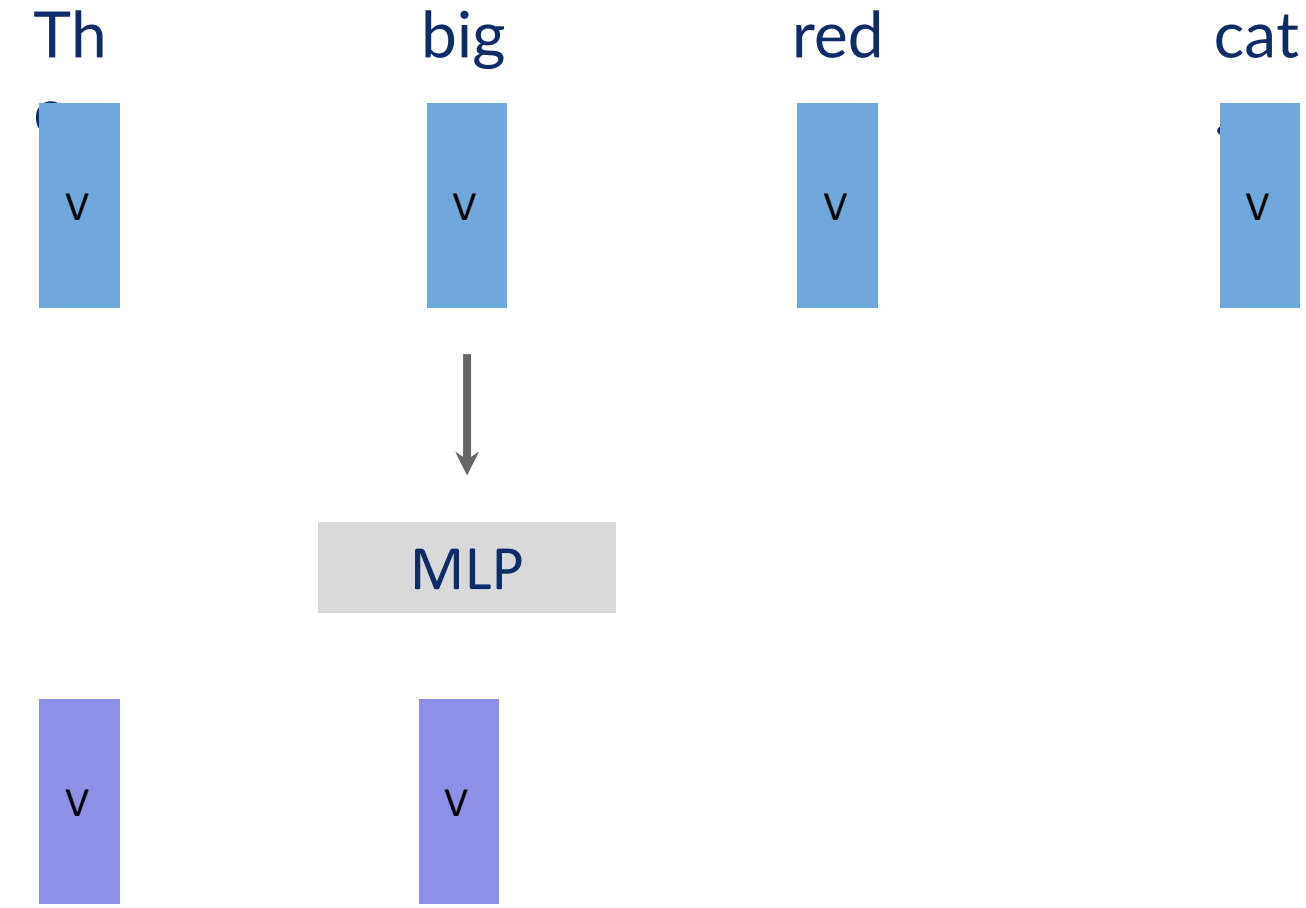


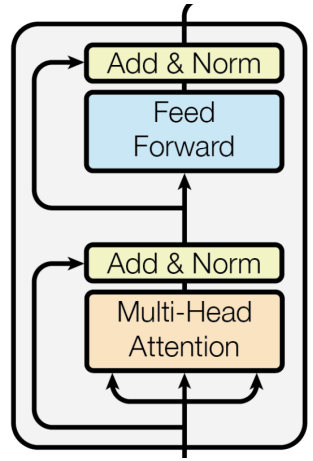


Representations after Multi-Head Attention and Residual + Layer norm

Position-wise feedforward/ 1x1 convolution

Attention Block

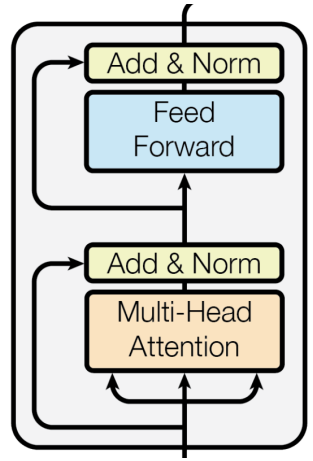


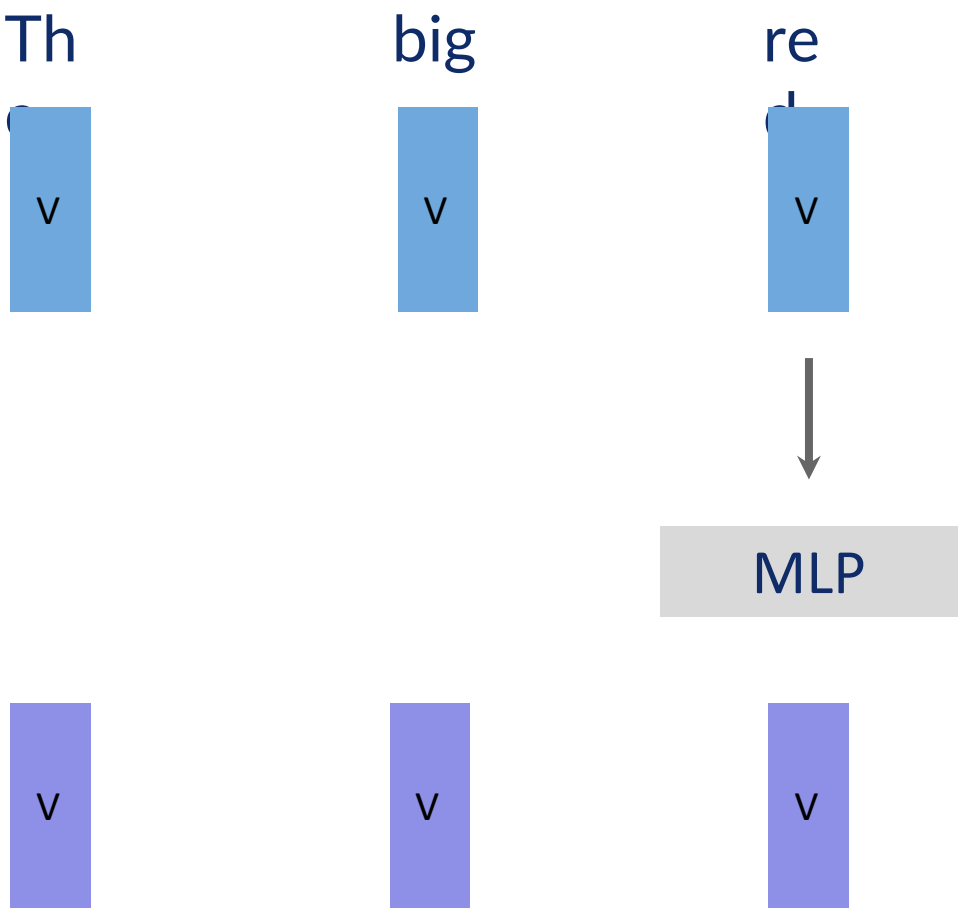


Representations after Multi-Head Attention and Residual + Layer norm

Position-wise feedforward/ 1x1 convolution

Attention Block

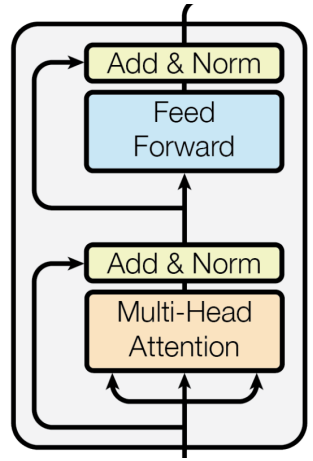




Representations after Multi-Head Attention and Residual + Layer norm

Position-wise feedforward/ 1x1 convolution

Attention Block



Th big re cat

v v v v

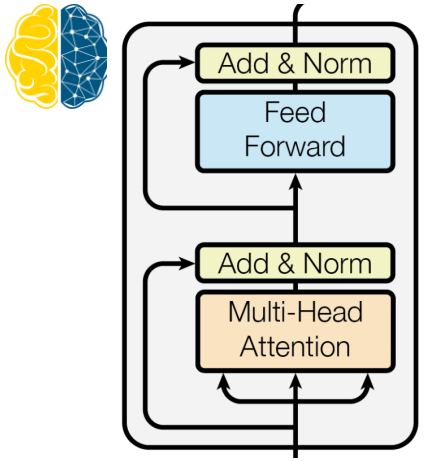
MLP

v v v v

Representations after Multi-Head Attention and Residual + Layer norm

Position-wise feedforward/ 1x1 convolution

Attention Block



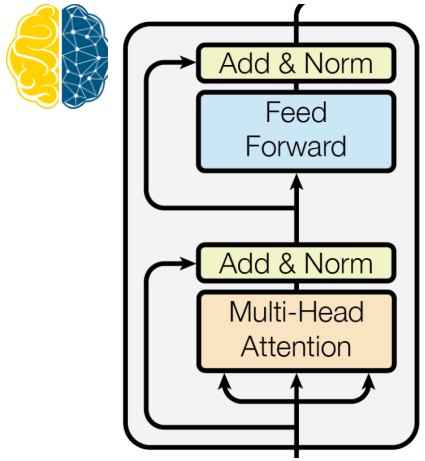
The big red cat .

v v v v

Residual + Layer Norm

E E E E

One more Residual + Layer norm to get transformer layer output!



V V V V

Th big re cat

d .

V V V V

Residual + Layer Norm

E E E E

Combine with MLP input (Residual)

Layer Normalization

Позиционные эмбеддинги

Обратите внимание: архитектура, описанная до сих пор, не является последовательной по своей сути!

- Все связано со всем остальным

Решение: добавьте к входным данным позиционные эмбеддинги, чтобы каждый токен мог отслеживать свой индекс в последовательности.

- Решение 1: синусоидальные эмбеддинги (вектор, основанный на фиксированном уравнении)
- Решение 2: выученные эмбеддинги

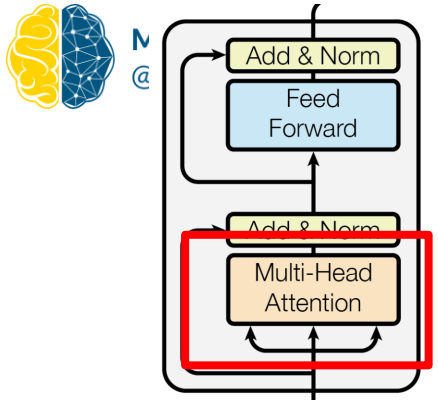
CLS Token

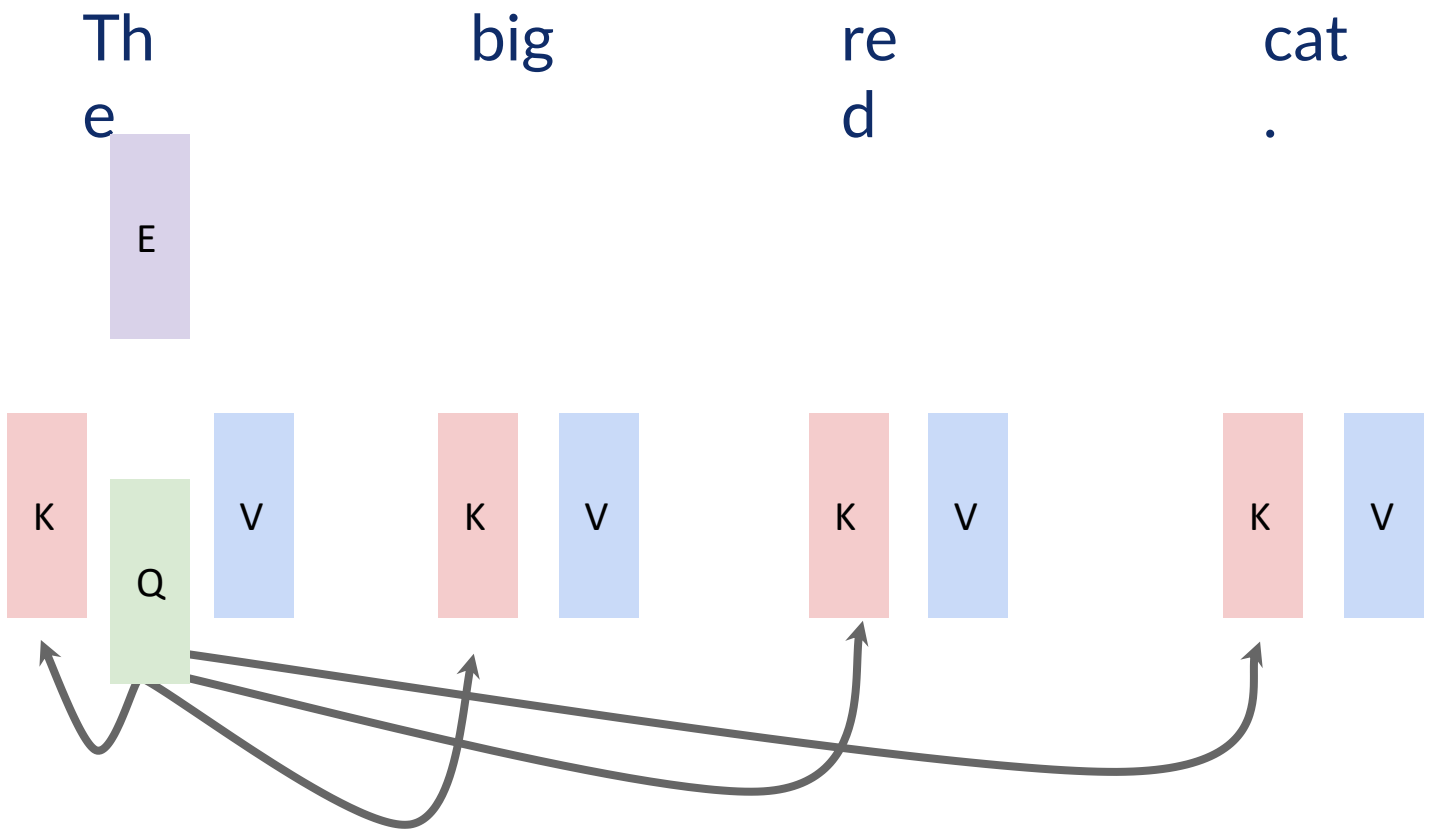
Часто к началу входного сигнала прикрепляется фиктивная лексема

Используйте выходное представление этого токена для классификации/других последующих задач.

- Например, передать его в сеть прямой передачи для предсказания:
  - позитивный/негативный настрой
  - следующий токен для языкового моделирования
  - и т. д.

The Math



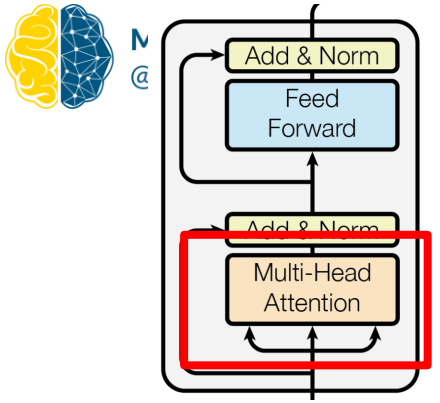


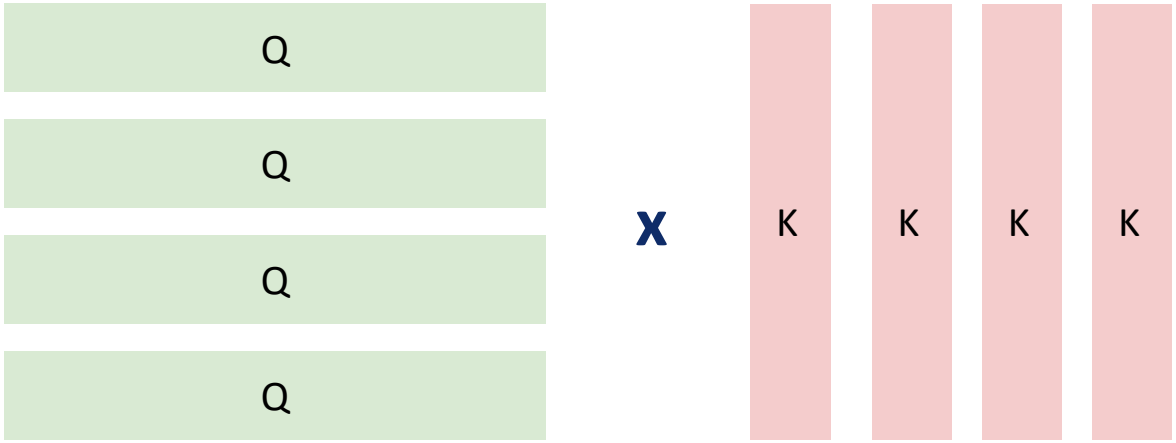
Dot product between Q and Ks

Let **Q_input**, **K_input**, **V_input** be represent Q, K, V for *each* input token where each entry of Q, K, V is a row in their respective matrix

If we multiply **Q_input** * **K_input**^{T}, we're just doing dot product attention for each token!

The Math

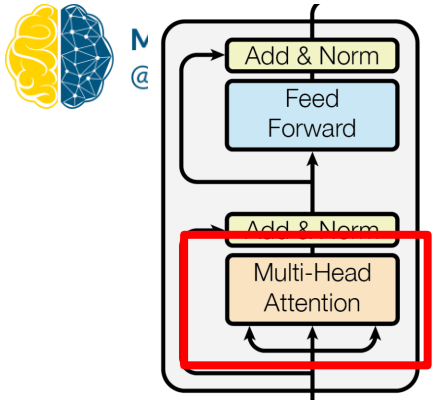


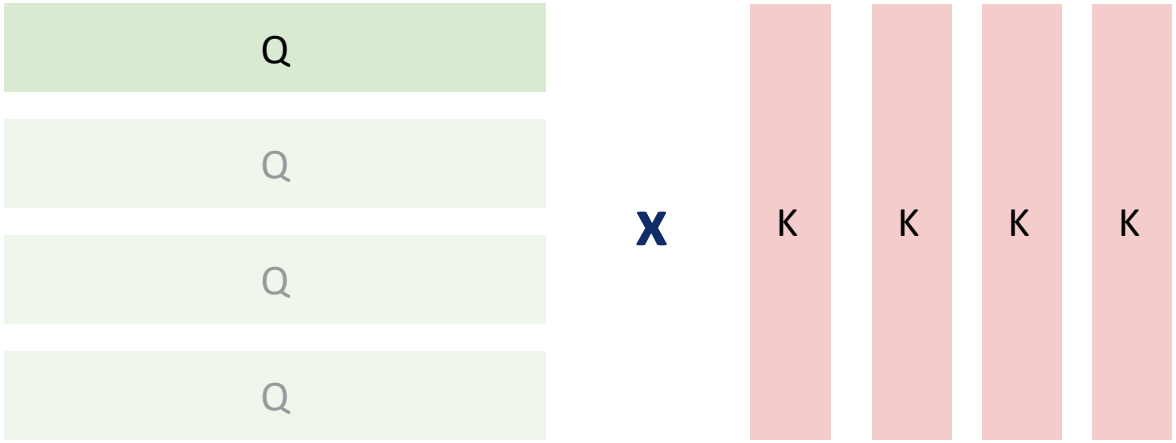


Let **Q_input**, **K_input**, **V_input** be represent Q, K, V for *each* input token where each entry of Q, K, V is a row in their respective matrix

If we multiply **Q_input * K_input^T**, we're just doing dot product attention for each token!

The Math

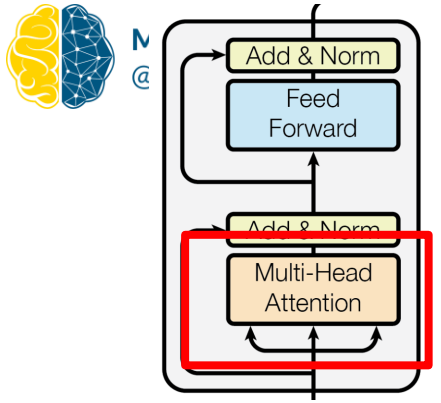




Let **Q_input**, **K_input**, **V_input** be represent Q, K, V for *each* input token where each entry of Q, K, V is a row in their respective matrix

If we multiply **Q_input** * **K_input^T**, we're just doing dot product attention for each token!

The Math



Let *i* be each QK

Q
Q
Q
Q

x

K	K	K	K

=

i	*i*	*i*	*i*
i	*i*	*i*	*i*
i	*i*	*i*	*i*
i	*i*	*i*	*i*

The Math

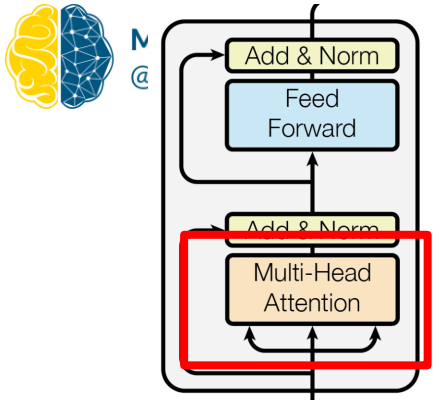
Let *i* be each QK

i	*i*	*i*	*i*
i	*i*	*i*	*i*
i	*i*	*i*	*i*
i	*i*	*i*	*i*

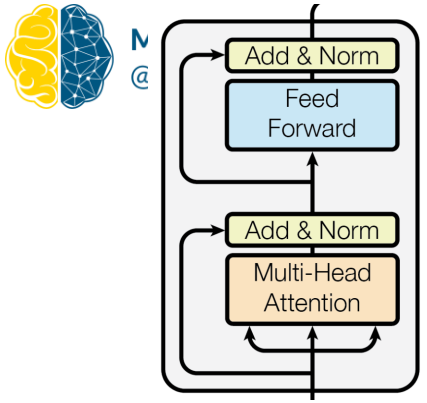
Calculate the weights with softmax

w = softmax(i)

w	*w*	*w*	*w*
w	*w*	*w*	*w*
w	*w*	*w*	*w*
w	*w*	*w*	*w*



The Math



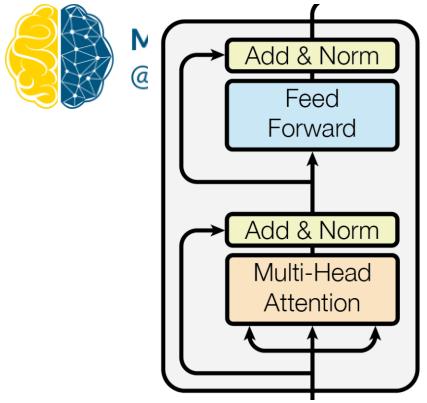
w	w	w	w
w	w	w	w
w	w	w	w
w	w	w	w

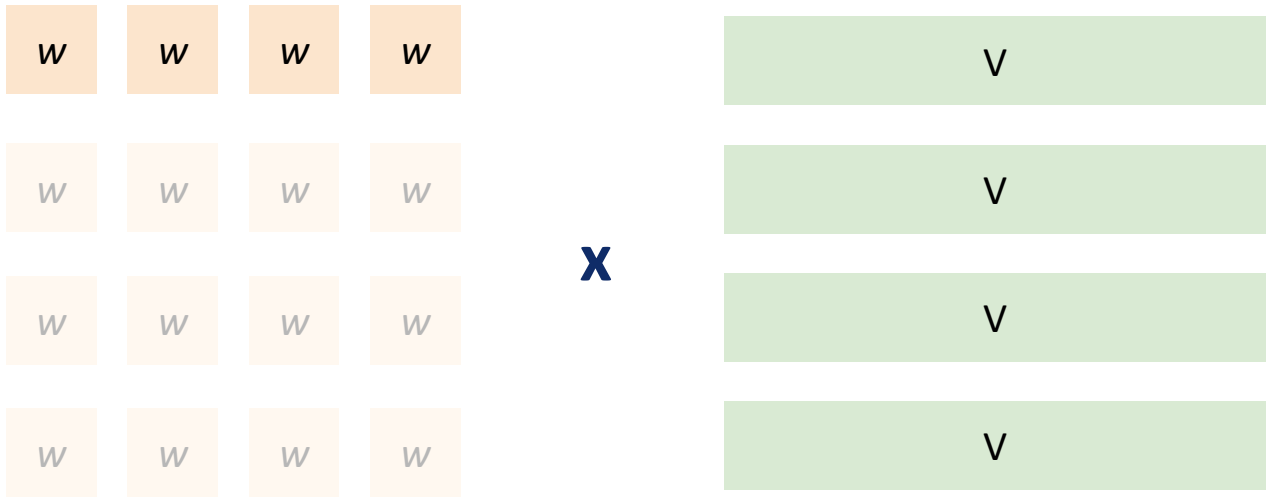
x

v
v
v
v

We can just do another matrix multiply between the weights matrix and **V_inputs** to get the weighted sum!

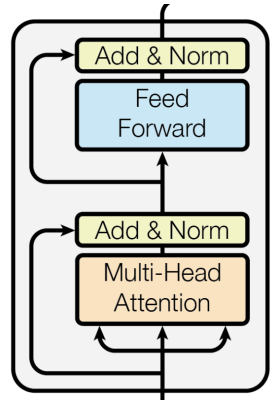
The Math





We can just do another matrix multiply between the weights matrix and **V_inputs** to get the weighted sum!

The Math



w	w	w	w
w	w	w	w
w	w	w	w
w	w	w	w

x

V
V
V
V

=

V
V
V
V

The Math

This is just

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Or

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\textbf{where } \text{head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Notice: Attention is just really large matrix multiplications!

The Math

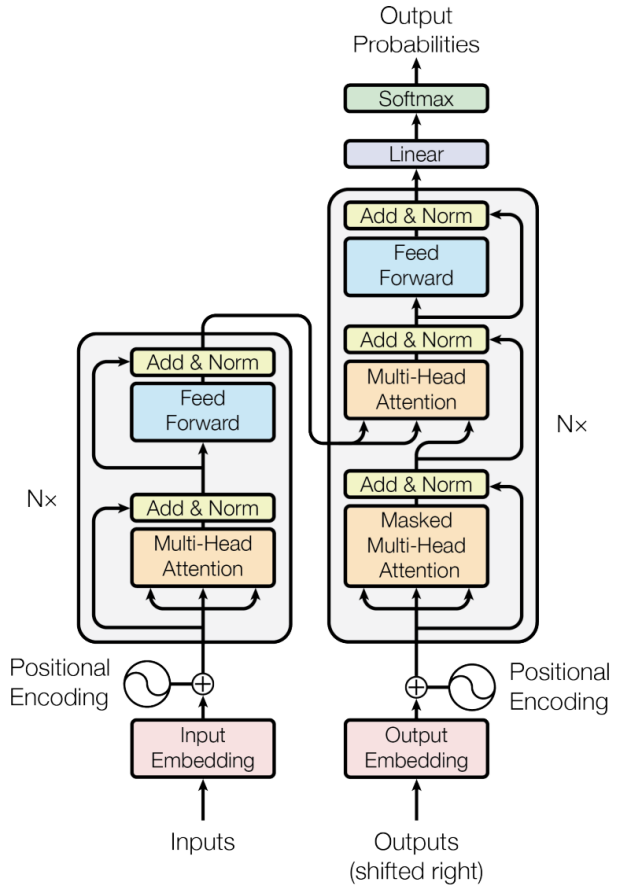


MACHINE LEARNING
@ BERKELEY

Разные архитектуры

Как и другие модели последовательности, вы можете иметь

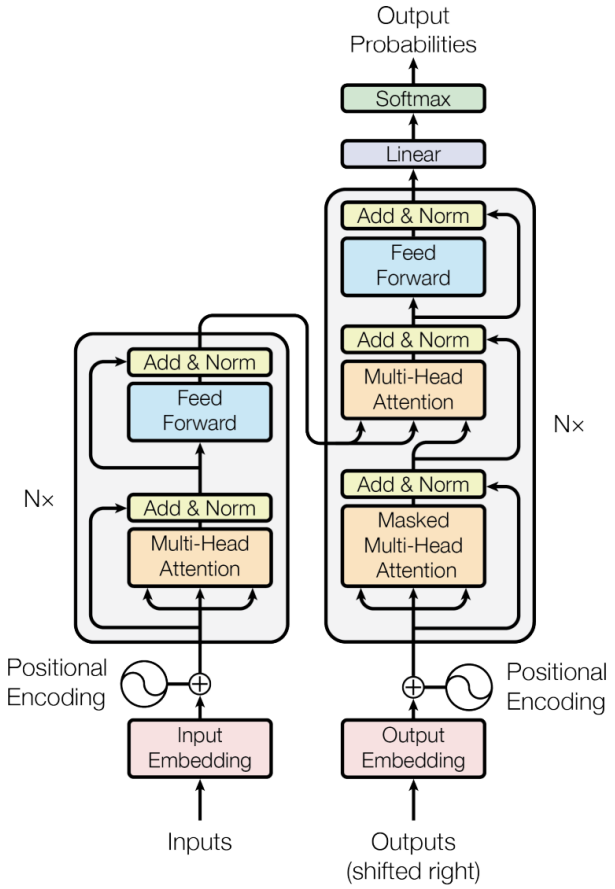
- Модели кодировщиков
  - Просто производят вкрапления из входных токенов
  - например. BERT
- Модели-декодеры
  - Авторегрессионная генерация слева направо
  - Осторожно с вниманием: нельзя использовать будущие лексемы для декодирования текущей лексемы во время обучения
    - Декодер не будет знать будущего во время вывода.
    - Чтобы игнорировать будущие лексемы во время обучения, используйте так называемое маскирующее внимание.
  - например. GPT, другие LLM



Разные архитектуры

Как и другие модели последовательности, вы можете иметь

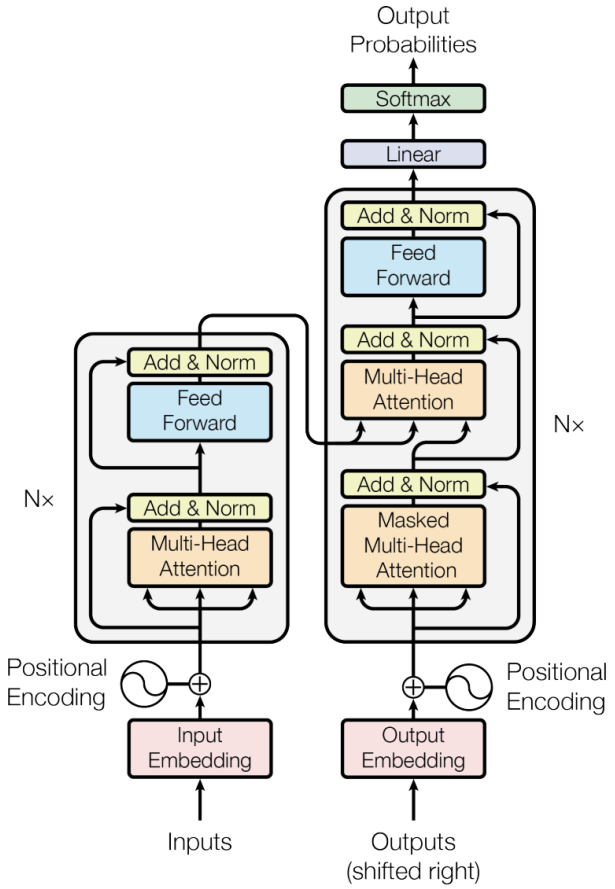
- Модели кодировщика-декодировщика
  - Модели Seq2seq
  - Перекрестное внимание
    - Запросы поступают с предыдущего слоя декодера, а ключи и значения - с выхода кодера
    - Вспомните механизмы внимания в моделях RNN.
  - например. T5



Резюме по трансформерам

Пересмотр проблем, которые мучили наши старые добрые RNN.

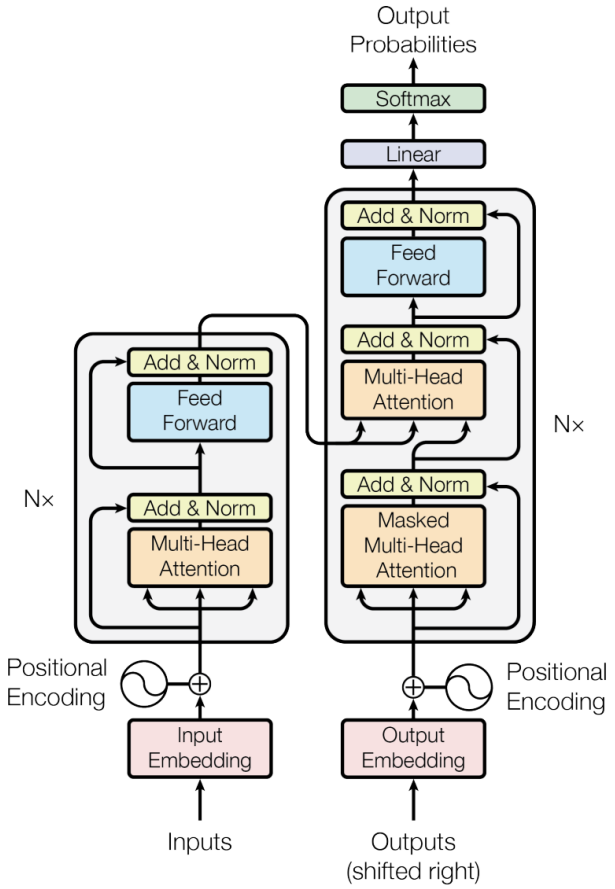
- Проблемы с моделированием длинных последовательностей?
  - Все относится ко всему в трансформаторе
  - Означает постоянные операции для произвольного расстояния между лексемами
- Последовательность → нераспараллеливаемость?
  - Матричные операции!



Резюме по трансформерам

Трансформеры - это очень общая архитектура.

- Даже не последовательная по своей сути!
- Может быть применена и к компьютерному зрению!
  - Мы увидим это на следующей лекции.
- Не кодирует очень много индуктивных предубеждений
  - Т.е. не предполагает ничего особенного в данных.
  - В отличие от RNN/CNN, которые навязывают последовательность/трансляционную инвариантность



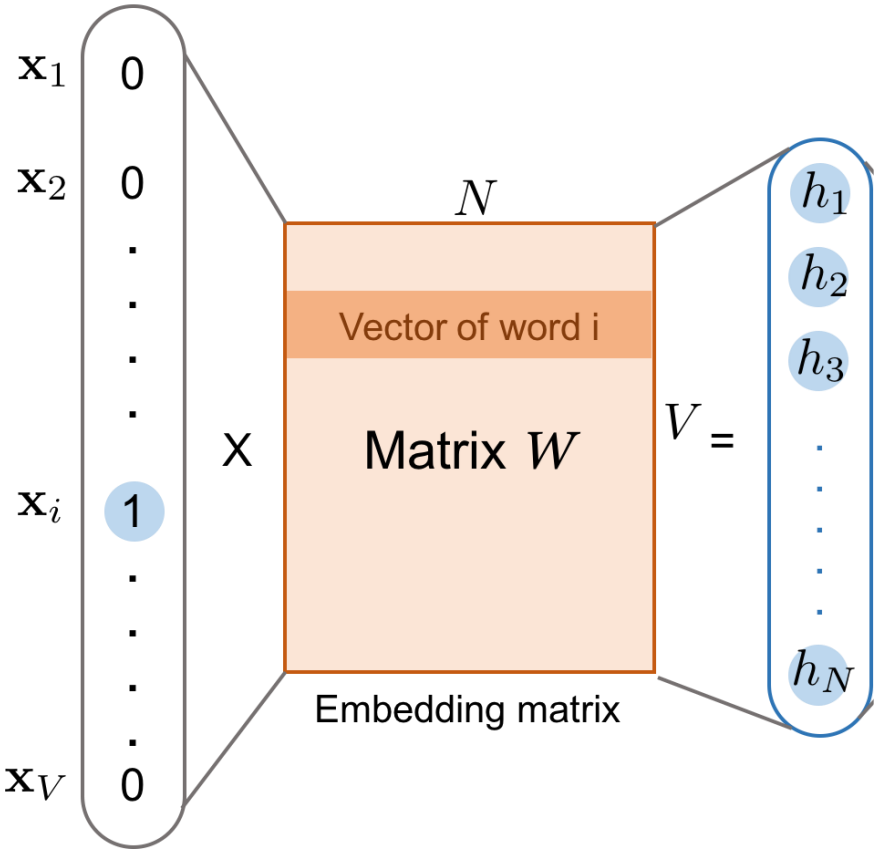
Эмбеддинги

Текстовые эмбеддинги

- Некоторые данные могут быть высокоразмерными
  - One-hot вектор для языка может быть очень высокоразмерным для больших словарей.
- Можно ли получить лучшее, более компактное представление?
  - Можно попробовать представить каждое слово в двоичном виде (представить каждое слово как вектор из 0 и 1 в соответствии с числом i в двоичном виде), а не в виде one-hot для компактности.
    - Можно даже закодировать с основанием 3 (троичное) вместо основания 2 (двоичное), и т. д., и т. п.
  - One-hot обычно полезен для отдельных классов, которые должны рассматриваться как ортогональные друг другу, но разные слова могут быть более или менее похожими (синонимы). Должно ли наше представление учитывать и это?
  - Вариантов много, но что будет лучше?

Текстовые эмбеддинги

- Просто выучите его!
- Вместо того чтобы вводить в модель на каждом шаге one-hot вектор для каждой лексемы, выбирайте соответствующий вектор из обучаемых.
- Мы можем выбрать предварительно обученные вкрапления (например, Word2Vec) или обучить эти векторы из end-to-end с помощью градиентного спуска.
- Последний вариант дает нам больше контроля над степенью сжатия, поскольку мы можем сами задавать размер этих эмбеддингов



Эмбеддинги

- Основная идея заключается в том, что мы учимся уменьшать размерность наших лексем для каждого элемента последовательности.
- Мы можем ожидать, что наши встроенные входные данные будут более интеллектуальными в своем представлении, поскольку мы можем ожидать, что наша модель будет сжимать их по своему усмотрению.

Позиционные эмбеддинги

Позиционные эмбеддинги

- Время имеет значение!
- Не все модели кодируют позиционную информацию последовательностей, но знание индекса лексемы в ее последовательности может иметь значение в некоторых случаях.
  - Это неважно для RNN.
  - Но станет очень важным, когда мы будем обсуждать трансформеры в следующий раз.
- Нам нужны позиционные кодировки, которые:
  - Выводили уникальную кодировку для каждого временного шага
  - Расстояние между двумя временными шагами было одинаковым для предложений разной длины.
  - Обобщались на более длинные предложения без каких-либо усилий
  - Они должны быть детерминированными.

Позиционные эмбеддинги

- По мере развития двоичного кодирования вы можете видеть, как каждая позиция переключается туда-сюда.
- Обратите внимание, что LSB быстро переключается туда-сюда, а старшие биты - медленно.
- Как мы можем использовать непрерывные числа, чтобы имитировать такое поведение перелистывания?

0 :	0	0	0	0	
1 :	0	0	0	1	
2 :	0	0	1	0	
3 :	0	0	1	1	
4 :	0	1	0	0	
5 :	0	1	0	1	
6 :	0	1	1	0	
7 :	0	1	1	1	
8 :	1	0	0	0	
9 :	1	0	0	1	
10 :	1	0	1	0	
11 :	1	0	1	1	
12 :	1	1	0	0	
13 :	1	1	0	1	
14 :	1	1	1	0	
15 :	1	1	1	1	

Позиционные эмбеддинги

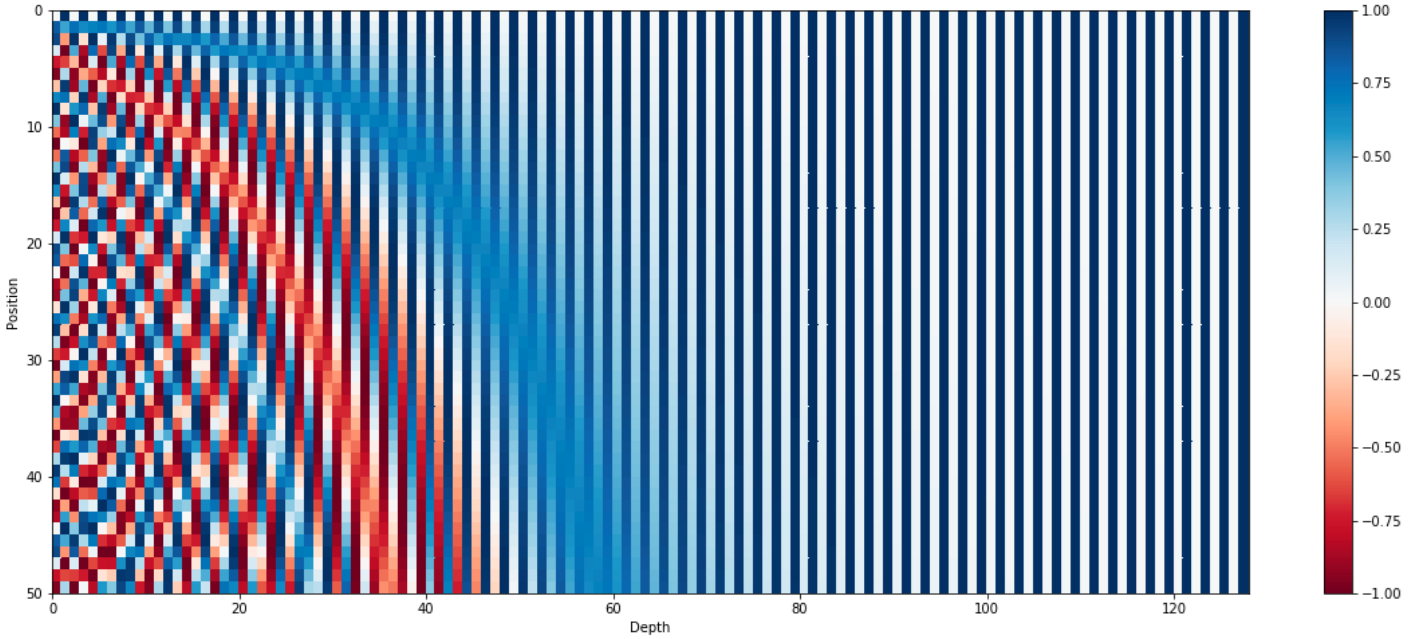
- Мы можем использовать синусоидальные и/или косинусоидальные волны!
- Раньше каждая следующая запись вектора переворачивалась вперед-назад с половинной скоростью.
- Просто замените каждую запись вектора синусоидальными и/или косинусоидальными волнами, которые будут перелистываться вперед-назад с половинной скоростью!
- Если мы хотим иметь возможность обрабатывать вдвое более длинные последовательности, просто заставьте все синусоиды чередоваться с половинной скоростью.
- Размер вектора кодирования менять не нужно!

$$\overrightarrow{p_t} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_{d \times 1}$$

p - вектор позиционного кодирования, w^i - «частота перелистывания».

Позиционные эмбеддинги

128-мерное позиционное кодирование



- Каждая строка - это позиционное кодирование
- Каждый столбец - это наша непрерывная версия 1/0, переключающаяся туда-сюда между -1 и 1

1 изолированная колонка

Это один из индексов позиционного кодирования на разных временных шагах - вы можете заметить, что он меняется взад-вперед, как волна sin/cos

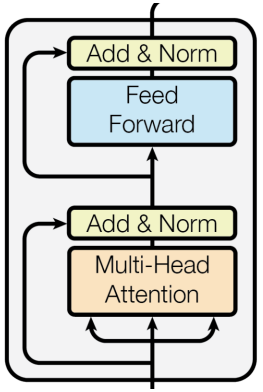
Содержание

1. Как картинки превратить в токены
2. Что на самом деле происходит с токенами дальше?

Мотивация

- Трансформеры хорошо работают с текстом → а что будет, если использовать их для изображений?
- Трансформеры обладают некоторыми приятными свойствами, которые могут быть полезны для компьютерного зрения
  - Например, масштабируемость, глобальные рецептивные поля

Вспоминаем: Архитектура трансформера

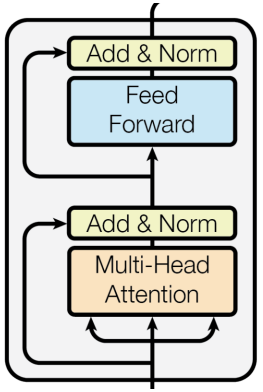


Начните с текстовой строки

1. → текстовые лексемы
2. → эмбеддиги текста (через словарь эмбеддингов)
3. → эмбеддинги текста/позиции
4. → стэк слоёв трансформации (самовнимание + нормализация + пропускающие связи + блоки MLP)
5. → токен CLS
6. → подключаем классификационную голову и выполняем предсказание и т.д.

Обычно обучаются в режиме self-supervised (например, предсказание следующей лексемы)

Проблема!



Начните с текстовой строки

1. → текстовые лексемы
2. → эмбеддиги текста (через словарь эмбеддингов)
3. → эмбеддинги текста/позиции
4. → стэк слоёв трансформации (самовнимание + нормализация + пропускающие связи + блоки MLP)
5. → токен CLS
6. → подключаем классификационную голову и выполняем предсказание и т.д.

Обычно обучаются в режиме self-supervised (например, предсказание следующей лексемы)

Нативное решение: imageGPT

Доклад: «Генеративное предварительное обучение на основе пикселей»

- Пиксели как бы дискретны - просто воспринимайте каждое значение цвета как отдельное слово в вашем словаре!
  - Каждый пиксель обычно представлен 24-битным значением (целые числа в диапазоне [0, 255] для каждого из 3 цветовых каналов).
  - Размер словаря 2^24 = 16 777 216!
- Кому вообще нужно столько цветов?
  - Используйте 9-битное представление (целые числа в диапазоне [0, 8] для каждого из 3 цветовых каналов)
  - Размер словаря 512
- Считайте пиксели из растрового порядка (ряд за рядом слева направо), чтобы получить входную последовательность

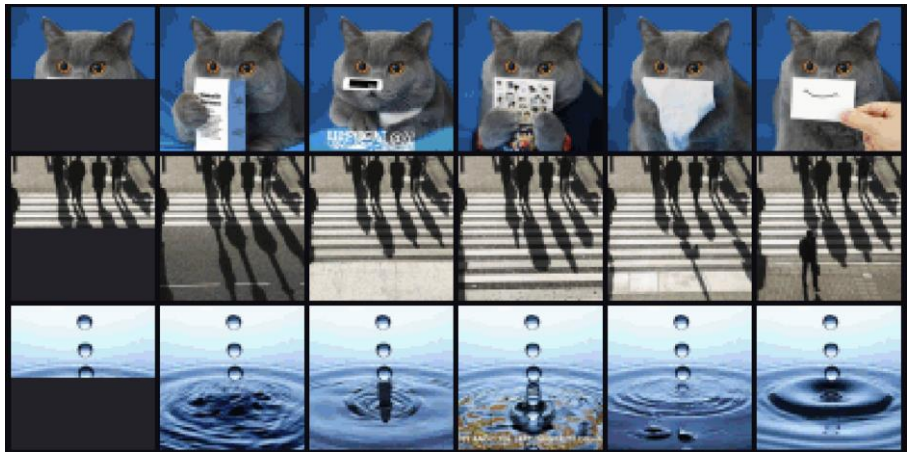
Нативное решение: imageGPT

- Другая проблема: временная сложность :
  - (Вспомним: трансформаторы имеют длину входа O(n^2)
  - И длина входного сигнала равна O(n^2) относительно длины каждой стороны
  - Изображение 256 x 256 => 65536 пикселей
  - Для справки, BERT имеет максимальную длину 512 лексем.
- Решение: просто используйте меньшие изображения
  - Максимальный размер 64 x 64
- Обучается с целью, аналогичной языковым моделям (предсказание следующего пикселя вместо предсказания следующей лексемы)

Хорошее

- Хорошие представления изображений
- SOTA по полунаблюдаемой классификации
  - Задача: классификация с ограниченным количеством помеченных образцов
  - Модель: линейный классификатор на представлениях iGPT
  - Конкурентоспособные результаты с наивным методом + много вычислений
- Хорошие генерации изображений
  - Эффективное моделирование визуальной информации

EVALUATION	MODEL	ACCURACY	PRE-TRAINED ON IMAGENET W/O LABELS	PRE-TRAINED ON IMAGENET W/ LABELS
CIFAR-10 Linear Probe	ResNet-152[50]	94.0		✓
	SimCLR[12]	95.3	✓	
	iGPT-L 32x32	**96.3**	✓	
CIFAR-100 Linear Probe	ResNet-152	78.0		✓
	SimCLR	80.2	✓	
	iGPT-L 32x32	**82.8**	✓	





Плохое

- Много считать

Плохое

- «Мы обучаем iGPT-S, iGPT-M и iGPT-L, трансформаторы, содержащие 76M, 455M и 1,4B параметров соответственно, на ImageNet. Мы также обучаем iGPT-XL, трансформатор с 6,8 млрд параметров, на смеси ImageNet и изображений из Интернета».
- «iGPT-L была обучена примерно за 2500 V100 дней, в то время как аналогичная модель MoCo может быть обучена примерно за 70 V100 дней».
  - Для справки, MoCo - это еще одна модель с самоконтролем, но в ее основе лежит ResNet, способная обрабатывать изображения с разрешением 224 x 224.
- И все это при разрешении всего 64x64!

Ну и зачем?

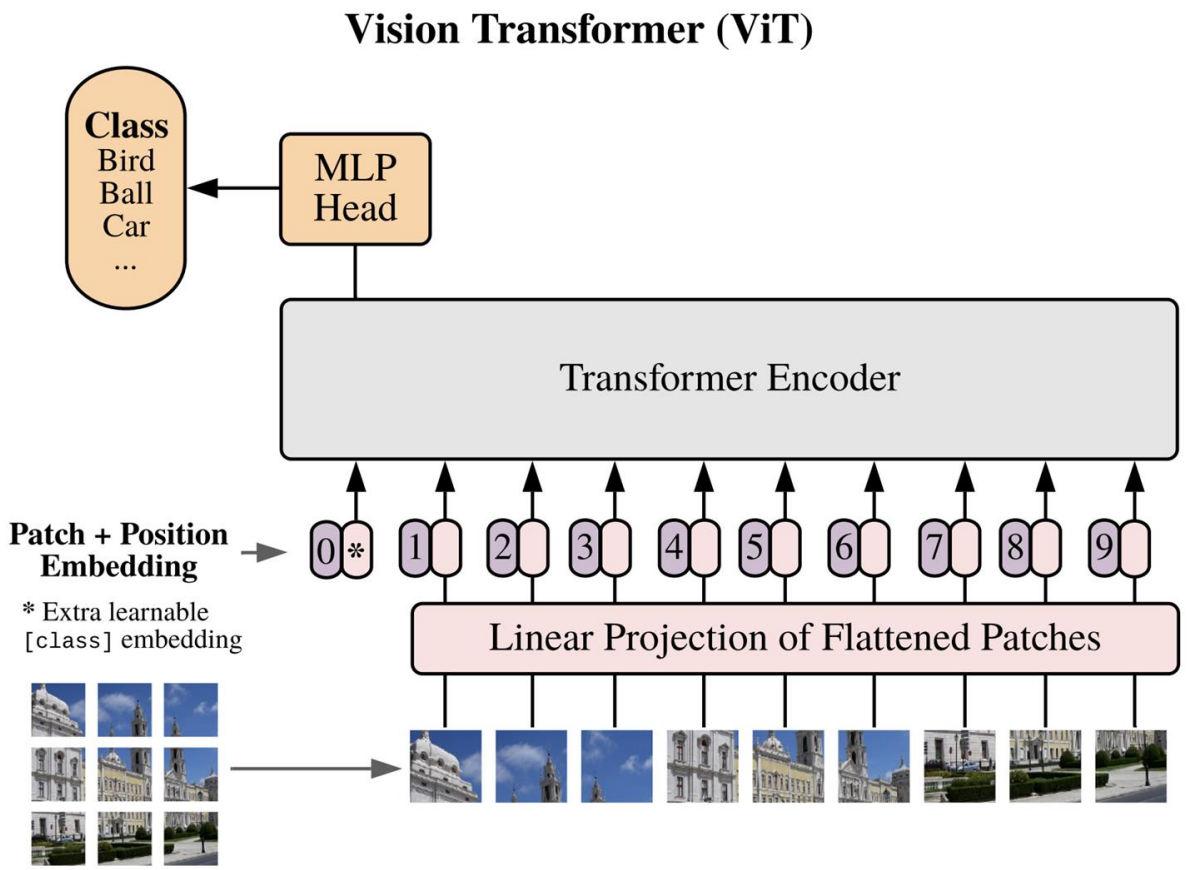
- В основном доказательство концепции
- Парадигма трансформаторов + самоконтролируемое предварительное обучение, но применительно к новой области
  - Общий метод обучения представлениям
  - Тот же метод, новые режимы

Кроме того

- *Есть* способы сделать трансформаторы более эффективными (с точки зрения архитектуры).
- Но напомним: основная привлекательность использования трансформаторов заключается в том, что они хорошо масштабируются относительно вычислений.
- Архитектуры трансформаторов должны быть простыми: само внимание - это просто огромные матричные умножения
  - огромные матричные умножения хороши для распараллеливания
  - хотим сохранить архитектуру как можно более простой

Более практичное решение

- Статья: «Изображение стоит 16x16 слов»
- Вместо того чтобы квантовать пиксели, «уменьшите масштаб» изображения, разбив его на участки, сгладив и затем линейно спроецировав их
  - Мысль: K, Q, V эмбеддинги
  - Каждый патч становится отдельным маркером последовательности
  - Более поздние архитектуры использовали слой свертки вместо патчей + линейные проекции для встраивания лексем



Более практичное решение

- Предварительное обучение при низком разрешении, тонкая настройка при высоком разрешении
  - Обучение с более низким разрешением удешевляет обучение / снижает вычислительные затраты
    - Дополнительную информацию можно найти в статье «Fixing the train-test resolution discrepancy».
- Бенчмарк на классификации ImageNet
  - Предварительное обучение на большом наборе данных, затем тонкая настройка на реальном ImageNet
  - Большие наборы данных включают ImageNet-22k (надмножество ImageNet с 14 млн изображений и 22 тыс. классов) и JFT-300M (300 млн изображений и 18 тыс. классов)

Nice.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

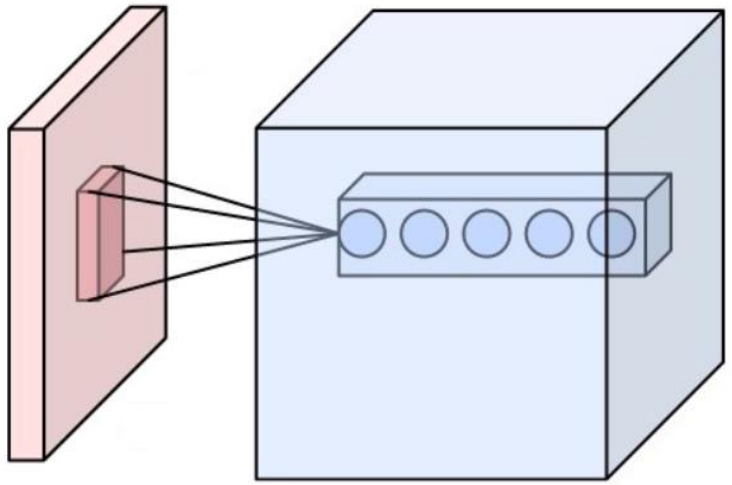
Индуктивные предубеждения

- По сути, это «предположения», сделанные моделью.
  - Если вы используете линейную модель, вы предполагаете, что данные линейны.
- ViT являются более «общими», поскольку в них меньше индуктивных предубеждений (подробнее об этом позже)

Индуктивные предубеждения

CNNs

- Локальность
- двумерная структура окрестностей
- эквивариантность перевода



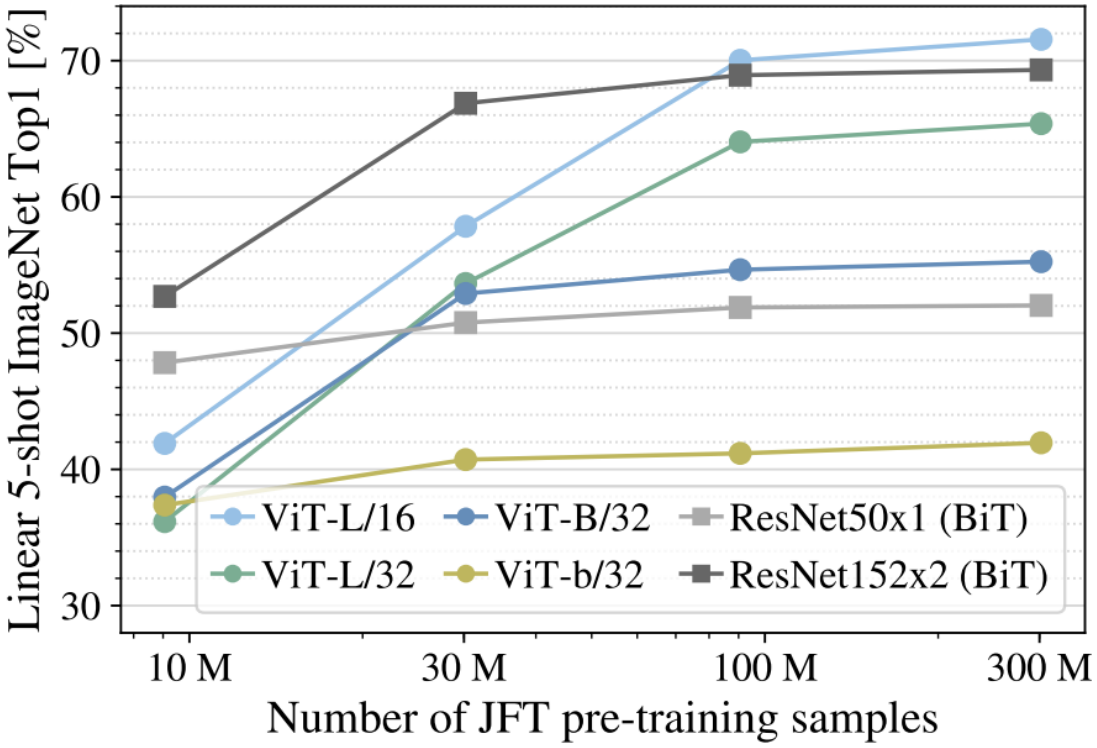
«Изображение стоит 16x16 слов»

- Патчи изображения
- Позиционная интерполяция встраивания при тонкой настройке + апскейлинг

т.е. гораздо меньше

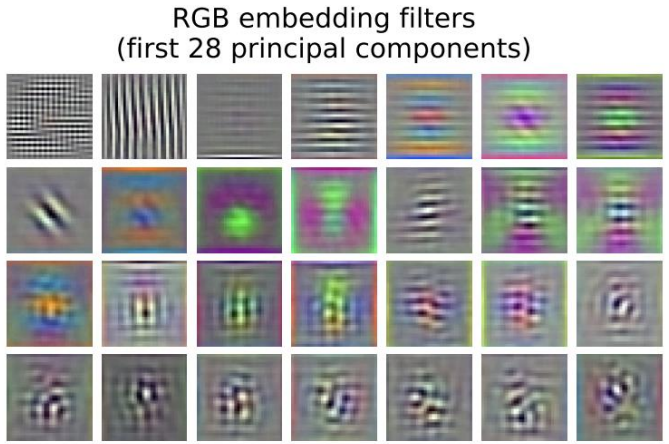
Tradeoff

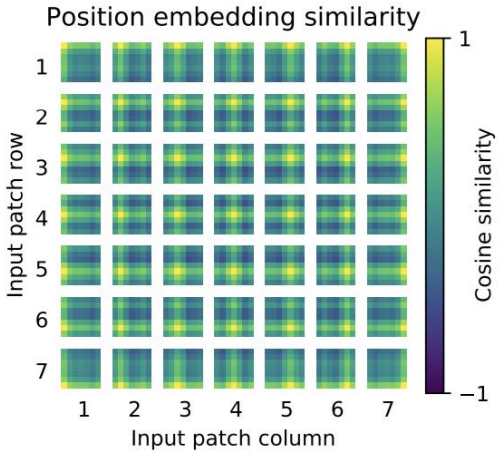
- Работает хуже, чем CNN, когда данных не так много
- Лучше, чем CNN, когда данных много

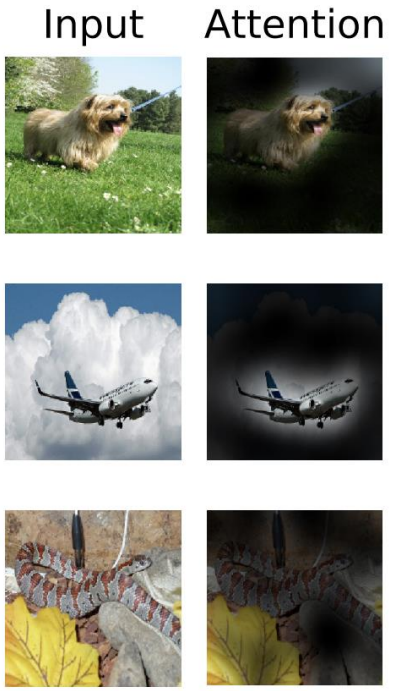


позвольте всему соотноситься со всем

- Почему происходит этот tradeoff?
- Отсутствие индуктивной предвзятости: если мы позволим соотноситься со всем во время обучения, модель должна научиться оптимальным представлениям для своей задачи
- т. е. узнавать предубеждения из данных, а не создавать их вручную

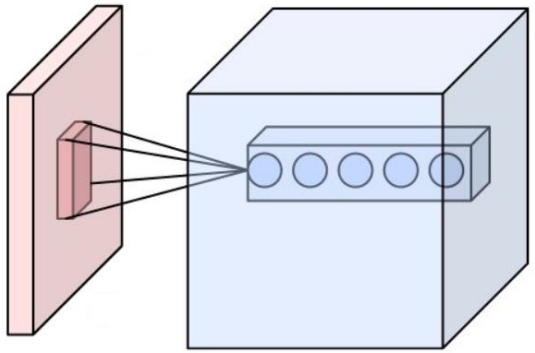


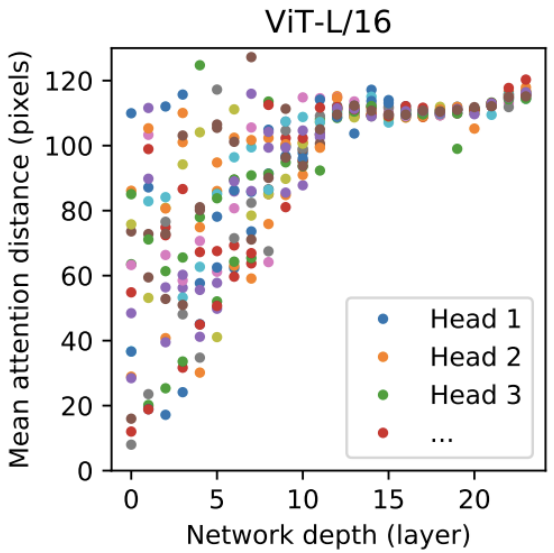




Глобальное рецептивное поле

- У CNN рецептивное поле ограничено размером ядра, особенно на ранних слоях.
- У ViT нет такого ограничения
- (Справа) Показано среднее расстояние между Q и K для разных слоев.
- График показывает, что ViT используют преимущества большего рецептивного поля





Минусы

- Патчи не очень точны.
- Представляет проблемы для таких задач, как сегментация, тонкая классификация и т.д.

Обобщение (выводы)

- Переход от индуктивных предубеждений к более общим моделям.
  - Любые специфические вещи должны изучаться на основе данных.
    - Почему?
    - Подумайте:
    - if statements против CNNs → CNNs + контролируемое обучение против трансформаторов + неконтролируемое обучение
      - Обучаемые алгоритмы, как правило, лучше (*если у нас достаточно данных).
    - Позволяет нам легко комбинировать домены. Если наша архитектура/схема обучения является общей, мы можем легко применить ее к тексту, изображениям, видео, аудио и т. д.
      - Текст, изображения... игры Atari - все моделируется с помощью трансформеров.
      - См: GATO

Обобщение (выводы)

На пути к самоконтролируемому и несамоконтролируемому обучению

- Почему?
  - Урок «An image is worth 16x16 words» - для работы трансформеров требуется много данных, но они очень хорошо работают, если они у вас есть.
    - Они также обладают совместимостью с GPU/TPU, что позволяет им масштабироваться настолько, чтобы действительно обрабатывать такой объем данных.
  - (См.: *если у нас достаточно данных*) У нас много ~данных~, но не так много помеченных данных.
    - Самоконтролируемые и «без учителя» задачи позволяют нам использовать немаркированные данные!

Трансформеры дают нам *обобщение*, где предубеждения выучиваются на основе данных, а не создаются вручную.

Т. е. пусть все соотносится со всем, и если мы дадим ему достаточно данных, оно само поймет, чем ему следует заниматься

Compute goes brrr

Статья : «Масштабные трансформеры видения»

- Та же арка, что и «Изображение стоит 16x16 слов».
- Представления о масштабировании ViTs
  - Лучшее представление масштабируется с вычислительным временем, размером модели и размером набора данных
  - Большие модели более эффективны с точки зрения выборки (т.е. им требуется меньше обучающих образцов для получения той же производительности)
- Некоторые инженерные решения для повышения эффективности обучения/точности настройки
- Модель 2B param получила ImageNet SOTA (на тот момент)

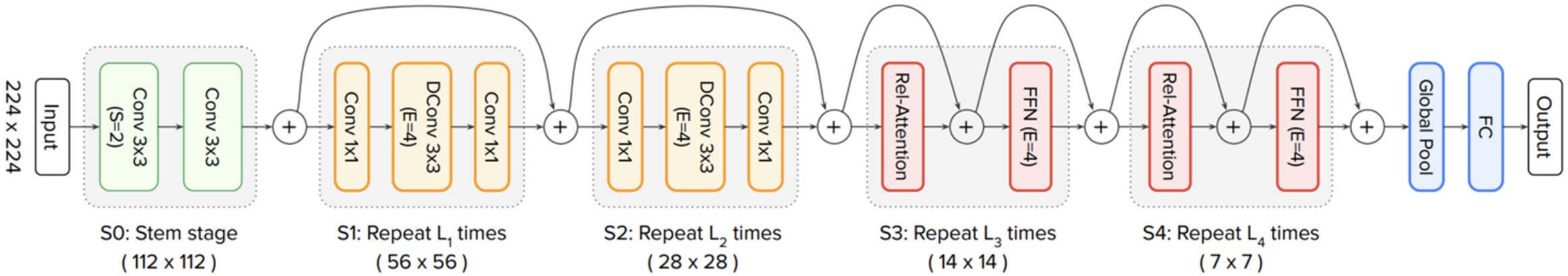
ViTs x CNNs

Статья: «CoAtNet: Объединение конволюции и внимания для всех размеров данных»

- Вспомните: отсутствие индуктивных предубеждений плохо, когда данных мало
- Добавьте *некоторые* индуктивные смещения в ViTs
- Две проблемы с трансформаторами
  - Отсутствие трансляционной инвариантности
    - -> добавить вес, зависящий только от относительного положения
  - Квадратичное время по отношению к пространственному размеру
    - Объединение в стиле CNN
- Архитектура представляет собой блоки MBConv + блоки относительного внимания
  - MBConv: блок конволюционных операций (из MobileNet V2)
- ImageNet SOTA (в то время)

$$y_i^{\text{pre}} = \sum_{j \in \mathcal{G}} \frac{\exp\left(x_i^\top x_j + w_{i-j}\right)}{\sum_{k \in \mathcal{G}} \exp\left(x_i^\top x_k + w_{i-k}\right)} x_j.$$

- ViTs x CNNs



Обратите внимание: Преобразования применяются позже, когда входные данные меньше

Размерность входного сигнала для каждого блока уменьшается (пулы перед входом)

Заключение

- ViT (и связанные с ними схемы обучения с самоконтролем) как «следующий шаг» в более и более общих моделях
  - Меньше индуктивных предубеждений и меньше меток
  - Ручная разработка признаков → CNNs → ViTs
  - Требует большого количества данных
- ViTs супермасштабируемы (позволяют нам использовать огромные наборы данных).
  - И они действительно хорошо работают!

Содержание

1. Введение
2. CLIP
3. Downstream-приложения
4. BLIP
5. OWL-ViT
6. SAM

Что такое мультимодальное обучение?

Что такое «мультимодальность»?

- У человека 5 органов чувств, и мы воспринимаем мир по-разному и целостно, используя каждое из них.
- Аналогично, у нас есть разные способы представления одних и тех же объектов
- Видео, аудио, язык, глубина и т. д.



“серая белочка”





“красная белочка”

Мультмодальные датасеты

- Наборы данных бывают в разных формах.
- При самоконтролируемом предварительном обучении мы не обращали внимания на подписи и метки.
- Можем ли мы использовать различные источники информации единым образом?



“серая белочка”



“красная белочка”

Почему мультимодальное обучение?

Мультмодальные датасеты

- Данные могут поступать из множества мест и множества наборов данных.
- Мы называем наборы данных «распределениями» по всем возможным изображениям.
- Ширина распределения каждого отдельного набора данных мала по сравнению с возможными



Проблемы обычного обучения

- «Смещение распределения» - это проблема
  - Если взять модель CV, которая была обучена на фотореалистичных данных, и оценить ее работу на эскизах, она покажет низкие результаты
  - Мы хотели бы, чтобы наша модель идеально обобщалась на всех видах данных.
- Управление различными модальностями





“Image of a tree”

Мультимодальное обучение

- У нас уже есть невероятное разнообразие данных!
- Мы хотим использовать различные связи между данными (а не рассматривать их по отдельности).
- Используя все доступные нам данные/отношения, мы сможем лучше обучаться на более разнообразных данных
- Мы можем изучать более осмысленные сжатые представления всего: изображений, текста, аудио, видео.

Сочетание различных модальностей

- Модальности не ограничиваются только текстом и изображениями
- Например, текст, глубина, аудио, изображения, тепловые карты - все это разные модальности.
- Допустим, вы хотите преобразовать звуки в визуальное представление - это будет относиться к двум различным модальностям



Мультимодальные задачи/цели

- Мы хотим объединить различные модальности, чтобы получить более полное представление о мире
- Объединение нескольких модальностей позволяет нам:
  - Максимально использовать наши данные
  - Выполнять задачи, которые было бы сложно или невозможно решить с помощью одной модальности

'Generate me an image of a fox in a grass field, in the style of a painting"



CLIP

CLIP

- Контрастивный языково-образный предварительный тренинг (2021)
- Соединяет изображения и текст
- Идея заключается в том, что если наши изображения соответствуют подписям, к которым они прилагаются, то представления, усвоенные для изображения и подписи, должны быть одинаковыми
- Мы хотим сравнить и сопоставить различные пары (изображение, подпись).
  - Примечание: В этой задаче нет инноваций в моделях и архитектуре... мы будем использовать трансформеры языка и трансформеры зрения, обученные с нуля.

CLIP

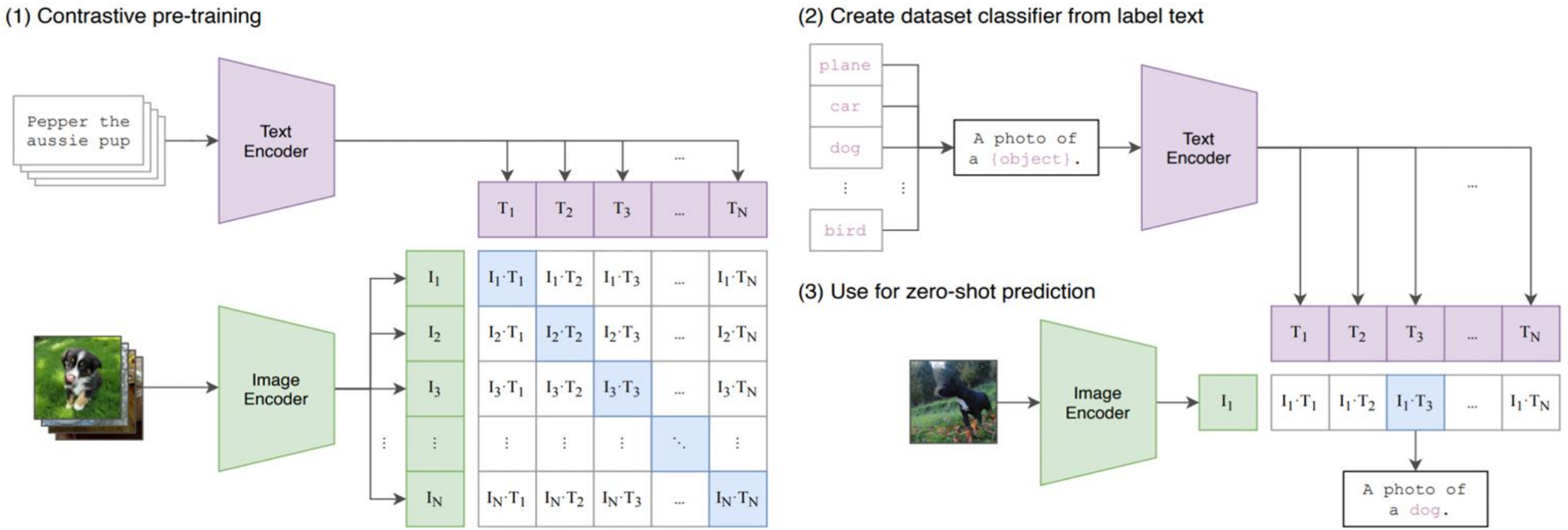


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

CLIP

- Black box визуальный трансформер и black box языковой трансформер
- Единственная уникальная вещь - это способ, которым мы формулируем нашу функцию потерь, сдвигая одинаковые вещи ближе друг к другу, а разные - дальше друг от друга

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

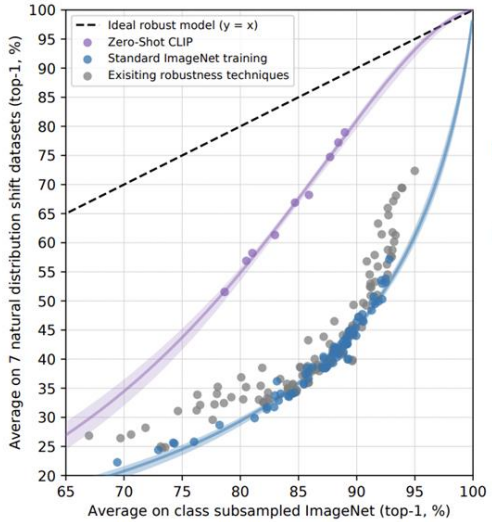
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

Эффект

- Характеристики CLIP чрезвычайно устойчивы к изменениям в домене, поскольку CLIP обучен на таком широком спектре данных.
  - Мы можем очень хорошо обобщать, нет необходимости в тонкой настройке
- Пример задачи – zero-shot классификация



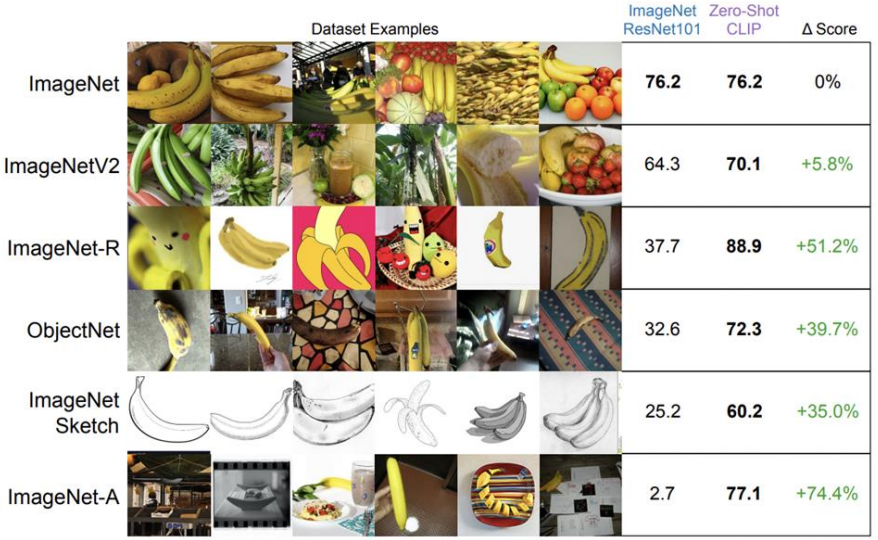
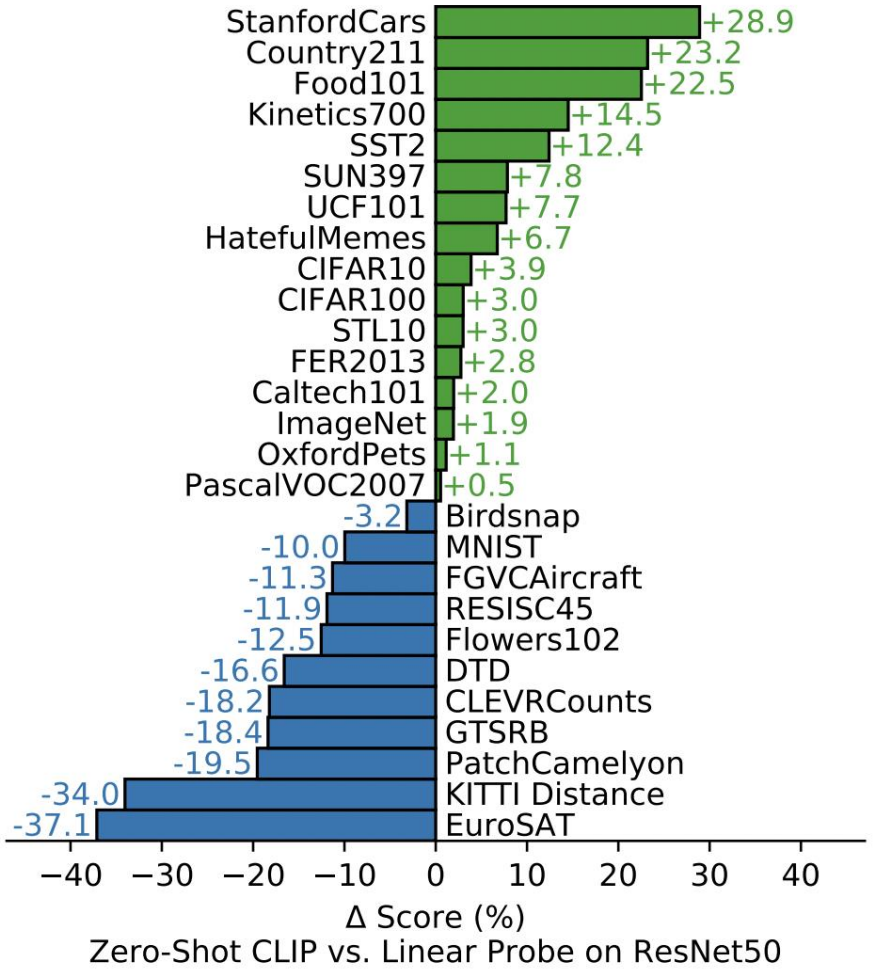


Figure 13. **Zero-shot CLIP is much more robust to distribution shift than standard ImageNet models.** (Left) An ideal robust model (dashed line) performs equally well on the ImageNet distribution and on other natural image distributions. Zero-shot CLIP models shrink this "robustness gap" by up to 75%. Linear fits on logit transformed values are shown with bootstrap estimated 95% confidence intervals. (Right) Visualizing distribution shift for bananas, a class shared across 5 of the 7 natural distribution shift datasets. The performance of the best zero-shot CLIP model, ViT-L/14@336px, is compared with a model that has the same performance on the ImageNet validation set, ResNet-101.

Эффект

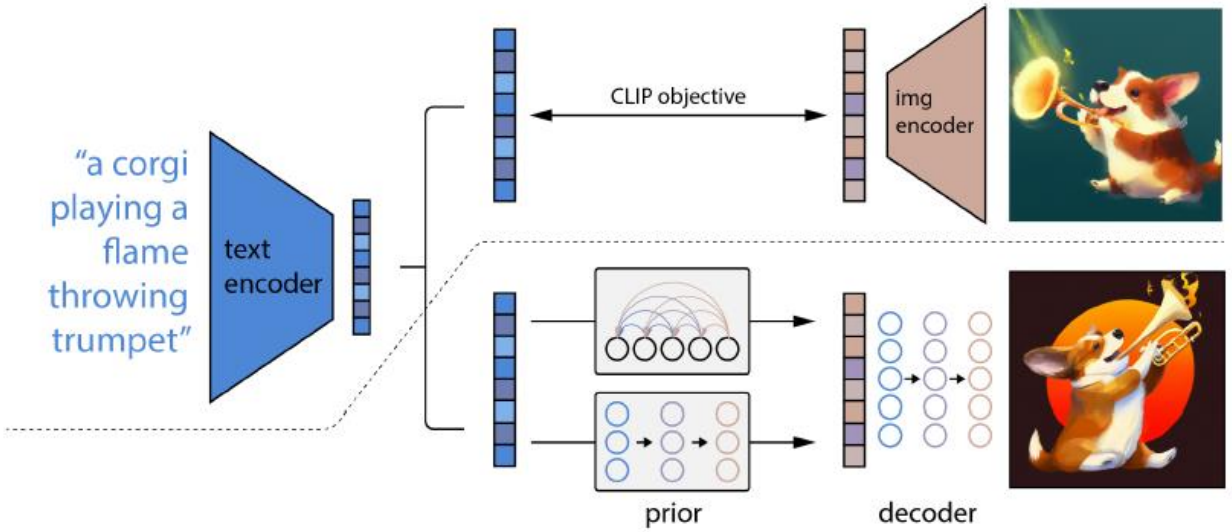
- Zero-shot латенты из CLIP даже конкурируют с контролируемым обучением на многих наборах данных
  - Для этой диаграммы линейный слой, обученный поверх предварительно обученного ResNet50, сравнивался с Zero-shot латентами из CLIP.
- Латенты CLIP очень общие!



Downstream CLIP задачи

Приложения

- Использование вкраплений CLIP в качестве экстракторов признаков, а затем передача этих вкраплений другим моделям.
- Может использоваться как форма слабого контроля, например, для создания субтитров к видео или распознавания действий.



Приложения

- Классификация изображений / легкая тонкая настройка
- Полезно для робототехники, где нам нужно компьютерное зрение для передачи в наши глубокие модели принятия решений.
- Эта идея обучения представлениям и обучения текст + изображение лежит в основе таких вещей, как стабильная диффузия





Приложения

- Можно использовать даже в 3D-видении!
  - Оптимизируйте radiance field, чьи рендеринговые изображения имеют латенты CLIP, соответствующие входной надписи.
  - Дорого для рендеринга большого количества полных изображений, но работает!
- Латенты CLIP практически универсальны

Zero-Shot Text-Guided Object Generation with Dream Fields

Ajay Jain[1,2*] Ben Mildenhall[2] Jonathan T. Barron[2] Pieter Abbeel[1] Ben Poole[2]

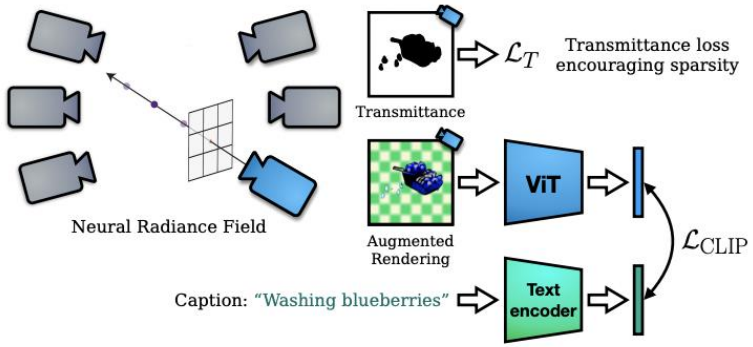


Figure 1. Given a caption, we learn a **Dream Field**, a continuous volumetric representation of an object's geometry and appearance learned with guidance from a pre-trained model. We optimize the Dream Field by rendering images of the object from random camera poses that are scored with *frozen* pre-trained **image** and **text** encoders trained on web images and alt-text. 2D views share the same underlying radiance field for consistent geometry.

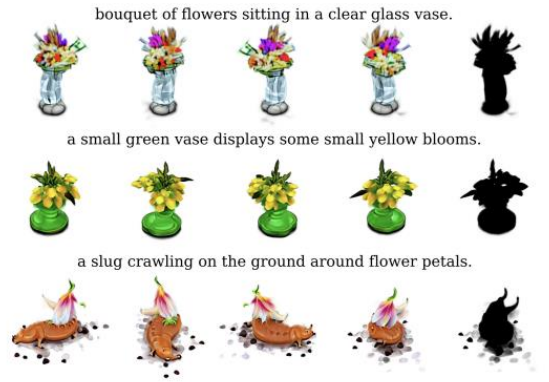


Figure 2. Example Dream Fields rendered from four perspectives. On the right, we show transmittance from the final perspective. We create diverse outputs using the compositionality of language; these captions from MSCOCO describe three flower arrangements with different properties like context and color.

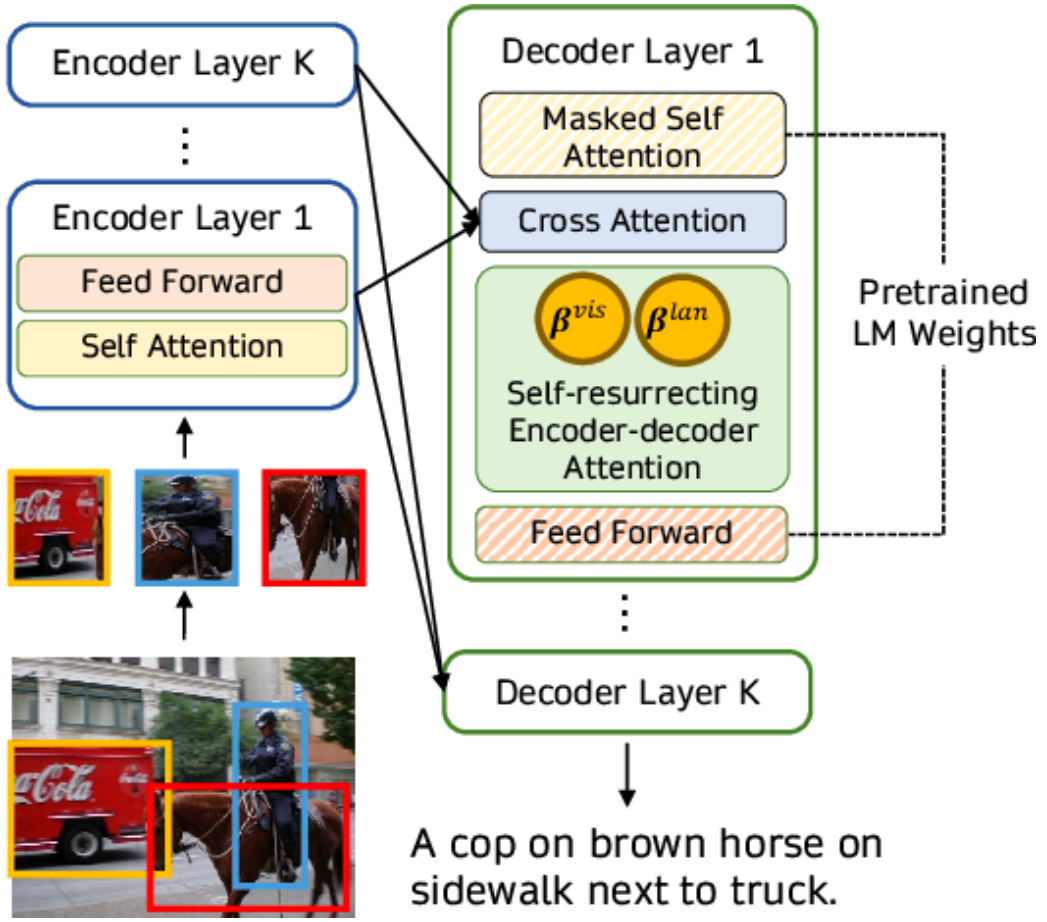
VLM, VLA и многое другое!

Языковые модели зрения (VLM)

- Обычно имеется кодировщик изображений, кодировщик текста и метод объединения информации от двух кодировщиков.
- Функции потерь обычно зависят от архитектуры и стратегии обучения

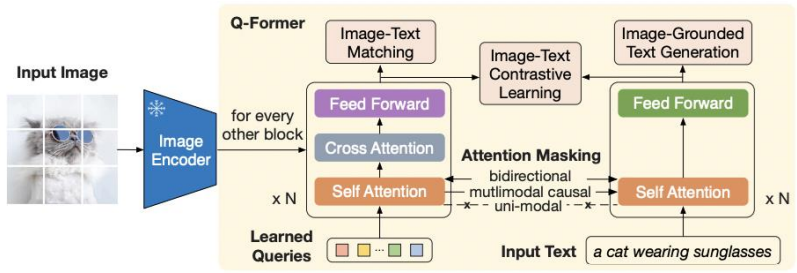
Языковые модели зрения (VLM)

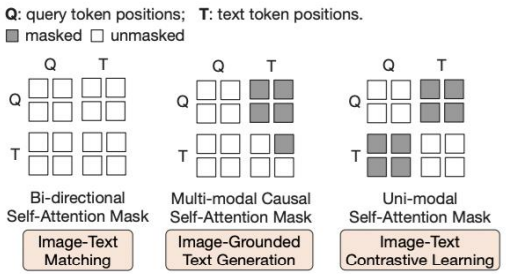
- **Contrastive learning**: Выравнивание изображений и текстов в совместном пространстве признаков контрастным способом
- **Мультимодальное объединение с перекрестным вниманием**: Включение визуальной информации в слои языковой модели с помощью перекрестного внимания
- **PrefixLM**: совместное обучение эмбеддингов изображений и текстов путем использования изображений в качестве префиксов в LM
- **MLM/ITM**: моделирование языка с маской, сопоставление изображения и текста

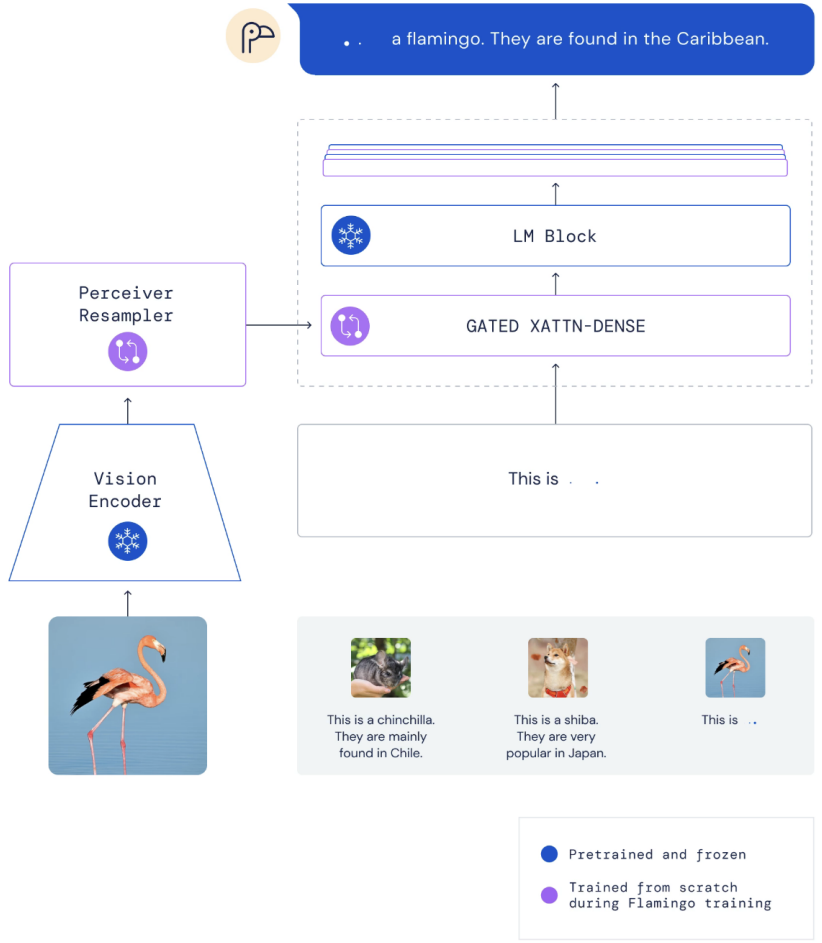


Языковые модели зрения (VLM)

- Активная исследовательская область, много новых разработок в последнее время
- BLIP2, Flamingo, Florence2 - хорошие примеры.





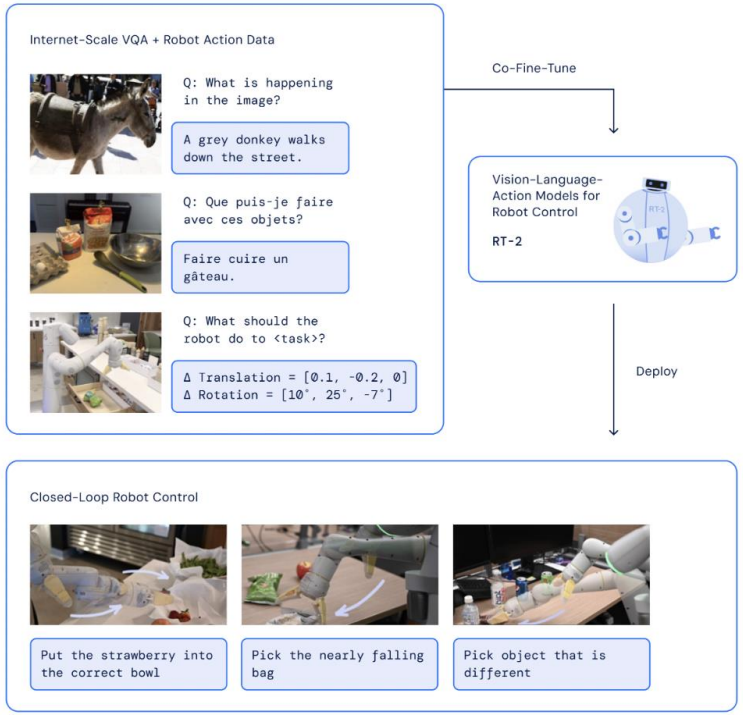


Vision Language Action Models (VLAs)

- Использует зрение и язык для связи с действиями роботов
- SayCan - ранняя попытка соединить язык с действиями.
- RT-2 - очень новая работа в этой области, которая объединяет зрение, язык и действия.



Figure 1: LLMs have not interacted with their environment and observed the outcome of their responses, and thus are not grounded in the world. SayCan grounds LLMs via value functions of pretrained skills, allowing them to execute real-world, abstract, long-horizon commands on robots.



Visual Question Answering (VQA) и генерация кода

- В последнее время был достигнут большой прогресс в решении VQA с помощью генерации кода.
- Мы разбиваем визуальные вопросы на более мелкие части и решаем их с помощью кода
- Некоторые примеры: ViperGPT, Visual Programming, CodeVQA

